# THE OMNI-COMPASS OPERATOR'S MANUAL

*How to Embed, Wire, Tune and Run Omni-Compass on Your Stack*

**AJ Dubra**
Owner and Developer

**THE OMNI-COMPASS LLC**
October 2026

**The Omni-Compass Operator's Manual**
How to Embed, Wire, Tune and Run Omni-Compass on Your Stack
October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

The Omni-Compass LLC reserves the right to change this book, the software it describes and their terms at any time, without notice. Every copy, export and printout of any part of this book carries this page, the LICENSE, the NOTICE and the Disclosures unchanged.

All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (no filing number is stated, and no grant, registration or approval is claimed). www.omni-compass.com

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

**License.** This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix O.

**Disclaimer.** The software is provided “as is”, without warranty of any kind. This book is the wiring and operating manual and carries no benchmark results; every published result carries its evidence class. A modelled result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Set in Liberation Serif and DejaVu Sans Mono, the text at 11 pt on a 16.5 pt line. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

# Contents

**Foreword** vi

**Preface** viii

**Prelude: A Night on the Cluster** x

**Introduction: The Problem, the Idea and What This Manual Covers** xii

**The key: GAIN, EVEN and LOSS** xv

**What You Are Looking At** xvii

**Quick Reference: The Card for the Glove Box** xx

**Disclosures, Declarations and Disclaimers** xxiii

**Part One. The Governor, from the Outside** 1

1. What Omni-Compass Is . . . . . 1
2. Your Stack as One Body: Muscles and Organisms . . . . . 5
3. Tuning: The Factory Tune, the Dial and the Settings You Write . . . . . 8

**Part Two. The Harness: Plugs, Wires and the Rules They Obey** 12

4. The Plug, the Adapters and the Wire Check . . . . . 12
5. The Rules the Wires Obey . . . . . 17

**Part Three. Wiring It onto Your Stack, Step by Step** 20

6. Before You Start, and the Eight Levels . . . . . 20
7. Stack by Stack . . . . . 28

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Part Four. Operating It** **38**

8. The OFF Switch, the Rules, and the Log . . . . . 38

9. Maintenance, Upgrades and Security . . . . . 42

**Part Five. Checking It on Your Own System** **44**

10. Paired Runs and Receipts on Your Own System . . . . . 44

11. Evidence Classes and How to Read a Receipt . . . . . 47

12. The Engine Version and the Three-Run Rule . . . . . 49

**Part Six. The Implementer's Guide: Omni-Compass on a Cloud** **51**

13. The Implementer's Path: From an Empty Account to a Governed Cluster . . . . . 51

14. Identity, Permissions and Least Privilege on a Managed Cluster . . . . . 56

15. Azure Kubernetes Service, Step by Step . . . . . 61

16. Google Kubernetes Engine, Step by Step . . . . . 66

17. Amazon Elastic Kubernetes Service, Step by Step . . . . . 70

18. The Response-Time Feed and the Line . . . . . 75

19. Running in the Cloud: the Machines Alone, the Bill and the Alarm Minutes . . . . . 78

20. Day Two: Running Omni-Compass in Production . . . . . 84

21. The Graphics Processor: Putting the Governor on a Card . . . . . 89

22. Observability: Feeding the Governor from Your Monitoring, and Watching It . . . . . 92

23. Change Management, Risk and Compliance . . . . . 98

24. Planning the Saving Before the Run: Capacity, Cost and the Shape of Load . . . . . 102

25. On Your Own Racks and in Private Clouds . . . . . 112

26. Hardening and the Supply Chain . . . . . 117

27. Rolling Out Across a Fleet: Many Clusters, Many Services, GitOps . . . . . 127

28. Failure Modes and the Drill Book . . . . . 133

29. A Complete Installation, Start to Finish . . . . . 139

30. Questions an Implementer Asks . . . . . 146

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Part Seven. License and Your Copy** **148**

31. License and Commercial Terms . . . 148

32. Verifying Your Copy . . . 149

**Back Matter** **150**

Glossary . . . 150

Appendix A. Command Reference . . . 156

Appendix B. The Files an Installation Touches . . . 160

Appendix C. The Gauges . . . 162

Appendix D. Troubleshooting . . . 165

Appendix E. The Muscles: What Each Is For, and How It Is Wired . . . 170

Appendix F. The Azure Account, Click by Click . . . 345

Appendix G. The Google Cloud Account, Click by Click . . . 347

Appendix H. The Amazon Web Services Account, Click by Click . . . 350

Appendix I. The Install Manifests . . . 353

Appendix J. The Implementer's Checklists . . . 365

Appendix K. The Kubernetes Governor's Settings . . . 368

Appendix L. The Audit, Record by Record . . . 374

Appendix M. The Three Clouds Side by Side . . . 379

Appendix N. The Paired Run from GitHub: Every Input . . . 384

Appendix O. The License . . . 388

Appendix P. Licensing: Questions and Answers . . . 404

Appendix Q. Third-Party Notices . . . 406

Contact . . . 408

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one governor that reads all of them at once, holds each of them inside its safe range, and puts every setting back exactly where it found it when it stops. That governor is Omni-Compass, and this book is how you put it in.

**Every machine already has a controller.** Kubernetes has its autoscaler. A graphics card has its firmware. A building has its thermostat loop, a battery has its charge controller, a power grid has its tap changers, and a robot has its servo. Engineers spent decades making these controllers safe, and they are. But each one was built to watch one thing and to protect that one thing. None of them sees the whole body. None of them knows that the cooling loop is holding back the processor, or that the autoscaler is adding machines the database cannot feed. So each one plays it safe in the middle of its range, because playing it safe is its whole job. The headroom between the middle and the edge is paid for every hour and almost never used.

**That gap is what you are wiring it to close.** Omni-Compass is a **supervisory governor**: one governor over the muscles you already run. It never replaces them and it never competes with them. It reads them, it decides, and it writes one setting at a time through a two-way plug: one wire in, one wire out. Before it writes anything, it takes a snapshot. When it stops, every setting goes back to that snapshot.

**The rule I will not bend: do no harm.** Omni-Compass must never cause a LOSS against the system's own native controller. Where it cannot bring a muscle a gain, that muscle stays native. Before a slow setting moves, the governor tries the step on that muscle and keeps it only if the muscle's own work costs no more for it. A step that costs more is undone.

**The max is the factory setting.** Omni-Compass ships tuned the way I tuned it: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought. It is never called a loss. Anyone can turn the dial down from the max, retune a muscle, or add one; nobody has to.

**One pedal.** I wanted it to drive like an electric car. Push the pedal for more, lift off for less. Lift off all the way and the settings drift back toward native. Reset hands everything back. If the governor ever crashes, its lease runs out and the plug walks every setting home, step by step. The kill switch is separate and instant, and it exists for security, not for daily driving.

**What this book holds.** This is the manual for the people who wire it: what the governor is from the outside, how to tune it, how the plug and the wires work, how to wire it onto each kind of stack one level at a time,

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

how to run it, how to switch it off, and how to check on your own system, with your own paired runs, what it does there. It is the book you keep in the glove box. When the software changes, this manual changes in the same commit.

**How I want it judged.** On your own receipts. Every paired run has exactly two arms: native, and Omni-Compass on top of native. Run it three separate times under rules you wrote down first, show every row, losses included, and read each row as one word, **GAIN**, **EVEN** or **LOSS**, by what the gauge is for. Where a receipt reads LOSS, look at the wiring first; this book tells you where.

**Where to begin.** Read the front matter first. It tells you what you are looking at before it asks you to touch anything. Then wire it one level at a time, watch it before you let it write, and judge it by your own receipts.

**AJ Dubra** Founder, The Omni-Compass LLC

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Preface

I wrote this book for the engineers who put Omni-Compass into a running system: how to embed it, how to wire it to every muscle of a stacked, clustered system or to any one muscle, how to tune it, how to run it, how to switch it off, and how to check on their own system, with their own paired runs, what it does there. It is the manual for the glove box, not a description of how the engine is built.

The book is in six parts, after the front matter.

| You are | Read |
|---|---|
| CTO, architect, head of platform | the front matter, Part One, the eight levels in Part Three, then Part Five |
| The engineer wiring it | everything, in order. Do not skip the wire check or the watch level |
| Platform engineer for one stack | Part Two, the chapter of Part Three for your stack, and Part Four |
| Data-center, facility or energy operator | the Quick Reference card, Part One, Part Three (levels five to seven, the processor and the data center under the servers), Part Four |
| Site reliability engineer, on call | the Quick Reference card, Part Four (the switches and the log), the troubleshooting appendix |
| Security and compliance | Part Four, the Disclosures, the license in the back matter |
| Procurement and counsel | the Disclosures, Part Six, the license and its questions and answers in the back matter |

**Part One, the governor from the outside,** says what Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned: the factory tune, the dial, each muscle's own card and the settings an operator writes. **Part Two, the harness,** is the universal plug, the adapters, the wire check and the rules every wire obeys. **Part Three** wires it onto a stack one level at a time and then stack by stack. **Part Four** is operating it: the OFF switch, the rules, the log, maintenance and security. **Part Five** is checking it on your own system: paired runs and receipts, evidence classes, the engine version and the three-run rule. **Part Six** is the license and how to verify a copy. The back matter holds the glossary, every command, the files an installation touches, the gauges, troubleshooting, and every muscle with its wiring and its own card.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Conventions.** `Code` is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a service reading's safe range. A step that writes to a system is marked **WRITES**. Every number in a receipt carries its evidence class: S for a model (modelled), L for live software, P for a physical meter.

**The standards every page of this manual keeps.** An installation wired by this manual should meet, and its receipts should show, the following discipline throughout:

1. **Two arms, always.** Every paired run compares native against omni, the same system with Omni-Compass on top, on the same machines, with the same work at the same moments. Omni-Compass is never run standalone and never compared as a competitor to the controller it sits on.
2. **Written down, then run.** The arms, the knob, its cover, the gauges, their aims and the readings are written down before the first counted run. Rules are fixed on one tuning case that is shown and never counted, then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt that carries the result says so.
3. **Three separate runs, one rule.** A result is run three times (A, B and C) on the same engine version, and every judged row reads exactly one of: GAIN in all three runs, LOSS in all three runs, no difference beyond the noise, or the runs disagree (chapter 12). Nobody outside The Omni-Compass LLC has reviewed or approved the published results, and every three-run table says so.
4. **Every row, losses included.** A row that went against Omni-Compass is in the receipt beside the rows that went for it. When Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; when that is fixed, the run is repeated and the loss stays in the record.
5. **Nothing read across engine versions.** Each engine version is fingerprinted (`python3 tools/omni_version.py`). A receipt names the version it ran on and is never read against a receipt of another.
6. **Do no harm.** Omni-Compass moves a slow knob only where a paired trial on the muscle itself shows the muscle never behind for it (section 1.2). Where it cannot bring a knob a gain, that knob stays native, and the receipt says "nothing for Omni to move".

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Prelude: A Night on the Cluster

This is a story, not a result. It follows one night on one cluster to show, in plain order, what each part of this book does. The shop, the times and the settings are made up for the story. A measured result belongs to a receipt, and chapter 10 says how to make one on your own system.

**9:00 p.m. The shop is quiet.** An online store runs on Kubernetes. Its native controllers are the ones almost everybody uses. The Horizontal Pod Autoscaler watches the average processor load and adds copies of the web service (pods) when that load passes its target. The cluster autoscaler adds machines when the pods no longer fit. A careful engineer picked their settings months ago, for the worst day she could imagine, and she picked them well. Tonight the cluster runs on its floor of two machines, and every customer gets an answer in a blink.

**A week earlier, Omni-Compass went on in watch mode.** For seven days it read every gauge and wrote nothing. It learned how fast each pod starts, how the response time moves when the load moves, and where each setting's safe band lies. Then the team ran the wire check: every reading came from the place it claimed, every lever moved the thing it claimed, and every write was read back. Only then did they turn it to govern. Every setting it would touch was saved in a snapshot first.

**9:40 p.m. The sale starts.** In two minutes the requests double. The native autoscaler does exactly what it was built to do, but it is built to wait: it needs the average load to climb, then it needs a decision, then each new pod needs time to start. While it waits, customers wait. With Omni-Compass on top, the compass sees the response time slide toward the edge of its band and into the cushion. It pushes. It raises the autoscaler's floor of pods, a setting the autoscaler already respects, so new pods start before the average catches up. It does not fight the autoscaler and it does not go around it. It moves one setting, through the plug, and reads it back.

**10:15 p.m. The database starts to strain.** More web pods mean more connections to the database, and the database has a limit of its own. A controller that watches only the web service would keep adding pods and make the jam grow. Omni-Compass sees the whole body. One body, one brain: before it takes the next step for the web service, it checks what that step would do to every other part. The step that would push the database past its own cushion is refused. Nothing is pushed into a loss to buy a gain somewhere else. The web service holds where the database can still keep up, and the queue drains instead of growing.

**11:30 p.m. The rush fades.** The pedal lifts. Omni-Compass does not drop everything at once, because a sudden drop is how systems stumble. It eases each setting back toward native along a smooth curve. The extra machines go away as the work goes away. The energy the cluster used during the rush shows up in the report as power given, beside the work it bought and the minutes customers did not spend waiting.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**1:00 a.m. Idle.** The pedal is lifted all the way. The cluster sits on its floor of two machines again, exactly where the engineer left it. Omni-Compass keeps reading every second. Staying native is its live decision, made again each second, not a verdict from some earlier run.

**3:12 a.m. Something breaks.** An upgrade restarts the machine that runs the governor, and Omni-Compass stops in the middle of its work. Nobody is awake. Nothing bad happens. The governor's lease runs out, and the plug walks every setting back to its snapshot, step by step. The native controllers keep running as they always have, because they never stopped. When the governor comes back, it starts from what it reads, not from what it remembers.

**7:00 a.m. The morning report.** On a live system, the proof does not come from a feeling that the night went well. It comes from paired runs: the same work, run once on native and once with Omni-Compass on top, and compared gauge by gauge. Every gauge gets one word. Time waited that fell reads **GAIN**. Work done that rose reads **GAIN**. A difference inside the noise reads **EVEN**, and the report says so plainly. A gauge that moved the wrong way reads **LOSS**, and it stays in the report until its cause is found and fixed. The energy spent on extra work reads **power given**, never a loss.

**What the night shows.** Every part of that night is a chapter of this book:

- the band, the cushions, the line and the factory tune are in chapter 3;
- the plug, the snapshot and the wire check are in chapter 4;
- one body, one brain, and blind means hold, are in chapter 5;
- the watch mode and the levels are in chapter 6, and the cluster's own wiring in section 7.1;
- the lease and the kill switch are in chapter 8;
- the paired runs and the three words are in chapters 10 to 12.

Read the rest of the front matter before you touch a wire. It tells you what you are looking at and how to read what it tells you.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Introduction: The Problem, the Idea and What This Manual Covers

**The problem.** Production systems are governed by many local feedback loops: horizontal autoscalers, firmware power and clock governors, building-management loops, battery-management systems, connection pools, servo controllers. Each loop is tuned in isolation, against its own setpoint, with margins sized for a worst-case disturbance. Three costs follow, and every operator pays them:

- **Reaction latency.** Reactive loops act on lagging, averaged signals. A Kubernetes Horizontal Pod Autoscaler recomputes its desired replicas from averaged utilization on a sync period (15 seconds by default) and damps its decisions with stabilization windows; new pods then need time to schedule and start. Queues form before capacity arrives.
- **Static over-provisioning.** Worst-case margins persist through the far more common nominal regime. The operating point sits well inside the safe region, and the headroom between it and the constraint boundary is paid for and rarely used.
- **Uncoordinated coupling.** Loops share resources: power, cooling, connections, memory, network. One loop's correction is another loop's disturbance. More web replicas saturate the database's connections; more cooling pushes a power feed toward its limit.

Operators close these gaps by hand, with fixed settings and runbooks. That is safe, and it leaves the region between the nominal operating point and the safe boundary unused nearly all of the time.

**The idea.** Close one supervisory loop around the native loops. Omni-Compass treats each setting a native controller already exposes (a replica floor, a power cap, a clock ceiling, a temperature setpoint, a pool size) as an actuator and each native telemetry stream as a measurement, and governs the whole body from above. Seen from the outside, every decision period runs the same five steps, and those five steps are all an installation has to provide for:

1. **Sense.** It reads each muscle's own gauges through the plug: utilization, response-time percentiles, queue depth, temperature, power, state of charge.
2. **Place.** It places each reading in its band, from calm to the line, with a 5% cushion at each edge, so that every muscle's state is read on the same scale whatever its units.
3. **Decide.** It chooses at most one bounded step per actuator, smooth and never hammering, clipped to the actuator's cover, the hard range nothing can leave.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. **Actuate.** It writes the step through the plug, idempotently, and reads the value back to confirm the write landed.
5. **Verify.** It compares the muscle with and without the step in a paired trial on the muscle itself, judged on the muscle's own cost per unit of work, and checks every coupled muscle against its own cushion. A step that fails either test is undone.

The installation supplies the wires (sense and actuate) and the settings (the band, the line, the cover and the tune); the governor does the rest. Nothing in an installation re-implements the decision, and nothing in this manual asks the installer to.

**What Omni-Compass is not.**

- It is **not a replacement** for your controllers. Native keeps running underneath, always. Omni-Compass only moves settings that native already exposes and respects.
- It is **not a black box**. Every decision is written to a log, with the reading that caused it and the setting it changed (chapter 8).
- It is **not a one-time tuning script**. It decides again every second, and staying native is one of its decisions.
- It is **not a cost-cutter first**. It gives power where the power buys work or speed, and it shows that power beside what it bought.

**The promise, in three parts.**

1. **Do no harm.** A step stays only where the muscle loses nothing for it, measured on the muscle itself. Where Omni-Compass cannot bring a gain, the muscle stays native.
2. **Everything comes back.** Every setting is saved before the first write and restored when Omni-Compass stops. If the governor crashes, its lease runs out and the settings walk home on their own. A separate kill switch stops everything at once.
3. **Every claim is checked.** Each paired run compares two arms, native and Omni-Compass on top of native, on the same machines with the same work, three separate times. Each reading is one word, GAIN, EVEN or LOSS, decided by what the gauge is for, and each carries its evidence class: **P** for a physical meter, **L** for live software, **S** for our own models. A model is never shown as proof of hardware.

**What it governs.** The catalog names 1,107 muscles in 70 families, the settings that operators everywhere already turn by hand: machine pools and replicas, power caps and clock limits, cache sizes and connection pools, cooling setpoints and battery reserves, robot speeds and drone-swarm settings, and many more. Appendix E lists every one: what it is, what Omni-Compass reads from it, what it moves, and the factory setting it runs on. Chapter 2 shows how the muscles of one stack are grouped into one body.

**What you need before you wire it.**

- **A license for any use beyond evaluation.** Production use requires a signed, paid Omni-Compass Enterprise License.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Read access first.** Omni-Compass starts in watch mode: it reads and computes, and writes nothing.
- **The wire check.** Every reading must come from where it claims, every lever must move what it claims, and every write must read back. Nothing writes until the check passes.
- **Your own service line.** The response time or other limit your service must meet comes from your own service objectives, not from ours.
- **Paired runs on your own system.** The only number that applies to your system is the one your own paired runs produce, on your own receipt.

**What this manual covers, and what it does not.** It covers how to embed, wire, tune, run and check Omni-Compass on a stacked, clustered system or on any one muscle, the way a vehicle's manual covers its controls, its settings, its service and its warning lights. It does not cover how the engine is built, and it carries no benchmark results: those are kept with the code, each in its own table with its evidence class.

**How to read what follows.** Next comes the Quick Reference card, the page to keep beside the system. Part I describes the governor from the outside and how to tune it; Part II, the plug and the rules its wires obey; Part III, the wiring, level by level and stack by stack; Part IV, operating it; Part V, checking it on your own system. Read in that order, nothing in the wiring chapters should surprise you.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# The key: GAIN, EVEN and LOSS

Every judged reading in Omni-Compass is one of three words: **GAIN**, **EVEN** or **LOSS**. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

| Word | What each letter stands for | Its sense in underwriting | What it means in a table |
|---|---|---|---|
| **GAIN** | **G** Growth (asset appreciation); **A** Advantage (risk reduction); **I** Improvement (margin expansion); **N** Net-positive (profitable outcome) | surplus and positive variance | the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut |
| **EVEN** | **E** Equalized (zero variance); **V** Verified baseline (expected norm); **E** Equilibrium (balanced risk); **N** Neutral (no net change) | the break-even baseline: equilibrium | native and Omni could not be told apart: the same, or no difference beyond the noise; a knob native already runs at the top of its range is EVEN, at its limit on native |
| **LOSS** | **L** Leakage (capital drain); **O** Off-track (adverse selection); **S** Setback (claim liability); **S** Subpar (negative variance) | actuarial deficit and exposure | the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures |

How the sign of a number becomes a word:

- **Time waited, missed or failed** (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- **Work done** (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- **A resource spent or held** (energy, machines, memory, processor time, a bill, wear on a part): a fall is a **cut**, and a cut is GAIN. A rise is **power given**: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves).
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads EVEN (the same as native); AT ITS LIMIT ON NATIVE reads EVEN (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads EVEN (inside the noise, within the guardrails); NOT ESTABLISHED reads EVEN (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

A reading counts only when it holds the same way in all three of our runs, A, B and C, with every 95% interval clear of zero: "GAIN in all three of our runs". The other readings are **EVEN, no difference beyond the noise** (in a run the interval includes zero, so native and Omni could not be told apart: that is the result), **the runs disagree** (clear runs pointing different ways: the test is unstable there and is looked into) and **shown, not judged** (a gauge declared in advance as shown only, such as the knob's own setting).

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, break-even or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (EVEN). Its first rule is do no harm, so **LOSS should never appear**: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, EVEN and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# What You Are Looking At

**What Omni-Compass is.** Omni-Compass is a supervisory governor. It is never a controller of its own and never a replacement for one: it sits on top of the controllers a system already runs (the Kubernetes autoscaler, a database's connection pool, a GPU's firmware, a building's battery controller, a power grid's tap changer, a robot's servo) and moves only the settings those controllers already expose to an operator, through one wire in (what it reads) and one wire out (what it writes). Everything the native controller does, it still does. Pull the plug and every setting walks back to where the operator left it (the lease); the kill switch returns them at once.

**What "native" means.** Native is the same stack, the same muscles, the same machines and the same work, run the way everybody runs it today: the shipped controllers at their shipped or operator settings. Every paired run has exactly two arms, native and omni, where omni is that same native stack with Omni-Compass on top. Nothing else differs between the arms, so whatever differs in the result is what Omni-Compass did.

**What a muscle and an organism are.** A muscle is one setting a native controller exposes (a pool's size, a power cap, a clock ceiling, a replica count, a battery's charge rate) together with the gauges that show what it does. An organism is a stack of muscles governed as one body: the muscles of one company's clusters, databases, caches, cards, cooling and power, read by one governor and judged together, so that no muscle is moved at another's expense. Chapter 2 sets out how a stack is grouped into one body; Appendix E lists every muscle Omni-Compass can be wired to, with what it reads and what it moves.

**What you see when it is hooked up.** Every paired run ends in a receipt with the same shape: the two arms side by side, every gauge of the system (the work done, the response times, the misses and failures, the machines and energy spent), the paired change with its 95% interval, and one word per gauge, GAIN, EVEN or LOSS (the key, above). A three-run table repeats the paired run three times, A, B and C, on the same engine version, and a word counts only when all three runs agree (chapter 12).

**How to read a receipt, in order.**

1. **The engine and the evidence class.** The header names the engine version (`python3 tools/omni_version.py`) and the class (P, L or S; chapter 11). A receipt is never read across engine versions, and S is never read as proof about real equipment.
2. **The work and the service rows first**: work done, response times, time over the line, failed requests. These say what the system delivered. A GAIN here is the point of the governor; a LOSS here is the first thing to look for.
3. **Then the resource rows**: machines, energy, memory, connections. A cut is a GAIN. A rise is **power given**: what Omni-Compass put in to buy the work and speed above it. Read the two together: power

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

given that bought nothing is a cost to find and fix, starting with the wiring.

4. **Then the noise.** EVEN, no difference beyond the noise, is a result, not a failure. The runs disagree means the test is unstable there and is looked into.
5. **Then the clock and the machine.** Where the test machine itself is the limit (its processor saturated, native alone failing a large share of requests), the receipt says so; a test machine's limit is never read as the governor's result, and the test is rebuilt so the machine no longer decides it.
6. **Every row is shown**, LOSS included. Nothing is cut from a receipt because it went against Omni-Compass.

**How the governor tells GAIN from LOSS.** The governor does not read a plus or a minus sign as good or bad. Before it moves anything, every gauge it reads is declared with its aim, and the aim decides the word: for time waited, missed or failed a fall is GAIN; for work done a rise is GAIN; for a resource spent or held a fall is a cut, and a cut is GAIN, while a rise is power given, kept only while it buys the work or the speed shown beside it. A slow knob moves only as far as a paired trial on that muscle has shown costs its own work nothing (section 1.2); a step that costs more is undone, and where no step passes the muscle stays native, which is EVEN. Inside a body, a step that would push another part past its cushion is refused, so no GAIN in one muscle is bought with a LOSS in another (chapter 5).

**Wiring it: who, what, when, where, why and how.** Read this before the wiring chapters; a governor wired without it may run and still not be the mechanism.

- **Why.** To get more work, faster, with fewer misses, from the machines and the energy already paid for, by letting one governor move the settings native controllers already expose, in the direction each gauge's aim asks, and only while no other gauge pays for it. If nothing can be gained, the purpose is met by leaving native alone.
- **What.** Two things per muscle and nothing else: one wire in (the gauges that show the muscle's state and its work, read live from the system, never typed into code) and one wire out (the one setting native already exposes to an operator, with its hard range read from the system). The governor does the arithmetic; the wiring never re-implements it.
- **Where.** On top of the native controller, never in place of it: on the autoscaler's target and ceiling, not on the pods; on the pool's size, not on the queries; on the card's power limit, not on its firmware. Every muscle of the stack is wired into the same governor, so the body is judged as one.
- **Who.** The operator who owns the system grants each wire, decides which knobs Omni-Compass may write and which it only watches, and keeps the kill switch. The governor writes only what it was granted; the native controller keeps doing its own job.
- **When.** Watch first: the governor reads every wire and writes nothing until the wire check passes and paired runs on the operator's own system show the receipt. Then one level at a time (the eight levels, chapter 6). It reads and decides once a second, all the time; staying native is a live decision, made again every second, never a setting left on.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **How.** The plug snapshots each setting once, at the start; the governor reads every gauge, places each muscle in its band, tries one step on one muscle at a time inside a body, keeps it only if the paired trial shows a GAIN and no LOSS, and hands every setting back to the snapshot on reset, on the lease after a crash, or at once on the kill switch. The proof is the receipt: native and Omni-Compass on the same work, every gauge, three runs, every row shown.

**What the end result is.** A system fully hooked up runs native exactly as before, with one governor reading every gauge of every muscle once a second and moving, muscle by muscle, only where a paired trial has shown a GAIN with no LOSS anywhere in the body; where it cannot help, the muscle stays native, which is EVEN, not a failure. The intended outcome, stated as the claim the receipts test: more work, faster, with fewer misses, on the same or fewer machines, with every unit of power given shown beside what it bought. Whether a given system gets there is answered only by its own receipts.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Quick Reference: The Card for the Glove Box

Keep this page where the people who run the system can reach it, the way a driver keeps the card in the glove box. Every line is explained in the section named beside it; nothing on this page replaces those sections.

**What it is.** A supervisory governor on top of the controllers you already run. It never replaces them: it moves only settings they already accept, inside a band you set, and hands every setting back when it stops (sections 1.1 and 4.1).

**The two names.** *Native* is your system as it runs on its own. *Omni* is the same system with Omni-Compass on top. There is no third arm and no other name (section 1.1).

**The modes** (section 1.3).

| Mode | What it does | When |
|---|---|---|
| Off (manual) | nothing; native runs alone | before installing, and whenever you choose |
| Watching | reads every gauge and logs what it would do; writes nothing | the first level of every installation (level 1) |
| Autopilot | one pedal: push for more, lift off for less, idle (lifted all the way) and reset, by its law | after the watch level has passed |
| Cruise | machines in service up to the pool's own top while a known queue waits | batch work |

**One pedal**, like an electric car's. *Push*: capacity added at once as traffic climbs, never held back. *Lift off*: capacity eased off a step at a time, handed back toward native, never below idle. *Idle*: the pedal lifted all the way, the floor of two machines in service, ready for the next burst. *Reset*: every setting handed back to where native had it, read back, and the record removed.

**The ways it stops, and what each leaves behind** (chapter 8).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| You want to, or it happens that | Do this | What is left |
|---|---|---|
| a run ends | nothing: the reset runs by itself at the end of every run | every setting at the operator's value, read back |
| stop everything at once, for any security reason | `python3 tools/omni_switch.py off` | every governor hands back and exits; none starts again until `python3 tools/omni_switch.py on` |
| stop one muscle | its kill file, or `OMNI_KILL=1` | that muscle handed back; the others run on |
| Omni-Compass dies without handing back (killed, crashed, hung) | nothing: the watchdog's lease hands back for it | every setting back at the operator's value, read back, as soon as the watchdog finds the process gone or its lease run out (chapter 8) |

**The eight levels of an installation** (section 6.3). Take them in order; go to the next only when the pass holds.

| Level | What it touches | Pass |
|---|---|---|
| 0 | nothing: `python3 verify.py` | `VERIFICATION: PASS` |
| 1 | nothing: watch mode, with a read-only identity | zero writes, and readings your operators agree with |
| 2 | the pods, through your autoscalers (**WRITES**) | p95, p99 and failed requests never behind native, over paired runs (chapter 10) |
| 3 | the machines, through the release gate and the verdict (**WRITES**) | fewer machines in service, with no LOSS on any service gauge |
| 4 | the replica decision itself (an option, not a step) | as level 3 |
| 5 | a GPU box, two wires: the clock ceiling and the power limit (**WRITES**) | work per energy up or equal; no request more than 2% slower at the median, p95 or p99 |
| 6 | the processor's clock and power on bare metal (**WRITES**) | energy per unit of work down, no LOSS on any service gauge |
| 7 | site power and the supply-air setpoint (**WRITES**) | the site meter's energy down, every rack inlet inside its band, no LOSS on any service gauge (section 7.10) |

**Before any level that writes:** run the wire check (section 4.4), start in watch, and keep the master switch in reach.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Warning signs, and the first thing to do.**

| You see | First | Then read |
|---|---|---|
| a service slower or failing more with Omni than without | `python3 tools/omni_switch.py off`: the system is native again | section 4.5, wired right or wired wrong |
| a setting at a value Omni-Compass did not write | nothing: it stops writing that lever by itself (one writer) | section 5.2 |
| a meter that is blind or late | nothing: blind means hold; it moves nothing it cannot see | section 5.3 |
| your results point the other way from the published ones | presume the wiring first, not the law | `DISCLOSURES.md`, Chapter 4 |

**Who to call.** Licensing, pilots and the Omni-Compass Enterprise License: The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, www.omni-compass.com. Security reports: the process in `SECURITY.md`, never a public issue.

**What the letters of the number mean.** Read in order, DEX-YLL-1 is the way the engine runs:

| DEX | YLL | 1 |
|---|---|---|
| **Data Exchange**: the input. In software, data exchange is how one system's readings reach another. Here it is the two-way plug, one wire in and one wire out: the system's own readings come in, Omni-Compass's settings go out. | **Yield Limit Line**: the boundary. In engineering science, the yield limit is the most load a material can carry and still spring back unharmed; past it, the damage is permanent. Omni-Compass runs every muscle up to its own yield limit line, read live, and never past it. | **1**: the output. In code, 1 is true, the finished state; in science, 1 is unity, the whole. Here it is one closed loop: one body, one brain, every result read back and the next step decided from it. |

Read as one sentence: Omni-Compass takes the system in through a Data Exchange (DEX), runs every setting up to its Yield Limit Line (YLL) and never past it, and closes it all into one loop (1). Science and software people read each part on sight.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Disclosures, Declarations and Disclaimers

This page is the one place every declaration about Omni-Compass is made. The README, the manual, the book, every guide and every report point here. Where any other page seems to say more than this page, this page governs.

## Rights

1. Omni-Compass, its engine, its mathematics, its software and its documentation are the property of The Omni-Compass LLC. All patents, copyrights and trademarks covering them have been filed in the United States by The Omni-Compass LLC. No filing number is stated here, and no grant, registration or approval is claimed.
2. The software is licensed for evaluation and simulation only (`LICENSE`). Running it in production, on any system beyond evaluation, or in any product or service, requires a signed, paid Omni-Compass Enterprise License.
3. Names of other companies and products (NVIDIA, Kubernetes, Red Hat OpenShift, Amazon EKS, Google GKE, Microsoft AKS, Karpenter, Lambda and others) are the property of their owners. They identify the systems Omni-Compass was tested with or connects to. No affiliation, endorsement or certification by any of them is stated or implied.
4. All rights reserved. The Omni-Compass LLC reserves the right to change the software, the documentation, the results and the terms at any time, without notice. Every copy, fork, export, report, archive or printout of any part of Omni-Compass carries the README's notice, `LICENSE`, `NOTICE` and this page unchanged. The Omni-Compass LLC, www.omni-compass.com

## What a result is, and what it is not

1. Every result carries its evidence class: **T** a theorem, **V** verified in code, **S** our own models (modelled), **L** live software (real Kubernetes on GitHub's machines), **P** a physical meter (a real card's own power meter). A model is a statement about the model. A software benchmark is a statement about the software and machine it ran on. Only a meter speaks for hardware.
2. The models were written by the same people who wrote the law. They show how the mechanism behaves; they are not an independent test.
3. On Kubernetes in kind every machine stays powered, so energy there is a declared model, not a meter.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

4. Every comparison is paired: the same seed, load and clock with and without Omni-Compass, in an order rotated by repetition. Intervals are 95% intervals over the paired repetitions. Ratios across seeds are summarised as geometric means. Labels are assigned by code from rules written and committed before the run (`docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md`), never by hand.
5. Losses and failed runs are kept beside the wins (`docs/EVIDENCE_LEDGER.md`, `results/live/`, git history). The first real-card run, for example, saved energy (+3.6% work per energy, proven) and made the slowest answers 58.5% slower; it is published as it happened.
6. No result is a promise of a saving on any other system. No dollar figure in this repository is a measured result. Nothing here is a vendor certification, a safety certification, legal advice or investment advice.

## The rule Omni-Compass keeps on every muscle

Omni-Compass sits on top of a muscle that already works. By default it moves the muscle's settings toward more work, faster, up to the muscle's own limit and never past it (the max); on the dial anyone can tune it down toward the same work with less (the tune-down). Either way it keeps a step only where the muscle is never behind native for it: a step that costs the muscle more for each piece of work, or loses more than **2%** in any measure (work done, response time at the median, p95 or p99, time over the service line, or the machine's own cost of running Omni-Compass), is undone. A resource spent that rose is **power given**, shown beside the work, the speed and the wear it bought, never called a loss. Where no move passes that test, Omni-Compass leaves the muscle exactly as it runs alone.

**The factory tune.** The engine in this repository is presented as The Omni-Compass LLC tuned it and ships it: a self-running engine, set at the factory. Every muscle in the catalog is wired, and each runs on the setting found for it: its own card where one was found on development seeds and kept under the strict rule (364 muscles, `realms/muscle_cards_v5.csv`), the engine's own setting for every other muscle, all frozen and fingerprinted by version (`python3 tools/omni_version.py`). Muscles differ: no single setting is right for every muscle, family, sector, domain or realm, and none is presented as universal. Every published result is this factory tune. As with any engine, a licensee may then add on, retune, upgrade or tune down any muscle or any combination of muscles (the dial, the cards, the bands and the service lines are all exposed); a changed tune is the licensee's own configuration, and its results are its own, not the published ones. On a live system the verdict (`omnicompass/verdict.py`) measures it with paired trials on the muscle itself before every step. Every result reports each measure, and where one read LOSS at all, by however little, the result says so and by how much.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The wiring declaration

**THE PUBLISHED BENCHMARKS ARE THE RECORD OF WHAT OMNI-COMPASS DID ON THE SYSTEMS, LOADS AND ENGINE THEY NAME, EACH WITH ITS EVIDENCE CLASS. WHERE A RESULT ON ANY OTHER SYSTEM DIFFERS FROM THEM, OR ANYTHING LOOKS WRONG, THE FIRST PRESUMPTION IS THE WIRING, THE SETTINGS, OR HOW OMNI-COMPASS WAS IMPLEMENTED OR READ, NOT THE LAW. CHECK THOSE FIRST (THE MANUAL'S "WIRED RIGHT OR WIRED WRONG") BEFORE DRAWING ANY CONCLUSION ABOUT OMNI-COMPASS.**

**Omni-Compass acts only through the wires it is given.** It reads the meters it is pointed at and moves the settings it is allowed to move. If a reading is the wrong one, slow, or blind; if a lever is the wrong one, already owned by another controller, or given a range (cover) that is wrong for the machine; if a band or a service line is set for a different workload; then Omni-Compass will do exactly what its law says with the wrong information, and the result will not be the published one.

Therefore:

1. **It cannot be slapped on.** Every installation goes through the levels of the manual in order (chapter 6), starting read-only, and every writing level begins with the wire check (manual, section 4.4) and a watch arm that must equal native.
2. **If your paired receipts differ from the published benchmarks in direction** (service LOSS, or no saving where one was shown), **the first presumption is wiring, not the law.** Confirm the installation with the checks in the manual's section 4.5 (*Wired right or wired wrong*) before drawing any conclusion about Omni-Compass.
3. **This has happened to us, and is published.** On the first real card (NVIDIA A10, 2026-10-02) the governor read being busy as trouble, set its power lid under the card's own working draw and allowed the clock under the card's own working clock; the card served bursts more slowly and the slowest answers were 58.5% slower. The cause was found in the card's own samples and corrected (`docs/GPU_PREREGISTRATION.md`, amendments 6 and 7). That is what a wiring fault looks like, and how it is found.
4. Omni-Compass never writes outside a lever's cover, never fights another writer (it stops and leaves that lever alone), reads back every write, and returns every lever to the value it read before its first write when it stops or when the OFF switch is used. These protect the machine; they do not make a wrong wiring right.

## Safety and responsibility

1. Run watch mode first; run the wire check before any write; keep the master OFF switch (`python3 tools/omni_switch.py off`, the whole harness at once) in reach at every level, and use it at the first sign of anything wrong, a suspected breach included.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2. Omni-Compass is a supervisory governor. The machines' own controls (firmware, autoscalers, safety systems) stay in place and keep their own protections; Omni-Compass sets only values they already accept.
3. Do not connect Omni-Compass to safety-critical systems (vehicles, medical devices, grid protection, life safety) except as a modelled study, and never without the qualified review those systems require.
4. The software is provided "as is", without warranty of any kind, as stated in `LICENSE`. The operator is responsible for its use on their systems.

## Changes to the work and to this page

1. The software, the law's settings, the benchmarks, the results, the documents, the licensing terms and this page may change at any time, without notice. The version in the repository's `main` branch at a given commit is the version that commit describes; `RELEASE_MANIFEST.json` and `python3 verify.py` identify it.
2. Statements about work in progress, planned features, expected results and future runs are expectations, not promises. Where a result has not yet been measured, the documents say so (`docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, Chapter 8).
3. A signed Omni-Compass Enterprise License governs its own terms for its own term.

## Privacy and data

Omni-Compass contains no telemetry. It sends nothing to The Omni-Compass LLC or anyone else, and makes no network connection of its own except to the systems an operator points it at. Its logs and receipts stay on the operator's machines.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Where everything is

| Question | Page |
|---|---|
| What has been measured, and how strongly | `docs/STATE_OF_PLAY.md`, `docs/DOSSIER.md`, `docs/EVIDENCE_LEDGER.md` |
| The rules written before each run | `docs/GPU_PREREGISTRATION.md`, `docs/REALMS_PREREGISTRATION.md`, `docs/K8S_COMPASS_PREREGISTRATION.md` |
| How to wire it, level by level, and how to confirm it is wired right | `docs/OMNI_COMPASS_MANUAL.md` (chapters 4 and 6), the book `docs/OMNI_COMPASS_MANUAL.pdf` |
| The license, in plain answers, and third-party components | `LICENSE`, `LICENSING_FAQ.md`, `ENTERPRISE_LICENSE.md`, `NOTICE`, `THIRD_PARTY_NOTICES.md`, `PATENTS.md`, `TRADEMARKS.md`, `SECURITY.md`, `.github/CLA.md` |
| The muscles, what each is for and how it is wired | `docs/MUSCLE_CATALOG.md` |
| Proof the code is the code that ran | `python3 verify.py`, `RELEASE_MANIFEST.json`, `results/SEAL.json` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART ONE

# The Governor, from the Outside

*What Omni-Compass is as an installer meets it, how the muscles of a stack become one body, and how it is tuned.*

CHAPTER 1

## What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, decides, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept.

Three facts about that process fix what an installation can and cannot expect of it. First, it is **one process per stack**: a controller that wakes on a fixed period, reads the stack's own gauges through the stack's own interface, decides, writes through the same interface, and reads back what the device took. It has no agent inside the pods, no kernel module and no hook in the request path, so it can add latency to nothing it does not govern. Second, every lever it may touch is **named before it starts**, read once into a snapshot, and handed back to that snapshot when it stops, is killed, or loses a sense; a lever it was not given it cannot reach, because the plug for it does not exist. Third, it has a **watching mode** in which it reads and decides and writes nothing, and every rollout begins there (chapter 6): the first thing an operator learns about Omni-Compass on their own stack is what it would have done, in a log, with the stack untouched.

What follows from those three is the shape of every receipt in Part V. Because the native controller keeps running underneath, every paired run is the native stack against the same native stack with Omni-Compass on top, and never Omni-Compass against anything else. Because the levers are the stack's own settings, every gain is one the operator could have reached by hand, if they had been able to read every gauge every second and move every knob without ever being wrong; the claim is not a new mechanism but a steady hand on the mechanisms that exist. And because the process is small and separate, its own cost (processor time on the host, reads of the API) is recorded in every run and counted against it where it shows.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Always on top of a native controller

The first design decision, and the one every other follows from, is that Omni-Compass is **supervisory**. It never replaces the controller a system already has, never bypasses it, and never runs in its place. The Kubernetes Horizontal Pod Autoscaler keeps scaling pods on its own CPU target; Omni-Compass moves that target inside a range the operator allows. The cluster autoscaler keeps adding and deleting machines; Omni-Compass marks machines idle and lets the autoscaler do the deleting. The card's firmware keeps boosting the clock and enforcing its power limit; Omni-Compass sets the ceiling the boost may reach and the limit the firmware enforces. PgBouncer keeps handing connections to clients; Omni-Compass sets its pool size through the pooler's own console. Kafka's consumer group keeps rebalancing partitions on its own; Omni-Compass changes how many consumers are in the group, which is what an operator does. Redis keeps evicting by its own rule; Omni-Compass sets the ceiling that rule works under.

This fixes how an installation is judged: every paired run has exactly two arms, **native** (the system with its own controller, as shipped and as the operator set it) and **omni** (the same system with Omni-Compass on top). There is no third arm in which Omni-Compass replaces the controller, and no comparison in which Omni-Compass is a rival to it. A gain is therefore always "what the governor added on top of what the native controller already did", and a loss is always "what the governor cost on top of it".

## Do no harm: the paired trial on the muscle

The second design decision is that Omni-Compass must never cause a LOSS. Before Omni-Compass moves a slow knob (a machine given back, a card's clock ceiling, a robot axis's speed, a drone's cruise override), it runs a **paired trial on the muscle itself**: the muscle's own cost per piece of work is measured at the deepest step already allowed and at one step past it, under the same traffic, with at least thirty pieces of work on each side; the step is allowed only if the trial's median cost is no more than 2% above the reference and no more than 2% above the cost first measured at native. A refused step is not tried again for a set number of decisions. Where no step passes, the trial reads **left native**, the knob is never moved, and the receipt for that muscle says "nothing for Omni to move". Gains are not capped; the 2% allowance caps only what may be spent in the trial to look for them.

For an installation this has two consequences. A knob the operator grants is not a knob Omni-Compass will move: it is a knob Omni-Compass may move once the muscle itself has shown the step costs it nothing. And a muscle that never passes a trial is not a fault: it is the right answer for that muscle on that stack, and it costs nothing.

## The pedals, the modes and the two switches

Omni-Compass drives a system the way a self-driving car drives itself: it feels, gauges and adjusts, with nobody in the seat. The vocabulary is used the same way everywhere in this manual:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **Off (manual).** Omni-Compass is not driving; native runs alone. This is the native arm of every paired run.
- **Watching.** Omni-Compass reads every gauge and logs the decision it would take, and writes nothing. Any difference between a watching arm and native is the cost of its being there (its processor time, its reads) plus noise, never a decision; a watching arm that writes anything fails its run. Watching is the first level of every rollout (chapter 6).
- **Autopilot.** One pedal: **push** (traffic climbs and capacity is added at once, never held back), **lift off** (traffic falls and capacity is eased off a step at a time, handed back toward native, never below idle), **idle** (the pedal lifted all the way: calm traffic, the engine at its floor of two machines in service, ready for the next burst), **reset** (every setting the governor ever wrote is handed back to where native had it, read back, and the record removed; every paired run's omni arm ends with a reset and checks it).
- **Cruise.** For work that comes as a pile: when pods have waited for a place for two decisions in a row, machines go into service up to the pool's own top (read live: every machine of a fixed pool at once, a growing pool a step a decision) and stay there, without second-guessing, until the line has been empty for two decisions. In cruise the governor also stops its five-second floor reads, because every machine is already in service and the reads would only cost processor time on the machines the queue is using.
- **Idle at once.** When the work is done and demand is at zero (the service at its autoscaler's own floor, wanting no more, at half its target or less), the pedal is lifted all the way in one move, straight to idle, with every safety check still holding.
- **The kill switch.** Separate from all of the above, and for **security only**: one switch in a human hand (`python3 tools/omni_switch.py off`) turns Omni-Compass's governing off across the whole machine at once, for anything rogue, or for anyone trying to drive a system through Omni-Compass. Every governor hands back and exits; none starts while the switch is off. The reset is the end of a run; the kill switch is the end of trust.

The floor of two machines is an operator's number; lifting off never takes a system below it, however far the pedal is lifted. There is no ceiling: every machine the operator has is usable, and the protection is the band inside each machine.

## Nothing is hardwired

Omni-Compass carries no table of per-stack special cases, and an installation adds none. The few numbers the governor holds are the same on every muscle and are declared, not chosen per benchmark: the centre of the band, the 5% cushion at each edge, and the 2% do-no-harm allowance. Everything else a muscle needs (how fast it answers, how often it may be moved, how far it may go) is read from the muscle's own declaration in the catalog or from the native controller's own settings at the start of the run, never typed in by hand. Where a stack needs a line, a cover or a step size (the MySQL pool moves in the server's own 128 MB chunks, the Redis ceiling in notches), the number comes from the stack being governed and is written down before the first counted run, so a reader of the receipt can see it was not chosen after the result.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

For an installer this means one thing: the settings an installation writes are the operator's own (chapter 3), and nothing else is to be set by hand. A behaviour seen in a receipt is the governor reading the wires and the settings it was given; where it looks wrong, the wires and the settings are the first place to look (section 4.5), because there is no hidden per-stack rule to blame.

## What it sets, muscle by muscle

Every muscle keeps its own control, and Omni-Compass sets what that control already accepts:

| Muscle | Its own control (kept) | What Omni-Compass sets |
|---|---|---|
| Kubernetes service | the HPA | the HPA's CPU target; replica floors |
| Kubernetes node pool | Cluster Autoscaler / Karpenter / MachineSet | how many machines stay in service (park and wake) |
| NVIDIA GPU | the card's firmware (boost, power and thermal limits) | the clock ceiling (up wire) and the power limit (down wire) |
| CPU | the kernel's frequency governor | the frequency ceiling |
| Data hall, building | the chiller and air handler loops | the supply-air setpoint; units in service |
| Battery site | the inverter | the reserve level; the peak ceiling |
| Robot joint, vehicle axis | the servo loop | speed and effort limits |
| Process loop, feeder | the PI controller | the setpoint inside its band |
| Database behind a pooler | PgBouncer's pool, the DBA's fixed setting | the pool size through the pooler's own console |
| Database's storage engine | MongoDB's WiredTiger cache, the operator's fixed setting | the cache size through the server's own console |
| Message broker | the consumer group at the operator's count | how many consumers are in the group |
| Cache | the operator's memory ceiling and eviction rule | the memory ceiling through the cache's own console |
| Substation | the tap changer's own controller | the tap position, one whole tap per move |
| Drone | the shipped autopilot | the cruise override inside the autopilot's limits |

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted: not to the last value it wrote, but to the value it read once, before its first write.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 2

# Your Stack as One Body: Muscles and Organisms

## The muscles you can wire

A muscle is anything with a meter and a setting: a compute pool and its machine count, a GPU and its clock ceiling, a chiller and its setpoint, a battery and its reserve, a feeder and its tap, a robot joint and its speed limit, a pump and its flow. Each muscle has a native controller of its own, which is what the muscle does when Omni-Compass is not there, and Omni-Compass wires to it through one wire in (the meter) and one wire out (the setting). The catalog (`realms/catalog_v5.csv`) lists **1,107 distinct muscles in 70 families**; Appendix E gives every one with what it is, what Omni-Compass reads from it, what it moves and the factory setting it runs on.

The seventy families, by realm, are these. **Compute, AI and cloud** (twenty-seven): AI inference serving, AI training, blockchain and ledger nodes, CI/CD and build farms, cloud virtual machines and capacity, container resources, cross-cluster and edge placement, DPU and programmable IO, distributed cluster managers, game servers and real-time interaction, GPU fabric and RDMA, GPU hardware, HPC and distributed compute, host CPU and memory, Kubernetes dynamic device allocation, Kubernetes placement and scheduling, Kubernetes workload scaling (the largest family, 32 muscles), Linux kernel and operating-system control, media streaming, node fleets and Karpenter-class control, OpenShift and the Machine API, quantum computing control, serverless functions, virtualization and hypervisors, work admission and demand shaping, and the body's gut (data-plane intake and waste) and skin (perimeter and admission). **Physics, robotics and autonomous** (eight): automotive EV and mobile powertrains, aviation and autonomous flight, elevators and vertical transport, marine propulsion and vessel automation, rail traction and train control, robotics fleets and warehouse automation, robotics motion control, and spacecraft and flight software. **Energy, facility and industrial** (seventeen): agriculture and irrigation, building and critical-environment HVAC, cooling and chillers, district heating and cooling, energy storage and microgrids, facility and grid optimisation, grid transmission and distribution, healthcare critical environments, industrial PLCs and process automation, mining and mineral processing, oil and gas pipelines, PDUs, UPS and electrical distribution, pharmaceutical and food manufacturing, power generation and turbine control, renewable generation and inverter control, semiconductor fabs and precision manufacturing, and water and wastewater pumping. **Distribution and**

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**specialized** (eighteen): cache and memory services, commerce and payments, content delivery and edge caches, data analytics and ETL, databases and transactions, financial markets and trading systems, medical imaging and clinical systems, messaging and streaming, network routing and switching, observability and telemetry, ports and maritime logistics, reliability, security and recovery, runtimes and applications, search, indexing and vector databases, service meshes and API reliability, block, file and object storage, telecom radio access and edge radio, and workflow, logistics and fulfilment. The open-source power and resource managers many operators already run (Kepler, Climatik, EAR, CruiseKube, Goldilocks, the Kubernetes power manager, PEAKS, adaptive GPU power capping and NVIDIA's platform power budget) are muscles too, each in the family its knob belongs to: Omni-Compass sits on top of them as it sits on any native controller.

## One body: the spine, the realms and the organism you build

An installation governs one body. The muscles of a stack, its clusters, databases, caches, cards, cooling and power, are read by one governor and judged together, so that a step on one muscle is refused when it would push another past its cushion (chapter 5). Under every realm's own muscles stands the same **spine** of 267 muscles (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution), because every realm's work runs on computers that must be cooled and powered. The catalog groups the muscles into four realms and stacks them into six organisms:

| Organism | Muscles | What it is |
|---|---|---|
| 1. Compute / AI / Cloud | 540 | clusters, GPUs, AI training and inference, cloud capacity, kernels, functions, media, games, ledgers, build farms |
| 2. Physics / Robotics / Autonomous | 404 | joints, fleets, vehicles, flight and spacecraft axes |
| 3. Energy / Facility / Industrial | 480 | data halls, buildings, batteries, UPS, process loops, feeders, hospitals, farms, pipelines, mines, district heat, power plants, renewables, pharmaceutical and food plants |
| 4. Distribution / Specialized | 484 | networks, storage, databases, commerce, trading, content delivery, workflows, radio networks, clinical systems, ports |
| 5. The four stacked, every duplicate kept | 1,908 | all four realms on one clock, the shared spine counted in each |
| 6. The whole tower, every muscle once | 1,107 | every distinct muscle on one clock |

**Building your own organism.** A real installation is rarely a whole realm. It is the muscles one company runs, wired in the order of chapter 6: the cluster first (its pods, then its machines), then the cards and the processors under it, then the databases, brokers and caches beside it, then the site under all of them. Three rules make the

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

result one body rather than a collection of separate governors:

1. **One governor per body.** Every muscle of the body is wired into the same governor, so every step is judged against every other muscle's cushion. Two governors on one body are two brains, and the coupling between them (more pods feeding a database at its connection limit, more cooling pulling a power feed toward its limit) is exactly what neither would see.
2. **Every shared resource is a muscle or a gauge.** Where muscles share a power feed, a cooling loop, a connection limit or a network link, that shared thing is wired in, as a muscle where it has a setting and as a gauge where it does not (a site limit read in watts, a database's connection count), so the governor can see the coupling it must respect.
3. **A muscle with nothing to gain stays wired.** A muscle whose knob never passes a trial stays native, but its gauges still reach the governor, because its state may be what makes another muscle's step unsafe. Watched is not the same as unwired.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 3

# Tuning: The Factory Tune, the Dial and the Settings You Write

## The factory tune

Omni-Compass ships tuned. The engine in the repository is presented as The Omni-Compass LLC tuned it and ships it: a self-running engine, set at the factory, the way a vehicle leaves the plant with its engine already mapped. Every muscle in the catalog is wired and runs on the setting found for it: its own card where one was found and kept (section 3.3), the engine's own setting for every other muscle. The whole tune is fixed and fingerprinted by version (`python3 tools/omni_version.py`), so an installation can always tell whether it is running the factory tune or a tune of its own. Every published result is the factory tune.

No setting is universal. Muscles differ, and no single setting is right for every muscle, family, sector, domain or realm; none is presented as such. What the factory tune fixes is the starting point an installation can trust: each muscle on the setting that was found for it, every muscle wired, nothing switched off.

## The max, and the dial that tunes it down

The factory setting is **the max**: every muscle runs up to its own limit, read live from the system, never past it and never below what native gives. It rides the edge of each muscle's range, not the middle. The energy it takes to do more work is **power given**, shown beside the work, the speed and the wear it bought, never called a loss; the max never takes from one muscle to give to another. A knob that native already runs at the top of its range is "at its limit on native", and the max leaves it there.

The **dial** turns the max down. It has two positions, the same on every live controller:

| Position | Switch | What it does |
|---|---|---|
| **max** (the default) | `--dial max`, or `OMNI_DIAL=max` | the most work, faster: the service held well inside its line so every rush finds its capacity already there; never a machine given back below what the system ran on its own; on a card, the clock ceiling never stepped below the top while the card serves |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Position | Switch | What it does |
|---|---|---|
| **save** (the tune-down) | `--dial save`, or `OMNI_DIAL=save` | native's work on the least resource: the service held nearer its line, machines and clock steps given back where the paired trial on the muscle shows the response time allows (section 1.2) |

The tune-down never does less work than native and never misses its line more often; it spends less to do the same. An installation chooses its position per governor, writes the choice down before its paired runs, and reads every receipt against the position it ran.

## Each muscle's own card

The governor is one, but its settings are each muscle's own. A muscle's **card** holds the numbers the max uses for that muscle alone: for a machine pool, how full it runs on its forecast before it calls for more and how many decisions of history it reads; for a battery, how many wobbles of margin it keeps under its line and, where it is wired by its power, how long it reads its line before it holds and how much of it it keeps clear. The cards were found by a search over every muscle on development cases only, today's engine setting always one of the choices, and a card was kept only where it beat the engine setting beyond the noise on two separate sets of cases and was never behind native on any case under any load (never over its line more often, never less work, never its line cleared later). 364 muscles carry one (`realms/muscle_cards_v5.csv`); every card is written out in plain words with its muscle in Appendix E; the rest run the engine setting. Inside a body, a battery wired by its power keeps the body's own margins on its line, because the line carries the whole body's swing, which the card chosen for the battery alone never saw.

An installation reads a muscle's card before it wires that muscle, so that the receipt can be read against the setting the muscle actually ran on.

## The settings an installation writes

Everything else an installation writes is the operator's own, taken from the operator's own system and objectives, and written down before the first counted run:

| Setting | What it is | Where it comes from |
|---|---|---|
| **The line** (`--slo-ms`, or the stack's own line) | the response time, or other limit, the service must meet | the operator's own service objective, never ours |
| **The band** | the reading's range from calm to the line; on a service, from a tenth of the line (the bare service time) to the line | the line, by rule |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Setting | What it is | Where it comes from |
|---|---|---|
| **The cushions** | the last 5% of the band at each edge, 10% in all; a reading in a cushion is treated as already too close to the edge | fixed, the same on every muscle |
| **The centre** (`--compass-center`) | where in the band the governor holds the reading: 0.3 at the max, 0.4 at the tune-down, unless the operator writes another | the dial, or the operator |
| **The cover** | the knob's hard range; every write is clipped to it | read from the system (the autoscaler's range, the card's supported clocks, the pooler's connection limit), or written by the operator inside it |
| **The ceiling of a growing lever** (`--replica-ceiling`, the machine lever's top) | how far a lever that can keep adding may go | read live by default: what the machines can hold, one under it; an operator may write a lower one, or 0 to keep native's own cap |
| **The floor** | the least a pool may run on | two machines by default; never below 1 for any muscle |
| **The allowance** (`--allow`) | the most a step on trial may cost the muscle's own work | 2%, the same on every muscle |
| **The site's limits** (`--site-limit-w`, `--cooling-min-c`, `--cooling-max-c`, `--cooling-restore-c`) | the facility's power limit and the setpoint band | the facility engineer, inside the equipment's ratings |

Three settings are deliberately not on that list. The governor's decision period, its damping and how fast each muscle answers are the muscle's own declaration, read from the catalog or measured from the system at the start; an installation does not tune them (on a Kubernetes cluster the declared decision period is 60 seconds and the fast checks run every 5 seconds, three times as fast as the autoscaler's own 15-second check; a shorter decision period changes rules the governor counts in decisions, so it is not an installation setting), and a receipt that needed them changed is a receipt to read as wiring first (section 4.5).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Profiles: service and batch

The governor has two **profiles**, service (the default) and batch, which differ only in what the pedals do with a pile of work: in the service profile the governor paces the slack; in the batch profile, cruise puts every machine in service while work waits for a place, and idle takes them straight to the floor when the queue is done (section 1.3). An installation running a queue of jobs chooses batch for that service and service for everything else, and writes the choice down beside the dial's position.

## A changed tune is your own configuration

As with any engine, a licensee may add on, retune, upgrade or tune down any muscle or any combination of muscles: the dial, the cards, the bands and the service lines are all exposed. A changed tune is the licensee's own configuration, and its receipts are its own, never the published ones. The discipline for a changed tune is the discipline for the factory tune: write the change down before the paired runs, run native and omni three times on it, show every row, and read the receipts against the tune that ran. Where a changed tune reads differently from the factory tune, the change is the first thing to look at, and then the wiring.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART TWO

# The Harness: Plugs, Wires and the Rules They Obey

*The universal plug, the adapters, the wire check that must pass before anything writes, and the rules every wire obeys.*

CHAPTER 4

## The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

### The plug (one design for every muscle)

Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same five-line contract, and an adapter for a new muscle is written to it:

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, placed in its band.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

### Adapters (one per standard plug)

Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Standard | Muscles | Adapter status |
|---|---|---|
| Kubernetes API | HPAs, deployments, pods, nodes | **built** (`python3 -m omni_controller.controller`) |
| NVIDIA NVML / `nvidia-smi` | GPU clock ceiling, power limit | **built** (`python3 -m omni_controller.gpu_compass`, two wires; `omni_controller.gpu_governor`, one wire) |
| Linux cpufreq, RAPL | CPU frequency ceiling, package watts | **built** (`--cpufreq-policy-root`, `--rapl-cmd`) |
| Site meter, building controller by command | site watts, supply-air setpoint | **built** (`--power-cmd`, `--cooling-cmd` command templates) |
| Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet) | machines | **built** through `--node-scale-cmd` templates |
| BACnet, Modbus, SNMP | chillers, air handlers, PDUs, UPS | designed; today reached through the command templates above |
| OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN | PLCs, drives, robots, vehicles | designed; reached today through a command template written to the plug's contract |
| IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP | substations, batteries, inverters, chargers | designed; reached today through a command template written to the plug's contract |

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

**The plug contract, line by line, and why each line is there.** *Attach* reads the knob once before any write and keeps that value as the snapshot. It is read once and never again, because a snapshot that moved with the knob would restore to wherever the governor last left it, and the whole point of the restore is to return to where the operator had it. *Read* takes the service reading (a response time, a temperature, a queue, an error) and places it in its band; the plug declares the band, the governor does not guess it. *Write* takes one value, clips it to the cover before it leaves the plug, sends it through the muscle's own interface, and then reads the lever back from the device and records both the sent and the taken value; a device that took something other than what was sent (a clock that rounds to 15 MHz steps, a pool size the pooler capped) is recorded as it is, not as it was asked. *One writer* compares the lever as found at every read with the value last written; a lever found elsewhere has another owner, after which the governor leaves that lever alone for the rest of the run and says so in its audit and its exit code. *Restore* writes the snapshot back on stop and reads it back; a restore that does not read back is an error (exit 3), never a silent success. The same five lines are implemented for Kubernetes

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

objects (annotations hold the snapshot on the object itself, so a watchdog can restore from the object without the governor's memory), for `nvidia-smi`, for PgBouncer's console, for Redis's `CONFIG SET`, for the consumer group and for the simulators' knobs. An adapter written for a new muscle keeps the same five lines, and is proved against a fake device of that muscle before it is pointed at a real one.

## Power-up order

Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order. In the Kubernetes controller the order is: the latency feed and the cluster's own metrics (read); the HPA target (a setting the HPA accepts and corrects within a minute); the pods' CPU limits (conveyance, in place); the machines (the slowest and most consequential, through the release gate and the verdict). On stop the machines are woken first, then the limits, then the target, so the cluster is never left with fewer machines than its pods need while a setting above them is still being handed back.

## The wire check (mandatory before any write)

For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

| Step | It proves |
|---|---|
| 1 read | every meter answers: watts, temperature, utilization, limit, clock, top clock |
| 2 snapshot | the start limit is recorded once |
| 3 up wire | with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it |
| 4 down wire | a lower power limit is reported back exactly; the start limit is reported back exactly |
| 5 restore | clocks reset and the start limit read back |
| 6 stop | the governor, stopped by its signal, hands both wires back |
| 7 other writer | a limit set by someone else while the governor runs is left alone, and the governor exits 5 |

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

## Wired right or wired wrong

The wire check proves the wires move. It does not prove the governor reads the right thing. A correct installation leaves signatures in its own receipts; a wrong one leaves others. Check them on your first paired runs, before you believe any number, good or bad.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Check | Wired right | Wired wrong, and what to look at |
|---|---|---|
| Watch arm against native | equal on every gauge (watch writes nothing) | different: the probe, the load or the machine differs between arms, not Omni-Compass |
| Requests served, failed requests | equal to native | fewer served or more failed: a lever is cutting capacity; check its cover and its sign |
| p95 and p99 response time | at native or faster | slower: the reading or the floors are wrong (rows below) |
| GPU: the card's clock while busy | at or above the clock the card reaches on its own | below it: the speed floor is missing or the lid is under the card's own draw |
| GPU: the lid while busy | at or above the card's own busy draw | at the envelope floor while busy: the lid is sized from a curve, not from the card's own meter |
| GPU: credit per write (`GPU_REPS.md`) | compass decisions pace the slack, race decisions cover the bursts; fail-up is rare | fail-up in most decisions: being busy is being read as a breach, or the service line is set too low |
| Kubernetes: machines given back | through the release gate, one per decision, with no pod waiting | machines given back while pods wait, or none at all while the service is far inside its band: check the latency feed, `--slo-ms` and the release gate's reasons in the audit |
| Kubernetes: the reset | every HPA target, replica range, CPU limit and worker back to native, no record left | anything left: an earlier run or another controller wrote the same objects |
| Every lever after the run | at its snapshot | not at its snapshot: another writer, or a restore that failed (exit 3): restore by hand and investigate |
| The feed itself | fresh samples every few seconds | stale or empty: Omni reads it as blind and fails up; fix the feed, not the governor |

**The rule.** If any row reads "wired wrong", the result of that run says nothing about Omni-Compass: fix the wiring, run the wire check and the watch arm again, and only then compare. The first real card run is the worked example: its receipts showed the card's busy clock under its own (736-768 MHz against 861-889), the lid at the 105 W floor under its own 135 W draw, and fail-up in 46% of decisions. Each of those is a row in this table.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The wire check on the other stacks

The GPU has a one-command wire check because its wires are the hardest to get right. The other stacks carry the same proof in a different place: every Kubernetes arm records `kubectl auth can-i` receipts for what the governor's identity can and cannot do (and fails if it can do anything it must not), checks that the cluster holds nothing but the harness, and ends with the reset check that every HPA target, replica range, CPU limit and worker is back at native with no record left; every database, broker and cache arm reads the knob back after every write and hands it back at the end, and the report says "handed back: yes" or the run is invalid; every simulator arm checks that every override was handed back. The watching arm, where it is run, is the wire check of the reading: a watching governor that disagrees with native on any gauge has found a difference in the probe, the load or the machine, not in Omni-Compass.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 5

# The Rules the Wires Obey

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them, the governor decides how much authority each muscle has at each moment, by four rules that an installer must know because they decide what a receipt can show:

- **Expand is always allowed** (except under a security hold, Chapter 9): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record:** giving anything back is allowed only while the muscle is calm and the service has been inside its line for the last three decisions; a continuous muscle gives back at most its calm share of its surplus per decision, a discrete one one unit at a time, through a release gate (section 5.1).
- **Fail up:** while the service is past the wall (95% of the band) the knob returns at once to full capacity, the operator's own setting or the cover's top, whichever the muscle's direction asks.
- **Blind means hold:** if a sense goes stale or unreadable, nothing is given back until it returns (section 5.3).

The rules are asymmetric on purpose. Adding is cheap to undo and expensive to delay, so it is never gated beyond the security hold. Taking back is cheap to delay and expensive to get wrong, so it is gated three ways: by calm, by the service record (inside the line for the last three decisions, not merely this one), and by the release gate, whose reasons are written into the audit every time it holds.

"Two-way" is a statement about wiring, and it is checked, not assumed. Each muscle's plug carries exactly one sensory wire and one motor wire, and the wire check that must pass before any write (section 4.4) proves that the wires move: a value written through the motor wire is taken by the device and read back as the value sent, and the snapshot is restored and read back at the end. No governor writes a knob whose wire check has not passed, because a knob that can be written but not read back cannot be handed back with proof. What the wire check does not prove, section 4.5 says plainly: that the governor is reading the right gauge for the knob; that is what the watching arm is for. The receipts of every run carry, for every write, the value sent and the value the device reported, side by side.

## The release gate

On Kubernetes the **release gate** lets one machine be given back only when every one of the following holds, and writes the first reason that fails into the audit, so a reader can see why a machine was kept:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

1. the remaining machines would stay under 95% full;
2. no pod is waiting for a place;
3. the pods are not scaling up;
4. the service is not past its line;
5. the muscle's own pressure is under its release threshold;
6. every sense is live;
7. the last command has landed (no new order goes on top of one the cluster has not yet carried out).

One machine per decision is the rule in the slow, reversible direction; past the wall, one machine up at once. The same shape holds on every discrete muscle: a consumer, a chunk of a buffer pool, a notch of a cache ceiling, a tap. The gate's records read, in the governor's own words, "release permitted" (with the utilisation the remaining machines would carry), "no contraction authority" (a calm system whose state has not yet earned the right to give back), or a list of reasons at once, such as "latency breached now; a sense is blind". A machine kept in service always has a stated cause.

## The snapshot, the read-back and the one-writer rule

The governor knows where its own levers are. The rule begins with the **snapshot**: the governor reads every knob it may move once, at the start, before its first decision, and that reading is the operator's setting for the whole run. Every reset, every hand-back and every kill returns the knob to the snapshot, never to a value computed later, and the receipt records the snapshot beside the final read-back so a reader can check the two are equal. The rule continues with the **read-back** after every write: the plug sends a value, waits for the device to take it, and reads the device's own report of what it holds. Where the device takes time to apply a setting, the plug knows that and does not mistake the transition for a foreign hand: MySQL's buffer pool is resized by the server in chunks over several seconds and reported as it goes, so the plug reads the server's own resize status, treats a pool caught mid-resize as its own write in progress, and waits for the server to report completion before it judges the write. The same shape holds on the autoscaler's bounds, Redis's memory ceiling and MongoDB's cache size: the value is read from the device's own console, and only a value the governor neither wrote nor is waiting on is read as someone else's.

If a lever is found at a value Omni-Compass did not write, someone else owns it: the governor stops writing that lever and leaves it alone, because restoring over it would fight the new owner. An operator who changed a knob by hand meant to, and a second controller that changed it is a conflict to be reported, not won. The lever is left where it was found, the audit records the value and the time, the governor goes on governing the other levers, and its exit code says so. A reader of the receipts who finds such a record knows at once that two hands were on the stack, and the paired run that contains it is read with that in mind. For an installer this rule has one consequence above all: **a knob may have only one writer besides its native controller**. A runbook, a cron job, a building schedule or a second tool that writes the same setting must be stopped or handed the knob before Omni-Compass is granted it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Blind means hold, and fail up

A sense that goes stale or unreadable is not read as calm. It is read as **blind**, and while any sense is blind nothing is given back; the knob holds, or, if the service was past its line when the sense went blind, returns to full capacity at once. Omni-Compass never acts on a reading it does not have. An API that answers a read with a server error produces a held decision, logged with the reason the API gave, and the next decision is taken on schedule when the API answers again. The same holds for a response-time feed that pauses: the governor holds through the gap and resumes when the feed returns. A stale or missing reading produces a hold, never a move, and the hold is in the record.

For an installer, the consequence is that the freshness of every feed is part of the wiring. A latency file that is written late, a meter that is polled less often than the governor decides, or an exporter that caches its readings will read as blind and keep the governor holding; the audit says so (`gate: a sense is blind`), and the fix is in the feed, never in the governor.

## One body, one governor

The alternative to one governor is a controller per layer, each tuned alone: an autoscaler that adds pods while a power cap is trying to hold the machines down, a cooling loop that chases heat the clock governor is about to remove. These controllers fight because none of them knows the others' intent. One governor that sets every knob from one reading of the body cannot fight itself: the step on each lever is decided from the same reading of the same body at the same moment, and a step that would push another part of the body past its cushion is refused. Nothing anywhere is pushed into a LOSS to move one thing to a GAIN; one trial runs at a time inside a body, so every receipt can name the step that produced it.

This is why chapter 2 asks for every muscle of a body, and every resource the muscles share, to be wired into the same governor. Where a shared resource is left out, the governor cannot see the coupling, and no rule in this chapter can respect a limit the wires do not carry.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART THREE

# Wiring It onto Your Stack, Step by Step

*The eight levels, from watching to governing the site, and the wiring of each kind of stack.*

CHAPTER 6

## Before You Start, and the Eight Levels

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## What your system needs

| You run | You need |
|---|---|
| Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind) | Kubernetes 1.34 or newer (in-place pod resize); metrics-server (`kubectl top nodes` works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service |
| A response-time feed (strongly recommended at every level) | a CSV per service, `elapsed_seconds,latency_ms,ok`, written continuously; `scripts/latency_probe.py` writes one against any HTTP endpoint |
| NVIDIA GPUs | a driver with `nvidia-smi`; root (or the capability to run `nvidia-smi -pl` and `-lgc`); on a VM, full GPU passthrough |
| CPU power control | Linux cpufreq with the schedutil governor; RAPL for package watts; usually bare metal |
| Site power and cooling | a command that prints site watts, and a command that sets the supply-air setpoint through your building management system |

**Why each requirement is there.** Kubernetes 1.34 or newer is required because the conveyance muscle resizes a pod's CPU limit in place, which older versions do only by restarting the pod, and a restart is a disruption Omni-Compass never causes. metrics-server is the HPA's own eyes and the governor's: without it neither knows what the pods use. An HPA with a CPU target on each governed service is the native controller Omni-Compass sits on; a service without one has nothing for level 2 to move. A readiness probe and a short preStop pause are what make a drain invisible to clients: the Service stops sending requests to a pod before it stops, and kube-proxy sends them to another ready endpoint. The response-time feed is the one wire a cluster does not already have: Kubernetes knows what its pods use, not what its users wait; the compass needs the second, and `scripts/latency_probe.py` writes it against any HTTP endpoint every five seconds. On the GPU, root is needed only for the two `nvidia-smi` writes, and a VM must pass the whole card through or the driver refuses to lock clocks (the wire check's step 3 is where that shows).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass.git
cd The-Omni-Compass
pip install -r requirements.txt
python3 verify.py                       # every check; ends with VERIF
    ↪ ICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t &lt;your-registry&gt;/omni-compass:&lt;tag
    ↪ &gt; .
docker push &lt;your-registry&gt;/omni-compass:&lt;tag&gt;
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

What `verify.py` checks is worth knowing before it is run, because its last line, `VERIFICATION: PASS` or `FAIL`, is the only thing an installer has to read. It runs every test in the repository (the plugs, the readings rule, every stack's harness against a fake server), checks every fingerprinted file of the engine against its registry so that a changed file fails before anything is installed, compares the release's files with its manifest, and checks that every path the documents name exists. It takes a few minutes on a laptop and needs no network beyond the clone. GitHub runs the same script on every push under Python 3.12, and each run is listed under the repository's Actions tab with the commit it checked; install only a commit whose run passed.

To know which engine a checkout carries, ask it:

```
python3 tools/omni_version.py                 # the engine version (f
    ↪ or example omni-v6), or every file that differs
python3 tools/omni_version.py --commit &lt;sha&gt;   # the same question of
    ↪ any pushed commit
```

Every receipt names the commit it ran on, and the three-run tools refuse to combine runs from different engine versions (chapter 12). No receipt is ever read across engine versions, and the command above is how an installer checks that for themselves.

## The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

The levels exist because a governor earns authority the way a new operator does: by watching first, then by touching the setting that is cheapest to undo, then the next. Level 0 touches nothing and proves the software is the software (`verify.py` checks every sealed file's fingerprint and runs every test). Level 1 watches a live

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

cluster for as long as the operator likes and logs every decision it would have taken, so the operators can read those decisions against what they would have done; nothing passes level 1 until they agree with its readings. Level 2 moves the HPA target, which the HPA itself corrects within a minute if the governor is wrong, and raises replica floors ahead of bursts; it is the first level that can show a gain and the first that can show a loss, and the paired receipts of chapter 10 are its pass condition. Level 3 moves machines, the slowest lever, through the release gate and the verdict; it is where the bill moves and where the most care is spent. Level 4 hands Omni-Compass the replica decision itself, with the HPA kept as the fast reflex upward; it is an option, not a step on the way, and it is not part of the factory tune. Levels 5 to 7 are the GPU, the CPU and the site, each with its own wire check and its own pass. An installation may stop at any level and run there indefinitely; most will stop at level 3.

**Level 0 - Evaluate without touching anything.**

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eva
    ↪ l     # the six organisms, modelled
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval
    ↪        # the two-wire card, modelled
```

Pass: `VERIFICATION: PASS`. Nothing in your systems is touched.

**Level 1 - Watch (read-only).**

```
kubectl apply -f deploy/install/omni-compass.yaml     # set the image
    ↪ line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

The install file grants a read-only identity; `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

**Level 2 - The pods, on top of your autoscalers. WRITES.**

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

| Muscle | Switch | What it does |
|---|---|---|
| convey | `--latency-file` and `--cap-deployments ns/name` | gives each machine's idle CPU to the serving pods on it |
| rightsize | `--rightsize-deployments ns/name` | each pod's CPU request follows its measured use times (1 + headroom), in place |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Muscle | Switch | What it does |
|---|---|---|
| coldstart | `--coldstart-deployments ns/name --coldstart-signal ns/configmap` | scales a service to zero while no work waits, wakes it when work arrives |
| batch | `--batch` | admits held Jobs labelled `omnicompass.io/batch=true` when there is load and power headroom |
| batch pace | `--batch-pace` | pauses Jobs labelled `omnicompass.io/pausable=true` under power or heat stress, resumes them after |
| rollout guard | `--rollout-guard ns/name` | pauses a rollout while change is not permitted |
| contain | `--contain-namespaces ns --contain-cpu-m <m>` | holds an agent namespace to a CPU budget |
| security hold | `--security-configmap ns/name` | key `hold: "true"` blocks every expansion |

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests never behind native, over paired runs (chapter 10).

**Level 3 - The machines. WRITES.**

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"` and `--node-restore-cmd "<command>"` for the OFF switch. For the compass on the same levers, use `--law compass` (it reads the mean response time of the latency window in the band from a tenth of `--slo-ms` to `--slo-ms` and holds it at the centre of section 3.4, 0.3 at the max and 0.4 at the tune-down; p95 at the line, a blind feed or a waiting pod adds capacity at once; the HPA target never goes above the operator's; at the max no machine the cluster had on its own is given back, and at the tune-down machines go back one at a time through the release gate and the paired trial).

| Your platform | The park/wake command |
|---|---|
| any cluster, bare metal, kind | `bash scripts/kind_nodepool.sh {n}` |
| Karpenter / EKS Auto Mode | the NodePool CPU limit at `{n}` times node CPU, parked nodes kept |
| Cluster Autoscaler node group | the group's desired size, scale-down through parking |
| OpenShift / OKD | the worker MachineSet replicas, parked |

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

LOSS on any service gauge.

**Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES.** Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

**Level 5 - A GPU box, two wires. WRITES.**

```
sudo python3 tools/gpu_wire_check.py --gpu 0                    # must
    ↪  end: WIRED RIGHT
# watch: computes both wires and writes neither
sudo python3 -m omni_controller.gpu_compass --mode watch --gpus 0 --au
    ↪ dit /var/log/omni/gpu.jsonl \
     --latency-file &lt;feed&gt; --slo-ms &lt;p95 target&gt; --floor-w &lt;your low
    ↪ t watts&gt;
# govern: writes the clock ceiling and the power limit
sudo python3 -m omni_controller.gpu_compass --mode cap --gpus 0 --audi
    ↪ t /var/log/omni/gpu.jsonl \
     --latency-file &lt;feed&gt; --slo-ms &lt;p95 target&gt; --floor-w &lt;your low
    ↪ t watts&gt;
# OFF: clocks reset, power limit back to the start, read back
sudo touch /tmp/omni-gpu-kill
```

Every `--interval` seconds (default 2) it reads the card's own meters and the response times, and:

- **learns the card on its own** first: while the ceiling is at the top and the card is busy, its busy clock and busy draw; until 15 such readings are in, neither wire moves;
- **races while work waits**: at 95% utilization or more, the ceiling to the top and the lid to the start limit;
- **at the max** (`--dial max`, the default): the clock ceiling is never stepped below the top while the card serves, and the lid sits above the card's own busy draw, never over the start limit;
- **at the tune-down** (`--dial save`), it paces the slack: the mean response time of the last 5 s, in the band from a tenth of `--slo-ms` to `--slo-ms`; the ceiling moves in whole 15 MHz steps, never under the card's own busy clock, and the lid never under the card's own busy draw plus 10%, never over the start limit; and before the ceiling may go one step lower, a paired trial measures the card's own time on each request (the workload's `service_ms`) at the top and at that step, and the step is allowed only if it adds at most `--allow` (2%). Where no step passes, the ceiling stays at the top: the card runs as it does alone, and the audit says so (`verdict_state: left native`);
- **fails up** when p95 reaches `--slo-ms`, a request fails, the feed goes blind or the card reports a heat slowdown.

The settings are the card's own (its busy clock and busy draw as learned), the line (`--slo-ms`), the allowance (`--allow`) and the dial. The workload must write `service_ms` (the card's own time per request) in its

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

latency file, as `tools/gpu_workload.py` does; without it no step is ever allowed and the card stays native. Pass: work per energy up or equal; requests served equal; no request more than 2% slower at the median, p95 and p99 at native or within 2%; every row of section 4.5 reads "wired right".

**Level 6 - CPU clock and power. WRITES.** On bare metal, add to the controller: `--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil --rapl-cmd "<prints CPU package watts>"`. The OFF switch writes every policy's recorded maximum back exactly. Pass: energy per unit of work down, no LOSS on any service gauge.

**Level 7 - Site power and cooling. WRITES.** `--power-cmd "<prints site watts>" --site-limit-w <limit>` brings site power stress into the engine; `--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22` lets it hold the supply-air setpoint in its band. CPU and GPU sharing one power budget, and groups of cards sharing a site budget, run as models today. Batteries are designed as a muscle of the site, not yet wired (section 7.10).

## Wiring for superiority: watch first, then the paired trial on every knob

Every knob is wired out first and in last, and the wire in is connected only by a measurement the governor makes on the knob itself. The steps, for any stack:

1. **Wire out, everything.** Every muscle's readings reach the governor, one wire out per muscle. Nothing is written. This is watch mode (section 1.3): where every installation starts, and where any knob returns the moment it stops paying.
2. **Run the observation.** With the stack together and under its real or replayed demand, let the governor watch for at least one full cycle of that demand. In watch mode the governor learns the band of every reading, where the native controllers hold them, and which readings leave calm and when. Nothing is decided yet; the audit says what the governor saw.
3. **Let the paired trial decide each knob, one trial at a time.** Switch the knobs the operator is willing to have moved into trial mode. A knob still writes nothing until the governor has run a paired trial on it (section 1.2): the knob is held at the operator's setting for a reference, then one notch away, under the same traffic, and the step is allowed only if the muscle's own cost did not rise under the declared objective. The objective is the operator's one declaration per knob: the resource reading (the service, the resource and the processor time weighed together) or the service reading (work and speed alone). A refused step is not retried for a set time; an allowed step widens the allowance by one notch, and the next trial probes the next. The governor then moves the knob inside the allowance, never outside.
4. **Keep checking.** The trials recur. A knob whose last step stops paying when the demand changes is refused on its recheck and the allowance narrows back; a knob that never passes a trial stays native for good, which is the right answer for that knob and costs nothing. New noise in the system is answered the same way: the governor reacts inside the allowance, and the allowance itself is re-earned by measurement.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

5. **Read the trials, muscle by muscle.** The audit carries the state of every knob every second (`verdict_phase`, `verdict_direction`, `allowed_low`, `allowed_high`, the cost sample), and the arm's record sums it. An operator who wants one knob wired both ways and another watched only sets the objective on the first and leaves the second in watch.

What this gives: no knob is ever written on faith. Where Omni-Compass can do more than the native controller on the muscle's own gauges, it will have shown it on that muscle before it moved it; where it cannot, the muscle runs by itself and Omni-Compass reads it. The body's result is then the sum of the knobs that earned it, not one turn of a single dial for the whole system. The live stacks of chapter 7 (the pooler, the broker, the cache, the storage engine's cache and the buffer pool) carry this trial on their knob, and on Kubernetes it guards every machine given back (section 1.2).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 7

# Stack by Stack

Every wiring below is the same plug (section 4.1): one wire in (the stack's own meter), one wire out (one setting the stack already accepts), a snapshot taken once before the first write, every write read back, everything restored on OFF. Native is always the stack's own controller left as shipped; omni is that controller with Omni-Compass on top.

| Stack | Levels | Notes |
|---|---|---|
| Vanilla Kubernetes, kind, Rancher | 1-7 | as written |
| Amazon EKS | 1-5 | machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances |
| Google GKE, Azure AKS | 1-5 | the provider's node-pool size as the park/wake command; on AKS the managed cluster autoscaler stays native and deletes the machines Omni-Compass idles (7.1) |
| Red Hat OpenShift / OKD | 1-5 | the same permissions through a Role; machines via the worker MachineSet |
| NVIDIA GPU servers without Kubernetes | 5 | the two-wire GPU governor alone |
| Bare-metal CPU servers | 6 | cpufreq and RAPL through sysfs |
| Slurm / HPC | 5 on the GPU nodes | a job-level Slurm connector is not built |
| Building management | 7 | through your BMS's command line or API, in the command templates |
| PostgreSQL behind PgBouncer (any database behind a pooler with a console) | one knob | the pool size through the pooler's own admin console (7.2) |
| A power grid's substation tap changer (pandapower, SimBench) | one knob | the tap position, one whole tap per move (7.3) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Stack | Levels | Notes |
|---|---|---|
| Buildings with batteries (CityLearn) | one knob a building | the battery's charge and discharge inside the simulator's own limits (7.3) |
| Robot arms (MuJoCo Menagerie) | one knob an axis | the joint speed where the axis has slack; never on an axis at its takt (7.3) |
| A message broker's consumer group (Apache Kafka) | one knob | the consumer count inside [1, partitions], read from the consumers' own records; the rebalance paid on every move (7.4) |
| A cache's memory ceiling (Redis) | one knob | `maxmemory` through the cache's own console, grown only while the cache is full (7.5) |
| Drone swarms (gym-pybullet-drones; PX4 and ArduPilot next) | one knob a drone | the cruise override inside the autopilot's limits; a separation wall; collisions void the cell (7.6) |
| A database's storage-engine cache (MongoDB under YCSB) | one knob | the WiredTiger cache size through the server's own console, grown only while the cache is full (7.7) |

## Managed Kubernetes on Azure: the bill as the gauge

On Azure Kubernetes Service the muscle is the same as on any cluster (chapter 6, levels 1 to 3) and one thing is new: the bill is real. The workflow `aks-metered` builds a fresh cluster for every arm the same way: a system pool of one machine, tainted so no workload lands on it (Azure's add-ons and the load generator live there; it is never measured and never governed), and one or several **work pools** under Azure's own cluster autoscaler, every pool labelled `omni-role=work`, every pool starting full. Native is that autoscaler alone. Omni on top is the same autoscaler with Omni-Compass idling the machines it gives back, which the autoscaler then deletes on its own schedule. The bill counts every work machine that exists, every 15 seconds, priced at Azure's list price for its family; each cluster is deleted when its arm ends, so nothing keeps billing.

**Sizing the fleet to the lever.** The smallest saving a fleet can show is one machine. On 4 workers one machine is 25% of the fleet, so only a saving of about 30% or more can clear the noise; on 15 workers about 10%; on 96 about 3%. A few percent of a 4-worker fleet is less than one machine, so a 4-worker receipt reading no difference beyond the noise on the machine rows is the honest reading of a fleet too small for the lever, not a result against it. A new subscription allows 200 vCPUs in a region but 10 per machine family, so one family gives five 2-vCPU workers; the way past it is **several families at once**: eight pools of four to five workers each make a fleet of 39 (`worker_pools` input, `size:max,size:max,...`; `pool_prices` names each family's price). The app's replica ceiling is raised with the fleet (`hpa_max`: nine 200-millicore pods fit one 2-vCPU worker, so 9 × workers lets the fleet fill), the same in both arms; Omni-Compass's replica room,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

read live from what the machines can hold, lets the omni arm raise that cap while it binds and hand it back once demand holds still (section 3.4). A cloud region can refuse a new cluster for hours at a time; check the subscription's allowances before building anything, so that a refusal costs cents and not a cluster. The account setup, click by click, is in `docs/AZURE_SETUP.md`.

**In the cloud, the bill is the gauge, and it is never raised.** In a public cloud every machine that exists is billed, whether it carries work or not, so the aim on a managed cluster is native's work for a smaller bill, never a bigger one. The cloud therefore runs at the tune-down (`dial save`, the `dial` input of `aks-metered` and its default) with the governor on the machines alone (`OMNI_MACHINES_ONLY=1`, which starts the governor with its one cloud flag, `--give-back-only`): it moves no pod autoscaler, conveys no processor and caps no power, so the pods are exactly native's and the cloud's own autoscaler alone makes machines; the governor parks a machine the cloud already made only after its own trial shows the response time holds, and wakes it on a breach. It can hold no more pods and no more machines than native: the bill can only fall or stay, and it falls in the quiet stretches between bursts and in a day's trough. The max (`dial max`) belongs where the machines are already paid for (your own racks, a chip whose heat line is its limit), where more work from the same machines is the gain; in a cloud it buys speed with machines, and every machine it adds is on the bill. The benefit of the max is taken before the cloud, in the service's own engine, or after it, in the chip. In the cloud, as everywhere, the governor's fast check (a parked machine woken for pods that wait) runs every 5 seconds, three times as fast as the autoscaler's own 15-second check, and its full decision every 60 seconds; a pod's start and a new machine's join are not shortened by deciding faster, so the governor acts on the reading while it climbs, before the line. On machines already paid for, the fast checks also give the running pods the processor their machine already has, which takes seconds and needs no new pod; in the cloud mode that lever is off, because a pod given more processor reads busier to its autoscaler, which then asks for more pods and, through them, more machines.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A database behind its connection pooler

PostgreSQL is wired through PgBouncer, the pooler most of its deployments already run. Nothing models the database. The wire in is the pooler's own report (`SHOW STATS`, `SHOW POOLS`), read once a second over one connection held open for the whole run, so that no process is launched for a reading: the larger of two figures, the time a transaction spent in the server plus the time clients waited for one, over the transactions completed, and the share of the pooler's clients queued for a server. The second figure is what makes the wire honest: a pool too small for the offered rate shows at the pooler as clients waiting even while its own service time reads calm. The wire out is one setting through the pooler's own admin console, `SET default_pool_size`, inside the cover [2, 90]: never under two server connections, never within ten of PostgreSQL's connection limit. The reading is held at 40% of a 50 ms line. Where the time was spent decides the direction: while clients waited for a server one percent or more of the last second, servers are added back, one for every percent of waiting, up to the pooler's own setting; a server is taken back only while clients waited under one percent of the second and a transaction was served, and only one the pooler itself shows idle. At 95% of the line the knob is handed back to the pooler's own setting at once. If the knob is found at a value Omni-Compass did not write, it stops writing (one writer). At the end of every omni arm the pool size is restored to the snapshot and read back. The same plug fits any pooler or proxy with a console and a pool-size setting (PgBouncer, Pgpool-II, ProxySQL, the application's own pool); the processor time the plug's own readings take on the host is recorded beside the host's, so the receipt shows what the wiring itself costs.

## Independent simulators, each with its own native controller

Three published simulators show how the plug is wired onto three more kinds of native controller, and the simulator's own controller is always native:

- **A building's batteries** (CityLearn): the knob is each building's battery charge and discharge inside the simulator's own limits; native is the simulator's own rule-based control.
- **A substation's tap changer** (pandapower on SimBench grids): the knob is the tap position, one whole tap per move and a week of evidence before stepping down; native holds the tap at 1.00 per unit as the grid ships.
- **A robot arm's joints** (MuJoCo Menagerie): the knob is the joint speed of an axis that has slack; a paired physics trial before the run leaves an axis at its takt native, so an arm whose axes are all at their takt has nothing for Omni-Compass to move.

The same three wirings apply to the real controllers they model: a building management system's battery dispatch, a substation's tap-changer controller, a robot's servo limits. Receipts from a simulator are evidence class S: statements about the simulator's model, never about hardware.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A message broker's consumer group

Apache Kafka is wired as an operator runs it: one broker as shipped, a topic of eight partitions, a consumer group at the operator's count of two. Nothing models the broker. The wire in is the group's own end-to-end latency: each message carries its production time, each consumer records when its handling ended, and the reading is the mean over the messages consumed in the last second; a second with nothing consumed while messages wait reads as the line (someone is waiting and no one is taking), and nothing consumed with nothing waiting reads as calm. The wire out is the count of consumers in the group, which is what an operator changes, inside the cover [1, 8]: never fewer than one, never more than the topic has partitions. The compass holds the reading at 40% of a 500 ms line with a response time of 3 s, because a consumer added or removed shows only after the group rebalances. Slow service with messages waiting adds consumers in proportion to the force; calm with nothing waiting gives back one idle consumer a second after a five-second dwell; at 95% of the line every consumer the topic can use starts at once. The rebalance that every move costs is paid by Omni-Compass and counted in the gauges. At the end of every omni arm the count is handed back to the operator's and read back. The receipt this wiring is built to produce has the queue kept short and no message lost, at the cost of consumers running, which reads power given by rule and stands beside the gain.

## A cache's memory ceiling

Redis is wired as Ubuntu ships it, with the two settings an operator sets for a cache, a memory ceiling of 64 MB and the allkeys-lru eviction rule. The application in front of it is ours and declared: 1,500 requests a second from 32 threads on a Zipf working set that widens and narrows one notch at a time; a miss costs a declared 5 ms trip to the store behind the cache and a write-back. The wire in is the application's own request latency, the mean over the last second (a hit about 0.3 ms, a miss about 5.5 ms). The wire out is one setting through Redis's own console, `CONFIG SET maxmemory`, inside the cover [16, 512] MB. The compass holds the reading at 40% of a 2 ms line. Its direction rule carries the do-no-harm gate in the form this stack needs: misses grow the ceiling by notches of 8 MB **only while the cache is full** (memory used at 90% of the ceiling or more), because a miss in a cache with room to spare is a cold miss that no ceiling can mend; calm with nothing evicted in the last second gives back one notch a second, after a five-second dwell since the last growth; at 95% of the line with the cache full a quarter of the cover is added at once. One writer: a ceiling found at a value Omni-Compass did not write stops it. At the end of every omni arm the ceiling is handed back to the operator's 64 MB and read back. The memory the compass holds for a wide working set is the resource this wiring trades and reads power given by rule; the gain is the hit rate and the work inside the line.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Drone swarms: the autopilot's cruise

In gym-pybullet-drones (University of Toronto, MIT License) every Crazyflie 2.x flies its shipped position controller along the planner's path at the planner's cruise: that is native. The wire in is each drone's tracking error, the distance between the drone and the point its autopilot was told to be at, on a band from 0 to a declared safe error of 0.25 m. The wire out is one knob a drone, the cruise override: the fraction of the planner's cruise the carrot moves at, never under 1.0 (the planner's own) and never over 2.0 (a quarter of the autopilot's own speed limit), moving by at most 0.01 a control tick. With slack the compass spends it as speed; as the error grows it eases; at 95% of the safe error the cruise is the planner's at once. Two guards belong to this stack: a **separation wall** (a drone with another within 0.5 m flies the planner's cruise, whatever the error reads) and the **paired physics trial** before the counted missions, which asks the simulator's own figures whether a faster mission costs less energy at all and keeps the error under the safe error; where it does not, the override is left native for that cell and the cell reads "nothing for Omni to move". Energy is a declared model from the simulator's own motor constants, said to be a model wherever it is printed; a collision voids the cell. At every landing and at the end of the run the override is handed back and read back.

## A database's storage-engine cache

MongoDB is wired as an operator runs it: the server as its publisher ships it, its configuration as shipped but for the one setting an operator sets for the storage engine, the WiredTiger cache (512 MB in the reference wiring). The wire in is the server's own mean read latency over the last second, from its `serverStatus` operation latencies, differenced, beside the cache's own counters (bytes in the cache, pages requested from it and pages read into it). The wire out is one setting through the server's own console, `setParameter wiredTigerEngineRuntimeConfig cache_size`, inside the cover [256 MB, 2,048 MB]: the server's own floor at one end, a quarter of the machine at the other. The line is set on the stack's own figures so that a hit reads calm and a full cache that misses reads slow (1 ms in the reference wiring, held at 40%). The direction rule is the cache's: slow reads grow the cache by notches of 64 MB only while the cache is full (bytes in it at 90% of its size or more) and missing (pages read into it one percent or more of the pages requested from it in the last second), because slow reads in a cache with room to spare, or in a cache that already holds its working set, are not the cache's to mend; calm gives back one notch a second after a five-second dwell, and only while the cache's miss share is under one percent, because "no page evicted" is true of every cold cache while it fills; at 95% of the line with the cache full a quarter of the cover is added at once. One writer, read-back and the hand-back at the end are the plug's, as everywhere. Where the data files fit in the operating system's own page cache, a storage-engine cache miss is a read from memory and a decompression, not a disk read, so the gain available to this knob is smaller than on a machine whose data does not fit in memory; say so beside the receipt. The same plug fits any store whose engine exposes a cache size at run time (InnoDB's buffer pool, RocksDB's block cache); a store that can only be sized at restart has no knob Omni-Compass moves.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A database's buffer pool, resized online in the server's own chunks

MySQL is wired as an operator runs it: the server as its distribution ships it, its configuration as shipped but for the settings an operator sets for InnoDB, the buffer pool (512 MB in the reference wiring), the chunk the server resizes in at its shipped 128 MB, one pool instance, and the performance schema on. The wire in is the server's own mean statement latency over the last second, from the performance schema's statement summary, differenced, beside the pool's own read requests and reads from disk. The wire out is one statement through the server's own console, `SET GLOBAL innodb_buffer_pool_size`, inside the cover [128 MB, 2,048 MB] and always in whole chunks, because that is how the server itself moves the pool. Two things about this knob differ from a cache the server resizes at once, and the plug carries both. The server resizes asynchronously and reports its progress in its own status variable, so the plug waits for the resize to complete before reading back, and a pool found mid-resize is the server still carrying out Omni-Compass's own write, not a foreign hand. And a shrink evicts pages while the resize runs, so the dwell after any move is ten seconds rather than five. The line is set on the stack's own figures, above a hit and under a miss (a 0.6 ms statement line in the reference wiring, held at 40%). Slow statements grow the pool by chunks only while the pool is full and missing (one percent or more of its read requests), at 95% of the line with the pool full four chunks are added at once, and calm gives one chunk back after the dwell only while the pool's misses are under one percent of its read requests and a read request was served, because InnoDB keeps stale pages resident, so neither "pages free" nor "no page read from disk" ever says that a pool holds its working set, and the miss share does. **The hand-back waits for the server.** Under load InnoDB finishes a shrink only when the pages the load has pinned are released, and it ignores a new pool size while a resize is in progress; the plug therefore waits for the resize the server is carrying out, writes the snapshot, writes once more if the server ignored the first write, and records the restore (what was in flight, the seconds waited, the writes, the final read-back) in every omni arm's receipt. A knob the device applies on its own time is handed back on the device's time, and the proof of the hand-back is the read-back, not the write.

## The machine itself: the kernel's frequency governor under the processor's meter

The lowest level a governor can sit on is the machine it runs on. Linux already has a controller for the processor's clock, the frequency governor (`schedutil`, or the processor driver's own), choosing the clock every few milliseconds under a ceiling the operator sets once; and the processor has its own energy meter (RAPL) ticking in microjoules. Omni-Compass sits on top of the governor exactly as it sits on the HPA: the governor keeps running, and Omni moves one knob, the frequency ceiling of every CPU, through the kernel's own files.

The reading is the service's own mean latency, held at 40% of a 20 ms line by the compass law; calm gives one notch of the ceiling back (5% of the top clock), a slow service raises it by notches, the wall puts the top on at once. Every notch down is tried on the machine itself first (section 6.4): the paired trial, eight cost samples at the deepest notch already allowed and eight one notch further, allowed only if the cost does not rise within

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

2%. The cost is the index's own reading for one machine (package power × latency / work inside the line), so a notch that saves less energy than it costs in speed is refused and the ceiling stays where the operator left it; a second declared objective, energy per request inside the line with the line as the guardrail, is run beside it. The snapshot of every CPU's governor and ceiling is taken once; the ceiling is handed back and read back at the end; a ceiling found at a value Omni-Compass did not write stops it; the master switch and the lease hand back if Omni-Compass dies.

The meter is the processor's own counter, class P with the caveat that it is the processor's calibrated counter and not an external meter; a smart plug on the wall socket (`WALL_METER`) adds the whole machine, read by the harness only. GitHub's own machines are virtual and expose neither the governor nor the meter. The paired run for this muscle runs on a machine on the metal as root, in one command (`sudo bash scripts/cpu_power_run.sh`): a workstation on Linux, a bare-metal server, or a self-hosted runner on either.

## The data center under the servers: cooling, power and batteries

Under every cluster in this book stands a building. Its native controllers are older and more conservative than any autoscaler: the building management system holds the supply-air temperature through the cooling units' own loops, the chilled-water plant runs its own sequence, the uninterruptible power supply holds its reserve, the power distribution units meter every rack, and every server's management controller and every card's firmware enforce their own power limits. Omni-Compass sits on top of them exactly as it sits on the HPA. They keep running; it moves only the settings they already accept, and only inside the band the operator writes down.

**What it may set, and what it never sets.**

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Cooling | the building management system and the cooling units' own loops | the supply-air setpoint, inside the operator's band | `--cooling-cmd "<sets {c}>" --cooling-min-c --cooling-max-c --cooling-restore-c` | built (level 7) |
| Site power | the site's own limit and its breakers | none: site watts are read, and the site limit enters the engine as a stress | `--power-cmd "<prints site watts>" --site-limit-w <limit>` | built (level 7) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Muscle | Native controller | The knob Omni-Compass may hold | Switch | Status |
|---|---|---|---|---|
| Servers' processors | the kernel's frequency governor and the processor's own limits | the clock ceiling and the power limit | section 7.9 | built (level 6) |
| Cards | the card's firmware | the clock ceiling and the power limit | level 5 | built |
| Uninterruptible power and batteries held as reserve | the UPS's own reserve | **none, ever**: a reserve held for an outage is never a lever (the physics criterion) | | native by design |
| Batteries held for shifting energy | the site's dispatch | the charge window | | designed as an organ, not yet wired |

**How the cooling muscle moves.** The setpoint is moved by the heat state between the operator's minimum and maximum: a warmer setpoint while every reading is cool, which saves the chiller's energy, and a colder one as heat rises. The band must sit inside what the equipment is rated for; for most data-center equipment that is the 18 to 27 °C inlet range of the ASHRAE thermal guidelines, and inside any narrower limit the equipment's own warranty sets. The restore value is the operator's own setpoint, handed back and read back on OFF, at the end of every run, and by the lease if the governor dies.

**The do-no-harm gate at the site meter.** A warmer setpoint saves chiller energy only up to the point where the servers' own fans spend the saving: fan power rises steeply with inlet temperature. That is why the cooling knob is judged at the site meter and not at the chiller: a step is allowed only where the paired trial shows the site's energy for the same work no higher, every rack inlet inside its band, and no LOSS on any service gauge. Where it does not, the setpoint stays the operator's, and the result says so.

**Wiring it, in order.**

1. **Watch.** Give Omni-Compass read access to the building management system's points (supply and return air, every rack inlet, the chilled-water temperatures) and to the site meter, and let it watch at least one full cycle of the site's demand. Nothing is written.
2. **Write the band.** The minimum, the maximum and the restore value of the supply-air setpoint, inside the equipment's ratings, signed off by the facility's engineer; the site limit in watts.
3. **The wire check** (section 4.4). The cooling command must read back the setpoint it wrote; a building schedule that rewrites the setpoint is another writer, and Omni-Compass stops writing that lever by itself (section 5.2).
4. **One hall first.** A test hall or a single row, paired against its own native weeks, before any other hall.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

5. **Keep the switches in reach.** The master switch and the kill file work at the facility level exactly as at every other level.

**What is proven, and what is not.** The facility muscles are modelled today (class S): the data-hall cooling family of the catalog, the power distribution family, and the site budget shared by groups of cards, which held out with no minute over its budget. No live facility has run yet. A class P result needs a building management system's point list, read access, a site meter and a test hall; until a facility pilot has produced its own paired receipts, this section is a wiring instruction, not a claim of savings.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FOUR

# Operating It

*The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.*

CHAPTER 8

# The OFF Switch, the Rules, and the Log

**The rules Omni-Compass keeps on your system.**

1. **The master switch, for the whole harness, in a human hand:** `python3 tools/omni_switch.py off` turns every Omni-Compass governor on the machine off at once (the Kubernetes controller and the GPU governors). Each puts every setting it ever wrote back to the value it read before its first write, reads it back and exits; the command waits until all have, and says so. While the switch is OFF, no governor will start. `python3 tools/omni_switch.py on` allows them to be started again (nothing restarts by itself); `status` shows the switch and every governor running. Use it the moment anything looks wrong, including a suspected breach. Each governor also keeps its own switch for one muscle at a time: the kill file (or `OMNI_KILL=1`, or SIGTERM to the GPU governor). **If a governor dies without handing back** (killed outright, the machine crashed, it hung), nothing it wrote is left in place: every governor records, before its first write, the exact commands that put every setting back, and renews a lease every decision. Run `python3 tools/omni_switch.py watchdog` as its own service next to the governors: it hands back for any governor whose process is gone or whose lease has run out (a hung one is stopped first). `status` names any governor that died without handing back.
2. It records before it acts (annotations on Kubernetes objects; the `snapshot` line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for `--slo-clear-s` seconds after (180 by default), nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

**On autopilot: why nobody babysits it.** Omni-Compass is an autopilot: a closed-loop supervisory controller that runs, steers and corrects itself inside the system it governs, as a self-driving car's controller does inside the car. Once the paired trial on a muscle has passed and the governor is set to govern, nobody turns a knob and nobody has to react. Every change is the governor's own, and the governor audits itself: every write is read back from the system, every machine it gives back is kept only after its own paired trial shows the response time holds, every reading, decision and reason is in its audit, and it hands every setting back at the end; nobody has to check its work either. It holds the response time at the centre of its band, under the line, and acts as the reading approaches the line, so the correction lands before the line is crossed, not after a person is paged. What a native stack pays in people for the same service (an engineer watching dashboards, paged while the service is over its line, retuning an autoscaler's target or a node pool by hand) is the **babysitting tax**, and every Kubernetes and cloud receipt counts it in both arms: the **alert minutes** (whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open; judged, the aim lower), the **settings commands the governor wrote on its own**, and the **share of its additions decided while the service was still inside its line** (both shown, not judged). No person touches either arm of a paired run. The aim in the cloud is **no alert minute at all**: a minute over the line is a person woken, so an alert minute in the omni arm is a LOSS of the aim, its cause found and fixed and the test run again. Where native itself is short of machines (over its line on its own), the receipt says so beside the row: the same machines can be made to carry more, but no governor creates compute that the machines do not have.

**The three ways a governor stops, and what each leaves behind.** A referee or an underwriter should know exactly what happens to a governed system in each of the three ways Omni-Compass can stop, because the answer is the same in all three: every setting at native, read back, with no record left.

- **The reset** (every setting handed back to native) is the planned end of a run. The governor sees its reset file, hands every setting back in reverse order of power-up, reads each back, writes its last audit line and exits 0. Every benchmark arm ends with one and then checks the cluster from outside the governor: the HPA target at the operator's, the replica range as set, every pod's CPU limit at its shipped value, every worker back in service, no `omnicompass.io/` annotation left on any object. An arm whose reset check fails is invalid and is left out of the table, never counted.
- **The kill switch** (security) is the unplanned end of trust. `tools/omni_switch.py off` writes the switch file and sends every registered governor SIGTERM; each performs the same hand-back as a reset and exits; the command waits until all have and reports any that have not. No governor starts while the switch is off. It is one switch for the whole machine, not one per muscle, because the case it exists for, something driving a system through Omni-Compass, is not a case for picking which muscle to stop.
- **A death without a hand-back** (killed outright, a crash, a hang) is the case the lease covers. Before its first write every governor registers itself with the exact command that puts every setting back

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

(`omni_controller.controller --restore-only`, which restores from the snapshot annotations on the objects themselves, so it needs no memory of the dead process), and renews a lease every decision. The watchdog (`tools/omni_switch.py watchdog`), run as its own service beside the governors, finds any governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore command, and records what it did. Drill it before production: kill the governor outright in the middle of a governed paired run on your own system, and record how many seconds pass before every setting is back at the snapshot and what the service saw in that gap.

**Reading the log.**

| Line | Meaning |
|---|---|
| `gate: a sense is blind` | a reading failed; nothing is given back until it returns |
| `gate: pods scaling up` | pods first, machines after |
| `decision failed (n in a row)` | the cluster could not be reached; nothing was written; turn it OFF for native |
| `decided_by: compass` | the compass set the wires this decision |
| `decided_by: fail_up` / `blind_fail_up` | the service crossed the 0.95 wall, or a sense went blind: full capacity at once |
| `decided_by: thermal_hold` | the card reported a heat slowdown; nothing was tightened |
| `foreign_writer` | someone else changed a knob; Omni-Compass now observes only |
| `restored ... ok: true` | the OFF switch put every setting back and read it back |
| `"watchdog": "handed back"` (in the watchdog's own log) | a governor died or hung without handing back; the watchdog ran its recorded restore command; `ok` says whether every command exited 0 |
| `cruise: on` / `cruise: off` | work waited for a place for two decisions: machines in service up to the pool's own top; the line has been empty for two: cruise ends |
| `idle_floor` | the work is done and demand is at zero: the pedal lifted all the way, straight to the floor in one move |
| `verdict_state: left native` | no step of the slow knob passed the paired trial (section 1.2): the knob is never moved on this muscle |
| `shield_interventions` | the shield clipped a write to its bounds or its step limit; the count says how often |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Line | Meaning |
|---|---|
| `overhead` (the last line of the audit) | the governor's own CPU over the window: its process and every command it ran |

**How to read an audit from end to end.** The audit (`audit.jsonl`, one JSON record a line, every line timestamped) is the governor's complete account of itself, and a reader can follow a run through it without the manual: the first lines are the snapshot (every setting as found, written to the objects as annotations so a watchdog can restore from them); then, every decision, the readings (the response time and its age, the pods waiting, the machines seen, whether any sense is blind), the reading's place in its band (`compass`), the authority granted and the release gate's reason (`authority`, `node_gate`), the decision itself (the machines recommended against the machines seen, the HPA target set), every write with the command that made it and the value read back, and the decision's wall time (`decision_ms`); in cruise, the floor step's silence; at the end, the reset's writes and read-backs and the `overhead` record. The paired-run harness prints a decision trail from the audit at the end of every omni arm, one line a decision, so a reviewer reading a run's log sees what the governor saw and did at each minute.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 9

# Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.

**What an upgrade is, and what it is not.** The engine is fingerprinted by version (chapter 12). An upgrade that changes any rule, guard, preset or muscle of the engine is by definition the next engine version, and every receipt an operator made on the old version must be made again on it before the two are read side by side; `tools/omni_version.py` says which version a checkout carries, and a checkout that matches none says which files differ. An upgrade that changes only the harness, the tools or this manual leaves the engine's fingerprint as it was and every receipt standing. An operator can therefore tell, from one command, whether a new checkout is the engine their receipts were made on or something that must prove itself again.

- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

**Least privilege, proven on every run.** The Kubernetes governor runs as its own service account (`deploy/kind/rbac-omni.yaml`) with exactly the verbs the level needs: patch on the governed HPA, nodes, pods and the governed deployment, get on its security ConfigMap, create on pod evictions. It cannot delete nodes, create or delete pods or deployments, touch the load generator or anything in `kube-system`, read any secret, create a namespace, or write its own security hold. Every paired arm records `kubectl auth can-i` for each of these and fails if a "cannot" reads yes or a "can" reads no (`rbac_omni.txt` in every paired run's files). The security hold itself is a ConfigMap another identity owns: when its key reads `hold: "true"` every expansion is blocked, and the governor cannot clear it.

**Credentials, and the notice.** Cloud credentials live only in an encrypted secret store and are used only by the jobs that need them; no credential, key or token belongs in any file, log, receipt or commit. Every source file carries its SPDX license identifier so that license scanners (Black Duck, FOSSA, Snyk) read the evaluation license correctly, and every generated report carries the legal notice at its head and foot.

**Supply chain.** Everything the paired-run harnesses download is pinned and checked: metrics-server by version and SHA-256 (`scripts/kind_bench.sh`), Apache Kafka against the Apache Software Foundation's own SHA-512 file for that release, Redis as the Ubuntu runner's own package, MongoDB from

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

its publisher's own repository signed by the publisher's key and YCSB against the SHA-256 written in its preregistration, gym-pybullet-drones at one named commit, Python packages by `requirements.txt`. A download whose checksum does not match stops the run before anything else happens.

**The upgrade drill, step by step.** An operator who receives a new checkout does five things, in order, and stops at the first that fails. First, `python3 tools/omni_version.py`: if it prints the version the operator's results were made on, the engine is unchanged and the rest of the drill is about the harness; if it prints a list of differing files, the checkout is a new engine and nothing the operator measured before applies to it until measured again. Second, `python3 verify.py`, which must end with its pass line; a checkout that fails it is not installed anywhere. Third, the wire check on the operator's own stack, which proves that every wire the new checkout would use still follows, reads back and returns. Fourth, watch mode for at least one full cycle of the operator's load, reading the log the governor would have written and comparing its would-be decisions against what the operator would have done. Fifth, the levels again from level one, each with its paired receipt. An upgrade that skips a step has not been upgraded; it has been installed.

**The security hold, and who holds it.** The hold is the one control that outranks the governor's own judgment, and it is deliberately not the governor's to hold. It lives in a ConfigMap owned by another identity (the operator's security team, in practice); while its key reads true, every expansion the governor would make is blocked and every contraction is still allowed, so a governor under a hold can only give capacity back, never take more. The governor reads the hold on every decision and records in its audit that it read it; it has no verb that could write it. The kill switch (chapter 8) is the blunter instrument beside it: the hold constrains a running governor, the switch stops every governor on the machine and hands everything back. An operator who suspects anything untoward uses the switch first, reads the audit second, and rotates the governor's identity third; the hold is for the slower case of a change window or an incident elsewhere during which no one wants capacity added.

**What has been reviewed, and what has not.** Tested on every paired run: the governor's permissions against the list above, read from the cluster itself; the refusal of every write past a cover, of every give-back while blind, and of every move under a hold; the lease and the watchdog's hand-back on a real cluster; the checksums of every download. Not yet done, and said so: an independent security review of the code by a party outside the company, and a penetration test of the deployed governor on a customer-like cluster. Until those are done, the operator's own review stands in for them, and this chapter tells the operator exactly what to look at.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART FIVE

# Checking It on Your Own System

*Paired runs and receipts on your own system, evidence classes, the engine version and the three-run rule.*

CHAPTER 10

## Paired Runs and Receipts on Your Own System

The method of every published benchmark is the method an installer uses on their own system, and it is deliberately simple: the same system, the same work, native and then omni back to back on the same machines, repeated with the order rotated, each gauge reported with the 95% interval of its paired difference. Pairing is what makes a difference of a few percent visible on a shared machine whose own noise is larger than that: two arms run minutes apart on one runner see the same neighbours and the same hour, so their difference is a difference between arms, while two arms on separate machines differ by about 15% in p95 on their own. The order rotates so that no arm always runs first, warm or cold. The work is held equal by an open-loop load, a fixed rate of requests at fixed moments, so that a lower resource count can never mean that less work was done. The steps below are the whole method.

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish). `scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: the wire check, a smoke run, then the counted run.
2. **The switch drill** after every run: turn Omni-Compass OFF; check every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change is proven only when it does.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / NOT ESTABLISHED / LESS

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

WORK PER ENERGY / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

5. **Write it down, then run.** The arms, the knob, the cover, the gauges and the readings are written down before the first counted run; the rules are fixed on one tuning case and then applied unchanged to the cases that are counted. A change made after a result has been seen is written down with its time and reason, and the receipt says so.
6. **Every row, losses included.** A row that went against Omni-Compass is reported with the others. If Omni-Compass loses, the first suspect is the wiring, then native's setup, then the scoring; that is fixed and the run repeated, and the loss stays in the record.
7. **Keep the raw files.** Every run's files (capture, response times, audit log, controller log, one SHA-256 manifest per repetition) are kept with the receipt, and every receipt names the runs and the commit it was made from, so that anyone holding the files can rebuild it.

**How many repetitions, and why.** The width of a paired interval shrinks with the square root of the number of pairs, $w \propto 1/\sqrt{n}$, so going from five pairs to ten narrows it by about a third ($1 - 1/\sqrt{2} \approx 0.29$), and from ten to forty by half again. On shared cloud runners, ten pairs of a 900 s window typically put the interval on a cluster's work inside the line at about ±10 points of native and on its 95th-percentile response time at ±15 to ±30 points: wide intervals, which clear zero only when the effect is tens of percent. A gauge computed from a declared formula over counts (an energy model over machines in service) has a much narrower interval, and a difference inside it is read as no difference, never as a small gain. A gain of a few percent on a cluster needs forty pairs or more to clear zero at these widths. On a stack with a finer knob (connections, consumers, megabytes) the intervals are narrower at three pairs than a cluster's at ten, because the knob moves in hundreds of small steps rather than in whole machines. Run five pairs first, look at the interval widths on the gauges you care about, and set the count for the counted run from those widths, never from the result.

**Why the load is open-loop.** A closed-loop load generator waits for each answer before sending the next request, so a slower system is offered less work and its throughput "improves" its own latency; two arms under closed-loop load are not doing the same work. An open-loop generator sends requests at fixed moments whatever the system does, so both arms are offered the same work to the request, a queue forms when the system falls behind, and the time over the line and the failed requests record the cost. Every paired-run harness (the cluster's load generator, pgbench's rate limit, Kafka's producer at a fixed rate, Redis and YCSB at a fixed target rate) offers its work open-loop, and a lower resource count in the omni arm can therefore never mean less work was asked. A burst is a schedule of open-loop rates written before the run and applied identically in both arms; only the load generator reads it, never the governor.

**What a receipt looks like.** One row per gauge: the gauge's name and direction; native's value and omni's; the paired change as a percentage of native with its 95% interval; the reading. A head naming the system, the knob, the cover, the window, the number of pairs, the order rotation, the engine's fingerprint and the commit; a foot saying whether every arm was handed back. `tools/live_reps.py` prints exactly this for a Kubernetes run and `tools/confirm_abc.py` prints it for three runs side by side; a receipt from any other

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

tool should be laid out the same way so that a reader of this manual can read it without learning a new shape.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

CHAPTER 11

# Evidence Classes and How to Read a Receipt

**One number, one word.** In every receipt a change keeps its raw sign: a response time that fell reads "-65%", a cache that grew reads "+300%", and the Reading column beside it gives its word, GAIN, EVEN or LOSS, decided by what the gauge is for and never by its sign (the key at the front of this manual): a response time cut 65% is GAIN, a cache grown 300% is **power given**, a fall in work is LOSS. Wherever a change is written in a sentence, it is written in words, never as a bare sign: "p95 cut 65% (GAIN)", so it cannot be misread against a receipt that shows the same change as "-65%". Read the Reading column, never the sign alone.

| Class | Rung | What it is | What it can show |
|---|---|---|---|
| T / V | E1 | deterministic tests of the software | the software does what its documentation says |
| S | E2 | a run on a modelled plant | what the governor does to the model, and where it breaks |
| L | E3 | real software, no hardware meter | real decisions on real software; energy there is a declared model |
| P | E4 | a physical meter (a card's own power reading, the processor's energy counters, a site meter) | the hardware's own answer |

Read every number with its class beside it. A modelled number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

**What each class can and cannot carry.** A class **T / V** result says that the software computes what its documentation says it computes; it says nothing about whether the governor helps any machine. A class **S** result says what the governor does to a model of a plant under that model's own native controller; it carries exactly the model's fidelity and no more, and every S line that reports energy says that the energy is a declared model. A class **L** result says what the governor did to real software making real decisions on real requests; where the machines are containers on one host, they are counted in machine-hours in service and their energy, where it is reported, is a declared formula, which the receipt says; the processor seconds of the host are

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

measured. A cloud bill is an L result with a real bill: the machines billed are the provider's own count and the price its list price, and the energy is still not a meter. A class **P** result says what a physical meter read, and only a P result speaks for hardware energy.

**How to read a row.** Every row in every three-run table has the same shape: the gauge and its direction; native's value in run A and omni's value in run A, so the size of the numbers is visible; the paired change in each of the three runs as a percentage of native with its 95% interval; and the reading. Read the reading first, then the intervals, then the sizes. A reading of "GAIN in all three of our runs" with intervals of −0.5% to −0.1% is a small, real effect; a reading of "no difference beyond the noise (3 of 3 runs)" with intervals of −30% to +25% is a test that could not see an effect of that size either way, which is a statement about the test's power, and the receipt should say so (a 4-worker fleet cannot show a saving smaller than one machine, section 7.1). A "LOSS in all three runs" row stands in the receipt.

**Three questions to ask of any class L result.** First, what was native, exactly: which controller, at which published version, with which settings, and were those settings an operator's or ours? (Write them down before the run; a native set anywhere other than the operator's own setting must say so.) Second, what did the governor spend: machines, memory, connections, CPU, its own cost? (Every receipt has the resource rows; a result with a gain and no resource row is not a receipt.) Third, who else could have produced the gain: the order of the arms, the warm-up, the runner's neighbour, the clock? (Rotation of the arm order, the warm-up before every window, the paired design on one machine and the clock rule are the answers; whatever residue they do not remove is written beside the receipt.)

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 12

# The Engine Version and the Three-Run Rule

**Every engine version has a fingerprint.** The engine is fingerprinted file by file: one SHA-256 per file and one digest over all of them, held in the version's registry (`OMNI_V5.json` for the engine that stands). Any change to a rule, guard, preset or muscle makes **the next version**, and every receipt is made again on it; no receipt is ever read across versions. When the engine is declared final, the engine that stands then is published as Omni-Compass 1.0.

```
python3 tools/omni_version.py                    # this checkout: its e
    ↪ ngine version, or every file that differs
python3 tools/omni_version.py --commit &lt;sha&gt;    # the engine at the co
    ↪ mmit any receipt ran on
```

A receipt names the commit it ran on, and every table made by rule checks each run's commit against the fingerprint and heads itself with a warning if any run is not the engine it claims or if the three are not separate runs. An operator's own tune (section 3.6) does not change the engine's fingerprint, and must therefore be written down beside the receipt.

**The three-run rule.** A result is run as **A, B and C**, three separate runs on the same engine version. Every judged row gets one of three readings, and all three runs are shown beside it:

- **GAIN** or **LOSS in all three runs** (a resource spent that rose reads **power given**, never a loss): the same sign in all three runs, each with its 95% interval clear of zero.
- **No difference beyond the noise** (EVEN): in at least one run the interval includes zero, so native and omni could not be told apart on that measure. That is the result, stated as such, with the count of runs it holds in.
- **The runs disagree**: runs clear of the noise point different ways. The test itself is then unstable on that measure, and it is looked into before anything is claimed.

The three-run table is made by a tool, never by hand: `tools/confirm_abc.py` for the paired Kubernetes and cloud runs, with one sibling tool per kind of raw record for the other stacks (`pgbench_abc.py`, `kafka_abc.py`, `redis_abc.py`, `ycsb_abc.py`, `sysbench_abc.py`), each checking the engine of every run it reads. A deterministic simulator's three runs must reproduce each other to the digit, or to the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

declared tolerance where the simulator is not bit-reproducible across machines.

**Why three separate runs, and why the readings are what they are.** One run of ten paired repetitions gives one interval, and an interval from one run can be narrow by luck: the machine was quiet that hour, the order of arms happened to favour one side. A second run on another day and another machine is a replication; a third makes a coincidence expensive. The three runs are separate runs, on separate machines, at separate hours, with separate installations of every dependency, so what agrees across them is not an artefact of one machine. The readings are designed so that no reading can be mistaken for a weaker form of "yes": a row counts only when all three runs move the same way with every interval clear of zero; a single run whose interval crosses zero makes the row "no difference beyond the noise", and the count says how many of the three; and two clear runs that disagree in sign are reported as a disagreement, which is an instability in the test to be found and fixed, not averaged away.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART SIX

# The Implementer's Guide: Omni-Compass on a Cloud

*For the engineer who installs it on Azure, Google Cloud or Amazon Web Services and keeps it in service: the path and its gates, the permissions, each cloud step by step, the response-time feed, the cloud mode, day two, the graphics processor, observability, change and risk, planning the saving, own racks and private clouds, hardening, a fleet, the failure drills, a complete installation, and the questions implementers ask.*

CHAPTER 13

# The Implementer's Path: From an Empty Account to a Governed Cluster

This part is written for the **implementer**: the platform, site-reliability or infrastructure engineer who is handed Omni-Compass and a cloud account and is responsible for installing it, proving it on the company's own cluster, putting it into service and keeping it there. It assumes fluency with Kubernetes objects, role-based access control (RBAC), the cloud provider's command-line tool and a container registry, and nothing about how the governor is built. Every step names the object it creates, the permission it needs, the evidence it leaves and the check that must pass before the next step begins. Where the three clouds differ, the step is written once per cloud (chapters 15 to 17); where they do not, it is written once here or in chapters 14, 18, 19 and 20.

## What the implementer delivers

At the end of this part an implementer has delivered five things, each with its own evidence:

| Deliverable | Evidence that it is done |
|---|---|
| A verified copy of the software, at a known engine version | `python3 verify.py` ends `VERIFICATION: PASS`; `python3 tools/omni_version.py` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Deliverable | Evidence that it is done |
|---|---|
| | names the version (chapter 32) |
| The governor installed in watch mode, writing nothing | its audit shows a `decision` record every period (the machines and the target it recommends) and no `write` record; every object it reads is listed in chapter 14 |
| A response-time feed for the service, and the service's line written down | the feed file advancing at least every 5 seconds; the line taken from the service's own objective (chapter 18) |
| A paired receipt on the company's own cluster: native against native with Omni-Compass on top | the receipt's rows, each read GAIN, EVEN or LOSS, and the run's commit and engine version (chapters 10 to 12) |
| The governor in service, with its OFF switch, its watchdog and its alarms wired | the switch tested, the watchdog's hand-back drilled, the alert route proven by a test page (chapter 20) |

None of the five is skipped, and they are delivered in that order. The order is not bureaucracy: each one is the evidence the next one stands on. A governor that has not watched cannot be judged before it writes; a receipt without a written line cannot be read; a governor in service without a drilled OFF switch is not in service, it is at large.

## What Omni-Compass needs from the cluster, and what it never touches

**What it needs.** A Kubernetes cluster on a supported minor version of the provider's managed service; the Kubernetes Metrics API (`metrics.k8s.io`, served by metrics-server: built into Azure Kubernetes Service and Google Kubernetes Engine, installed by the implementer on Amazon Elastic Kubernetes Service); a HorizontalPodAutoscaler (HPA) on each service it governs, scaling on CPU utilisation; a cluster autoscaler on the work node pools (the provider's own, or Karpenter on Amazon); a namespace and a service account for the governor; and a response-time feed for each service whose line it holds (chapter 18). In-place pod resize (the `pods/resize` subresource) is needed only where the governor conveys processor to running pods, which it never does in the cloud mode of chapter 19.

**What it never touches, in any mode.** The governor never creates, deletes, evicts or restarts a pod; never changes a container image, a command, an environment variable or a request; never changes a node's machine type or a pool's configuration in the provider; never holds a cloud credential in the cloud mode (it acts through the Kubernetes API alone); and never writes a setting it has not first recorded on the object it writes, so that every setting can be put back from the object itself (chapter 8). In the cloud mode it also never moves a pod autoscaler, never conveys processor and never caps power: it moves the machines alone (chapter 19).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The eight phases and their gates

| Phase | What happens | Gate to the next phase | Typical time |
|---|---|---|---|
| 0. Verify | the copy is checked against its manifest and engine version | `VERIFICATION: PASS`, the version named | under an hour |
| 1. Account and cluster | the cloud account, quotas, the cluster with a system pool and work pools | the cluster autoscaler scales a work pool up and down on its own | half a day |
| 2. Identity | namespace, service account, the read-only role | `kubectl auth can-i` answers exactly the table of chapter 14 | an hour |
| 3. Watch | the governor installed in watch mode | an audit with decisions and no write, for at least one full day of the service's own load | one to seven days |
| 4. Feed and line | the response-time feed and the written line | the feed's age under 15 seconds at every decision; the line signed off by the service's owner | a day |
| 5. Paired run | native against native with Omni-Compass on top, on the company's own cluster | a receipt read row by row, every LOSS explained or fixed and rerun | two to five days |
| 6. Govern | the governing role granted, the governor switched to govern, the OFF switch and the watchdog wired | the OFF switch drilled: every setting back and read back within 60 seconds | a day |
| 7. Day two | upgrades, rollbacks, audits, alarms, fleet roll-out | quarterly drill of the OFF switch and the watchdog | ongoing |

Phases 3 and 5 are the long ones, and they are long on purpose: a governor is judged on the service's own load, which has its own daily and weekly shape. A watch phase shorter than one full cycle of the service's own load (a day for most services, a week for a service with a weekly peak) has not seen the service.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Roles, approvals and the separation of duties

| Role | Holds | Signs off |
|---|---|---|
| Implementer | the namespace, the service account, the install, the feed | phases 2 to 5 |
| Service owner | the service's line (its response-time objective) and its HPA | the line (phase 4) and the receipt (phase 5) |
| Platform owner | the cluster, its pools and its autoscaler profile | the governing role (phase 6) |
| Security | the RBAC review and the OFF switch | the role table (phase 2) and the OFF switch drill (phase 6) |
| Finance (cloud cost) | the bill's gauge and its list prices | the bill rows of the receipt (phase 5) |

The OFF switch (`python3 tools/omni_switch.py off`, chapter 8) is held by someone who is not the implementer. It is the one control in this part whose purpose is to stop the governor when the people who installed it are not available or not trusted, and it is only worth what that separation makes it worth.

## What it costs to implement

The governor's own cost is small and is measured in every receipt (the row "Omni's own CPU (cores), mean"): one process, a request of 50 millicores and 128 MiB, a limit of 500 millicores and 256 MiB in the shipped manifest. The cost of an implementation is the cost of the evidence: a cluster for the paired run, sized like the service's own, for the hours the run takes. On the managed services in this part, a two-vCPU worker costs between about USD 0.07 and USD 0.10 an hour at list price (Google's e2-standard-2, Azure's Standard_D2s_v4 and Amazon's m5.large in their eastern United States regions; check each provider's price list on the day), a control plane costs nothing or about USD 0.10 an hour depending on the provider and tier, and a paired run of five repetitions on four workers is a few tens of dollars. The run builds a fresh cluster for every arm and deletes it when the arm ends, so nothing keeps billing.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## How to read the rest of this part

Chapter 14 sets the permissions, identically on every cloud. Chapters 15, 16 and 17 build the cluster and install the governor on Azure, Google Cloud and Amazon Web Services, click by click and command by command. Chapter 18 sets up the response-time feed and the line, on any cloud. Chapter 19 is the cloud mode itself: why the governor moves the machines alone in a cloud, what that guarantees about the bill, and how to read the bill and the alarm minutes. Chapter 20 is day two: running the governor as a service, upgrades, rollbacks, the watchdog, alarms and a fleet of clusters. Chapter 21 is the graphics processor, written for the implementer who will put the governor on a card, rented or owned. Chapter 22 connects the governor to the company's own monitoring in both directions, and chapter 23 maps the installation onto change management, risk and compliance. Chapter 24 bounds, before any run, the saving a cloud pool can show and where in the day it lives. Chapter 25 is for clusters on the company's own racks and in private clouds; chapter 26 hardens the installation and its supply chain; chapter 27 rolls it out across a fleet and under continuous delivery; chapter 28 lists every failure to expect, with the drills that rehearse them; and chapter 29 is the whole path at once, on one cluster, start to finish. Chapter 30 is the implementer's questions, answered. The accounts and the paired run from GitHub are in Appendices F to H, the install manifests in Appendix I, and the checklists in Appendix J.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 14

# Identity, Permissions and Least Privilege on a Managed Cluster

The governor acts on a cluster only through the Kubernetes API, as one service account, with the verbs its level needs and no more. This chapter gives those verbs exactly, level by level, the same on every cloud; the cloud-specific steps (registry, cluster credentials, the provider's own identity) are in chapters 15 to 17.

## One namespace, one service account, one process

The governor runs as one Deployment of one replica in its own namespace, `omni-compass`, under the service account `omni-compass`. One replica is a rule, not a default: the governor is the one writer of every setting it moves (section 5.2), and two copies would be two writers. A second replica for availability is not needed and must not be added; if the one process dies, the watchdog of chapter 20 puts every setting back from the records on the objects themselves, and the native controllers carry the service as they did before the governor was installed.

The pod runs as a non-root user (UID 10001) with a read-only root filesystem, no Linux capabilities, no privilege escalation and the runtime's default seccomp profile. It writes only two places: its audit log (an `emptyDir` volume mounted at `/var/log/omni`, or a persistent volume where the audit must outlive the pod, section 20.4) and `/tmp` (an `emptyDir` for its kill file). It needs no secret, no host path, no host network and no service exposed to anything.

## The verbs, level by level

The table is the complete list. A verb not in it is not needed at that level and must not be granted.

| Level | API group | Resources | Verbs | Why |
|---|---|---|---|---|
| Watch (every level) | core | `nodes`, `pods` | `get`, `list` | read the machines, their allocatable processor and the pods on them |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Level | API group | Resources | Verbs | Why |
|---|---|---|---|---|
| Watch | `metrics.k8s.io` | `nodes`, `pods` | `get`, `list` | read processor in use |
| Watch | `autoscaling` | `horizontalpodautoscalers` | `get`, `list` | read each autoscaler's target, range and current replicas |
| Watch | `apps` | `deployments`, `replicasets`, `daemonsets` | `get`, `list` | read what each autoscaler scales, and tell workload pods from per-node agents |
| Watch | core | `configmaps` (the one security-hold ConfigMap, by name) | `get` | read the security hold, where one is wired |
| Machines (the cloud mode) | core | `nodes` | `patch` | park a machine (a `PreferNoSchedule` taint) and wake it (remove the taint); a machine someone else cordoned is never uncordoned |
| Machines | core | `pods` | `patch` | mark the pods on a parked machine first to go (the `pod-deletion-cost` annotation), so the autoscaler's own scale-down removes them first |
| Pod target (not in the cloud mode) | `autoscaling` | `horizontalpodautoscalers` (each governed one, by name) | `patch`, `update` | move the autoscaler's CPU target inside its cover, and record the operator's own target on the object |
| Processor conveyance (not in the cloud mode) | core | `pods/resize` | `get`, `patch` | resize a running pod's processor limit in place; no pod is restarted |
| Batch admission (where wired) | `batch` | `jobs` | `patch` | admit held work |

Two verbs are absent from every level and are refused by review if they appear: `delete` on anything, and `create` on `pods/eviction`. The governor does not delete or evict; a parked machine is emptied by the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

cluster autoscaler's own scale-down, as it would be without the governor, and only once its pods have gone by the autoscaler's own rules.

## The cloud mode's role, complete

For the cloud mode of chapter 19 (the machines alone), the grant is the watch role plus two verbs. Apply it in place of the level-2 and level-3 files of chapter 6:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-machines}
rules:
  - apiGroups: [""]
    resources: [nodes]
    verbs: [patch]
  - apiGroups: [""]
    resources: [pods]
    verbs: [patch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: {name: omni-compass-machines}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-machines}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

The watch role is in the install manifest (`deploy/install/omni-compass.yaml`, printed in Appendix I); it grants only `get` and `list`. Together the two files are everything the governor can do in the cloud mode, and every write it makes is in its audit with the command and the reason.

## Proving the grant before the governor runs

Before the governor is started at any level, the implementer proves the grant from outside it, as the service account, and keeps the output with the installation record:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
SA=system:serviceaccount:omni-compass:omni-compass
for v in "get nodes" "list pods" "get horizontalpodautoscalers.autosca
    ↪ ling" "list deployments.apps" \
         "patch nodes" "patch pods" "patch horizontalpodautoscalers.au
    ↪ toscaling" "patch pods/resize" \
         "delete pods" "create pods/eviction" "delete nodes" "create p
    ↪ ods"; do
  printf '%-48s %s\n' "$v" "$(kubectl auth can-i $v --as=$SA -A)"
done
```

In the cloud mode the answers are `yes` for the four reads and for `patch nodes` and `patch pods`, and `no` for everything else, including `patch horizontalpodautoscalers.autoscaling` and `patch pods/resize`. Any other answer is a grant to correct before the governor starts, not after.

## No cloud credential

In the cloud mode the governor holds no credential of the cloud provider: no Azure service principal, no Google service account key, no Amazon access key, no workload-identity binding. It parks and wakes machines through the Kubernetes API; the provider's own cluster autoscaler, with the provider's own identity, makes and deletes machines. This is a security property worth stating to a reviewer in one sentence: an attacker who took the governor's pod would hold a Kubernetes service account that can read the cluster's shape and taint nodes and annotate pods, and nothing that can spend money in the provider, open a network path or read a secret.

The paired runs from GitHub (Appendices F to H) are different: there the workflow itself builds and deletes clusters, and it holds a provider credential as a GitHub secret (`AZURE_CREDENTIALS`, `GCP_CREDENTIALS`, `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`). That credential belongs to the test harness, never to the governor, and it is removed when the evaluation ends.

## Network and data

The governor talks to one endpoint: the cluster's API server, through a single local proxy it starts itself (one `kubectl proxy` for the life of the process; `--no-api-proxy` falls back to one `kubectl` call per read, at a higher cost of its own). A NetworkPolicy may restrict its egress to the API server and, where the response-time feed is read over the network, to the feed; it accepts no ingress. It sends nothing off the cluster. The audit holds object names, replica counts, machine counts, response-time statistics and the governor's reasons; it holds no request payload and no personal data, because the governor never sees either.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The record an installation keeps

| Item | Where |
|---|---|
| The engine version and the copy's verification receipt | `python3 tools/omni_version.py`, `results/VERIFY_RECEIPT.txt` (chapter 32) |
| The grant as applied, and the `kubectl auth can-i` output of section 14.4 | the installation record |
| The governor's command line (its arguments in the Deployment) | the Deployment object, kept with the installation record |
| The service's line, and who signed it | the installation record (chapter 18) |
| The audit of every decision and every write | the audit volume (chapter 20) |
| Every OFF-switch drill, with the seconds to hand back | the installation record (chapter 20) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 15

# Azure Kubernetes Service, Step by Step

This chapter takes an implementer from an Azure subscription to Omni-Compass watching an Azure Kubernetes Service (AKS) cluster, ready for the governing switch of chapter 19. Every command runs in Azure Cloud Shell (Bash) or on a workstation with the Azure CLI (`az`), `kubectl` and Docker; nothing else is needed. Replace the names in angle brackets.

## The subscription and its allowances

1. **Sign in and choose the subscription.** `az login`, then `az account set --subscription <subscription-id>`.
2. **Register the provider once:** `az provider register --namespace Microsoft.ContainerService --wait`.
3. **Read the allowances before building anything.** Azure counts vCPUs per region and, separately, per machine family in that region; a cluster fails on the family's allowance, not the region's. Read both:

``bash az vm list-usage -l <region> -o table | grep -Ei "Total Regional vCPUs|Standard DSv4|Standard DSv5|Standard Dv4" ``

A new pay-as-you-go subscription commonly has 10 vCPUs a family and a few hundred a region. A work pool of four two-vCPU machines plus a one-machine system pool needs 10 vCPUs of the families used. Raise an allowance in the portal (**Quotas**, then **Compute**, the family and region, **Request quota increase**) before the cluster is built, not after a half-built cluster fails.

4. **Pick the machine family the region offers.** `az vm list-skus -l <region> --size Standard_D2 -o table` lists what the subscription may create there. A size marked "NotAvailableForSubscription" is refused at build time.

## The cluster: a system pool, a work pool, the autoscaler

The layout is the one every receipt in this manual is made on, and it is the right layout for production too: the cluster's own add-ons and the governor on a small **system pool** that no workload may land on, and the service on one or more **work pools** under Azure's cluster autoscaler, each labelled `omni-role=work`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
RG=&lt;resource-group&gt;; NAME=&lt;cluster&gt;; LOC=&lt;region&gt;; SIZE=Standar
    ↪ 4
az group create -n $RG -l $LOC -o none
az aks create -g $RG -n $NAME -l $LOC --tier standard --node-count 1 -
    ↪ -node-vm-size $SIZE \
  --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoS
    ↪ chedule --nodepool-labels omni-role=system \
  --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down
    ↪ -delay-after-add=10m scan-interval=10s \
  --generate-ssh-keys -o none
az aks nodepool add -g $RG --cluster-name $NAME -n work0 --mode User -
    ↪ -node-vm-size $SIZE --labels omni-role=work \
  --node-count 2 --enable-cluster-autoscaler --min-count 2 --max-count
    ↪  &lt;pool top&gt; -o none
az aks get-credentials -g $RG -n $NAME
kubectl get nodes -L omni-role,agentpool
```

Four choices in those lines matter, and each is the operator's, not the governor's:

- **The tier.** `standard` (about USD 0.10 an hour a cluster, with Azure's uptime agreement) or `free`. The control plane is not in the bill the receipts count, which is worker machines only. In some regions at busy hours Azure refuses new free-tier clusters ("creating a new cluster is unavailable at this time in region"); the standard tier is the way past.
- **The autoscaler profile.** `scale-down-unneeded-time` is how long a machine must be unneeded before Azure deletes it, and so how long a machine the governor has parked keeps billing once its pods are gone. Azure's default is 10 minutes; the paired receipts in this manual used 2 minutes in both arms. Choose it for the service, write it down, and keep it the same in both arms of any paired run. The governor never changes it.
- **The pool's floor.** `--min-count` is the autoscaler's own floor. The governor's floor of two machines in service (`MIN_NODES`, section 3.4) is applied on top of it; set the pool's floor to at least two for a service that must survive one machine's loss.
- **The pool's top.** `--max-count` is the most machines Azure will make. The governor never asks Azure for a machine; it parks and wakes the ones Azure made (chapter 19), so the pool's top is the operator's cost ceiling, as it is without the governor.

Several machine families can be pooled, each as its own work pool with its own top, all labelled `omni-role=work`; that is how a fleet gets past a ten-vCPU allowance a family, and the governor treats all labelled pools as one set of machines.

**Prove the autoscaler before anything else.** Scale a test Deployment past the pool's capacity and back, and watch Azure add and delete a machine (`kubectl get nodes -w`). A cluster whose autoscaler has not

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

been seen to work both ways is not ready for a governor that relies on it to delete the machines it parks.

## 15.3 metrics-server and the service's autoscaler

AKS runs metrics-server in every cluster; check it with `kubectl top nodes`. The service must have a HorizontalPodAutoscaler on CPU utilisation, with requests set on its containers (an HPA cannot compute utilisation without them):

```
kubectl autoscale deployment &lt;service&gt; -n &lt;namespace&gt; --cpu-percent=&lt
    ↪ arget&gt; --min=&lt;min&gt; --max=&lt;max&gt;
kubectl get hpa -n &lt;namespace&gt;
```

The target, the minimum and the maximum are the service owner's, written down before the governor is installed. In the cloud mode the governor never moves them; they are recorded because the receipt's native arm runs on them.

## The registry and the governor's image

```
ACR=&lt;registry-name&gt;
az acr create -g $RG -n $ACR --sku Basic -o none
az aks update -g $RG -n $NAME --attach-acr $ACR -o none
az acr build -r $ACR -t omni-compass:&lt;version&gt; -f deploy/Dockerfile .
az acr build -r $ACR -t omni-compass-cloud:&lt;version&gt; -f deploy/Dockerf
    ↪ ile.cloud \
  --build-arg BASE=$ACR.azurecr.io/omni-compass:&lt;version&gt; .
```

The first image is the shipped one (`deploy/Dockerfile`): the governor, non-root, read-only, with `kubectl`. The second adds what the cloud mode needs to move machines and to feed itself, and is written once for every cloud in section 19.4. Tag images with the engine version printed by `python3 tools/omni_version.py` and the commit, never `latest`, so the audit of any decision can be tied to the exact code that made it.

## Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ ACR.azurecr.io/omni-compass:&lt;version&gt;
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

The install manifest (Appendix I) creates the namespace, the service account, the read-only role and one Deployment running `--mode observe`. In watch mode the governor computes every decision it would make and writes none: each `decision` record carries the machines and the pod target it recommends (`nodes_recommended`, `hpa_target_recommended`), and no `write` record appears. Leave it watching for at least one full cycle of the service's own load (section 13.3). To keep the audit past a pod restart, mount a persistent volume at `/var/log/omni` (section 20.4); on AKS the `managed-csi` storage class provides one.

## The paired run on Azure

The receipt that decides whether the governor goes into service is a paired run on the company's own service: native against native with Omni-Compass on top, the same load in both arms, a fresh cluster per arm, repeated (chapter 10). The repository's own workflow `aks-metered` runs exactly that from GitHub, on clusters it builds and deletes itself, and counts Azure's billed machines every 15 seconds at list price; Appendix F connects it to a subscription, click by click. To run it on a company's own service instead of the reference service, the service's manifests replace `deploy/kind/demo.yaml` in a fork, and its line replaces the reference line of 500 ms (chapter 18).

## Tearing down

`az group delete -n $RG --yes --no-wait` deletes the cluster, its machines and its registry. Every paired run of the workflow deletes its own resource group at the end of every repetition, whatever happened, so nothing keeps billing; a run cancelled by hand runs the same deletion as its last step.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## What goes wrong on Azure, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `QuotaExceeded` or `OperationNotAllowed` on `az aks create` or `nodepool add` | the family's vCPU allowance in the region | request the family's increase; or add a second work pool in another family |
| "creating a new cluster is unavailable at this time in region" | Azure's free-tier capacity in the region | `--tier standard`, or another region |
| A size "is not available for this subscription" | the family is not offered there | `az vm list-skus` and choose an offered size (the receipts used Standard_D2s_v4 and Standard_D2_v4 in eastus) |
| Pods `Pending`, no machine added | the pool is at its `--max-count`, or the pods do not fit any machine size in the pool | raise the top, or add a pool of a larger size |
| A parked machine keeps billing for minutes | `scale-down-unneeded-time` (10 minutes by default) | expected; it is native's own setting, kept the same in both arms |
| The governor's image will not pull | the registry is not attached | `az aks update --attach-acr` |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 16

# Google Kubernetes Engine, Step by Step

This chapter takes an implementer from a Google Cloud project to Omni-Compass watching a Google Kubernetes Engine (GKE) cluster, ready for the governing switch of chapter 19. Every command runs in Cloud Shell or on a workstation with the Google Cloud CLI (`gcloud`), `kubectl` and Docker. The layout and the order are those of chapter 15; only the provider's commands differ.

## The project and its allowances

1. **Sign in and choose the project.** `gcloud auth login`, then `gcloud config set project <project-id>`.
2. **Enable the two services once:** `gcloud services enable container.googleapis.com compute.googleapis.com artifactregistry.googleapis.com`.
3. **Read the allowances.** Compute Engine counts CPUs per region (the `CPUS` quota) and, for some families, per family (for example `N2_CPUS`); a new project's regional CPU quota is often 24 or less. Read them with `gcloud compute regions describe <region> --format="table(quotas.metric,quotas.limit,quotas.usage)"` and raise them in the console (**IAM and Admin**, **Quotas**) before the cluster is built.
4. **Pick the zone and machine type.** The receipts' reference is `e2-standard-2` (two vCPUs) in `us-central1-a`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The cluster: a system pool, a work pool, the autoscaler

```
NAME=&lt;cluster&gt;; ZONE=&lt;zone&gt;; MACHINE=e2-standard-2
gcloud container clusters create $NAME --zone $ZONE --num-nodes 1 --ma
    ↪ chine-type $MACHINE \
  --node-taints CriticalAddonsOnly=true:NoSchedule --node-labels omni-
    ↪ role=system \
  --autoscaling-profile balanced --release-channel regular
gcloud container node-pools create work0 --cluster $NAME --zone $ZONE
    ↪ --machine-type $MACHINE \
  --num-nodes 2 --enable-autoscaling --min-nodes 2 --max-nodes &lt;pool t
    ↪ op&gt; --node-labels omni-role=work
gcloud container clusters get-credentials $NAME --zone $ZONE
kubectl get nodes -L omni-role
```

The default node pool created with the cluster becomes the system pool: one machine, tainted so that no workload lands on it, labelled `omni-role=system`. The work pool carries the label `omni-role=work`, which is how the governor knows which machines it may park.

**The autoscaling profile.** GKE offers two: `balanced` (Google's default) and `optimize-utilization`, which removes underused machines sooner and so lets a parked, emptied machine leave the bill sooner. The receipts used `optimize-utilization` in both arms. As on Azure, the profile is native's own setting: choose it for the service, write it down, keep it the same in both arms of any paired run. The governor never changes it.

**Prove the autoscaler.** As in section 15.2: push a test Deployment past the pool and back, and watch a machine come and go. GKE's autoscaler may take several minutes to remove a machine even under `optimize-utilization`; that time is the time a parked machine keeps billing after its pods have gone, and it is the same in both arms.

## 16.3 metrics-server and the service's autoscaler

GKE runs metrics-server; check with `kubectl top nodes`. The service needs a CPU HorizontalPodAutoscaler and container requests, exactly as in section 15.3. GKE Autopilot clusters are a different product: their nodes are managed by Google and not visible as a pool the governor can park, so the governor's machine lever has nothing to move there. Use a Standard cluster.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The registry and the governor's image

```
REPO=&lt;region&gt;-docker.pkg.dev/&lt;project-id&gt;/omni-compass
gcloud artifacts repositories create omni-compass --repository-format=
    ↪ docker --location=&lt;region&gt;
gcloud auth configure-docker &lt;region&gt;-docker.pkg.dev
docker build -t $REPO/omni-compass:&lt;version&gt; -f deploy/Dockerfile . &amp;&a
    ↪  docker push $REPO/omni-compass:&lt;version&gt;
docker build -t $REPO/omni-compass-cloud:&lt;version&gt; -f deploy/Dockerfil
    ↪ e.cloud \
  --build-arg BASE=$REPO/omni-compass:&lt;version&gt; . &amp;&amp; docker push $R
    ↪ /omni-compass-cloud:&lt;version&gt;
```

The cluster's nodes pull from an Artifact Registry repository in the same project with their default service account; a repository in another project needs `roles/artifactregistry.reader` granted to the nodes' service account there.

## Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ REPO/omni-compass:&lt;version&gt;
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

The governor watches and writes nothing (section 15.5). For an audit that outlives the pod, the `standard-rwo` storage class provides a persistent volume (section 20.4).

## The paired run on Google Cloud

The workflow `gke-metered` is the same design as Azure's, on clusters it builds and deletes itself, with Google's billed machines counted every 15 seconds at the list price of the machine type; Appendix G connects it to a project, click by click. It runs at the tune-down with the governor on the machines alone, as every cloud set does (chapter 19), and its run title carries the load and the dial.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Tearing down

`gcloud container clusters delete $NAME --zone $ZONE` deletes the cluster and its machines; `gcloud artifacts repositories delete omni-compass --location=<region>` deletes the images. Every paired run deletes every cluster it made, whatever happened, as its last step.

## What goes wrong on Google Cloud, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `Quota 'CPUS' exceeded` | the region's CPU quota | request an increase; or a smaller machine type |
| `ZONE_RESOURCE_POOL_EXHAUSTED` | the zone has no capacity of that type now | another zone in the region, or another machine type |
| The work pool never shrinks | the autoscaler sees a pod it may not move (local storage, a pod without a controller, a PodDisruptionBudget at zero) | find the pod in `kubectl describe` of the autoscaler's status ConfigMap; it holds native too |
| The governor's image will not pull | the nodes' service account cannot read the repository | grant `roles/artifactregistry.reader` |
| No pool for the governor to park on an Autopilot cluster | Autopilot hides the nodes | use a Standard cluster |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 17

# Amazon Elastic Kubernetes Service, Step by Step

This chapter takes an implementer from an Amazon Web Services account to Omni-Compass watching an Amazon Elastic Kubernetes Service (EKS) cluster, ready for the governing switch of chapter 19. Every command runs in AWS CloudShell or on a workstation with the AWS CLI (`aws`), `eksctl`, `helm`, `kubectl` and Docker. EKS differs from the other two in one respect that matters: it ships neither a cluster autoscaler nor metrics-server, and the implementer installs both.

## The account and its allowances

1. **Sign in.** `aws configure` with an identity allowed to create EKS clusters, EC2 instances, IAM roles and CloudFormation stacks (eksctl builds the cluster through CloudFormation).
2. **Read the allowances.** EC2 counts vCPUs of on-demand standard instances per region (the quota "Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances"). Read it with `aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A --region <region>` and raise it in the Service Quotas console before building.
3. **Pick the region and instance type.** The receipts' reference is `m5.large` (two vCPUs) in `us-east-1`.

## The cluster: a system node group, a work node group, the autoscaler

eksctl reads the whole cluster from one file. The system group carries the add-ons and the governor; the work group is tagged for the cluster autoscaler's auto-discovery and labelled `omni-role=work`:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata: {name: &lt;cluster&gt;, region: &lt;region&gt;}
iam:
  withOIDC: true
  serviceAccounts:
    - metadata: {name: cluster-autoscaler, namespace: kube-system}
      wellKnownPolicies: {autoScaler: true}
managedNodeGroups:
  - name: system
    instanceType: m5.large
    minSize: 1
    maxSize: 1
    labels: {omni-role: system}
    taints: [{key: CriticalAddonsOnly, value: "true", effect: NoSchedu
    ↪ le}]
  - name: work0
    instanceType: m5.large
    minSize: 2
    maxSize: &lt;pool top&gt;
    labels: {omni-role: work}
    tags: {"k8s.io/cluster-autoscaler/enabled": "true", "k8s.io/cluste
    ↪ r-autoscaler/&lt;cluster&gt;": "owned"}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
eksctl create cluster -f cluster.yaml
helm repo add autoscaler https://kubernetes.github.io/autoscaler &amp;&amp; he
    ↪ lm repo update
helm upgrade --install cluster-autoscaler autoscaler/cluster-autoscale
    ↪ r -n kube-system \
  --set image.tag=v1.&lt;minor&gt;.0 --set autoDiscovery.clusterName=&lt;cluste
    ↪ r&gt; --set awsRegion=&lt;region&gt; \
  --set rbac.serviceAccount.create=false --set rbac.serviceAccount.nam
    ↪ e=cluster-autoscaler \
  --set extraArgs.scale-down-unneeded-time=10m --set extraArgs.scale-d
    ↪ own-delay-after-add=10m \
  --set "tolerations[0].key=CriticalAddonsOnly" --set "tolerations[0].
    ↪ operator=Exists" \
  --set "nodeSelector.omni-role=system" --wait
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/rel
    ↪ eases/download/v0.7.2/components.yaml
```

- **The autoscaler's version** must match the cluster's Kubernetes minor version (`kubectl version` prints it); the autoscaler publishes one release a minor.
- **The autoscaler's identity** is the one place a cloud credential enters the cluster, and it belongs to the autoscaler, never to the governor: eksctl creates an IAM role for the `cluster-autoscaler` service account with the autoscaling policy and nothing else.
- **The scale-down timing** (`scale-down-unneeded-time`, 10 minutes by default) is native's own, kept the same in both arms of any paired run, never changed by the governor; the receipts used 2 minutes in both arms.
- **metrics-server** goes on the system machine: patch its Deployment with the same toleration and node selector, or it will take a work machine's room.

**Karpenter.** Where the cluster runs Karpenter instead of the cluster autoscaler, the governor's machine lever works the same way: it parks a machine with a `PreferNoSchedule` taint and marks its pods first to go, and Karpenter's consolidation removes the machine once it is empty. Karpenter's NodePool must carry the label `omni-role=work` on the machines it makes, and its disruption budget and consolidation policy are native's own settings, kept the same in both arms.

## The service's autoscaler

As in section 15.3: a CPU HorizontalPodAutoscaler and container requests, the target and range written down by the service owner. Check that metrics-server answers with `kubectl top nodes` before the governor is installed; without it the governor reads the service's processor as blind and holds (section 5.3).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The registry and the governor's image

```
ACCT=$(aws sts get-caller-identity --query Account --output text); REG
    ↪ =$ACCT.dkr.ecr.&lt;region&gt;.amazonaws.com
aws ecr create-repository --repository-name omni-compass --region &lt;reg
    ↪ ion&gt;
aws ecr create-repository --repository-name omni-compass-cloud --regio
    ↪ n &lt;region&gt;
aws ecr get-login-password --region &lt;region&gt; | docker login --username
    ↪  AWS --password-stdin $REG
docker build -t $REG/omni-compass:&lt;version&gt; -f deploy/Dockerfile . &amp;&amp;
    ↪ docker push $REG/omni-compass:&lt;version&gt;
docker build -t $REG/omni-compass-cloud:&lt;version&gt; -f deploy/Dockerfile
    ↪ .cloud \
  --build-arg BASE=$REG/omni-compass:&lt;version&gt; . &amp;&amp; docker push $REG/
    ↪ mni-compass-cloud:&lt;version&gt;
```

eksctl's managed node groups carry `AmazonEC2ContainerRegistryReadOnly` on their instance role, so the nodes pull from ECR in the same account without further grants.

## Watch first

```
kubectl apply -f deploy/install/omni-compass.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ REG/omni-compass:&lt;version&gt;
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 5 /var/log
    ↪ /omni/audit.jsonl
```

The governor watches and writes nothing (section 15.5). For an audit that outlives the pod, the EBS CSI driver's `gp3` storage class provides a persistent volume once the driver add-on is installed (section 20.4).

## The paired run on Amazon

The workflow `eks-metered` is the same design as Azure's and Google's, on clusters it builds with eksctl and deletes itself, Amazon's billed instances counted every 15 seconds at list price; Appendix H connects it to an account, click by click. It runs at the tune-down with the governor on the machines alone (chapter 19).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Tearing down

`eksctl delete cluster --name <cluster> --region <region> --wait` deletes the cluster, its node groups and the CloudFormation stacks; `aws ecr delete-repository --repository-name omni-compass --force` deletes the images. Every paired run deletes every cluster it made as its last step, whatever happened.

## What goes wrong on Amazon, and what to do

| Symptom | Cause | Fix |
|---|---|---|
| `VcpuLimitExceeded` | the region's on-demand vCPU quota | request an increase |
| eksctl stops with a CloudFormation `ROLLBACK_COMPLETE` | a quota, a subnet or an IAM permission | read the stack's events in the CloudFormation console; delete the stack before retrying |
| The work group never grows or shrinks | the autoscaler's version does not match the cluster, or the group is not tagged for discovery | match the image tag to the cluster's minor; check the two tags |
| `kubectl top` fails | metrics-server is not installed or cannot reach the kubelets | install the pinned release; check its logs |
| A parked machine is never removed | Karpenter's or the autoscaler's disruption rules hold it (a PodDisruptionBudget, a do-not-disrupt annotation) | the same rule holds native; it is the operator's |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 18

# The Response-Time Feed and the Line

The governor holds a service at its line by reading the service's response time. The **feed** is how that reading reaches it; the **line** is the response time the service must meet, written down by the service's owner. Both are the implementer's to provide, on any cloud, and both are decided before the governor watches, because a governor watching a wrong feed against a wrong line produces an audit that teaches nothing.

## The line

The line is a response-time objective in milliseconds, passed as `--slo-ms`. It is the service owner's, never the implementer's and never the governor's, and it is written down with its basis before the first counted run:

- **The percentile it binds.** The governor reads the line against the 95th percentile of recent responses (a breach is the 95th percentile at or over the line) and holds the mean of recent responses inside its band below the line. An objective written on another percentile is translated by the owner, not by the implementer.
- **What counts as a response.** In the receipt a failed request (a timeout, a connection refused, a 5xx where the owner says so) counts over the line, whatever its timing. The governor builds its own reading from the served responses of its window; a window in which no request was served reads as past the line, and every machine is held.
- **Where it is measured.** At the edge the customer sees (the load balancer or ingress), or behind it; the line and the feed must be measured at the same place.

A line set far above the service's real response times leaves the governor nothing to hold and reads as "no difference beyond the noise" on every row; a line set below what the service can meet on native reads as a breach in every decision and the governor holds every machine. Both are wiring faults, and Appendix D lists their symptoms.

## The feed's format

The feed is a text file of comma-separated values that the governor reads with `--latency-file`. Its first line is the header and every later line is one response:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
elapsed_seconds,latency_ms,ok
0.0,212.4,1
0.3,198.0,1
0.6,5003.1,0
```

- `elapsed_seconds`: seconds since the feed began, increasing; the governor reads only rows newer than the last it read.
- `latency_ms`: the response's time, in milliseconds.
- `ok`: 1 for a response the owner counts as served, 0 for a failure.

The governor reads the window of recent rows (the last 60 seconds for its decision, the last 15 to 30 seconds for its fast reflexes) by the feed's own clock. **A feed whose file has not changed for twice the window, or whose window holds no served response, reads as blind**, and a blind sense makes the governor hold every machine in service and give nothing back until the feed returns (section 5.3). That is the safe direction, and it is why a broken feed costs savings, never service.

## Feed one: the probe

The repository's probe (`scripts/latency_probe.py`) times real HTTP requests to one URL and writes exactly this file:

```
INTERVAL=5 N=5 DURATION=31536000 python3 scripts/latency_probe.py http
    ↪ ://&lt;service&gt;.&lt;namespace&gt;.svc.cluster.local/ /feed/latency.csv
```

It sends `N` requests back to back, pauses `INTERVAL` seconds, and repeats for `DURATION` seconds; each request has a 10-second timeout, and a timeout or an error is written as `ok=0`. In the cloud mode it runs as a second container in the governor's own pod (section 19.4), writing to a volume both containers share. A probe measures a synthetic request; it is the right feed where the service has one representative endpoint, and the wrong one where the customer's requests differ widely in cost.

**A probe restart.** The probe's clock starts again at zero when its container restarts. The governor's reading follows the newest rows by their own clock and carries on, but a give-back trial counts only rows newer than the last it counted, so no trial completes, and no machine is given back, until the new clock passes the old one. That is the safe direction. Restart the governor's pod so both clocks start together (the governor hands every setting back on its way out; section 20.3), and alert on a restarted probe container (section 20.5).

## Feed two: the service's own timings

Where the service already measures its responses (an ingress controller, a service mesh, the application's own histogram), an adapter can write those timings in the same three columns. The adapter is a small program the implementer owns: it appends one row per response, or one row per short interval carrying that interval's 95th percentile with `ok=0` rows for the interval's failures. Two rules keep it honest:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

1. **The same place as the line.** Timings from the ingress for a line written at the ingress; timings from the application for a line written at the application.
2. **No smoothing the governor does not know about.** A row is a response or an interval's own statistic, never a moving average of earlier rows; the governor builds its own windows, and a pre-smoothed feed makes it late.

## Checking the feed before the governor watches

| Check | How | Pass |
|---|---|---|
| The file advances | `tail -f /feed/latency.csv` for a minute | a new row at least every 5 seconds |
| The clock is monotonic | the first column never decreases | yes, across a full day |
| The failures are counted | stop the service's endpoint for 30 seconds in a test window | rows with `ok=0` appear, and the governor's audit reads the latency sense as blind or breached: either way past the line, every machine held |
| The line is plausible | the feed's 95th percentile on a normal day against `--slo-ms` | the 95th percentile sits below the line on a normal day, and reaches it only under real pressure |
| Blind is safe | stop the probe for two minutes in a test window | the audit shows `blind` and no machine given back until the feed returns |

## One feed for one governor

A governor reads one feed (`--latency-file`), and a cluster has one governor on its machines (the one-writer rule, section 5.2). Where several services share the governed machines, the feed must speak for all of them: either the feed of the service most sensitive to a lost machine, or an adapter (section 18.4) that writes, for each short interval, the worst of the services' own 95th percentiles against their own lines, scaled to one line. Outside the cloud mode `--sensed` names the services whose autoscalers the feed speaks for, and only those are moved; in the cloud mode no autoscaler is moved at all, and the feed is the guard on the machines the governor parks: a machine is parked only while the line holds, and a breach wakes it at the next decision.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 19

# Running in the Cloud: the Machines Alone, the Bill and the Alarm Minutes

## Why the governor moves the machines alone in a cloud

In a public cloud every machine that exists is billed, whether it carries work or not, so the benefit a cloud can show is the bill: the same work as native for a smaller bill, never a bigger one, and never more alarms. The governor has three levers a cloud would let it pull besides the machines: the pod autoscaler's target, the processor of running pods and their power. In a cloud each of them can only raise the bill: a lower pod target asks for more pods, and more pods ask the cloud for machines; more processor for a pod makes it read busier to its autoscaler, which then asks for more pods; and a power cap saves nothing a cloud bills. So in a cloud the governor moves the machines alone:

- **No pod autoscaler is moved.** Every HPA keeps the operator's own target, range and cap, so the pods are exactly the pods native would run.
- **No processor is conveyed and no power is capped.**
- **The machines.** The governor parks a machine the cloud's autoscaler already made, only after its own trial shows the service's response time holds without it (within 2% of the response time with it, through the release gate, one machine at a time), and wakes it on a breach: at the next full decision (within 60 seconds) for a response time over the line, and at the next fast check (within 5 seconds) for a pod that cannot be placed. A pod never waits for either: a parked machine's taint is `PreferNoSchedule`, so a pod that finds the open machines full lands on a parked one at once. A parked machine whose pods have gone is deleted by the cloud's own autoscaler, on the cloud's own schedule.

The governor never asks the cloud for a machine (its machine lever cannot ask for more machines than the cluster has), and it can hold no more pods than native's autoscaler gives it. **The bill can only fall or stay.** It falls where native holds machines the service does not need: the quiet stretches between bursts, the hours of a day's trough, the weekend. It stays where native is already short of machines; there the governor has nothing to give back, and the receipt reads EVEN on the bill.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The dial in the cloud

The cloud runs at the tune-down (`--dial save`): native's work on the least resource. The max (`--dial max`) is for machines already paid for (a company's own racks, a card whose heat line is its limit), where more work from the same machines is the gain; it is never used in a cloud, and a run of the max in a cloud is never read as a bill result.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The settings the cloud mode writes

| Setting | Value in the cloud mode | Why |
|---|---|---|
| `--mode` | `nodepool` | the machine lever |
| `--law` | `compass` | the compass holds the response time in its band and gives machines back through the trial |
| `--dial` | `save` | the tune-down |
| `--sensed` | `none/none` | names no service, so no HPA is moved; every HPA keeps the operator's target |
| `--cap-deployments`, `--power-cmd` | absent | no processor conveyed, no power capped |
| `--latency-file`, `--slo-ms` | the feed and the line (chapter 18) | the guard on every machine parked |
| `--node-scale-cmd` | `bash /app/scripts/kind_nodepool.sh {n}` | parks and wakes machines that exist; never asks for one |
| `--node-restore-cmd` | `bash /app/scripts/kind_nodepool.sh 100000` | on a reset, wakes every parked machine (the script caps the number at the machines that exist) |
| `--active-nodes-only` | on | counts only machines in service |
| `--min-nodes` and `MIN_NODES` | 2, or the operator's floor | never fewer machines in service |
| `--max-node-step` | 1 | one machine a decision |
| `--max-nodes` | the work pools' top (the sum of their autoscaler maxima) | the most machines the lever counts on; it never asks for more than exist |
| `--interval`, `--floor-interval` | 60 and 5 | the full decision every minute, the fast checks every 5 seconds |
| `--closure` | `/app/law/closure.json` | shipped in the image |
| `--audit`, `--kill-file` | `/var/log/omni/audit.jsonl`, `/tmp/omni.kill` | the record and the switch |
| `WORKER_SEL` (environment) | `omni-role=work` | which machines may be parked |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The cloud image and the governing Deployment

The cloud image adds five things to the shipped one: `jq` (the machine lever reads the cluster's JSON with it), the machine lever's script, the probe, the Prometheus feed adapter (section 22.7), and the switch tool that the in-pod watchdog runs (section 20.2). It is `deploy/Dockerfile.cloud`:

```
ARG BASE=omni-compass:local
FROM ${BASE}
USER root
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends jq &amp;&a
    ↪  rm -rf /var/lib/apt/lists/*
COPY scripts/kind_nodepool.sh /app/scripts/kind_nodepool.sh
COPY scripts/latency_probe.py /app/scripts/latency_probe.py
COPY tools/omni_switch.py /app/tools/omni_switch.py
COPY tools/prom_feed.py /app/tools/prom_feed.py
USER 10001
```

Build the shipped image first and pass it as the base (`--build-arg BASE=<registry>/omni-compass:<version>`). The governing Deployment is `deploy/install/omni-compass-cloud.yaml` (printed in Appendix I). Apply the machines role of section 14.3 (`deploy/install/rbac-machines.yaml`) first, set the image, the service's URL and the line, and replace the watch Deployment:

```
kubectl apply -f deploy/install/rbac-machines.yaml
kubectl apply -f deploy/install/omni-compass-cloud.yaml
kubectl -n omni-compass set image deployment/omni-compass controller=&lt;
    ↪ registry&gt;/omni-compass-cloud:&lt;version&gt; \
  probe=&lt;registry&gt;/omni-compass-cloud:&lt;version&gt;
kubectl -n omni-compass set env deployment/omni-compass -c probe TARGE
    ↪ T_URL=http://&lt;service&gt;.&lt;namespace&gt;.svc.cluster.local/
kubectl -n omni-compass rollout status deployment/omni-compass
```

**Dry run first.** The manifest ships with `--dry-run` on the governor's command line: it decides as it will, writes nothing, and records every command it would have run as a `write` record marked `"dry_run": true`. Read a day of its audit (section 22.6 traces one decision) before removing `--dry-run`. A dry run checks the commands, not their effect: because no machine is parked, a trial during a dry run compares the service with itself, and its `step allowed` is no evidence that the service holds without the machine. That evidence is the paired run's (phase 5). The first write after `--dry-run` is removed is the first machine the governor parks, in its audit with the trial that allowed it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## What to expect, hour by hour

| Hour of a typical day | Native | With Omni-Compass on top, machines alone |
|---|---|---|
| The peak | every machine the pods need | the same machines; nothing parked while the line is at risk |
| The descent after a peak | machines held until the autoscaler's own scale-down finds them unneeded | machines parked one at a time as each trial passes; their pods leave first on the autoscaler's next scale-down |
| The trough (night, weekend) | the pool held at what the pods' requests need | parked down to the floor where the trial allows; each empty parked machine deleted by the cloud |
| A new burst | the autoscaler adds machines when pods cannot be placed | pods that find the open machines full land on a parked machine at once (its taint only prefers against it); the governor wakes it within 5 seconds if pods wait, or at the next decision on a breach; the cloud adds machines past those, as for native |

The savings come from two places: the machines the governor parks before native's autoscaler would find them unneeded, and the empty machines the cloud deletes sooner because their pods were marked first to go. Neither changes how many pods run, and neither delays a pod: a parked machine carries a `PreferNoSchedule` taint, so a pod that finds the open machines full lands on a parked one at once.

## Reading the bill and the alarm minutes

Every paired receipt on a cloud carries these rows, and they are read by the rules of chapter 11:

| Row | Its aim | How to read it |
|---|---|---|
| compute bill at list price ($) | lower | a cut is GAIN; EVEN where native was already short of machines; a rise is power given, and in the cloud mode it is a fault to find, because the mode cannot hold more machines than native |
| machines billed, machine-hours | lower | as the bill |
| response time, 95th and 99th percentile | lower | must read EVEN or GAIN; a LOSS is a wiring fault to find first (section 4.5) |
| failed requests | lower | must read EVEN or GAIN |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Row | Its aim | How to read it |
|---|---|---|
| alert minutes | lower | the whole minutes whose 95th percentile, failed requests counted over, was over the line: the minutes an on-call page would be open. The aim in a cloud is none at all; an alert minute on the omni side is found and fixed |
| settings commands the governor wrote on its own | shown | the machines it parked and woke, each in its audit with its reason; no person touched either arm |
| HPA replicas | shown | must match native's within the noise in the cloud mode; a difference is a wiring fault (an HPA moved) |

**A run of one day of real traffic says more than any burst.** The bill's saving lives in the quiet hours; a paired run of a steady load at the service's peak can read EVEN on the bill, honestly, because at the peak there is nothing to give back. The day-shaped load of the repository's own cloud sets (the Google 2011 trace on one to nine load generators) is the test that shows the trough.

## What the cloud mode never does

It never raises a pod autoscaler's cap, target or range; never resizes a pod; never caps power; never asks the cloud for a machine; never deletes a machine (the cloud's autoscaler does, on its own schedule, once a machine is empty); never evicts or deletes a pod; and never holds a cloud credential. Each of those is checked from outside the governor at the end of every paired arm: every HPA back at the operator's, every pod's limit at its shipped value, every machine back in service, no `omnicompass.io/` record left on any object.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 20

# Day Two: Running Omni-Compass in Production

An installation is in production when the governor writes, the service depends on the result, and nobody watches it by hand. This chapter is what keeps that state safe: the watchdog and the lease, the OFF switch drill, the audit as a record, the alarms, upgrades and rollbacks, a fleet of clusters, and the runbooks for the incidents an implementer should expect.

## The governor's lifecycle on a cluster

| Event | What the governor does | What the implementer checks |
|---|---|---|
| Start | records every setting it may move, on the object itself (an `omnicompass.io/` annotation), before its first write; registers its lease | the first audit lines: the snapshot |
| Every decision | renews its lease, reads, decides, writes (or holds), reads back every write | the decision trail (Chapter 8) |
| Reset (planned stop) | hands every setting back in reverse order of power-up, reads each back, writes its last audit line, exits 0 | no `omnicompass.io/` annotation left; every machine in service |
| OFF switch (security) | the same hand-back at once, from a signal; no governor starts while the switch is off | `python3 tools/omni_switch.py status` |
| Death without hand-back (killed, crashed, hung) | nothing; the watchdog acts for it | the watchdog's log: `handed back` with `ok` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The watchdog and the lease, in a cluster

A governor that dies without handing back leaves its settings in place: in the cloud mode, machines parked. The lease covers that case. Before its first write the governor registers the exact command that puts every setting back (`python -m omni_controller.controller --restore-only`, which restores from the annotations on the objects themselves, so it needs no memory of the dead process) and renews a lease every decision. The watchdog (`python3 tools/omni_switch.py watchdog`) finds a governor whose process is gone or whose lease is older than its limit (five decision periods, at least two minutes), stops a hung one, runs its recorded restore and records what it did.

In a cluster, the watchdog runs as a second container in the governor's own pod, with the pod's process namespace shared (`shareProcessNamespace: true`), as the cloud manifest ships it (Appendix I). It must: the watchdog judges a governor dead by its process, and a process is visible only inside its own process namespace, so a watchdog in a separate pod would find every governor dead and hand back over a governor that is running. In the pod, the watchdog sees the governor's process and its lease (on the pod's shared `/tmp`); when the governor's container dies or hangs, the watchdog stops a hung one and runs its recorded restore, which wakes every parked machine, and the kubelet restarts the governor's container, which begins again from the cluster as it finds it. When the whole pod is lost (its machine fails, or it is evicted), the Deployment starts a new pod; until it does, the parked machines stay parked, which never blocks a pod (a `PreferNoSchedule` taint), and the cloud's autoscaler adds machines for any pod that cannot be placed. No failure of the governor or the watchdog can leave a pod unscheduled.

## The OFF switch drill

In a cluster the OFF switch has three forms, and the person who holds it (section 13.4) knows all three:

| Form | Command | What happens | How long it holds |
|---|---|---|---|
| Scale to zero (the OFF switch of record) | `kubectl -n omni-compass scale deployment/omni-compass --replicas=0` | the pod receives `SIGTERM`; the governor hands every setting back at once (every parked machine woken, its marks on pods removed), reads each back, writes its last audit line and exits | until the holder scales it back to 1 |
| **Form** | **Command** | **What happens** | **How long it holds** |
| The kill file | `kubectl -n omni-compass exec deploy/omni-compass -c controller --` | at its next full decision (within 60 seconds) the governor hands every setting back, then writes | until the file is removed; the file lives in the pod's own `/tmp`, so a replaced pod starts ON |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Form | Command | What happens | How long it holds |
|---|---|---|---|
| | `touch /tmp/omni.kill` | nothing; each decision's audit reads `killed` | |
| Revoke the grant | `kubectl delete clusterrolebinding omni-compass-machines` | every write the governor attempts is refused by the API server and recorded | until the binding is applied again |

Revoke the grant only after one of the first two forms has handed everything back: a governor without its grant cannot wake a machine it parked. (A parked machine never blocks a pod, so even then nothing waits; but its taint stays until the grant returns or a person removes it.)

The OFF switch is drilled before production and every quarter after, by its holder, on the production cluster, in a quiet hour:

1. Note the time and the machines parked: `kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints[*].key`.
2. Scale to zero.
3. Time it: no `omnicompass.io/idle` taint left on any machine, no `controller.kubernetes.io/pod-deletion-cost` mark left on any pod of the governed pools (unless another controller of the company's own sets that mark), the governor's pod gone. The limit is 60 seconds.
4. Scale back to 1 (`--replicas=1`) and record the drill: the date, the holder, the seconds, anything left behind.

A drill that leaves anything behind, or takes more than 60 seconds, stops the installation until the cause is found.

## The audit as a record

The audit (`audit.jsonl`, one JSON record a line, each timestamped) is the governor's complete account of itself: every reading, decision, write, read-back and reason. In production it is kept like any other operational record:

- **Storage.** Mount a persistent volume at `/var/log/omni` (Azure `managed-csi`, Google `standard-rwo`, Amazon `gp3` through the EBS CSI driver). An `emptyDir` loses the audit with the pod.
- **Shipping.** A log collector on the node reads the file and ships it to the company's log store; every line is already JSON. Keep the audit at least as long as the change record the company keeps for any production change.
- **Size.** One decision a minute writes a few records; a month is tens of megabytes. Rotate by size or day.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- **What to search for.** `"write"` (every change), `"refused"` (a write the cluster refused), `"error"` (a failed read or decision), `"proprioception"` in the `authority` record written with each decision (the drift between what the governor last commanded and what the cluster shows: a drift that does not close means a command did not land or another writer moved the setting), `"verdict"` (each give-back trial and its outcome), `"blind"` (a sense lost).

## Alarms an implementer wires

| Alarm | Source | Why |
|---|---|---|
| Alert minutes on the service | the company's own monitoring of the line | the line is the service's; an alert minute is a person woken, whether or not Omni-Compass is installed |
| The governor's pod not running, or restarting | the cluster's own pod status | the watchdog hands back; someone must know why |
| The probe container restarted | the cluster's own pod status | no machine is given back until the governor's pod restarts (section 18.3) |
| `"error"` records in the audit, more than a few in an hour | the log store | the API is unreachable or a grant is missing; the governor holds |
| A `proprioception` drift that does not close over several decisions | the log store | a command did not land, or another controller or a person moves the same setting; find it (the one-writer rule) |
| A machine parked longer than a day with pods on it | `kubectl get nodes` and the taints | expected only in a long quiet stretch; worth a look otherwise |

## Upgrades and rollbacks

An upgrade is a new image of the governor at a new engine version (chapter 12):

1. Verify the new copy (chapter 32) and read what changed in its version notes.
2. Run the new version in watch mode beside nothing: replace the Deployment with the new image and `--mode observe` for at least a day; read its decisions against the old version's.
3. Run a paired check on the new version where the receipts matter (chapter 10): receipts are never read across versions.
4. Switch it to govern. The old image stays in the registry, tagged, for the rollback.

A rollback is the same in reverse, and safe at any moment: stop the governor (its reset hands every setting back, or the watchdog does), set the old image, start it. The governor carries no state between processes except the annotations it reads back at start, so an old version starting after a new one begins from the cluster as it finds it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## A fleet of clusters

One governor per cluster, never one governor for several clusters: each cluster has its own autoscaler, its own machines and its own line. A fleet is rolled out cluster by cluster, each through the phases of section 13.3, with a paired receipt on at least one cluster of each kind (each provider, each machine family, each service class) before the rest of that kind go to govern. The governor's settings are the same text in every cluster of a kind; keep them in the company's configuration repository and deploy them like any other workload, and keep one audit store for the fleet.

## Runbooks

**A breach while machines are parked.** Expected and handled: a breach wakes a parked machine at the next full decision (within 60 seconds), and a pod that cannot be placed wakes one at the next fast check (within 5 seconds). If alert minutes persist with machines parked, turn the governor OFF (section 20.3), which wakes every machine, and read the audit for the decision that parked the last one: its trial, its readings and the feed's age.

**The governor parks nothing for days.** In the cloud mode at a steady peak that is the right answer: there is nothing to give back. Otherwise read `"verdict"` records: trials abandoned for too little work mean the feed is too thin (section 18.3); trials refused mean the service is slower without the machine, and the governor is right to keep it.

**The cloud's bill did not fall.** Read the receipt's machine-hours beside the autoscaler's own profile: a long `scale-down-unneeded-time` keeps a parked, empty machine billing; a PodDisruptionBudget or a pod with local storage can keep the autoscaler from removing one. Both hold native too, and both are the operator's settings.

**Someone else moves a setting.** The `proprioception` drift in each decision's `authority` record does not close: the machines in service are not the machines the governor commanded. Find the other writer (a GitOps controller resetting node taints, a second autoscaler or descheduler, a person running `kubectl uncordon`); there must be one writer for each setting, and until there is, turn the governor OFF.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 21

# The Graphics Processor: Putting the Governor on a Card

A graphics processor (a GPU card) is the other end of the cloud from the bill: the card is paid for by the hour or owned outright, and the benefit there is **more work inside the card at native's own heat, never hotter**. This chapter is written for the implementer who will put the governor on one card or on every card of a server, rented or owned. It is complete and ready to run; it is not run until the processor side is finished, and the first run is given one exact commit.

## What the governor moves on a card

Two wires, both the card's own settings, both written through the vendor's own management interface (`nvidia-smi`, the NVIDIA System Management Interface) and read back after every write:

| Wire | What it is | Its range |
|---|---|---|
| The clock ceiling (up wire) | the highest graphics clock the card may run (locked clocks) | the card's own supported clocks, read from the card |
| The power limit (down wire) | the card's own power cap, in watts | the card's own minimum and default limits, read from the card |

It moves nothing else: not the application, not the model, not the batch, not the card's fan curve, not its memory clock, not its persistence mode, not another process's settings. Native is the card as it ships: the vendor's own firmware and driver deciding its clock under the default power limit.

## What the card needs

- **Bare metal or full passthrough.** A virtual machine that hides clock locking or the power limit from the guest cannot be governed; the wire check (section 21.4) fails on it and says which wire.
- **The vendor's driver**, recent enough to support locked clocks and power limits on the card; `nvidia-smi -q -d CLOCK,POWER` lists the supported clocks and the power-limit range.
- **Root**, for the governor's writes; the governor is the only process allowed to set the card's clocks and limit while it runs, and the machine check refuses to start if another management agent holds them.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

- **A response-time feed** for the work on the card (chapter 18): the serving engine's own per-request timings, or the probe against the serving endpoint.
- **The heat sensor** the card reports (`nvidia-smi --query-gpu=temperature.gpu,clocks_throttle_reasons.active`), read by the governor every decision; a thermal slowdown reported by the card holds every tightening (Chapter 8, `thermal_hold`).

## Rented or owned

| | A rented card (for example a Lambda Cloud instance) | An owned card |
|---|---|---|
| Who pays for idle | the renter, by the hour | the owner, in power and heat |
| The gain | more work in the hours rented: fewer card-hours for the same work | more work from the same card at native's heat, and the same work at less power where the card is not the bottleneck |
| The meter | the card's own power reading (`power.draw`), class P | the card's own reading and, where fitted, a wall meter on the server (`tools/wall_meter.py`), class P |
| Access | root on the instance; full passthrough is standard on bare-metal rentals | root on the host |

## The order on a card, step by step

1. **Verify the copy** (chapter 32) and record the commit; a card run is given one exact commit and run on it alone.
2. **The machine check.** One copy of the harness only, nothing else on the card, the card's default limit and full clock range restored. The harness's own scripts do this and refuse to start otherwise.
3. **Declare the envelope** once, before any trial: `python3 tools/declare_envelope.py envelope.json --gpu 0` writes the power range the governor may set, read from the card itself: from the larger of the card's own minimum and 70% of the limit read at declaration, up to that limit. The file is kept with the run.
4. **The wire check:** `sudo python3 tools/gpu_wire_check.py --gpu 0` must end `WIRED RIGHT`. It moves each wire and reads the card back, proving the up wire raises and lowers the clock and the down wire moves the power. Its failures are listed in Appendix D with their fixes.
5. **Watch:** `sudo python3 -m omni_controller.gpu_compass --mode watch --gpus 0 --latency-file <feed> --slo-ms <line>` computes both wires every decision and writes

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

neither.

6. **The paired run** on the card: native against native with Omni-Compass on top, the same work, repeated, the card's own power reading and the response times counted in both arms (the harness's `scripts/gpu_paired.sh` on an owned card, `scripts/gpu_rented_run.sh` for the whole rented sequence in one command, `scripts/gpu_8card.sh` for every card of an eight-card server at once).
7. **Govern:** `--mode cap` in place of `--mode watch`, with the OFF switch drilled (section 21.6).

## The heat line

The aim on a card is more work at native's own heat. The governor reads the card's temperature and its own reported slowdown every decision and never tightens while the card reports a thermal slowdown; the card's own firmware protects the silicon underneath whatever the governor writes. Each receipt on a card reports the card's temperature in both arms beside the work and the power, so an implementer sees directly whether the omni arm ran hotter than native. A receipt in which it did is read as a wiring fault first: the clock ceiling raised where the card's cooling could not carry it.

## OFF on a card

`sudo touch /tmp/omni-gpu-kill` (one card's governor), `SIGTERM` to the governor, or the master switch (`sudo python3 tools/omni_switch.py off`, every governor on the machine) each restore the card at once: clocks reset to the card's own range (`nvidia-smi -rgc`), the power limit back to the value read at start, both read back. A restore that does not read back exits with code 3 and must be completed by hand before anything runs again (Appendix D). The fault drill of the rented sequence kills the governor outright mid-run and times the watchdog's hand-back; an implementer repeats it on every new card model.

## Cards inside Kubernetes

Where the cards are nodes of a Kubernetes cluster (the vendor's device plugin exposing `nvidia.com/gpu`), the card governor runs as a privileged agent on each GPU node (it writes the card's own settings, which needs the host's management interface), one per node, with its feed taken from the serving pods on that node. It never schedules or moves a pod; the cluster's own scheduler and autoscaler place the work, and the cloud mode of chapter 19 governs the machines, if the cards are in a cloud. The two governors are different writers of different settings and never touch the same one.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 22

# Observability: Feeding the Governor from Your Monitoring, and Watching It

A company that runs a cloud cluster already measures it: an ingress or a service mesh times every request, a metrics system (Prometheus, Azure Monitor, Google Cloud Monitoring, Amazon CloudWatch, Datadog) stores the series, and a log store keeps the clusters' logs. This chapter connects the governor to that apparatus in both directions: the company's own response times as the governor's feed (section 18.4), and the governor's own audit as series and alarms the company's engineers already read.

## The feed from the company's own response times

The governor's feed is a file of rows `elapsed_seconds,latency_ms,ok` (section 18.2). An adapter that writes it from the company's own measurements has four duties: read the measurements at the point where the line is written; write rows whose own distribution is the served requests' distribution; keep the feed as dense as the probe's (the probe writes five rows every five seconds; a feed thinner than twenty rows a minute leaves the governor's give-back trials short of samples, and the audit shows them as `trial abandoned (too little work)`, section 20.8); and never smooth. A metrics system stores counters and histograms, not single requests, so the adapter works in windows. For each window of length Δ ending at time t:

1. Read the count of requests served in the window, n[ok], and the count failed, n[fail], from the request counter's increase over exactly that window (not a rate over a longer one).
2. If n[ok] > 0, read the served requests' quantiles q[0.05], q[0.15], …, q[0.95] (the midpoints of ten equal shares of the requests) from the response-time histogram's bucket increases over the same window, and write one row with `ok` = 1 for each. Each row then stands for a tenth of the window's served requests, so the rows' own mean and percentiles are the window's, to the precision of the histogram's buckets.
3. Write one row with `ok` = 0 for each failure, up to a cap (twenty a window is enough for the governor to see a failing service; the receipt counts failures from its own probe or the service's own logs, not from the adapter).

Δ must be at least twice the metrics system's scrape interval, or the counter's increase over it is undefined (two samples are needed); with a 15-second scrape, a 30-second window read every 15 seconds gives forty rows a minute.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

With Prometheus and the NGINX ingress controller, for example, n[ok] and n[fail] are the increases of `nginx_ingress_controller_requests` (status below 500, and 500 and above) for the service's ingress over the window, and the quantile q is `histogram_quantile(q, sum by (le) (increase(nginx_ingress_controller_request_duration_seconds_bucket[30s])))`. A histogram's buckets bound the precision of every quantile: a service whose line is 500 ms needs buckets around 250, 400, 500 and 750 ms, or the quantiles jump between bucket edges and the governor reads steps that are not there. A service mesh (Istio, Linkerd) exports the same two series under its own names; the application's own histogram is the closest measurement of all when the line is written at the application. Section 22.7 gives an adapter that does all of this.

**What the adapter must not do.** It must not average across windows, carry a value forward when a window is empty (an empty window is no row; a feed with no new row for twice the governor's window reads as blind, which is the correct reading of a service that answered nothing), or rewrite the file from the start (the governor reads rows newer than the last it read, by the feed's own clock; a restarted adapter continues the clock where it stopped, or the governor's pod is restarted with it, section 18.3).

## The governor's own series

The audit is JSON, one record a line (section 20.4). A log store that parses JSON (Loki, Elasticsearch or OpenSearch, Azure Log Analytics, Google Cloud Logging, CloudWatch Logs, Datadog Logs) turns it into series without any code in the governor:

| Series | From the audit | What it shows |
|---|---|---|
| machines in service, machines seen | each `decision`: `nodes_observed`, `nodes_recommended` | the governor's view of the pool, decision by decision |
| machines parked and woken | each `write` whose `why` is `node pool size` (a decision) or `scheduling floor (pending pods)` (a fast check) | every move of the machine lever |
| trials and their outcomes | each `verdict`: `trial`, `step allowed`, `step refused`, `trial abandoned` | the give-back trials; abandoned trials point to a thin feed |
| the response time the governor read | each `decision`: `latency_p95_ms` | the governor's own reading, beside the company's own |
| senses | the `senses` of the `authority` record written with each decision: `blind`, `latency_age_s` | a feed lost, or late |
| proprioception | the `proprioception` of the same `authority` record | a command that did not land, or another writer |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Series | From the audit | What it shows |
|---|---|---|
| decision time | each `decision_ms` | the governor keeping its period |
| own cost | the last `overhead` record | the governor's processor over its life |

## A dashboard an on-call engineer can read in ten seconds

Four panels, on the service's own dashboard, beside the service's own response time:

1. **The line and the response time.** The service's 95th percentile against its line, from the company's own measurement, with the governor's own reading (`latency_p95_ms`) as a second line. The two should agree; where they do not, the feed is measuring something else (section 18.4).
2. **Machines.** Machines in the pool (from the cluster), machines in service and machines parked (from the audit). A parked machine is never a problem in itself; a parked machine during a breach is impossible by design and is a fault to report.
3. **Trials.** Allowed, refused and abandoned trials an hour. Many abandoned trials: the feed is too thin. Many refused: the service is slower without the machine, and the governor is right to keep it.
4. **Health.** Blind decisions, errors, decision time and the governor's own processor.

## Alarms, routed where the company's alarms already go

The alarms of section 20.5 are written once in the company's alerting system (Prometheus Alertmanager, Azure Monitor alerts, Google Cloud alerting policies, CloudWatch alarms, Datadog monitors) against the series above, and routed to the same on-call rotation as the service. The alert that matters most is not the governor's: it is the service's own line. An installation in the cloud mode is judged first by whether that alert fires more often with the governor than without it, and the paired receipt's alert-minutes row (section 19.6) is the same measure taken under control.

## The bill, from the provider's own records

The receipt's bill is the machines the cluster held, counted every 15 seconds, at list price (chapter 11). In service, the company reads the same thing from the provider's own billing records, and should, because list price is not what every company pays:

| Provider | Where the machine cost lives | How to separate the governed pools |
|---|---|---|
| Azure | Cost Management (cost analysis), or the cost export to a storage account | the node resource group of the cluster, and a tag on each work pool (`az aks nodepool update --tags`) |
| Google Cloud | the Cloud Billing export to BigQuery (detailed usage cost) | GKE cost allocation, enabled on the cluster, by namespace and node pool label |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Provider | Where the machine cost lives | How to separate the governed pools |
|---|---|---|
| Amazon | the Cost and Usage Report, or Cost Explorer | a cost allocation tag on each node group's instances, activated in the Billing console |

The comparison that means something is the same service, the same weeks of the year, before and after the governor went into service, in machine-hours of the governed pools, with the load (requests served) beside it; a bill that fell because traffic fell is not the governor's saving, and the paired receipt, run on the same day for both arms, exists to remove exactly that doubt.

## Tracing a single decision end to end

An auditor's or an engineer's question about one moment ("why was this machine parked at 03:12?") is answered from the audit alone, in this order: the `verdict` record that allowed the step (the trial's response times with and without the machine); the `authority` and `decision` records of that minute (the senses, the release gate's reason, the machines seen and recommended); the `write` record (the command, its reason, whether it was a dry run); and the next decision's `authority` record, whose `proprioception` shows whether the command landed. If any of the four is missing, the governor did not park the machine, and the cause is elsewhere (the cluster autoscaler's own scale-down, a person, another controller).

## A complete adapter: Prometheus to the feed

The repository's `tools/prom_feed.py` is the adapter of section 22.1 for any metrics system that answers Prometheus's HTTP query API (Prometheus itself, Thanos, Cortex, Grafana Mimir, and the managed services of the three clouds that expose that API). It reads three queries the implementer writes for the service, every `--interval` seconds over the last `--window`, and appends the rows of section 22.1 to the feed. It continues the feed's clock across its own restarts and never truncates the file; it writes nothing for an empty window or when the metrics system does not answer, so the governor reads a silent adapter as blind. It is in the cloud image (`deploy/Dockerfile.cloud`) and runs in place of the probe container:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
      - name: feed                    # in place of the probe: the se
    ↪ rvice's own response times, from Prometheus
        image: &lt;registry&gt;/omni-compass-cloud:&lt;version&gt;
        command: ["python", "/app/tools/prom_feed.py"]
        args:
          - "--prometheus=http://prometheus-operated.monitoring.svc:
    ↪ 9090"
          - "--served=sum(increase(nginx_ingress_controller_requests
    ↪ {ingress=\"checkout\",status!~\"5..\"}[{w}]))"
          - "--failed=sum(increase(nginx_ingress_controller_requests
    ↪ {ingress=\"checkout\",status=~\"5..\"}[{w}]))"
          - "--quantile=histogram_quantile({q}, sum by (le) (increas
    ↪ e(nginx_ingress_controller_request_duration_seconds_bucket{ingre
    ↪ ss=\"checkout\"}[{w}])))"
          - "--unit-ms=1000"
          - "--interval=15"
          - "--window=30s"
          - "--out=/feed/latency.csv"
        resources: {requests: {cpu: 10m, memory: 32Mi}, limits: {cpu
    ↪ : 100m, memory: 64Mi}}
        securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
        volumeMounts: [{name: feed, mountPath: /feed}]
```

In the queries, `{w}` is replaced by the window and `{q}` by each quantile; every other brace is PromQL's own. The network policy of section 26.5 then allows the metrics system's address in place of the service's. Before the governor relies on the adapter, check its feed as section 18.5 checks any feed, and run it beside the probe for a day: the two feeds' 95th percentiles should agree within the histogram's precision, and the dashboard's two lines (section 22.3) should coincide.

| Setting | Default | What it is |
|---|---|---|
| `--prometheus` | (required) | the metrics system's base URL |
| `--served`, `--failed`, `--quantile` | (required; `--failed` may be empty) | the three PromQL templates |
| `--quantiles` | 0.05, 0.15, ..., 0.95 | the quantiles read, one row each |
| `--unit-ms` | 1000 | milliseconds per unit of the histogram (1000 for seconds) |
| `--interval` | 15 s | seconds between reads |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Default | What it is |
|---|---|---|
| `--window` | `30s` | the range of each read: at least twice the scrape interval |
| `--fail-cap` | 20 | the most failure rows a window |
| `--out` | (required) | the feed the governor reads (`--latency-file`) |
| `--once`, `--duration` | off, 0 | read one window and exit; run for a number of seconds |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 23

# Change Management, Risk and Compliance

A governor that writes to production is a change to production, and companies with a change-management process will treat its installation as one. This chapter maps the phases of section 13.3 onto that process, lists the risks a risk owner will ask about with the control that answers each, and names the evidence a compliance review will want.

## The installation as changes, phase by phase

| Phase | Change class it usually is | Why |
|---|---|---|
| 1. The cluster | the company's own class for new infrastructure | the governor is not involved yet |
| 2. Identity | standard (pre-approved) | a namespace, a service account and a read-only role |
| 3. Watch | standard | the governor writes nothing; its role cannot write |
| 4. Feed and line | standard, or normal where an adapter is new code | the line is a decision of the service owner, recorded |
| 5. Paired run | none in production | the paired run is on its own clusters, built and deleted by the run |
| 6. Govern | normal (reviewed and approved) | the first grant that can write, and the first write |
| 7. Upgrades | normal, with the same gates as phase 6 | a new engine version is a new governor (chapter 12) |

The normal change of phase 6 carries, as its evidence, the paired receipt of phase 5 read row by row, the grant and its `kubectl auth can-i` proof (section 14.4), the OFF switch drill (section 20.3), and the rollback plan, which is short: turn the governor OFF, and every setting it ever wrote is back at the operator's, read back, within 60 seconds.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The risks and the controls that answer them

| Risk | Control | Evidence |
|---|---|---|
| The governor makes a decision that harms the service | it gives back a machine only after its own trial shows the response time holds; a breach wakes it at the next decision; nothing is given back while a sense is blind | the `verdict` and `decision` records; the paired receipt |
| The governor raises the bill | in the cloud mode it moves no pod autoscaler and never asks the cloud for a machine (chapter 19) | the grant (no autoscaler write); HPA replicas equal to native's in the receipt |
| The governor dies holding machines parked | the lease and the watchdog hand everything back from the records on the objects (section 20.2); a parked machine never blocks a pod | the watchdog drill |
| Someone else moves the same setting | the one-writer rule; the drift is visible in every decision | the `proprioception` field of each `authority` record |
| A compromised governor is used against the cluster | least privilege: read, patch nodes and patch pods; no delete, no eviction, no secrets, no cloud credential (chapter 14) | the role and its proof |
| A tampered copy is installed | the copy is verified against its manifest and seal before it is installed (chapter 32) | the verification receipt |
| The feed lies | the feed is checked before the governor watches (section 18.5); a stopped feed reads as blind and the governor holds | the feed checks |
| The audit is lost | the audit on a persistent volume, shipped to the log store (section 20.4) | the log store's retention |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## What a compliance review will ask for

Frameworks differ in their words, not in the questions. A review against the company's change-management control (for example SOC 2's change-management criterion, CC8.1, or ISO/IEC 27001:2022 Annex A control 8.32), its logging control (Annex A control 8.15) and its privileged-access control (Annex A control 8.2) asks the same five things of the installation, and each has its answer in the installation record (section 14.7):

1. **Who approved the change?** The phase-6 change record, with the service owner's and the platform owner's sign-off.
2. **What exactly can the system do?** The role, the `kubectl auth can-i` output, and the statement that it holds no cloud credential.
3. **How is every action recorded?** The audit: every decision, write and reason, on a persistent volume, shipped to the log store, kept for the retention period.
4. **How is it stopped?** The OFF switch, its holder, and the drill records.
5. **How do you know it works?** The paired receipt, read row by row on the company's own cluster, on the engine version in service.

## Separation of duties, restated for auditors

The implementer installs; the service owner writes the line and signs the receipt; the platform owner grants the governing role; security holds the OFF switch; finance signs the bill rows (section 13.4). No one person can both grant the governor its writes and be the only person able to stop it.

## Records an installation keeps, and for how long

| Record | Keep for |
|---|---|
| The verification receipt and engine version of every version installed | the life of the installation |
| The grant as applied, with its proof | the life of the installation, and every change to it |
| The line, its basis, and who signed it | the life of the line |
| Every paired receipt that decided a phase | the life of the engine version it was made on |
| The audit | the company's retention period for production change records |
| Every OFF-switch and watchdog drill | the company's retention period for control tests |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## Communicating the installation

The people who will notice the governor are the service's on-call engineers, and they are told before it governs: what it moves (in the cloud mode, only which machines are in service), what it never moves, how a parked machine looks in `kubectl get nodes` (a `PreferNoSchedule` taint, `omnicompass.io/idle`), where its audit is, how to turn it OFF, and who holds the switch. An engineer who finds a tainted node at three in the morning and does not know why is the failure this section exists to prevent.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 24

# Planning the Saving Before the Run: Capacity, Cost and the Shape of Load

A paired run costs days of the implementer's time and the price of two clusters. Before it is spent, the implementer can bound, from the company's own monitoring and with arithmetic alone, how large a saving the cloud mode could show on the pools it will govern, and where in the day that saving lives. This chapter gives that bound, the settings that move it, and the pricing terms that decide whether a machine-hour saved is a dollar saved. Nothing in it predicts what the governor will do: the bound is a ceiling that no machine lever on top of the same pods can pass, and the paired run (phase 5) is the only measurement of how much of it the governor takes while the line holds.

## Where the bill of a governed pool comes from

In every provider's on-demand pricing, a work pool's compute bill over a period T is the sum, over the machines that existed in it, of each machine's price per hour times the hours it existed:

$B = \Sigma_m \, p_m \, h_m$, $h_m = \int_0^T 1[\text{m exists at t}] \, dt$ .

For a pool of one machine family at list price p, this is $B = p \int_0^T N(t) \, dt$, where N(t) is the number of machines in the pool at time t. The cloud mode lowers B in one way only: it makes machines exist for fewer hours. It parks a machine (a `PreferNoSchedule` taint, chapter 19), marks that machine's pods first to go (`controller.kubernetes.io/pod-deletion-cost`), so that the next scale-down of the service's own HPA removes them first, and leaves the machine to empty; the cloud's own cluster autoscaler then deletes the empty machine on its own schedule. Every term of B that the governor reduces is a machine the cluster autoscaler deleted. Two consequences follow, and an implementer plans with both:

1. **The saving is realised by the cluster autoscaler.** A parked machine still exists, and is still billed, until the autoscaler deletes it. The autoscaler's own profile (section 24.4) sets how long that takes.
2. **The saving needs the pods to leave.** A parked machine whose pods stay keeps serving and keeps billing. Pods leave when the service's HPA scales down (the ReplicaSet controller removes the pods with the lowest deletion cost first) or when the cluster autoscaler drains an underused machine on its own rules. A service whose replica count never falls gives a parked machine no way to empty.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## The room: an upper bound on what any machine lever can save

Define, for each sampling instant t of the governed pools:

- N(t): the machines in the pools;
- Q[c] (t) and $Q_m$(t): the processor and memory **requests** of the pods running on them, DaemonSet pods excluded (a DaemonSet runs one pod on every machine whatever the count, so it is a per-machine overhead, not work);
- C[c] and $C_m$: one machine's allocatable processor and memory (`kubectl get node <name> -o jsonpath='{.status.allocatable}'`) less the requests of the DaemonSet pods on it;
- N[f]: the floor of machines in service (`--min-nodes`, never fewer than 2).

The fewest machines that could hold the pods' requests at instant t, packing perfectly, is

$$N[req](t) = \max(N[f], \lceil \max((Q[c](t))/(C[c]), Q_m(t)/C_m) \rceil),$$

and the **room** over the period is the machine-hours above it:

$$R = \int_0^T (N(t) - N[req](t))^+ \, dt \approx \Sigma_k \, \Delta t \, (N(t_k) - N[req](t_k))^+,$$

where $(x)^+ = \max(x, 0)$ and $\Delta t$ is the sampling interval. R is an upper bound for three reasons, each of which a real cluster exhibits: perfect packing is not achievable (pods have affinity rules, topology spread constraints and sizes that do not tile a machine); the scheduler places pods by its own scoring, not by packing; and a machine that is parked must also empty before the cloud deletes it. The cloud mode never asks the cloud for a machine and never holds more pods than native's autoscaler gives it, so the governed pool can never fall below N[req] by the governor's action, and it does not try to: every machine it parks first passes its own trial against the line.

**The billable room.** Where the company holds a commitment that pays for K machines of the family whether they exist or not (section 24.5), only machine-hours above K reach the bill:

$$R[\$] = p \, \Sigma_k \, \Delta t \, (N(t_k) - \max(K, N[req](t_k)))^+ .$$

**The governor's own floor.** The governor never parks a machine that would leave the processor requests of the pods on the machines in service, with the spare of its `--headroom` setting (half again, by default; Appendix K), or their processor in use, without a place. With Q′[c] (t) the requests of every pod on the machines in service (DaemonSet pods included), U[c] (t) the processor in use, and C′[c] one machine's allocatable processor, it keeps at least

$$N[gov](t) = \max(N[f], \lceil ((1+h) \, Q'[c](t))/(C'[c]) \rceil, \lceil (U[c](t))/(C'[c]) \rceil), \; h = 0.5,$$

so the room the cloud mode can take is at most $R[gov] = \int_0^T (N(t) - N[gov](t))^+ \, dt \le R$. A pool whose pods' requests, with half again to spare, already fill the machines the cluster autoscaler keeps has R[gov] = 0: there is nothing to park, and a paired run on it reads EVEN on the bill, correctly.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

**An arithmetic illustration (not a measurement).** A pool holds 12 machines from 08:00 to 20:00 and 8 machines overnight; the pods' requests need 9 machines at the day's level and 3 overnight; the floor is 2 and there is no commitment. Then R = 12 h × (12 − 9) + 12 h × (8 − 3) = 36 + 60 = 96 machine-hours a day. The overnight trough holds almost two thirds of the room; a paired run made only at the day's level could show at most 36 of the 96, and a run at a steady peak where N = N[req] could show none. The illustration's numbers are invented to show the arithmetic; the company's own samples replace them.

## Measuring the room on the company's own cluster

The samples need nothing the cluster does not already have. The following loop, run from any workstation or as a CronJob with the watch role of chapter 14, writes one row every five minutes for the pools labelled `omni-role=work`:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
#!/usr/bin/env bash
# One row every 5 minutes: time, machines in the governed pools, the r
    ↪ equests of the pods on them (DaemonSet pods
# excluded: processor in millicores, memory in MiB), and the processor
    ↪  requests of every pod on them (for N_gov).
SEL="omni-role=work"; OUT="room.csv"
[ -s "$OUT" ] || echo "epoch,machines,cpu_req_m,mem_req_mib,cpu_req_al
    ↪ l_m" &gt; "$OUT"
while true; do
  nodes=$(kubectl get nodes -l "$SEL" -o json | jq -c '[.items[].metad
    ↪ ata.name]')
  kubectl get pods -A --field-selector status.phase=Running -o json |
    ↪ jq -r --argjson nodes "$nodes" '
    def mcpu: if endswith("m") then .[:-1] | tonumber else tonumber *
    ↪ 1000 end;
    def mib:  if endswith("Ki") then .[:-2] | tonumber / 1024 elif end
    ↪ swith("Mi") then .[:-2] | tonumber
              elif endswith("Gi") then .[:-2] | tonumber * 1024 elif e
    ↪ ndswith("k") then .[:-1] | tonumber * 1e3 / 1048576
              elif endswith("M") then .[:-1] | tonumber * 1e6 / 104857
    ↪ 6 elif endswith("G") then .[:-1] | tonumber * 1e9 / 1048576
              else tonumber / 1048576 end;
    [.items[] | select(.spec.nodeName | IN($nodes[]))] as $on
    | [$on[] | .spec.containers[].resources.requests // {}] as $all
    | [$on[] | select(all(.metadata.ownerReferences[]?; .kind != "Daem
    ↪ onSet"))
             | .spec.containers[].resources.requests // {}] as $r
    | [now | floor, ($nodes | length),
       ($r | map(.cpu // "0" | mcpu) | add // 0 | floor), ($r | map(.m
    ↪ emory // "0" | mib) | add // 0 | floor),
       ($all | map(.cpu // "0" | mcpu) | add // 0 | floor)]
    | @csv' &gt;&gt; "$OUT"
  sleep 300
done
```

Two weeks of rows cover two weekly cycles. The room then follows from the formulas of section 24.2:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
import csv, math
C_CPU, C_MEM = 3860, 12800      # one machine's allocatable less its D
    ↪ aemonSet requests (millicores, MiB): read them
FLOOR, DT_H, K = 2, 300 / 3600, 0  # the floor, the sampling interval
    ↪ in hours, the committed machines
rows = list(csv.DictReader(open("room.csv")))
need = [max(FLOOR, math.ceil(max(int(r["cpu_req_m"]) / C_CPU, int(r["m
    ↪ em_req_mib"]) / C_MEM))) for r in rows]
room = sum(DT_H * max(0, int(r["machines"]) - n) for r, n in zip(rows,
    ↪  need))
billable = sum(DT_H * max(0, int(r["machines"]) - max(K, n)) for r, n
    ↪ in zip(rows, need))
C_ALLOC, H = 3860, 0.5             # one machine's allocatable process
    ↪ or (millicores), the governor's headroom
gov = [max(FLOOR, math.ceil((1 + H) * int(r["cpu_req_all_m"]) / C_ALLO
    ↪ C)) for r in rows]
room_gov = sum(DT_H * max(0, int(r["machines"]) - g) for r, g in zip(r
    ↪ ows, gov))
print(f"{len(rows) * DT_H:.0f} h sampled: room {room:.0f} machine-hour
    ↪ s, billable room {billable:.0f}, "
      f"room under the governor's own floor {room_gov:.0f}")
```

The constants `C_CPU`, `C_MEM` and `C_ALLOC` above are placeholders for a machine with four virtual processors; each installation reads its own. The use term of N[gov] is left out of the script, because on a pool that holds its requests it rarely binds; it can be added from `kubectl top nodes`. The same rows, plotted by hour of the week, show where the room lives, and so which load the paired run must carry (section 24.6).

## The settings that bound the room, and whose they are

None of the settings below is the governor's, and none is changed by it. They are the operator's, they shape native and omni alike, and a paired run holds each of them identical in both arms.

| Setting | Where | What it does to the room |
|---|---|---|
| The floor (`--min-nodes`, `MIN_NODES`) | the governor's command line | the least the governor keeps in service; set it to at least the number of zones the service's availability design needs serving at every moment, never fewer than 2 |
| The pools labelled `omni-role=work` | the node pools' labels | only these machines may be parked; a pool left unlabelled is never touched |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Where | What it does to the room |
|---|---|---|
| The HPA's minimum replicas | the service's HPA | a replica floor holds pods, and their machines, at every hour |
| The HPA's scale-down behaviour (`behavior.scaleDown`, stabilization window 300 s by default) | the service's HPA | a long window keeps pods after a peak, so parked machines empty later |
| The cluster autoscaler's scale-down profile | the provider (table below) | how long an empty or underused machine is kept before it is deleted |
| PodDisruptionBudgets | the service's manifests | a budget of zero disruptions keeps the cluster autoscaler from draining a machine; native is held the same way |
| Pods the cluster autoscaler will not move (`cluster-autoscaler.kubernetes.io/safe-to-evict: "false"`, local storage on some providers, bare pods with no controller) | the workloads | such a pod pins its machine until it ends |
| Spot or preemptible pools | the node pools | billed at the spot price; parking one saves that price; the provider may reclaim it on its own |

The cluster autoscaler's scale-down profile is set per provider:

| Provider | How the profile is set | The settings that matter |
|---|---|---|
| Azure (AKS) | `az aks update -g <rg> -n <cluster> --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down-delay-after-add=10m scale-down-utilization-threshold=0.5 scan-interval=10s` (the values shown are AKS's defaults) | how long a machine must be unneeded before deletion; how long after a scale-up no scale-down happens; the utilisation under which a machine counts as unneeded |
| Google (GKE) | `gcloud container clusters update <cluster> --autoscaling-profile balanced` or `optimize-utilization` | the profile as a whole; `optimize-utilization` removes underused machines sooner |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Provider | How the profile is set | The settings that matter |
|---|---|---|
| Amazon (EKS), Cluster Autoscaler | the flags on the autoscaler's own Deployment: `--scale-down-unneeded-time`, `--scale-down-delay-after-add`, `--scale-down-utilization-threshold`, `--scan-interval` | as on Azure |
| Amazon (EKS), Karpenter | the NodePool's `spec.disruption`: `consolidationPolicy` (`WhenEmpty` or `WhenEmptyOrUnderutilized`) and `consolidateAfter` | how soon an empty or underused machine is consolidated away |

A shorter scale-down profile also lowers native's bill. The comparison the governor is judged on is always at the same profile in both arms; a saving that came from a profile changed for one arm only is not the governor's, and a paired run that changes the profile between arms is not a paired run.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## Pricing terms, and whether a machine-hour saved is a dollar saved

| Pricing term | What a deleted machine-hour saves | Planning rule |
|---|---|---|
| On-demand, billed by the second or the minute (the common case on all three providers; read the provider's own page for the machine family) | its full price | the billable room is the room |
| A reservation, committed-use discount or savings plan covering K machines of the family | nothing below K; the full price above K | size commitments to the governed pool's floor and trough, not its peak; R[$] of section 24.2 |
| Spot or preemptible | the spot price | govern the on-demand pools first; a spot pool's machines can vanish on the provider's notice, governor or not |
| Software licensed per machine or per core (a commercial operating system, an agent billed per node) | that licence too, where it is billed by the hour the machine exists | add it to p |
| The cluster's own fee (the control plane), storage, network egress, load balancers | nothing | never in the bill rows of a receipt; the cloud mode moves none of them |

The bill rows of a receipt (section 19.6) price machine-hours at the provider's list price for the family, because list price is the one price both arms share and anyone can check. The company's own discounts change the dollars, never the machine-hours; finance converts the machine-hour row to the company's own price after the run (section 22.5).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Choosing the load for the paired run from the shape of the room

| Shape of the company's load | Where the room lives | The paired run that can show it |
|---|---|---|
| Steady at a level the pool was sized for | nowhere: N ≈ N[req] | none; a steady run reads EVEN on the bill, honestly, and is still worth running for the response-time and alert rows |
| A daily cycle (business hours, consumer evenings) | the trough and the descent after the peak | a replay of the company's own day, compressed only as far as the autoscaler's profile allows (a 10-minute unneeded time cannot be compressed below 10 minutes), or the repository's own day-shaped load (the Google 2011 trace on one to nine load generators) |
| Bursts separated by quiet stretches | between bursts, as native's autoscaler holds machines through its scale-down delay | the burst steps of the service's own traffic; in the repository, the burst sets of section 19.6 |
| A weekly cycle with a quiet weekend | the weekend | a run long enough to include a quiet stretch as long as the profile's scale-down time several times over |
| Batch windows | after each window | the batch itself, with the service's line on the jobs' own completion time where that is the objective |

**The run must be long enough for the autoscaler to act.** A quiet stretch shorter than the cluster autoscaler's unneeded time plus its scan interval gives neither arm a chance to delete a machine, and the bill reads EVEN for a reason that has nothing to do with the governor. Plan each quiet stretch of the paired run to last at least three times the unneeded time.

## What the plan delivers to the paired run

Before phase 5 starts, the implementer writes, and the service owner and finance sign:

1. the room and the billable room from two weeks of samples, with the constants used;
2. the hours of the week that hold most of the room;
3. the load the paired run will carry, chosen from section 24.6, with its quiet stretches sized to the profile;
4. the settings of section 24.4 as they stand, identical in both arms;

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

5. the price used for the bill rows (list price) and the company's own price for the conversion afterwards.

A receipt that then reads GAIN on the bill, with response time, failed requests and alert minutes reading EVEN or GAIN, has measured a share of a room that was known beforehand; a receipt that reads EVEN on the bill where the plan found room is a wiring question to answer first (section 4.5): the feed, the line, the autoscaler's profile, a pod that pins its machine.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 25

# On Your Own Racks and in Private Clouds

Many companies that run a public cloud also run Kubernetes on machines they own: bare-metal clusters in their own data centers, Red Hat OpenShift, VMware's Kubernetes on vSphere, Rancher's distributions, clusters at the edge, and clusters with no route to the internet at all. This chapter is for the implementer who installs Omni-Compass there. The plug, the grant, the feed, the audit, the OFF switch and the phases are the same as in the cloud; what changes is what a machine costs, and so which way the dial stands.

## What changes when the machines are already paid for

In a public cloud every machine that exists is billed by the hour, so the benefit is the bill and the dial stands at the tune-down (chapter 19). On machines the company owns, the machine is paid for whether it works or idles; an idle machine saves only the difference between its idle power and its working power, and nothing at all of its purchase price. The benefit there is **more work from the same machines**: the dial stands at the max (section 3.2), every muscle run at its own limit and never past it. The max never gives back a machine the cluster had on its own; it works the pods, the processor and, where wired, the power, each at its limit.

| | Public cloud | Own racks or a private cloud |
|---|---|---|
| What an idle machine costs | its full hourly price | its idle power, and its share of cooling |
| The dial | the tune-down (`--dial save`) | the max (`--dial max`), the default |
| The levers | the machines alone (chapter 19) | the pods, the processor, the power and the machines, level by level (section 6.3) |
| What the receipt is judged on | the bill, at native's work, speed and failures | work, speed, misses and failures; the power it took shown beside them as power given |
| The meter | the provider's bill at list price | the machine's own meter, a wall meter, or the facility's (evidence class P) |

An owner who wants the power an idle machine draws back, rather than more work, turns the dial down to the tune-down on that cluster; the machines then go back one at a time through the trial, exactly as in the cloud, and the receipt reads the machines and the power instead of the bill.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Bare-metal Kubernetes

A cluster built on the company's own machines (with kubeadm, RKE2, k3s, Talos or a vendor's distribution) usually has no cluster autoscaler: the machines exist all the time. The wiring follows section 6.3 level by level, with three points specific to bare metal.

**No machine is created or deleted by the governor.** The shipped machine script (`scripts/kind_nodepool.sh`) parks and wakes machines that exist, with a `PreferNoSchedule` taint; a parked machine stays powered, joined and Ready, and a pod that needs it lands on it at once. On bare metal, parking concentrates the work on fewer machines, and a parked machine whose pods have gone idles at its idle power.

**Powering a parked machine down is the operator's own command, and its price is the boot.** The governor's machine lever runs whatever command the operator writes in `--node-scale-cmd`, with `{n}` replaced by the number of machines to keep in service. An operator may write a command that, in addition to parking, powers an empty parked machine down through its baseboard management controller (Redfish or IPMI, or a provisioning system such as Metal3 or MAAS) and powers it up before waking it. The command's contract is the shipped script's:

| The command must | Why |
|---|---|
| keep exactly `{n}` machines in service, never fewer than the floor and never more than exist | the governor reads the count back as proprioception (section 20.4) |
| be idempotent: the same `{n}` twice changes nothing the second time | the governor issues the count again at its next decision while the machines in service differ from it |
| exit 0 when the count stands, and non-zero when it cannot be met | a non-zero exit is recorded as `refused`, and the governor learns the pool's top from it |
| never move, evict or delete a pod, and never power down a machine carrying a pod | parking never disturbs work |
| never touch a machine someone else cordoned | a cordon is a person's or an upgrade's, never the governor's (section 28.1) |
| wake a powered-down machine and return only when it is Ready | a woken machine must take pods |

A powered-down machine takes minutes to boot and join; the shipped script's parked machine takes no time at all. A wake that takes minutes turns the governor's answer to a breach from seconds into minutes. An installation that powers machines down therefore runs its paired run (phase 5) with the power-down command in place, so that the boot time is in the receipt's response-time and alert-minute rows, and keeps powered-down machines only beyond a number of parked, powered machines large enough to carry the service's largest burst.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The meter.** On owned machines the power the governor gives or takes is measured, not priced. The repository's wall meter reader (`tools/wall_meter.py`) reads a networked power meter or a power distribution unit; Kepler (the Kubernetes efficiency exporter) and the processor's own energy counters (RAPL on x86 servers) read per-machine estimates. A receipt that reports power names its meter and its evidence class (chapter 11).

## Red Hat OpenShift and OKD

OpenShift adds three things an implementer must account for.

**Security context constraints.** OpenShift admits pods through security context constraints (SCCs). The default for a service account, `restricted-v2`, assigns each pod a user ID from the namespace's own range and refuses a pod that asks for a fixed one, so the governing Deployment, which runs as user 10001 (chapter 19), is refused under it. Grant the governor's service account the `nonroot-v2` constraint, which admits any non-root user ID and keeps every other restriction of `restricted-v2`:

```
oc new-project omni-compass
oc apply -f deploy/install/omni-compass.yaml
oc adm policy add-scc-to-user nonroot-v2 -z omni-compass -n omni-compa
    ↪ ss
oc -n omni-compass get pod -l app=omni-compass -o jsonpath='{.items[0]
    ↪ .metadata.annotations.openshift\.io/scc}'
# prints: nonroot-v2
```

No other constraint is needed for the governor on a cluster: it is not privileged, mounts no host path and adds no capability. (The card governor of section 21.7 is a privileged agent on each graphics node and is a separate decision for the cluster's security owner.)

**Machine sets and the cluster autoscaler.** On an installer-provisioned OpenShift cluster the machines are made by the Machine API from MachineSets, and the cluster autoscaler is configured by a `ClusterAutoscaler` resource and one `MachineAutoscaler` per MachineSet. Native is that autoscaler. The governed machines are those of the MachineSets whose node template carries the label `omni-role: work`:

```
# in the MachineSet: the label every node it makes will carry
spec:
  template:
    spec:
      metadata:
        labels:
          omni-role: work
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

The governor itself runs on the infrastructure or control-plane machines, never on a machine it may park: set its `nodeSelector` to the infrastructure label the cluster uses (for example `node-role.kubernetes.io/infra: ""`) and add the matching toleration, in place of the manifest's `omni-role: system`.

**`oc` and `kubectl`.** The governor's image carries `kubectl`, which works against OpenShift's API unchanged. Commands in this manual written with `kubectl` work with `oc` in their place.

## VMware vSphere, Rancher and Cluster API

Where the cluster's machines are virtual machines on a private cloud and a cluster autoscaler makes and deletes them (VMware's Kubernetes on vSphere through Cluster API, Rancher's RKE2 with its node-pool autoscaling, OpenStack's Magnum, or any Cluster API provider with the cluster autoscaler's Cluster API integration), the cloud mode of chapter 19 applies as written: the work pools are labelled `omni-role=work`, the system pool carries the governor, and native is the cluster autoscaler. What changes is the price. A private cloud's bill is the company's own internal charge (chargeback or showback) per machine-hour, or, where there is none, the power and the licences per host; the receipt's machine-hour row is the measurement, and finance prices it.

Where the private cloud licenses its hypervisor per host and hosts are not released when virtual machines are deleted, deleting a virtual machine saves nothing until the platform consolidates hosts. Read the platform's own consolidation behaviour (for vSphere, Distributed Power Management) before claiming a saving from machine-hours alone.

## Clusters with no route to the internet

An air-gapped cluster pulls nothing from outside, so every artifact the installation needs is carried in:

| Artifact | How it is carried in |
|---|---|
| The verified copy | the repository archive, its manifest and its seal, verified on a connected machine (`python3 verify.py`) and again inside (`python3 verify.py` and `python3 tools/seal.py --check` need no network) |
| The base images | `python:3.12-slim` and `registry.k8s.io/kubectl` (the shipped Dockerfile's two bases), pulled on a connected machine, pinned by digest, saved (`docker save`), carried in and pushed to the internal registry |
| The Python packages | the wheels for `requirements.txt` downloaded on a connected machine (`pip download -r requirements.txt -d wheels`) and installed inside from that directory (`pip install --no-index --find-links wheels -r requirements.txt`) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Artifact | How it is carried in |
|---|---|
| `jq` for the cloud image | the distribution's package carried in with the base image, or a static `jq` binary copied into the image |
| The built images | built inside from the carried-in bases, with the `FROM` lines pointed at the internal registry |

The governor needs no network beyond the cluster's own API server and, for the probe, the service it measures. It sends nothing anywhere: no telemetry, no licence check, no update check.

## Clusters at the edge

A cluster of a few machines at a factory, a store or a cell tower has a floor problem: the governor never keeps fewer than two machines in service, and a three-machine cluster can park at most one. The gain at the edge is usually not machines but work and power: the max on the pods and the processor, and, where a site has a power limit, the site's watts read into the engine (`--power-cmd`, `--site-limit-w`, level 7). The OFF switch, the watchdog and the audit are the same; the audit should be shipped off the site, because an edge site's disks are the first thing lost.

## Own machines: what to check before claiming a gain

| Claim | What backs it |
|---|---|
| More work from the same machines | the paired receipt's work rows (requests served, jobs completed) reading GAIN, with response time and failures EVEN or GAIN, on the same machines in both arms |
| The same work on fewer machines | the machine rows reading GAIN at the tune-down, with every service row EVEN or GAIN |
| Power | a meter of class P named in the receipt; the power the max took shown as power given beside the work it bought, never called a loss |
| A dollar figure | the company's own price per kilowatt-hour or per machine-hour, applied by finance to the measured rows, never by the governor's receipt |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 26

# Hardening and the Supply Chain

A governor that can write to production is a privileged workload, however narrow its grant. This chapter takes the installation of chapter 19 from correct to hardened: the image built, described, scanned and signed; only that image admitted; the governor's writes narrowed below what role-based access control alone can express; its network reduced to the two endpoints it needs; and its every write cross-checked against the API server's own record. Each step is optional in the sense that the governor works without it, and expected in the sense that a security review will ask for it.

## What the governor's grant allows, read as an attacker would

Least privilege (chapter 14) limits the governor to reading the cluster and patching nodes and pods. A security reviewer reads that grant by asking what a stolen copy of the governor's service-account token could do, not what the governor does with it:

| With the cloud-mode grant, a stolen token could | Consequence | The control that closes it |
|---|---|---|
| read nodes, pods, deployments, HPAs and config maps cluster-wide | the cluster's shape is disclosed | the token is short-lived and projected; the namespace admits only the signed image (section 26.3); no person or other workload uses the service account |
| taint a node `NoSchedule`, or cordon it | new pods cannot land there; enough of them and new pods wait | the admission policy of section 26.4 admits only the governor's own `omnicompass.io/idle` taint with the `PreferNoSchedule` effect, and no cordon |
| change a node's labels or annotations (for example, disable the cluster autoscaler's scale-down on it) | scheduling or the bill is distorted | the admission policy of section 26.4 |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| With the cloud-mode grant, a stolen token could | Consequence | The control that closes it |
|---|---|---|
| change a pod's labels | the pod leaves its Service's selector and stops receiving traffic | the admission policy of section 26.4 admits, on a pod, only the `controller.kubernetes.io/pod-deletion-cost` annotation |
| change a mutable field of a pod's spec (its container image, its active deadline, its tolerations) | code the company did not ship runs, or pods end | the admission policy of section 26.4 admits no change to a pod's spec |

Role-based access control cannot express the right-hand column: its verbs are whole-object (`patch` on `nodes` is every field of every node). The Kubernetes admission policy language can, and section 26.4 uses it. With it, the governor's token is worth, to an attacker, what it is worth to the governor: the ability to mark a machine "prefer not" and to mark a pod "first to go".

## The image: built from the verified copy, described, scanned, signed

1. **Build from the verified copy** (chapter 32), with the base images pinned by digest rather than by tag:

```bash python3 verify.py # VERIFICATION: PASS, on the commit being built docker build -t <registry>/omni-compass:<version> -f deploy/Dockerfile . docker build -t <registry>/omni-compass-cloud:<version> -f deploy/Dockerfile.cloud \

```
  --build-arg BASE=<registry>/omni-compass:<version> .
```

```

Pin `python:3.12-slim` and `registry.k8s.io/kubectl` by digest in a copy of `deploy/Dockerfile` kept in the company's own repository (`FROM python:3.12-slim@sha256:<digest>`), so that a rebuild of the same commit yields the same layers. Tag every image with the engine version and the commit; never deploy `latest`.

2. **Describe it** with a software bill of materials, in SPDX or CycloneDX: `syft <registry>/omni-compass-cloud:<version> -o spdx-json > sbom.spdx.json`. Keep the bill with the installation record.
3. **Scan it** for known vulnerabilities before it is pushed and on a schedule after: `trivy image <registry>/omni-compass-cloud:<version>` or `grype <registry>/omni-compass-cloud:<version>`. The image holds a Python runtime, its packages, `kubectl`, `bash` and `jq`; a finding in any of them is handled under the company's own vulnerability policy, by rebuilding on a patched base, never by editing the engine.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

4. **Sign it** in the registry, so that admission can prove where it came from: with Sigstore's `cosign` (`cosign sign --key <key> <registry>/omni-compass-cloud@sha256:<digest>`, or keyless signing bound to the company's build identity), or with Notation and the cloud's key service (Azure Key Vault, Google Cloud KMS, AWS Signer). Sign the digest, not the tag.
5. **Push to the company's private registry** (Azure Container Registry, Google Artifact Registry, Amazon ECR), with tag immutability turned on, and pull from it only.

## Admitting only the signed image, and only the restricted profile

**Only the signed image.** Each provider offers an admission control that refuses an unsigned or wrongly signed image:

| Provider | Admission control for image signatures |
|---|---|
| Azure (AKS) | Image Integrity (Ratify with Azure Policy), verifying Notation signatures |
| Google (GKE) | Binary Authorization, with an attestor bound to the company's signing key |
| Amazon (EKS) | Kyverno's `verifyImages` rule or the Sigstore policy controller, verifying cosign or Notation signatures |
| Any cluster | Kyverno `verifyImages`, the Sigstore policy controller, or Gatekeeper with Ratify |

Scope the rule to the `omni-compass` namespace at least; the governor's pod then cannot be replaced by a pod running an image the company did not build and sign.

**The restricted Pod Security Standard.** The governing Deployment meets the `restricted` profile of the Kubernetes Pod Security Standards as shipped: it runs as a non-root user, forbids privilege escalation, drops every Linux capability, uses the runtime's default seccomp profile, mounts only `emptyDir` volumes (or a persistent volume claim, section 20.4) and has a read-only root filesystem. Enforce the profile on its namespace, so that no later edit can loosen it:

```
kubectl label namespace omni-compass \
  pod-security.kubernetes.io/enforce=restricted pod-security.kubernete
    ↪ s.io/enforce-version=latest
```

On OpenShift the security context constraints of section 25.3 play this part.

## Narrowing the writes below the role: an admission policy

A `ValidatingAdmissionPolicy` (Kubernetes 1.30 and later, which every current managed Kubernetes version satisfies) evaluates rules written in the Common Expression Language (CEL) against every write a chosen identity makes, and refuses a write that breaks them, after role-based access control has allowed it. The two policies below bind the governor's service account, and only it, to the writes the cloud mode makes: on a node, its own idle taint and nothing else; on a pod, its own deletion-cost mark and nothing else.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
# The governor's writes on machines: its own idle taint (PreferNoSched
    ↪ ule), nothing else.
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: {name: omni-compass-nodes}
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - {apiGroups: [""], apiVersions: ["v1"], operations: ["UPDATE"],
    ↪  resources: ["nodes"]}
  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
    ↪ :omni-compass:omni-compass'"
  validations:
    - expression: &gt;-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &amp;&amp;
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)
      message: "the governor changes no label of a machine"
    - expression: &gt;-
        has(object.metadata.annotations) == has(oldObject.metadata.ann
    ↪ otations) &amp;&amp;
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons == oldObject.metadata.annotations)
      message: "the governor changes no annotation of a machine"
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    - expression: &gt;-
        (has(object.spec.unschedulable) &amp;&amp; object.spec.unschedulable)
    ↪ ==
        (has(oldObject.spec.unschedulable) &amp;&amp; oldObject.spec.unschedul
    ↪ able)
      message: "the governor never cordons or uncordons a machine"
    - expression: &gt;-
        (!has(object.spec.taints) || object.spec.taints.all(t,
           (t.key == 'omnicompass.io/idle' &amp;&amp; t.effect == 'PreferNoSch
    ↪ edule') ||
           (has(oldObject.spec.taints) &amp;&amp; oldObject.spec.taints.exists
    ↪ (o, o.key == t.key &amp;&amp; o.effect == t.effect)))) &amp;&amp;
        (!has(oldObject.spec.taints) || oldObject.spec.taints.all(o, o
    ↪ .key == 'omnicompass.io/idle' ||
           (has(object.spec.taints) &amp;&amp; object.spec.taints.exists(t, t.
    ↪ key == o.key &amp;&amp; t.effect == o.effect))))
      message: "the governor adds or removes no taint but its own omni
    ↪ compass.io/idle:PreferNoSchedule"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-nodes}
spec: {policyName: omni-compass-nodes, validationActions: [Deny]}
---
# The governor's writes on pods: its own deletion-cost mark, nothing e
    ↪ lse.
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
metadata: {name: omni-compass-pods}
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - {apiGroups: [""], apiVersions: ["v1"], operations: ["UPDATE"],
    ↪  resources: ["pods"]}
  matchConditions:
    - name: the-governor
      expression: "request.userInfo.username == 'system:serviceaccount
    ↪ :omni-compass:omni-compass'"
  validations:
    - expression: "object.spec == oldObject.spec"
      message: "the governor changes nothing in a pod's spec"
    - expression: &gt;-
        has(object.metadata.labels) == has(oldObject.metadata.labels)
    ↪ &amp;&amp;
        (!has(object.metadata.labels) || object.metadata.labels == old
    ↪ Object.metadata.labels)
      message: "the governor changes no label of a pod"
    - expression: &gt;-
        (!has(object.metadata.annotations) || object.metadata.annotati
    ↪ ons.all(k,
           k == 'controller.kubernetes.io/pod-deletion-cost' ||
           (has(oldObject.metadata.annotations) &amp;&amp; k in oldObject.meta
    ↪ data.annotations &amp;&amp;
            oldObject.metadata.annotations[k] == object.metadata.annot
    ↪ ations[k]))) &amp;&amp;
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
        (!has(oldObject.metadata.annotations) || oldObject.metadata.an
    ↪ notations.all(k,
           k == 'controller.kubernetes.io/pod-deletion-cost' ||
           (has(object.metadata.annotations) &amp;&amp; k in object.metadata.a
    ↪ nnotations)))
      message: "the governor changes no annotation of a pod but contro
    ↪ ller.kubernetes.io/pod-deletion-cost"
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata: {name: omni-compass-pods}
spec: {policyName: omni-compass-pods, validationActions: [Deny]}
```

**Prove the policies before the governor relies on them.** A policy is proved the way the grant is (section 14.4): by asking the API server, as the governor, to do what the policy forbids and what it allows, and reading the answers. The implementer's own account needs the right to impersonate the service account, which a cluster administrator has:

```
SA=system:serviceaccount:omni-compass:omni-compass; N=&lt;a work machine&gt;
    ↪ ; P=&lt;a pod of the service&gt;; NS=&lt;its namespace&gt;
kubectl --as=$SA label node $N omni-test=1
    ↪        # refused: changes no label of a machine
kubectl --as=$SA cordon $N
    ↪        # refused: never cordons
kubectl --as=$SA taint node $N omni-test=1:NoSchedule
    ↪        # refused: no taint but its own
kubectl --as=$SA taint node $N omnicompass.io/idle=true:PreferNoSchedu
    ↪ le     # admitted
kubectl --as=$SA taint node $N omnicompass.io/idle:PreferNoSchedule-
    ↪        # admitted
kubectl --as=$SA label pod $P -n $NS omni-test=1
    ↪        # refused: changes no label of a pod
kubectl --as=$SA annotate pod $P -n $NS controller.kubernetes.io/pod-d
    ↪ eletion-cost=-1000 --overwrite   # admitted
kubectl --as=$SA annotate pod $P -n $NS controller.kubernetes.io/pod-d
    ↪ eletion-cost-                    # admitted
```

Each refusal names the policy and its message. Keep the eight answers with the installation record beside the grant's proof. Run the drill in a quiet hour: the two admitted taint commands park and wake the machine `$N` for a moment, and the two admitted annotations mark and unmark one pod.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**What `failurePolicy: Fail` means here.** If the policy itself cannot be evaluated, the governor's write is refused, never admitted. A refused write is recorded as `refused` in the governor's audit (section 20.4); a refused park leaves a machine in service, and a refused wake leaves a machine parked, which never blocks a pod. The policy can therefore stop the governor from saving, never make it harm.

The cloud mode needs nothing the policies forbid: the shipped machine script parks and wakes machines with the idle taint alone, marks pods with the deletion-cost annotation alone, and never cordons or uncordons a machine (section 28.1).

## The network: two endpoints

The governor's pod talks to two endpoints: the cluster's API server (through `kubectl`, from the controller and the watchdog) and the service it measures (from the probe). It receives no connection at all. A `NetworkPolicy` can hold it to exactly that, where the cluster's network plugin enforces network policy (Azure Network Policy Manager or Cilium on AKS, GKE Dataplane V2 or Calico on GKE, the Amazon VPC CNI's network policy agent or Calico on EKS):

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: omni-compass, namespace: omni-compass}
spec:
  podSelector: {matchLabels: {app: omni-compass}}
  policyTypes: [Ingress, Egress]
  ingress: []                                     # nothing connects to
    ↪ the governor
  egress:
    - to:                                         # the cluster's DNS
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.nam
    ↪ e: kube-system}}
          podSelector: {matchLabels: {k8s-app: kube-dns}}
      ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
    - to:                                         # the API server: its
    ↪ address from `kubectl get endpoints kubernetes`
        - ipBlock: {cidr: &lt;api-server-address&gt;/32}
      ports: [{protocol: TCP, port: 443}]
    - to:                                         # the measured service
    ↪ , for the probe
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.nam
    ↪ e: &lt;namespace&gt;}}
          podSelector: {matchLabels: {&lt;the service's selector&gt;}}
      ports: [{protocol: TCP, port: &lt;the service's target port&gt;}]
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

Three details decide whether this policy works on a given cluster: the API server's address, which on a managed cluster is outside the pod network and is read from `kubectl get endpoints kubernetes -n default` (and some providers translate it, so test from the pod); the DNS pods' label, which differs by provider (`k8s-app: kube-dns` on all three as of this writing); and the measured service's pods' labels and target port. Prove the policy as the grant is proved: from the governor's pod, the API server and the service answer, and nothing else does (`kubectl -n omni-compass exec deploy/omni-compass -c probe -- python -c "import urllib.request; urllib.request.urlopen('https://example.com', timeout=5)"` must fail).

## The API server's own record of every write

The governor's audit records every write it makes; the API server's audit log records every write it receives. Read together, they prove that the governor wrote nothing it did not record, and that nothing wrote as the governor that was not the governor. Turn the control plane's audit log on, and filter it by the service account `system:serviceaccount:omni-compass:omni-compass`:

| Provider | The API server's audit log |
|---|---|
| Azure (AKS) | diagnostic settings on the cluster, category `kube-audit` (or `kube-audit-admin` for writes only), to a Log Analytics workspace |
| Google (GKE) | Cloud Audit Logs, Admin Activity (always on) for writes, in Cloud Logging |
| Amazon (EKS) | control plane logging, log type `audit`, to CloudWatch Logs |

Every `patch` on a node or a pod by the service account must match a `write` record in the governor's audit within the same few seconds, with the same object; every `write` record that is not marked `"dry_run": true` must match a patch. A patch by the service account with no `write` record behind it is a security incident: the token was used by something other than the governor.

## The token, and no other secret

The governor holds no secret of any kind: no cloud credential (section 14.5), no password, no key; its role reads no `Secret` object. Its only credential is the Kubernetes service-account token that the kubelet projects into its pod, short-lived and rotated by the kubelet, bound to the pod's lifetime. No person and no other workload uses the `omni-compass` service account; a request by it from anywhere but the governor's pod is visible in the API server's audit log (section 26.6) by its source address and its user agent.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## The hardening record

| Item | Evidence kept |
|---|---|
| The image | its digest, its software bill of materials, its last scan, its signature |
| Admission | the signature rule and the namespace's Pod Security label |
| The writes | the two admission policies, and the eight answers of their proof |
| The network | the network policy and the proof that only the two endpoints answer |
| The record | the API server's audit log turned on, and one quarter's cross-check against the governor's audit |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 27

# Rolling Out Across a Fleet: Many Clusters, Many Services, GitOps

Section 20.7 sets the rule for a fleet: one governor per cluster, rolled out cluster by cluster through the phases of section 13.3, with a paired receipt on at least one cluster of each kind. This chapter is the practice behind the rule: how the governor's configuration is kept as code, what differs from cluster to cluster and what must not, how a continuous-delivery controller interacts with the OFF switch, and how a fleet of audits is read as one.

## The unit of rollout

| Unit | Rule |
|---|---|
| The cluster | one governor, one feed, one line, one set of governed pools; never one governor for several clusters, never two governors on one cluster |
| The kind | clusters on the same provider, machine families, autoscaler profile and service class; one paired receipt per kind decides whether the kind may govern |
| The engine version | one engine version across the fleet at a time, apart from the wave that is upgrading; a receipt is read only against the version it was made on (chapter 12) |
| The wave | an ordered group of clusters that moves from one phase to the next together, after the wave before it has run a full cycle of its load without a fault |

A conventional order of waves is: one non-production cluster of the kind; one production cluster with the smallest blast radius; a tenth of the kind; the rest. Each wave starts at phase 3 (watch), never at phase 6, even when the kind already has a receipt: watching on a new cluster shows that its feed, its line and its pools are what the receipt assumed.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## What differs from cluster to cluster, and what must not

| Setting | Per cluster | Why |
|---|---|---|
| `TARGET_URL` (the probe), or the adapter's source | yes | the service's address in that cluster |
| `--slo-ms` | per service, usually the same across clusters | the service owner's line; a cluster with a different line is a different kind |
| `--max-nodes` | yes | the sum of that cluster's work pools' autoscaler maxima |
| `--min-nodes` and `MIN_NODES` | per kind | the floor, at least the zones the service needs serving |
| `WORKER_SEL` | per kind | the label of the governed pools, `omni-role=work` by convention |
| The image digest | **never** | one engine version, one build, one signature |
| `--mode`, `--law`, `--dial`, `--sensed`, `--interval`, `--floor-interval`, `--max-node-step`, the node commands | **never** | the cloud mode as section 19.3 writes it; a cluster that differs is not in the cloud mode and its receipt is its own |

## The configuration as code

The repository's manifests are the starting point, not the company's deployable unit. Two of them cannot be applied together as written: `deploy/install/omni-compass.yaml` carries the identity (the namespace, the service account, the watch role and its binding) and the watch Deployment, and `deploy/install/omni-compass-cloud.yaml` carries the governing Deployment under the same name. In the company's own configuration repository, split the first file into its identity half and its watch half, and keep a base for each phase:

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
omni-compass/
  base/identity/        namespace, service account, watch role and bin
    ↪ ding (from deploy/install/omni-compass.yaml)
  base/watch/           the watch Deployment (from deploy/install/omni
    ↪ -compass.yaml)
  base/govern/          rbac-machines.yaml, omni-compass-cloud.yaml, t
    ↪ he network policy and admission policies (chapter 26)
  overlays/&lt;cluster&gt;/   kustomization.yaml: identity + watch, or ident
    ↪ ity + govern, and this cluster's settings
```

An overlay for one cluster in phase 6:

```
# overlays/aks-eastus-prod-1/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [../../base/identity, ../../base/govern]
images:
  - name: omni-compass-cloud             # the image name in the mani
    ↪ fest, without its tag
    newName: &lt;registry&gt;/omni-compass-cloud
    digest: sha256:&lt;the signed digest&gt;
patches:
  - target: {kind: Deployment, name: omni-compass}
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/1/env/0/value
        value: http://checkout.shop.svc.cluster.local/
      - op: replace
        path: /spec/template/spec/containers/0/args
        value: ["--mode", "nodepool", "--law", "compass", "--dial", "s
    ↪ ave", "--sensed", "none/none",
                "--active-nodes-only", "--interval", "60", "--floor-in
    ↪ terval", "5",
                "--min-nodes", "3", "--max-nodes", "24", "--max-node-s
    ↪ tep", "1",
                "--node-scale-cmd", "bash /app/scripts/kind_nodepool.s
    ↪ h {n}",
                "--node-restore-cmd", "bash /app/scripts/kind_nodepool
    ↪ .sh 100000",
                "--latency-file", "/feed/latency.csv", "--slo-ms", "40
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ 0",
                "--closure", "/app/law/closure.json",
                "--audit", "/var/log/omni/audit.jsonl", "--kill-file",
    ↪  "/tmp/omni.kill"]
```

The patch replaces the governor's whole command line rather than single arguments, so that the line in Git is the line that runs, readable in one place; its absence of `--dry-run` is the change that the phase-6 change record approves (section 23.1). The containers are addressed by position (0 the governor, 1 the probe, 2 the watchdog), as the manifest orders them; `MIN_NODES` in the governor's and the watchdog's environment must be patched to the same floor as `--min-nodes`.

## Continuous delivery and the OFF switch

A continuous-delivery controller that keeps the cluster equal to Git (Argo CD with automated sync and self-heal, Flux with its default reconciliation) undoes a change made directly in the cluster. Two of the OFF switch's forms (section 20.3) are such changes, and the holder must know what the controller will do with them:

| OFF form | What a self-healing controller does | What the installation does about it |
|---|---|---|
| Scale to zero (`kubectl scale ... --replicas=0`) | puts the replica count back to Git's 1 within its reconciliation interval, and the governor starts again | either the OFF switch of record is a commit to Git (`replicas: 0`), or the controller is told to leave the replica count alone: in Argo CD an `ignoreDifferences` entry on the Application for `/spec/replicas` of the `omni-compass` Deployment; in Flux, `flux suspend kustomization <name>` as the first step of the drill |
| The kill file | nothing: the file is in the pod's `/tmp`, which Git does not describe | none needed; but a pod replaced for any reason, including a sync that changes the pod template, starts ON |
| Revoke the grant (`kubectl delete clusterrolebinding omni-compass-machines`) | applies the binding again from Git | revoke in Git, or suspend the controller for the binding first |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

Write the chosen form into the runbook and drill it as it is written (section 20.3). A drill on a GitOps cluster that scales to zero and then watches the controller start the governor again has found the fault this section exists to prevent.

**Other reconcilers of the same objects.** The governor writes two things: a taint on a node and an annotation on a pod. A tool that reconciles node taints to a declared list (some node-management controllers, some provider settings applied to a pool) removes the governor's `omnicompass.io/idle` taint, and a tool that reconciles pod annotations would remove the deletion-cost mark; either is a second writer of the governor's setting (section 20.8) and shows as a `proprioception` drift that does not close. Exclude the key from that tool's reconciliation, or leave that pool ungoverned. Pods are created by their controllers, not by Git, so a continuous-delivery controller never sees the deletion-cost mark.

## Many services on one cluster

The cloud mode governs machines, and machines are shared. Where several services run on the governed pools, the governor's single feed must speak for all of them (section 18.6): the feed of the service most sensitive to a lost machine, or an adapter that writes the worst of the services' own percentiles against their own lines. Three practical rules follow:

1. **Govern a pool, not a service.** Label `omni-role=work` on the pools whose machines the feed speaks for, and leave pools that serve other services unlabelled. A pool dedicated to one service (a node pool with a taint the service tolerates) is the simplest case and the cleanest receipt.
2. **One line per pool.** If two services on the same pool have lines that cannot be scaled to one, they are not governed together: split the pool, or govern only by the stricter line.
3. **Batch and stateful work.** Pods that the cluster autoscaler will not move (section 24.4) pin their machines; the governor may park such a machine, but it empties only when those pods end. A pool that carries long jobs and serving pods together gives the governor little to give back, honestly; the receipt shows it as EVEN on the bill.

## Regions, zones and failover

Each region's clusters are separate units. A governor in one region never reads another region's feed, and a regional failover (traffic moved from one region's clusters to another's) is, to each governor, a change in its own service's load: the receiving region's governor sees its line approach and wakes what it parked; the sending region's governor sees the load fall and, after its trials, parks what the trough allows. Two settings deserve attention in a region that must be able to absorb another's traffic at once:

- **The floor.** A region that must carry a failover without waiting for the cloud to create machines sets its floor (`--min-nodes`) to the machines the failover needs at its first minute, not to the trough's need; the bill's room then starts above that floor (section 24.2).
- **The zones.** The governor does not read zones, and the machine script chooses which machines to park by the work on them. A service that needs a minimum number of serving machines in every zone keeps that

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

minimum through its own pods' topology spread constraints and a floor of at least that number times the zones; the `PreferNoSchedule` taint never prevents a pod from landing on a parked machine in the zone it needs.

## Reading a fleet's audits as one

Every governor writes its own audit (section 20.4); a fleet's audits become one record when the log collector adds the cluster's name to every line on its way to the log store (Fluent Bit's Kubernetes filter, the Azure Monitor agent, Google Cloud's logging agent and the CloudWatch agent each attach the cluster's identity, or can be configured to). Four fleet views answer most questions:

| View | From | Shows |
|---|---|---|
| Machines parked, by cluster and hour | `write` records with `why` = `node pool size` | where and when the fleet gives machines back |
| Trials refused or abandoned, by cluster | `verdict` records | clusters whose feed is thin or whose service is short of machines |
| Blind and error decisions, by cluster | `authority` records' `senses`, and `error` records | clusters with a feed or API problem |
| Drift that does not close, by cluster | `authority` records' `proprioception` | clusters with a second writer |

The fleet's bill is read from the provider's own billing records by cluster and pool (section 22.5), never summed from receipts: a receipt is a controlled comparison on a day, and a bill is the fleet's actual cost over a period.

## Upgrading a fleet

An engine upgrade moves through the fleet in the same waves as the first installation (section 20.6 for one cluster): each wave watches on the new version for a full cycle of its load, a paired receipt on the new version decides each kind, and only then does the wave govern on it. During the upgrade the fleet runs two versions, and every audit line, dashboard and receipt names the version it came from (`python3 tools/omni_version.py` on the build, the image tag in the cluster). The rollback of a wave is the rollback of one cluster repeated (section 20.6), safe at any moment because the governor carries no state between processes beyond the marks it reads back from the cluster.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

CHAPTER 28

# Failure Modes and the Drill Book

Every component of an installation fails eventually: the governor's process, its pod, the machine under it, its feed, the API server it talks to, the service it measures, and the other controllers that share the cluster with it. This chapter lists each failure an implementer should expect, what the governor does when it happens, what the audit and the cluster show, and what the operator does. Section 28.2 turns the most important ones into drills, each with the command that causes the failure on purpose, so that the first time the on-call engineer sees it is not the night it happens.

The design rule behind every row is the same, and it is worth stating once: **a failure of the governor never leaves a pod without a place to run.** A parked machine carries a `PreferNoSchedule` taint, which the scheduler treats as a preference and never as a refusal, so the worst a dead or blind governor can leave behind is machines that are still in the cluster, still Ready and still usable, with a mark asking new pods to go elsewhere first.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## What fails, what the governor does, and what you do

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| The governor's process is killed (out of memory, a signal) | nothing; the watchdog in the same pod finds the process gone within 10 seconds and runs the governor's recorded restore, waking every parked machine and removing its marks; the kubelet restarts the governor's container | the watchdog's `handed back` record; the container's restart count rises; the restarted governor's first decisions | find why it died (`kubectl describe pod`, the container's last state); an out-of-memory kill means the limit is too low for the cluster's size |
| The governor hangs | its lease stops being renewed; after five decision periods (300 seconds at the cloud mode's 60) the watchdog kills it and runs the restore | the watchdog's record with `hung: true` | as above |
| The governor's pod is deleted or evicted gracefully (a drain of the system pool, an upgrade, `kubectl delete pod`) | on `SIGTERM` it hands every setting back, reads each back and exits; the Deployment starts a new pod, which starts ON | the audit record `master switch: OFF; every setting handed back; exiting`, then a new pod's first decisions | nothing, unless the new pod cannot be placed |
| The machine under the governor fails abruptly | nothing can act for it at that moment; the parked machines stay parked, which blocks no pod; the Deployment starts a new pod on another system machine, which begins from the cluster as it finds it | the pod rescheduled; if the system pool has one machine, the pod waits for the provider's repair | run the system pool on at least two machines in production, so the governor's pod always has somewhere to go |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| The feed stops (the probe's container dies, the adapter stops writing) | after twice its window with no new row the governor reads the feed as blind: it gives nothing back, treats the reading as past its wall, and wakes a parked machine each decision while the sense stays blind | `authority` records whose `senses.blind` names `latency`, with a growing `latency_age_s`; the probe container's restarts | restore the feed (section 18.3 on a probe restart); the governor resumes on its own |
| The service stops answering (every request fails) | no served row means no reading: blind, as above | as above, and the service's own alert | the service's own incident; the governor has already stopped giving back and is waking what it parked |
| The feed lies (a reading that does not track the service) | the governor cannot know; it acts on the reading it is given | the dashboard's two response-time lines disagree (section 22.3) | turn the governor OFF until the feed is fixed; this is why the feed is checked before the governor watches (section 18.5) |
| The API server is unreachable or throttling | each decision that cannot read the cluster writes an `error` record (with the count of consecutive failures) and changes nothing; the fast checks record their own errors | `error` records; the provider's own control-plane status | nothing on the governor's side: it resumes when the API answers; parked machines stay usable meanwhile |
| **Failure** | **What the governor does** | **What you see** | **What you do** |
| A write is refused (the role, an admission policy, a conflict) | the command's failure is recorded as `refused` with its exit code; with `--max-nodes` set, as the cloud manifest sets it, the governor's top is | `refused` records | read the reason; a policy refusal of a write the cloud mode needs means the policy is wrong (chapter 26); a role refusal means the grant is |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| | unchanged, and it issues the count again at its next decision while the machines in service differ from it | | missing (chapter 14) |
| A person cordons a governed machine (maintenance) | the machine script leaves a machine someone else cordoned alone: it never uncordons it, never counts it in service or parked, and removes its own idle mark from it, if any | the machine `SchedulingDisabled`; a `proprioception` drift while the cordoned machine still carries work | none needed; for a long maintenance, also remove the machine's `omni-role=work` label so the governor no longer sees it at all, and put the label back afterwards |
| The provider upgrades the cluster or a node pool (cordon, drain, surge machines) | as for a cordon; surge machines join open and are counted | the upgrade's own events | scale the governor to zero before the upgrade starts (its hand-back wakes everything) and back to one when the upgrade completes, so the upgrade runs exactly as it does without the governor |
| The cluster autoscaler deletes a parked, empty machine | nothing; that is the saving (section 24.1) | the machine gone; the next decision counts one fewer | nothing |
| The cluster autoscaler adds machines | they join open; the governor counts them at its next read | new machines | nothing |
| A spot or preemptible machine is reclaimed | as a machine failure; parked or open, it is gone | the provider's eviction notice | nothing on the governor's side |
| **Failure** | **What the governor does** | **What you see** | **What you do** |
| A descheduler evicts pods from parked machines | nothing; the descheduler is a second actor on the same machines | pods moved off machines with the `omnicompass.io/idle` taint | exclude the taint: the descheduler's `RemovePodsViolatingNodeT` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Failure | What the governor does | What you see | What you do |
|---|---|---|---|
| | | | `aints` plugin acts on `PreferNoSchedule` taints only when told to, and takes an `excludedTaints` list |
| Someone else removes the governor's taint, or uncordons a parked machine | the machine counts as open at the next decision; the governor may park it again | a `proprioception` drift; machines parked and unparked repeatedly | find the other writer (section 20.8); until there is one writer, turn the governor OFF |

## The drills

Each drill is run first on a non-production cluster, then in production in a quiet hour, by the people who will handle the failure for real, with the result recorded as the OFF switch drill is (section 20.3). The pass condition of every drill is the same: no pod waited for a place, no alert minute was caused, every setting was back at the operator's within the stated time, and the audit and the cluster show what this chapter says they show.

**Drill 1: the governor's process killed outright.** The watchdog's hand-back is the lease (section 20.2), and this is its test. From the watchdog's container, which shares the governor's process namespace, send `SIGKILL` to every governor the lease registry lists:

```
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "
import glob, json, os, signal
for f in glob.glob('/tmp/omni-compass/running/*.json'):
    os.kill(json.load(open(f))['pid'], signal.SIGKILL)"
kubectl -n omni-compass logs deploy/omni-compass -c watchdog --tail=5
    ↪      # the 'handed back' record, ok: true
kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.n
    ↪ ame,TAINTS:.spec.taints[*].key
```

Pass: within 30 seconds of the kill, no `omnicompass.io/idle` taint remains, the watchdog's record reads `ok: true`, and the governor's container has restarted and is deciding again.

**Drill 2: the pod deleted.** `kubectl -n omni-compass delete pod -l app=omni-compass`. Pass: the old pod's audit ends with the hand-back; every parked machine is woken before it exits (within the 60-second grace period); the new pod is deciding within two minutes.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Drill 3: the feed stopped.** Pause the probe without touching the service or replacing the pod: from the watchdog's container, which shares the pod's process namespace, stop the probe's process, and later let it continue:

```
PAUSE='import os, signal, sys
for p in os.listdir("/proc"):
    if p.isdigit() and b"latency_probe.py" in open("/proc/%s/cmdline"
    ↪ % p, "rb").read():
        os.kill(int(p), getattr(signal, sys.argv[1]))'
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "$PAUSE" SIGSTOP    # the feed stops
# ... three decision periods later ...
kubectl -n omni-compass exec deploy/omni-compass -c watchdog -- python
    ↪  -c "$PAUSE" SIGCONT    # the feed resumes
```

Pass: within three decision periods every `authority` record shows the feed blind; no machine is parked while it is blind; parked machines are woken one per decision; after `SIGCONT` the feed's rows resume on the same clock and the governor reads it again.

**Drill 4: a write refused.** With the admission policies of section 26.4 in place, refuse the governor's next write by binding a policy that denies every node update from its service account for the drill's duration (a copy of `omni-compass-nodes` whose only validation is `false`). Pass: `refused` records appear; no other record changes; the service is untouched; when the drill policy is removed, the next decision's write lands.

**Drill 5: a maintenance cordon.** Cordon one parked machine and one open machine (`kubectl cordon <name>`). Pass: the governor's next decisions neither uncordon nor count either of them; the parked one loses its `omnicompass.io/idle` taint; the OFF switch drill run now leaves both cordoned. Uncordon them afterwards.

**Drill 6: the OFF switch under continuous delivery.** On a cluster whose governor is deployed by Argo CD or Flux, run the OFF switch drill exactly as the runbook writes it (section 27.4). Pass: the governor stays off until the holder turns it on, and the delivery controller does not start it again.

## What the drill book is for

The drills prove three claims the installation makes to the people who run the cluster: that the governor can always be stopped, that its failure never strands a pod, and that every failure is visible in the audit with its cause. The records of the drills, kept with the installation record (section 14.7), are the evidence a reviewer asks for under "how do you know it fails safe?" (section 23.3), and the quarterly repetition of drills 1, 2 and 6 is part of day two (Appendix J).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 29

# A Complete Installation, Start to Finish

The chapters of this part each take one step in depth. This chapter is the whole path at once, on Azure Kubernetes Service, as one engineer would run it: every command in order, what each should print, and the gate each must pass. The same sequence runs on Google Kubernetes Engine and Amazon EKS with the provider commands of chapters 16 and 17 in place of the Azure ones; every Kubernetes command is the same on all three. The example service is a web shop's checkout, `checkout` in the namespace `shop`, with a line of 400 ms at the 95th percentile, written down by its owner. Every value in angle brackets is the installation's own.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Day 0: the copy, the cluster and the service

```
# The copy (phase 0, chapter 32)
git clone &lt;the licensed repository&gt; omni-compass &amp;&amp; cd omni-compass
python3 verify.py                        # ends: VERIFICATION: PASS
python3 tools/omni_version.py            # prints the engine version: r
    ↪ ecord it with the commit
git rev-parse HEAD

# The cluster (phase 1, section 15.2): a system pool of two machines,
    ↪ one work pool under the autoscaler
RG=shop-prod-rg; NAME=shop-prod; LOC=eastus; SIZE=Standard_D4s_v5
az group create -n $RG -l $LOC -o none
az aks create -g $RG -n $NAME -l $LOC --tier standard --node-count 2 -
    ↪ -node-vm-size Standard_D2s_v5 \
  --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoS
    ↪ chedule --nodepool-labels omni-role=system \
  --cluster-autoscaler-profile scale-down-unneeded-time=10m scale-down
    ↪ -delay-after-add=10m scan-interval=10s \
  --generate-ssh-keys -o none
az aks nodepool add -g $RG --cluster-name $NAME -n work0 --mode User -
    ↪ -node-vm-size $SIZE --labels omni-role=work \
  --node-count 3 --enable-cluster-autoscaler --min-count 3 --max-count
    ↪  12 -o none
az aks get-credentials -g $RG -n $NAME
kubectl get nodes -L omni-role               # 2 system, 3 work
```

The system pool has two machines (section 28.1: the governor's pod must always have somewhere to go). The work pool's floor is 3 because the checkout spreads across three zones; its top is 12, the operator's cost ceiling. **Gate:** a test Deployment scaled past the pool and back shows Azure adding and deleting a machine (section 15.2).

The service's own autoscaler is the operator's and stays so: `kubectl -n shop get hpa checkout` shows its target, minimum and maximum, which are written into the installation record (section 14.7).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## Day 1: identity, image and watch

```
# Identity (phase 2, chapter 14): the namespace, the service account,
    ↪ the watch role; the restricted profile
kubectl apply -f deploy/install/omni-compass.yaml
kubectl label namespace omni-compass pod-security.kubernetes.io/enforc
    ↪ e=restricted
for verb in get list; do for r in nodes pods horizontalpodautoscalers
    ↪ deployments; do
  echo "$verb $r: $(kubectl auth can-i $verb $r --as=system:serviceacc
    ↪ ount:omni-compass:omni-compass -A)"; done; done
kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass
    ↪ :omni-compass       # no (not yet granted)

# The images (section 15.4), built from the verified copy and tagged w
    ↪ ith the version and the commit
ACR=shopprodacr; V=&lt;version&gt;-&lt;commit&gt;
az acr create -g $RG -n $ACR --sku Standard -o none &amp;&amp; az aks update -
    ↪ g $RG -n $NAME --attach-acr $ACR -o none
az acr build -r $ACR -t omni-compass:$V -f deploy/Dockerfile .
az acr build -r $ACR -t omni-compass-cloud:$V -f deploy/Dockerfile.clo
    ↪ ud --build-arg BASE=$ACR.azurecr.io/omni-compass:$V .

# Watch (phase 3, section 15.5)
kubectl -n omni-compass set image deployment/omni-compass controller=$
    ↪ ACR.azurecr.io/omni-compass:$V
kubectl -n omni-compass rollout status deployment/omni-compass
kubectl -n omni-compass exec deploy/omni-compass -- tail -n 3 /var/log
    ↪ /omni/audit.jsonl
```

The last command prints `decision` records, one a minute, each with `nodes_observed` and `nodes_recommended`, and no `write` record. **Gate:** at least one full cycle of the checkout's load watched (a week, because the shop's peak is weekly), and the platform owner agrees with what the governor would have done.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Day 2: the feed and the line

The checkout has one representative endpoint, so the probe is the feed (section 18.3). Its URL and the line go into the installation record, signed by the service owner. Before governing, the feed is checked on its own (section 18.5): the probe is run beside the watch Deployment for an hour, and its rows advance every five seconds, its 95th percentile tracks the shop's own monitoring of the same endpoint, and a stopped endpoint shows as failed rows.

At the same time the room is measured (chapter 24): two weeks of the sampling loop of section 24.3, from which the plan records where the room lives (here: the nights and the hours after the weekday peak), the billable room (the shop holds no reservation on this machine family) and the load the paired run will carry: a replay of one recorded day of checkout traffic, compressed no further than the autoscaler's ten-minute profile allows.

## Days 3 to 7: the paired run

The paired run (phase 5, section 15.6) builds two identical clusters for each repetition, one native and one with Omni-Compass on top in the cloud mode, drives the same recorded day through both, and deletes both. Its receipt is read row by row (section 19.6), and before the installation proceeds it must show: the bill and the machine-hours reading GAIN or EVEN; response time, failed requests and alert minutes reading EVEN or GAIN; HPA replicas equal to native's within the noise. A LOSS row stops the installation until its cause is found in the wiring and the run is repeated (section 4.5). The receipt is signed by the service owner and by finance.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## Day 8: hardening

```
# The admission policies (section 26.4) and their proof: eight answers
    ↪ , kept with the record
kubectl apply -f omni-compass-policies.yaml
SA=system:serviceaccount:omni-compass:omni-compass
kubectl --as=$SA label node &lt;work machine&gt; omni-test=1          # refu
    ↪ sed by omni-compass-nodes
# ... the remaining seven commands of section 26.4 ...

# The network policy (section 26.5), with this cluster's API server ad
    ↪ dress and the checkout's labels and port
kubectl get endpoints kubernetes -n default -o jsonpath='{.subsets[0].
    ↪ addresses[0].ip}'
kubectl apply -f omni-compass-netpol.yaml

# The API server's own audit log to the workspace the platform team re
    ↪ ads (section 26.6)
az monitor diagnostic-settings create --name aks-audit --resource $(az
    ↪  aks show -g $RG -n $NAME --query id -o tsv) \
  --workspace &lt;log-analytics-workspace-id&gt; --logs '[{"category":"kube-
    ↪ audit-admin","enabled":true}]'
```

The image's signature rule (section 26.3) is applied by the platform team's existing admission control; the cloud image's digest is the one it allows in the `omni-compass` namespace.

## Day 9: govern, with the dry run first

The governing manifest is the repository's, with this cluster's settings, a persistent volume for the audit, and the probe pointed at the checkout. Kept in the company's configuration repository as an overlay (section 27.3), its patch for this cluster is:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# The audit on a persistent volume (section 20.4) instead of the manif
    ↪ est's emptyDir
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: omni-audit, namespace: omni-compass}
spec: {accessModes: [ReadWriteOnce], storageClassName: managed-csi, re
    ↪ sources: {requests: {storage: 5Gi}}}
---
# patch on Deployment omni-compass (JSON 6902)
- {op: replace, path: /spec/template/spec/volumes/0, value: {name: aud
    ↪ it, persistentVolumeClaim: {claimName: omni-audit}}}
- {op: replace, path: /spec/template/spec/containers/1/env/0/value, va
    ↪ lue: "http://checkout.shop.svc.cluster.local/"}
- {op: replace, path: /spec/template/spec/containers/0/env/1/value, va
    ↪ lue: "3"}     # MIN_NODES, the governor
- {op: replace, path: /spec/template/spec/containers/2/env/1/value, va
    ↪ lue: "3"}     # MIN_NODES, the watchdog
```

and the governor's command line, written whole (section 27.3), carries `--min-nodes 3 --max-nodes 12 --slo-ms 400` and, for the first day, `--dry-run`. Then:

```
kubectl apply -f deploy/install/rbac-machines.yaml              # the
    ↪ machines role (section 14.3), approved in the change record
kubectl -n omni-compass delete deployment omni-compass           # the
    ↪  watch Deployment goes; the governing one replaces it
kubectl apply -k overlays/shop-prod                              # the
    ↪  governing Deployment, with --dry-run
kubectl -n omni-compass get pod -l app=omni-compass              # REA
    ↪ DY 3/3: governor, probe, watchdog
kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass
    ↪ :omni-compass       # yes
```

A day of the dry run's audit is read: every `write` record is marked `"dry_run": true` and asks for a count between 3 and the machines that exist, one from the last (Appendix J, phase 6). Then the change record's approved commit removes `--dry-run` from the overlay, the delivery controller applies it, and the first real write is read when it comes: a `verdict` record with `step allowed`, then a `write` record with `why: node pool size`, then the next decision's `authority` record, whose `proprioception` shows that the command landed. In the cluster, one work machine now carries the `omnicompass.io/idle:PreferNoSchedule` taint.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

**Gate:** the OFF switch drilled by its holder in the form the runbook names (section 20.3, section 27.4 for a GitOps cluster): every parked machine woken and every mark removed within 60 seconds; the drill recorded.

## Day 10 onward: the first month

| When | What is read | Expected |
|---|---|---|
| Every morning, first week | the dashboard of section 22.3 | machines parked overnight and woken before the morning's rise; the two response-time lines agree; no blind decisions |
| Each weekly peak | alert minutes on the checkout's line | none caused by the governor; any alert minute is traced with section 22.6 to its cause |
| End of week one | the drills of section 28.2, numbers 1, 2, 3 and 5 | each passes as written |
| End of month one | the provider's billing records for the work pool (section 22.5) against the same weeks before installation, with requests served beside them | machine-hours lower in the hours the plan found room (chapter 24); higher only where traffic was higher |
| End of month one | the API server's audit against the governor's (section 26.6) | every patch by the service account matched by a `write` record |
| Every quarter | the OFF switch and drills 1, 2 and 6; the engine version against the receipts kept | as Appendix J, phase 7 |

## What the installation record holds at the end

The record of section 14.7, complete: the verification receipt and the engine version and commit; the image digests, the software bill of materials, the scan and the signature; the grant as applied and its proof; the admission policies and their eight answers; the network policy and its proof; the line and its signature; the room and the plan; the paired receipt, signed; the governing command line as it runs; the change records of phases 2 and 6; and the records of every OFF switch drill and every failure drill. An installation whose record is complete can be audited by a person who was not there, which is the standard this manual holds every installation to.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

CHAPTER 30

# Questions an Implementer Asks

The questions below are the ones platform teams ask in the first week, answered from the wiring as it stands. Each answer points to the section that holds the detail.

**Does Omni-Compass replace our cluster autoscaler or our HPA?** No. It sits on top of both and never replaces either (section 1.1). In the cloud mode it moves neither: the HPA keeps the operator's target, and the cluster autoscaler keeps making and deleting machines on its own schedule; the governor parks and wakes machines between them (chapter 19).

**Can it make our bill bigger?** In the cloud mode, no: it moves no pod autoscaler, so the pods are native's; it never asks the cloud for a machine, so the machines are native's or fewer. The bill can only fall or stay (section 19.1). Outside the cloud mode, on machines already paid for, the max may spend resource to buy work; that spending is shown as power given beside the work it bought, and it is never used in a cloud.

**Can it wake our on-call engineer?** It is built so that it should not: it acts on the reading as it climbs toward the line, a breach wakes a parked machine at the next decision, and nothing is given back while the line is at risk. The receipt counts alert minutes in both arms (section 19.6), and the aim in a cloud is none on the omni side; an alert minute there is a fault to find, never a cost to accept.

**How fast does it act?** Its fast checks run every 5 seconds (a machine for pods that wait, a parked machine woken), three times as fast as the HPA's own 15-second check; its full decision runs every 60 seconds, the period it was built and tested at (section 3.4). Native's own checks run underneath, untouched, whatever the governor's period.

**What happens if the governor crashes?** Nothing harmful. The watchdog puts every setting back from the records on the objects themselves (section 20.2); a parked machine never blocks a pod even before that, because its taint is `PreferNoSchedule`; and the native controllers carry the service as they did before the governor was installed.

**Does it need cloud credentials?** Not in the cloud mode (section 14.5). It acts through the Kubernetes API with a service account that can read the cluster and patch nodes and pods, and nothing else.

**Does it work with Karpenter, KEDA, the Vertical Pod Autoscaler or a descheduler?** Karpenter: yes, in place of the cluster autoscaler; consolidation removes a parked machine once it is empty (section 17.2). KEDA: yes; KEDA drives the HPA, and in the cloud mode the governor does not touch the HPA at all. The

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

Vertical Pod Autoscaler: yes, if it does not also act on the nodes; it changes requests, which the governor reads. A descheduler that uncordons or rebalances nodes is a second writer of the setting the governor moves, and must be configured to leave tainted nodes alone (section 20.8).

**Can two governors run on one cluster for high availability?** No. One writer per setting is a rule (section 5.2), and the watchdog, not a second governor, is the availability story (section 20.2).

**How much does the governor itself cost?** One pod: a request of 50 millicores and 128 MiB, a limit of 500 millicores and 256 MiB, plus a probe container of 10 millicores and 32 MiB. Every receipt reports the governor's own processor use (the row "Omni's own CPU (cores), mean").

**What does it log, and is any of it sensitive?** Object names, counts, response-time statistics, its decisions and its reasons (section 20.4). No request payload and no personal data, which it never sees.

**Can we tune it?** The factory tune is the engine as shipped (section 3.1); an installation writes the settings of section 3.4 (the line, the floor, the dial, the ceiling of a growing lever) from its own system. A changed tune is the installation's own configuration, and its receipts are its own (section 3.6).

**How do we know it is working?** The audit shows every decision and every write with its reason; the receipt of a paired run on the company's own cluster shows what it did against native, row by row, read GAIN, EVEN or LOSS (chapters 10 and 11). An installation whose receipt it has not seen is not yet an installation.

**What if our receipt shows a LOSS?** Suspect the wiring first (section 4.5): the feed, the line, the grant, the autoscaler profile, a second writer. Fix it and run the receipt again. A LOSS is never accepted and never cut from the receipt.

**What do we need to keep for an audit by our own risk team?** The installation record of section 14.7: the engine version and verification receipt, the grant and its proof, the governor's command line, the signed line, the audit, and every drill of the OFF switch.

**Can we run it on our own machines, not a cloud?** Yes: chapter 7 wires every stack the governor knows, from a cluster on the company's own racks to a data center's cooling and power. There, the machines are already paid for, and the max (the most work from the same machines) is the default.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

PART SEVEN

# License and Your Copy

*The terms the software and this manual are licensed under, and how to verify a copy before it is installed.*

CHAPTER 31

## License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (`LICENSE`): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in `.github/CONTRIBUTING.md`, which assign their rights to The Omni-Compass LLC.

The copyright in Omni-Compass belongs to The Omni-Compass LLC. The source is visible under the Evaluation License, which is not an open-source license; seeing the source grants no right beyond it. The owner of a copyright may grant it under more than one license at once (the Evaluation License to everyone, a paid Enterprise License to a customer), may set the terms of each new version, and may assign the copyright; copies already received keep the terms they were received under. The license texts (`LICENSE`, `NOTICE`, `DISCLOSURES.md`) govern.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

CHAPTER 32

# Verifying Your Copy

Before a copy is installed anywhere, and again after every upgrade, it is checked in three ways, and an installation proceeds only when all three pass:

1. **The whole check.** `python3 verify.py` runs every test and every fingerprint check and ends with `VERIFICATION: PASS` or `FAIL`. A copy that fails it is not installed.
2. **The engine version.** `python3 tools/omni_version.py` prints the engine version the copy carries, or every file that differs from it. Receipts made on one version are never read against another (chapter 12).
3. **The seal and the manifest.** `python3 tools/seal.py --check` confirms that every sealed file is the file that was sealed, and the release manifest (`RELEASE_MANIFEST.json`) names the commit, the engine and the verification receipt the copy was released with. A copy whose files do not match its manifest is not the release it claims to be.

A reader holding any one of the three can tell whether the files in front of them are the files a receipt was made on. Every copy, export or printout carries the legal notice at its head and foot, with `LICENSE`, `NOTICE` and `DISCLOSURES.md` beside it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

# Back Matter

BACK MATTER

## Glossary

| Term | Meaning |
|---|---|
| Authority | how far the governor lets a muscle move this decision; adding is always allowed, giving back needs calm and a clean service record (chapter 5) |
| Band | a reading's or a knob's safe range |
| Card | a muscle's own settings, found for it and kept only where it was never behind native; the factory tune for that muscle (section 3.3) |
| Dial | max (the default) or save (the tune-down), the same switch on every governor (section 3.2) |
| Factory tune | the engine as The Omni-Compass LLC tuned and ships it: every muscle on its own card or the engine's setting (section 3.1) |
| Line | the limit the service must meet, from the operator's own objectives (`--slo-ms`) |
| Engine version | the engine as fingerprinted file by file; a receipt belongs to the version its commit carries (chapter 12) |
| Compass | the band seen as a position from 0 (calm) to 1 (the line), with a 5% cushion at each edge and the centre where the reading is held (section 3.4) |
| EVEN | Equalized, Verified baseline, Equilibrium, Neutral: native and Omni could not be told apart (the same, or no difference beyond the noise); the key, front matter |
| GAIN | Growth, Advantage, Improvement, Net-positive: a gauge moved the way its declared aim asks, against native; a resource cut is a GAIN; the key, front matter |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| GAIN in all three of our runs | the same sign in all three runs (A, B, C), every 95% interval clear of zero, and that sign a GAIN for the gauge's aim |
| LOSS | Leakage, Off-track, Setback, Subpar: a gauge moved against its declared aim (less work, a longer wait, more misses); it should never appear, and where it does it is shown, its cause found and fixed; the key, front matter |
| Power given | a resource spent or held that rose: what Omni put in to buy the work and speed beside it; shown, never called a loss |
| Cover | a knob's hard range; every write is clipped to it |
| Cushion | the 5% at each edge of the compass, 10% in all |
| DEX-YLL-1 | the company's number, 1-888-DEX-YLL-1, read in order as the engine runs: DEX, Data Exchange (the input, the two-way plug); YLL, Yield Limit Line (the boundary: every muscle up to its own yield limit, never past it); 1, one closed loop (the output; in code 1 is true, in science 1 is unity) |
| Fail up | the return to full capacity the moment service crosses the wall or a sense goes blind |
| Governor | the process that reads, decides and writes; Omni-Compass |
| Muscle | any machine, service or controller Omni-Compass reads and sets |
| Native | the system as it runs without Omni-Compass |
| No difference beyond the noise | an interval over zero in at least one run: native and omni could not be told apart; the result, stated as such |
| Plug | the two-way connection to one muscle: read, write, read back, restore |
| Pool (Azure) | one machine family under Azure's autoscaler with its own ceiling; several pools make the fleet |
| Profile | service (the default) or batch: what the pedals do with a pile of work (section 3.5) |
| Race | full speed at once while work waits, so a burst is never served slowly |
| Yield limit line | a muscle's own limit, read live: Omni-Compass takes it right up to the line and never past it, as an engineer loads a material up to its yield limit, the most it carries and still springs back unharmed |
| Speed floor | the clock a card reaches on its own while busy, learned from its own meter; the governor never sets the ceiling under it |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Receipt | the paired record of native against Omni-Compass for one run |
| Snapshot | a knob's value read once before the first write; the restore point |
| The runs disagree | runs clear of the noise point different ways; the test is unstable on that measure and is looked into |
| Wire check | the test that proves every wire follows, reads back and returns before anything runs |
| Work per energy | work done divided by energy used |
| Work inside the line | the requests, transactions or messages a second answered within the response line; the product number on every real stack |
| One pedal | push for more, lift off for less, idle (the pedal lifted all the way), reset (everything back to native): how the governor moves capacity under autopilot |
| Cruise | every machine in service while work waits for a place, held without second-guessing until the line has been empty for two decisions |
| Idle at once | the work done and demand at zero: the pedal lifted all the way in one move, every safety check still holding |
| Kill switch | security only: one switch in a human hand that turns every governor on the machine off at once; never the reset |
| Lease | the record every governor renews each decision; a lease older than its limit, or a dead process, makes the watchdog hand back for it |
| Watchdog | the service beside the governors that hands back for any governor that died or hung without doing it itself |
| Verdict, paired trial | the trial on the muscle itself that allows a slow knob to move only where the cost is within 2%; "left native" where no step passes (section 1.2) |
| Tuning case | the one workload or case the rules were fixed on; shown in every receipt and never counted |
| Untouched case | a workload or case the rules were applied to unchanged; the judged rows |
| Amendment | a change to the written rules of a paired run after they were written, with its time and reason; one made after a result was seen says so |
| Dwell | the seconds after a knob moves during which the law will not move it back |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Notch | the smallest step a knob moves in one decision (one consumer, 8 MB of cache, one tap) |
| Evidence class P, L, S | a physical meter; live software with no meter; a model (chapter 11) |
| Handed back | the knob read back at the operator's value at the end of an arm; every receipt says whether every arm was |
| Working set | the records a workload actually touches over a window; a cache benchmark measures nothing unless the working set reaches the cache |
| Full-cache gate | the MongoDB and Redis rule: slow reads grow the cache only while the cache is full (nine tenths used), because a miss in a cache with room to spare is not the cache's to mend |
| Buffer pool | InnoDB's cache of table and index pages, the one memory setting every MySQL operator sizes; resized online by the server in whole chunks |
| Chunk | the unit in which MySQL resizes its buffer pool (128 MB as shipped); the knob's notch on that stack |
| Page cache | the operating system's own cache of file contents; where the data fits in it, a storage-engine miss is a read from memory and a decompression, not a disk read, and the gain available to the cache knob is smaller |
| Cloud mode | the governor on top of a public cloud's cluster at the tune-down, moving machines alone: no pod autoscaler moved, no processor conveyed, no power capped, no machine asked for; the bill can only fall or stay (chapter 19) |
| Parked machine | a machine the governor has closed to new work with the taint `omnicompass.io/idle:PreferNoSchedule`; it stays Ready and usable, its pods are marked first to go, and the cloud's autoscaler deletes it once it is empty |
| Taint, `PreferNoSchedule` | a mark on a machine that the scheduler treats as a preference against placing new pods there, never as a refusal; the only taint the governor writes |
| Cordon | a machine marked unschedulable (`kubectl cordon`), by a person, an upgrade or a repair; the governor never cordons or uncordons a machine, and leaves a cordoned machine alone |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Term | Meaning |
|---|---|
| Pod deletion cost | the annotation `controller.kubernetes.io/pod-deletion-cost`, by which a ReplicaSet removes the lowest-cost pods first when it scales down; the governor marks a parked machine's pods with it so that they leave first |
| Cluster autoscaler | the cloud's own controller that adds a machine when pods cannot be placed and deletes a machine that has been unneeded for its configured time; native, never changed by the governor (Karpenter plays the same part on Amazon) |
| HorizontalPodAutoscaler (HPA) | the Kubernetes controller that sets a service's replica count from its pods' measured use against a target; native, and in the cloud mode never moved |
| Room | the machine-hours a pool holds above what its pods' requests need; the upper bound on any machine lever's saving, measured before any run (section 24.2) |
| Billable room | the room above the machines a commitment already pays for: the part of the room that can reach the bill (section 24.2) |
| Feed | the file of rows `elapsed_seconds,latency_ms,ok` the governor reads its response times from: the probe's, or an adapter's (chapters 18 and 22) |
| Blind | a sense with no fresh reading (the feed silent for twice its window, or no served request in it); the governor gives nothing back while any sense is blind |
| Lease | the record each governor renews every decision; a lease out of date, or a process gone, makes the watchdog hand back for it (section 20.2) |
| Watchdog | the process that hands every setting back for a governor that died or hung; in a cluster, a container in the governor's own pod (section 20.2) |
| Proprioception | the governor's reading of whether its last command landed: the drift between the machines it commanded and the machines in service, in each `authority` record |
| Dry run | `--dry-run`: the governor decides and records every command it would run, marked `"dry_run": true`, and writes none |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Term | Meaning |
|---|---|
| Admission policy | a rule the API server applies to every write after the role allows it; section 26.4 binds the governor's writes to its own taint and its own pod mark |
| System pool | the pool of machines that carries the cluster's add-ons and the governor, tainted so no workload lands there, never governed |
| Work pool | a pool labelled `omni-role=work`, under the cloud's autoscaler, whose machines the governor may park |
| OFF switch of record | the form of the OFF switch the runbook names for a cluster: scaling the governor to zero, or a commit to Git where continuous delivery reconciles it (sections 20.3 and 27.4) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix A. Command Reference

| Task | Command | | | |
|---|---|---|---|---|
| Verify the copy | `python3 verify.py` (ends `VERIFICATION: PASS`) | | | |
| Which engine version a checkout or a commit carries | `python3 tools/omni_version.py [--commit <sha>]` | | | |
| Seal check | `python3 tools/seal.py --check` | | | |
| Evaluate without touching anything: the organisms, modelled | `python3 tools/run_scale.py --runs N --scale K --out <dir>` | | | |
| Evaluate without touching anything: the two-wire card, modelled | `python3 tools/run_gpu_card.py <dir>` | | | |
| Kubernetes, watch (read-only) | `kubectl apply -f deploy/install/omni-compass.yaml` (it runs `--mode observe`) | | | |
| Kubernetes, govern | `python3 -m omni_controller.controller --mode target | nodepool --latency-file <feed> --slo-ms <line> [--law compass] [--dial max | save ]` | | |
| A response-time feed against any HTTP endpoint | `python3 scripts/latency_probe.py <url> <out.csv>` | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| GPU wire check | `sudo python3 tools/gpu_wire_check.py --gpu 0` (must end `WIRED RIGHT`) | | | |
| GPU governor, two wires | `sudo python3 -m omni_controller.gpu_compass --mode watch | cap --gpus 0 --latency-file <feed> --slo-ms <line> [--dial max | save ]` | |
| GPU, the whole paired run in one command | `sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &` | | | |
| The processor's clock under its meter, the paired run | `sudo bash scripts/cpu_power_run.sh` | | | |
| Kubernetes paired runs, then the receipt | `scripts/kind_paired.sh`, then `python3 tools/live_reps.py <dir>` | | | |
| The three-run table (Kubernetes, cloud) | `python3 tools/confirm_abc.py "<title>" <run A dir> <run B dir> <run C dir> --out <table.md>` | | | |
| The three-run table (pooler, broker, cache, storage engine, buffer pool) | `tools/pgbench_abc.py`, `tools/kafka_abc.py`, `tools/redis_abc.py`, `tools/ycsb_abc.py`, `tools/sysbench_abc.py`, the same arguments | | | |
| OFF, one muscle (Kubernetes) | `touch /tmp/omni.kill` | | | |
| OFF, one card | `sudo touch /tmp/omni-gpu-kill` | | | |
| The whole harness OFF, ON, status, watchdog | `python3 tools/omni_switch.py off | on | status | watchdog ` | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| Cloud mode: the machines role, then the governing Deployment (dry run first) | `kubectl apply -f deploy/install/rbac-machines.yaml`, then `kubectl apply -f deploy/install/omni-compass-cloud.yaml` (chapter 19) | | | |
| Cloud mode: the cloud image | `docker build -f deploy/Dockerfile.cloud --build-arg BASE=<registry>/omni-compass:<version> -t <registry>/omni-compass-cloud:<version> .` | | | |
| Cloud mode: OFF, the switch of record | `kubectl -n omni-compass scale deployment/omni-compass --replicas=0` (section 20.3) | | | |
| Cloud mode: OFF from inside the pod | `kubectl -n omni-compass exec deploy/omni-compass -c controller -- touch /tmp/omni.kill` | | | |
| Cloud mode: the grant, as the governor sees it | `kubectl auth can-i <verb> <resource> --as=system:serviceaccount:omni-compass:omni-compass -A` (section 14.4) | | | |
| Cloud mode: machines parked now | `kubectl get nodes -l omni-role=work -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints[*].key` | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Command | | | |
|---|---|---|---|---|
| Cloud mode: the last decisions | `kubectl -n omni-compass exec deploy/omni-compass -c controller -- tail -n 20 /var/log/omni/audit.jsonl` | | | |
| Planning: the room on a cluster | the sampling loop and the arithmetic of section 24.3 | | | |
| A feed from the company's own Prometheus | `python3 tools/prom_feed.py --prometheus <url> --served <q> --failed <q> --quantile <q> --out <feed>` (section 22.7) | | | |
| The paired run on a cloud, from GitHub | the `aks-metered`, `gke-metered` or `eks-metered` workflow (Appendix N) | | | |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

# Appendix B. The Files an Installation Touches

| Path | What it is for |
|---|---|
| `verify.py`, `tools/omni_version.py`, `tools/seal.py`, `RELEASE_MANIFEST.json` | checking the copy before it is installed (chapter 32) |
| `deploy/Dockerfile` | the container image: non-root, read-only root filesystem, no Linux capabilities |
| `deploy/install/omni-compass.yaml` | the watch-mode install with a read-only identity (level 1) |
| `deploy/install/rbac-machines.yaml` | the cloud mode's machines role: patch nodes and patch pods, nothing else (section 14.3) |
| `deploy/install/omni-compass-cloud.yaml` | the cloud mode's governing Deployment: the governor, its probe and its watchdog in one pod, shipped with `--dry-run` (chapter 19, Appendix I) |
| `deploy/Dockerfile.cloud` | the cloud image: the shipped image with `jq`, the machine script, the probe, the Prometheus feed adapter and the switch tool |
| `scripts/kind_nodepool.sh` | the machine script: keeps `{n}` machines in service by the idle taint alone; never creates, deletes, cordons or uncordons a machine, and never moves a pod |
| `tools/prom_feed.py` | the response-time feed from the company's own Prometheus (section 22.7) |
| `deploy/rbac-target.yaml`, `deploy/kind/rbac-omni.yaml` | the permissions of levels 2 and 3, exactly the verbs each level needs |
| `scripts/latency_probe.py` | the response-time feed against any HTTP endpoint |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Path | What it is for |
|---|---|
| `scripts/kind_paired.sh`, `tools/live_reps.py`, `tools/confirm_abc.py` | the paired runs, the receipt and the three-run table on Kubernetes |
| `tools/gpu_wire_check.py`, `scripts/gpu_rented_run.sh`, `tools/gpu_workload.py` | the card's wire check, its paired run, and a workload that writes the card's own time per request |
| `scripts/cpu_power_run.sh` | the processor's paired run on bare metal |
| `tools/omni_switch.py` | the master switch, the status and the watchdog (chapter 8) |
| `realms/catalog_v5.csv`, `realms/muscle_cards_v5.csv`, `docs/MUSCLE_CATALOG.md` | every muscle, its wiring and its own card (Appendix E) |
| `docs/AZURE_SETUP.md`, `docs/GOOGLE_SETUP.md`, `docs/AMAZON_SETUP.md` | each cloud's account and its secrets, click by click (Appendices F to H) |
| `.github/workflows/aks-metered.yml`, `gke-metered.yml`, `eks-metered.yml`, `aks-metered-full.yml` | the paired runs on each cloud, from GitHub (Appendix N) |
| `LICENSE`, `NOTICE`, `DISCLOSURES.md`, `SECURITY.md` | the license, the notices, every declaration and disclaimer, and how to report a security issue |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix C. The Gauges

Every gauge in a receipt, where it comes from, and whether it is measured or modelled:

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| work inside the response line (requests, transactions or messages a second answered within the line) | the probe's own records, pgbench's log, the consumers' records, the application's records | measured | higher; the product number |
| response time, mean, median, 95th and 99th percentile | the same | measured | lower |
| time over the response line (share of samples) | the same, against the written line | measured | lower |
| failed requests, lost messages | the same | measured | lower; any increase is marked |
| alert minutes: whole minutes whose 95th percentile, failed requests counted over, was over the line | the probe's own records, minute by minute | measured | lower: the minutes an on-call page would be open |
| settings commands the governor wrote on its own | the governor's audit | measured | shown, not judged (no person touched either arm) |
| the governor's additions decided while the service was still inside its line (% of its additions) | the governor's audit | measured | shown, not judged |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| worker machines in service, mean; node-hours | the cluster's own node records every 15 s | measured (machine-hours on a local cluster; the provider's own count in a cloud) | lower; a rise is power given |
| compute bill at list price | the provider's own machine count × list price per family | measured (a real bill) | lower; a rise is power given |
| energy (Wh), the standby model | a declared formula over the machines in service | **modelled**, said so on every line | lower; a rise is power given |
| energy, the machine's own meter | the processor's RAPL counters, a card's own power reading, a site meter | measured (class P) | lower; a rise is power given |
| host CPU busy; host CPU-seconds; CPU-seconds per 1,000 units inside the line | `/proc/stat` of the real machine under the run | measured | lower |
| Omni's own CPU (cores), mean | the governor's audit (its process and every command it ran) | measured | shown beside the receipt |
| connections held open, consumers running, memory ceiling held, cache size held | the pooler's, the group's, the cache's and the server's own reports | measured | lower; a rise is power given |
| cache hit rate; consumer lag | Redis's INFO; the consumers' records | measured | higher; lower |
| pages read into a cache, the cache's miss share | the server's own counters | measured | shown, not judged |
| HPA replicas, pods started, pod start wait | the API server's own records | measured | shown, not judged except pod start wait (lower) |
| seconds from a kill to every setting back at the operator's | the harness polling the system once a second from the kill | measured | lower; past 60 s marked; never, INVALID |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Gauge | Source | Measured or modelled | Its aim |
|---|---|---|---|
| handed back (every arm) | the knob read back at the end of the arm | measured | yes in every arm, or the run is invalid |

**Three rules for reading any gauge in this appendix.** A measured gauge is one the system under test, or the host's own kernel, reported about itself; a modelled gauge is one computed from a declared formula over measured inputs, and every receipt says which it is on the line itself. A resource gauge (machines, connections, consumers, memory, cache) is always read against an aim of lower, even where the governor's purpose was to add the resource to buy service, so that a gain bought with a resource is shown as power given in the same receipt. And a gauge marked "shown, not judged" never enters a reading; it is there so that a reader can see what the governor did (how often it moved, how many pages it read, how many pods it started), not to be counted for or against it.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix D. Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Wire check `FAIL 3 up wire (lock)` | the driver or VM refuses clock locking | use bare metal or full passthrough; or run the one-wire governor |
| Wire check `FAIL 3 up wire (follows down)` | the card ignores the ceiling | update the driver; check for another management agent holding clocks |
| Wire check `FAIL 4 down wire` | the power limit is refused or capped by the board | check `nvidia-smi -q -d POWER`; run as root |
| `another copy of this test is already running` | a second copy was started | wait for the first or reboot; start once |
| `something else is using the GPU` | another process holds the card | stop it; the paired run must run alone |
| Governor exit 5 | another writer changed a knob | find the other controller; Omni-Compass left its value alone (section 5.2) |
| Governor exit 3 | a restore did not read back | restore by hand (`nvidia-smi -rgc`, `-pl <start>`); investigate before running again |
| `decision failed (n in a row)` | the cluster API is unreachable | turn it OFF; native runs on |
| Energy saved but p95 slower than native | the card served bursts below its own clock, or the lid sat under its own draw | read section 4.5; in the audit compare `telemetry.clock_mhz` while busy with `native_busy_clock_mhz`, and `want_w` with `native_busy_draw_w` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| Fail-up in most decisions | `--slo-ms` set too low for the workload, or a stale feed read as blind | set `--slo-ms` from the workload's own target; check the feed's age (section 5.3) |
| No saving at all, service unchanged | the card is saturated almost all the time (it races), or the dial is at the max | expected: the max spends where it buys work; the tune-down (section 3.2) saves in the quiet stretches |
| Kubernetes: no machine ever given back | the dial is at the max (no machine below the cluster's own is given back), or the release gate refuses (its reason is in the audit) | expected at the max; at the tune-down read `node_gate.reason` in the audit and check `--slo-ms` and the latency feed |
| A pooler's clock reads calm while users wait | the pooler cannot see a client waiting behind its own schedule | wire the queue share beside the service time and read the larger (section 7.2) |
| Broker: consumers climb to the cover and stay | messages wait at every step (the producer is past the group's capacity) | check the offered rate against the group's measured capacity in the run's log |
| Cache: the ceiling grows while little memory is used | cold misses read as pressure | the full-cache gate is the rule (used ≥ 90% of the ceiling); check `used_mb` beside `maxmemory_mb` in the run's log |
| Cache: the ceiling never grows | the working set fits the operator's ceiling | expected: a cache that fits has nothing for the ceiling to mend, and the row reads inside the noise |
| Storage-engine cache: the cache never fills, whatever the load, and omni reads as a free memory saving | the requests concentrate on a few hot records, or the run asks for records the load did not write, so the reads are served from the index | a result that flatters the governor is the first thing to suspect: check the bytes in the cache against its size; draw requests over the whole key space and use the same key order in load and run; a knob with nothing to do is not a saving |
| **Symptom** | **Cause** | **Fix** |
| Storage-engine cache or buffer pool: the governor only ever reads calm | the line is set far above the stack's own latencies | set the line on the tuning case's own figures so that a hit is calm and a full-cache miss is slow, and |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Symptom | Cause | Fix |
|---|---|---|
| | | write the figures down before any counted run |
| Buffer pool: not handed back at the end of an arm | the server was still carrying out a shrink and ignored the restore | the plug waits for the resize and writes again (section 7.8); check the restore record in the arm's receipt |
| A three-run table heads itself with a warning | a run's commit is not the engine version it claims, or the three are not separate runs | run `tools/omni_version.py --commit <sha>` on each; make the table only from runs on one engine version |

| Cloud: the governor's pod shows `2/3` or restarts its watchdog | the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp` | apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 20.2) |
|---|---|---|
| Cloud: parked machines are woken every few seconds, then parked again | the watchdog runs in a separate pod, or a second governor runs on the cluster | one governor, its watchdog in its own pod (section 20.2); `kubectl get pods -A -l app=omni-compass` must list exactly one pod |
| Cloud: the pod is refused at admission with a Pod Security message | the namespace enforces a profile the pod does not meet, or a later edit loosened the pod | the shipped pod meets `restricted` (section 26.3); compare the applied pod with Appendix I |
| OpenShift: the pod is refused with `unable to validate against any security context constraint` | `restricted-v2` refuses the fixed user 10001 | grant `nonroot-v2` to the service account (section 25.3) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| **Cloud: the governor's pod shows `2/3` or restarts its watchdog** | **the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp`** | **apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 20.2)** |
|---|---|---|
| Cloud: every decision shows the feed blind | the probe cannot reach the service (its URL, a network policy, the service's port), or the adapter's queries return nothing | `kubectl -n omni-compass logs deploy/omni-compass -c probe` (or `-c feed`); test the URL from the pod; check the network policy (section 26.5) and, for the adapter, run each query in the metrics system's own console |
| Cloud: `trial abandoned (too little work)` in most `verdict` records | the feed is thinner than twenty rows a minute, or the service is idle | densify the feed (section 22.1); at an idle service, nothing is given back on trial, by design |
| Cloud: `step refused: the muscle is slower there` in most `verdict` records | the service is slower without the machine | the governor is right to keep it; the pool has no room at this load (section 24.2) |
| Cloud: the governor parks nothing for days and its trials are never begun | the pool already holds only what the pods' requests need with the governor's spare, or the floor is the pool | read N[gov] from the room sampler (section 24.3); EVEN on the bill is then the honest receipt |
| Cloud: `refused` records with code 1 on every park | the machines role is not bound, or an admission policy refuses the taint | `kubectl auth can-i patch nodes --as=system:serviceaccount:omni-compass:omni-compass`; the policy's message names the rule (section 26.4) |
| Cloud: a parked machine is never deleted though its pods are gone | the cluster autoscaler's unneeded time is long, a DaemonSet or a pod it will not move remains, or a PodDisruptionBudget allows no disruption | read the autoscaler's own status (its config map or events, `kubectl -n kube-system describe configmap cluster-autoscaler-status`) for the machine's reason (section 24.4) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Cloud: the governor's pod shows `2/3` or restarts its watchdog | the pod was applied without `shareProcessNamespace: true`, or the watchdog lacks the shared `/tmp` | apply the manifest as shipped (Appendix I); a watchdog that cannot see the governor's process judges it dead and hands back over it (section 20.2) |
|---|---|---|
| Cloud: the audit is empty after a restart | the audit was on the pod's `emptyDir` | mount a persistent volume at `/var/log/omni` and ship the file (section 20.4) |
| Cloud: the governor starts again minutes after it was scaled to zero | a continuous-delivery controller restored Git's replica count | make the OFF switch a Git change, or tell the controller to ignore the replica count (section 27.4) |
| Cloud: HPA replicas differ from native's in a cloud receipt | an autoscaler was moved, which the cloud mode never does | check the command line for `--sensed none/none` and the absence of `--cap-deployments` and `--power-cmd` (section 19.3); a run with them is not in the cloud mode |
| Cloud: the cloud image build fails at `FROM` | the base image argument was not given | pass `--build-arg BASE=<registry>/omni-compass:<version>` (section 19.4) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

APPENDIX

# Appendix E. The Muscles: What Each Is For, and How It Is Wired

A **muscle** is one setting on one machine that already has its own control: a replica target, a node pool's size, a GPU's clock ceiling, a chiller's setpoint, a battery's reserve, a joint's effort. Omni-Compass does not replace that control. It reads the machine's meters and, at its default, the max, moves the setting the machine already accepts toward the machine's own limit for the most work, never past it and never below native, through a plug that reads the setting once before the first write, reads back every write, steps aside if another controller moves it, and walks it back to native at the end. A knob native already runs at the top of its range is at its limit on native and is left there.

This chapter lists all 1107 muscles of the catalog (`realms/catalog_v5.csv`, engine omni-v5). For each one it says what kind of machine it is, which of the four kinds of knob it is, what Omni-Compass reads and does with it, how such a muscle is reached in a real stack, and which organisms it belongs to. The plants behind the benchmark numbers are models of these machines (evidence class **S**); a muscle in this list is wired on a real system only through the levels and the checks of the manual (chapters 4 and 6). See `DISCLOSURES.md`.

## How to read an entry

**The four kinds of knob.**

- **capacity**: how much of the machine is in service (replicas, instances, units, speed). At the max Omni-Compass pushes it to the machine's own limit for the most work, never past it and never below native.
- **admission**: what is let in (queues, rate limits, admission control). Native already lets everything in: the knob is at its limit on native, and Omni-Compass reads it and leaves it native.
- **power**: the power share or limit the machine may draw. Native already runs it at full: at its limit on native, read and left native.
- **setpoint**: the target the machine's own loop holds (a temperature, a reserve, a process value). At the max a room is cooled ahead of the heat and a battery's charge is held for its line, inside the safe band, never behind native on its line.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

**The five kinds of plant.** Every muscle is modelled on one of five plants, each with its own service reading and lever:

| Plant | What it is | What Omni-Compass reads | What Omni-Compass moves |
|---|---|---|---|
| `compute_pool` | a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs | how far its queue and its load sit toward the service line (queue or lateness, and load above half) | the capacity it may use: its replica or instance count (through its autoscaler's target or floor), its power share, or one unit given back through the release gate |
| `thermal_zone` | a data hall or building zone cooled by chillers, air handlers or fans | its temperature against its limit | its supply setpoint (colder is more cooling), its power share, or one cooling unit given back when the room has margin |
| `energy_storage` | a battery, UPS or microgrid store with a load and a grid connection | its draw on the grid connection and its reserve | its reserve setpoint (how much charge it holds back); its power limit is left native |
| `motion_axis` | a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor | its tracking error and its load | its speed and effort share, or its power share |
| `process_loop` | a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint | how far the process sits from its band | its setpoint within its safe band, its actuator range, or its power share |

**Every muscle its own card.** The brain is one, its settings each muscle's own. A muscle's card holds the settings the max uses for that muscle (a machine pool: how full it runs on its forecast before it calls for more, and how many decisions of history it reads; a battery: how many wobbles of margin it keeps under its line, and, wired by its power, how long it reads its line before it holds and how much of it it keeps clear). The cards were found by an automatic search over every muscle on the development seeds, today's global setting always one of the choices, and kept only where the card beat the global setting beyond the noise on two separate sets of seeds and on no seed under any load fell behind native: never over its line more often, never less work, never its line cleared later (`tools/tune_cards.py`, `realms/muscle_cards_v5.csv`, `results/cards_v5/`). Inside a body, a battery wired by its power keeps the body's own margins on its line. 364 muscles carry a card of their own; every other muscle runs the global setting (machines started at 95% full, the last 20 decisions read, 3.09 wobbles of margin on a battery's line, 5% of it kept clear). Each muscle's card is in the last column of its family's table.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**The organisms.** The 1107 muscles build six organisms: each of the four realms (every muscle whose realm list includes it), the four stacked with every duplicate kept (1,908), and the whole tower with every muscle once (1107). A muscle of the shared spine sits in all four realms.

| Organism | Muscles |
|---|---|
| Compute / AI / Cloud | 540 |
| Physics / Robotics / Autonomous | 404 |
| Energy / Facility / Industrial | 480 |
| Distribution / Specialized | 484 |
| The four stacked, every duplicate kept | 1,908 |
| The whole tower, every muscle once | 1107 |

**How a muscle is reached in a real stack.** The class of machine decides the wire:

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `batch` | batch and queued jobs | the job queue (Kubernetes Jobs, Slurm, Spark) |
| `batch_reactor` | a GMP batch reactor, fermenter or food process | the batch control system (ISA-88, OPC UA) |
| `building` | a building zone | the building management system |
| `chamber` | a controlled chamber (clean room, kiln, reactor) | the PLC or DCS |
| `clinical` | clinical systems: imaging archives, records, interface engines, monitoring gateways | the application's own scaling and admission settings (Kubernetes, the vendor console) |
| `commerce` | a customer-facing service with bursts | the Kubernetes API |
| `cpu_host` | a CPU host whose clock and power can be set | Linux cpufreq and RAPL |
| `data_hall` | a data hall cooling plant | the building management system (BACnet, Modbus) or its command line |
| `database` | a database cluster | the database operator (replicas, connection pools) |
| `district_heat` | a district heating or cooling network | the network control system and substation controllers (Modbus, M-Bus) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `elevator_hoist` | an elevator or escalator drive and its group control | the lift controller and destination dispatch (modelled only) |
| `ev_traction` | an electric vehicle traction motor | the vehicle's motor controller (CAN; modelled only) |
| `fabric` | a high-speed fabric (RDMA, DPU, SmartNIC) | the fabric manager's API |
| `facility` | a facility battery behind the meter | the energy management system |
| `feeder_voltage` | a distribution feeder's voltage | the distribution management system (IEC 61850, DNP3; modelled only) |
| `flight_axis` | a flight-control axis | the flight controller (modelled only) |
| `gpu` | GPU serving with a response-time target | nvidia-smi (clock ceiling and power limit) or the serving autoscaler |
| `gpu_batch` | GPU training and batch work | nvidia-smi, the job scheduler's pause and resume |
| `hospital` | a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites) | the hospital's building management system (BACnet) |
| `inverter` | a plant of grid-support inverters or wind turbines | the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547) |
| `irrigation` | pumps, pressure and climate on a farm or in a greenhouse | the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud) |
| `marine_propulsion` | a ship's propulsion shaft and power plant | the vessel automation and power management system (modelled only) |
| `microgrid` | a microgrid battery and loads | the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR) |
| `mill` | a grinding, flotation or materials-handling circuit | the plant DCS or PLC (OPC UA) |
| `network` | network routing and switching | the network controller (SDN, routing API) |
| `node` | a fleet of machines or VMs that boot in minutes | the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Class of machine | What it is | The wire in a real stack |
|---|---|---|
| `pipeline` | a pipeline segment with its compressors, pumps and valves | pipeline SCADA (DNP3, Modbus, OPC UA) |
| `port` | a container terminal's cranes and vehicles taking moves | the terminal operating system and fleet controllers |
| `process` | an industrial process loop | the PLC or DCS (OPC UA, EtherNet/IP, PROFINET) |
| `qpu` | a quantum or specialised accelerator queue | the accelerator's job queue |
| `rail_traction` | a train's traction and auxiliary systems | the train control and management system and ATO (modelled only) |
| `ran` | a radio access network cell or site | the RAN controller (O-RAN interfaces) |
| `reaction_wheel` | a spacecraft reaction wheel | the attitude controller (modelled only) |
| `robot_fleet` | a fleet of robots or vehicles taking tasks | the fleet manager's dispatch API |
| `robot_joint` | a robot joint servo | the robot controller (ROS 2, EtherCAT, the drive's fieldbus) |
| `server` | an application service on servers or pods | the Kubernetes API (HPA target and floor, pod resize) |
| `storage` | block, file or object storage | the storage system's QoS and tiering API |
| `turbine` | a generating unit's governor, boiler or excitation loop | the plant DCS and governor (OPC UA, IEC 61850; modelled only) |
| `ups` | an uninterruptible power supply | the UPS and PDU management interface (SNMP, Modbus) |
| `water` | a water or pumping process | the SCADA system (Modbus, DNP3) |
| `workflow` | a workflow or pipeline engine | the workflow engine's concurrency settings |

Wires marked *modelled only* exist as plants in the benchmark and are not built for live use.

## The shared spine: 267 muscles in all four realms

The machines every realm stands on: servers, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution. Each spine muscle is part of every realm's organism, counted once in the tower and four times in the stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Cloud VM & Capacity (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 97 | vm count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 98 | vm size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 99 | vm start stop | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 100 | vm migrate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 101 | instance family | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 102 | asg min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 103 | asg max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 104 | spot mix | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 105 | reservation mix | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 106 | zone selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 107 | region selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 108 | architecture selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 109 | accelerator selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 110 | boot disk class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 111 | placement group | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 112 | interruption response | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1038 | target tracking policy | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1039 | scaling cooldown warmup | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1114 | vm cpu cap by priority | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1120 | predictive scaling | capacity | starts the machines the forecast needs before the rush lands | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1121 | instance refresh | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Container Resources (25 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 33 | cpu request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 34 | cpu limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | than native runs; hands back what it no longer needs at once | | |
| 35 | memory request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 36 | memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 37 | ephemeral storage | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 38 | hugepages | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 39 | io weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 40 | pids limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 41 | memory high | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 42 | memory reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 43 | swap limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 44 | cgroup io max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 45 | cpu weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 46 | cpu quota | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 47 | cpuset | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 48 | runtime class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1017 | in place resize | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1019 | namespace quota | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1020 | default limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1022 | memory hard limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1023 | memory protection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1024 | io latency target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1025 | freeze | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 2176 | cruisekube request rightsizing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 2177 | goldilocks vpa request recommendation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Host CPU & Memory (26 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 113 | cpufreq min | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 114 | cpufreq max | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 115 | rapl package power | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 116 | uncore frequency | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 117 | energy perf preference | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 118 | cpu idle policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 119 | memory bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 120 | numa balance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 121 | irq affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 122 | llc allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 123 | memory pressure gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 124 | page reclaim rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 125 | transparent hugepages | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 126 | core online offline | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 127 | thermal throttle policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 128 | host power profile | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1040 | frequency governor | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1041 | turbo boost | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1042 | dram power limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1043 | swappiness reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1112 | core packing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1115 | per pod cpu frequency range and c state access | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1116 | uncore frequency per node | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2174 | ear cpu frequency policy | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2175 | ear global energy budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 2178 | k8s power manager core profile | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Kubernetes Placement & Scheduling (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 49 | node selector | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 50 | node affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 51 | pod anti affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 52 | topology spread | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 53 | numa placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 54 | gpu topology | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 55 | storage locality | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 56 | network locality | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 57 | taint toleration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 58 | priority class | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 59 | preemption policy | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 60 | device claim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 61 | failure domain spread | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 62 | scheduler backoff | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 63 | gang admission | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 64 | deschedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1026 | scheduler scoring strategy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1027 | scheduler plugin weights | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 1028 | scheduling gates | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1029 | cpu manager | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 2179 | peaks power aware placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Kubernetes Workload Scaling (32 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 17 | replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 18 | hpa cpu target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 19 | hpa memory target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 20 | hpa custom target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 21 | vpa apply | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 22 | scale to zero | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 23 | keda threshold | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 24 | rollout rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 25 | max surge | capacity | starts the machines the forecast needs before the | all four realms; stack; | machines started at 70% full; reads the last 10 |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | tower | decisions |
| 26 | max unavailable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 27 | deployment pause resume | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 28 | rollout abort | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 29 | pod eviction | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 30 | pdb policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 31 | scheduler queue priority | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 32 | api priority fairness | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1001 | hpa min replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 1002 | hpa max replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1003 | hpa scale down stabilisation window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1004 | hpa scale up stabilisation window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1005 | hpa scaling rate policies | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1006 | hpa policy selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1007 | hpa tolerance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1008 | hpa sync period | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1009 | vpa minmax allowed resources | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1010 | keda polling interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1011 | rollout progress deadline | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1012 | rollout readiness delay | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1014 | unhealthy pod eviction policy | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1015 | graceful termination | capacity | starts the machines the forecast needs before the rush lands and clears work | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1016 | probes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 2172 | kepler energy target autoscaling | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *GPU Hardware (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 129 | gpu allocate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | longer needs at once | | |
| 130 | gpu power limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 131 | gpu sm clock | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 132 | gpu memory clock | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 133 | gpu persistence mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 134 | gpu compute mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 135 | mig mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 136 | mig geometry | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 85% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 137 | gpu timeslice | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 138 | gpu mps | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 139 | gpu quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 140 | gpu reset | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 141 | gpu thermal limit | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 142 | gpu ecc response | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 143 | gpu job power budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1044 | application clocks | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1045 | gpu temperature target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1117 | sync boost | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1118 | target clocks | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2173 | climatik inference power cap | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2180 | adaptive gpu power cap | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 2181 | nvidia platform power budget | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Node Fleet & Karpenter-Class Control (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 65 | node desired | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 66 | node pool min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 67 | node pool max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 68 | node provision | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one | all four realms; stack; tower | machines started at 75% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 69 | node cordon | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 70 | node drain | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 71 | node consolidate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 72 | node replace | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 73 | node shutdown | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 74 | node power on | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 75 | nodepool weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 76 | disruption budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 77 | consolidation policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 78 | consolidate after | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 90% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 79 | expire after | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 80 | capacity class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1030 | scale down utilisation threshold | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1031 | scale down unneeded time | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1032 | scale down delay after add | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1033 | expander | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 1034 | node pool limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1035 | kubelet eviction thresholds | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1036 | kubelet reserved resources | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1037 | max pods per node | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Network Routing & Switching (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 225 | lb weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 226 | route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 227 | rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 228 | bandwidth limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 229 | qos class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 230 | connection limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 231 | failover route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 232 | nic rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 233 | nic queue count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 234 | queue discipline | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 235 | congestion control | capacity | starts the machines the forecast needs before the rush lands and clears work waiting | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 236 | egress budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 237 | ingress budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 238 | ecmp weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 239 | path selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 240 | network isolation | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1046 | nic interrupt coalescing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1047 | nic ring size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 1048 | port link power | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |

### *Observability & Telemetry (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 373 | collector memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 374 | export concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 376 | cardinality budget | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 377 | retention window | setpoint | starts the machines the forecast needs before the rush | all four realms; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 378 | remote write queue | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 379 | telemetry shed | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1053 | metric scrape interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1054 | batching | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 80% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1119 | per pod energy attribution | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |

### *Reliability, Security & Recovery (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 385 | restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 386 | rollback | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 388 | traffic divert recovery | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 389 | degraded mode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 390 | actuator freeze | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 391 | workload isolate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 393 | credential rotation gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 394 | policy enforcement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 395 | rate abuse gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 396 | fault domain isolate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 397 | backup trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | all four realms; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 400 | kill switch | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

### *Storage Block/File/Object (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 257 | volume size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 258 | iops limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 259 | throughput limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 260 | replica count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 261 | storage tier | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 262 | volume placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 263 | snapshot trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 264 | rebalance | capacity | starts the machines the forecast needs before the rush lands and | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 265 | recovery rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 266 | backfill rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 267 | compaction pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 268 | cache allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 269 | object replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 270 | erasure code profile | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |
| 271 | storage admission | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 272 | degraded storage gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1049 | scrub schedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 1050 | io scheduler | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1051 | disk power spin down | power | at its limit on native (the full clock): read and left native | all four realms; stack; tower | global |
| 1052 | lifecycle tiering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | all four realms; stack; tower | global |

### *Cooling, Chillers & Thermodynamics (18 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `data_hall` (a data hall cooling plant), reached through the building management system (BACnet, Modbus) or its command line. Omni-Compass reads its temperature against its limit. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 545 | supply air temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 546 | return air target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 547 | coolant supply temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 548 | coolant flow | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 549 | pump speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 550 | fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 551 | chiller setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 552 | compressor authority | power | at its limit on native (the site envelope at full): read and left native | all four realms; stack; tower | global |
| 553 | cooling tower fan | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 554 | cooling capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 555 | rack thermal budget | power | at its limit on native (the site envelope at full): read and left native | all four realms; stack; tower | global |
| 556 | gpu thermal envelope | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 557 | cpu thermal envelope | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | all four realms; stack; tower | global |
| 558 | thermal workload migrate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 559 | thermal load shed | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1055 | server fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 1056 | economiser | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |
| 1122 | chiller plant supervisory control rl with safety layer | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | all four realms; stack; tower | global |

### *PDU, UPS & Electrical Distribution (18 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `ups` (an uninterruptible power supply), reached through the UPS and PDU management interface (SNMP, Modbus). Omni-Compass reads its draw on the grid connection and its reserve. It belongs to every organism.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 529 | server power cap | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 530 | rack power cap | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 531 | pdu branch power limit | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 532 | pdu outlet control | capacity | holds the charge for the line and clips only the draw that | all four realms; stack; | 0 wobbles of margin on its |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | would cross it; a UPS lends only the charge above its outage floor | tower | line |
| 534 | ups operating mode | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 535 | ups charge rate | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 536 | ups discharge rate | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 537 | phase balance | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 538 | load transfer | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 539 | reactive power target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 540 | voltage target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0.25 wobbles of margin on its line |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 541 | generator dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 542 | electrical isolation | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 543 | breaker trip gate | admission | at its limit on native (nothing paced or turned away): read and left native | all four realms; stack; tower | global |
| 1057 | ups shutdown battery test | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | all four realms; stack; tower | 0 wobbles of margin on its line |
| 1110 | hierarchical power budget row pdu switchboard controllers | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 1111 | priority aware capping throttle low priority first | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |
| 1113 | power oversubscription with prediction | power | at its limit on native (full battery power): read and left native | all four realms; stack; tower | global |

# Realm: Compute / AI / Cloud (273 muscles of its own, 540 in its organism with the spine)

## *AI Inference Serving (24 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 161 | model replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 162 | model route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 163 | model load | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 164 | model unload | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 165 | model instance count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 166 | continuous batching | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 167 | max batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 168 | batch queue delay | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 169 | inference concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 170 | inference max tokens | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 60 decisions |
| 171 | kv cache budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 172 | prefix cache budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 173 | speculative decode budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 174 | model precision | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 175 | inference priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 176 | inference slo gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1058 | autoscaling target concurrency qps | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1059 | scale to zero panic window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 1060 | request priority levels | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1061 | kv cache swap space | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1062 | chunked prefill | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 1063 | tensor parallel degree | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 1064 | disaggregated prefill decode | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1065 | lora adapter load | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *AI Training (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu_batch` (GPU training and batch work), reached through nvidia-smi, the job scheduler's pause and resume. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 177 | training workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 178 | global batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 179 | microbatch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |
| 180 | gradient accumulation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 181 | data parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 182 | tensor parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 183 | pipeline parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 184 | expert parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| 185 | checkpoint interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 186 | checkpoint trigger | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 187 | training preempt | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 188 | training gang size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 189 | elastic worker count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |
| 190 | straggler mitigation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 191 | training precision | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 192 | compute comm overlap | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 40 decisions |
| 1066 | activation checkpointing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 1067 | dataloader workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Cross-Cluster, Multi-Region & Edge (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 609 | multi cluster dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | once | | |
| 610 | region dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 611 | zone dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 612 | edge dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 613 | cloud capacity class | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 614 | workload migrate region | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 615 | data residency gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 616 | latency region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 617 | cost region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 618 | carbon region gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 619 | global failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 620 | federation quota | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 621 | cross cluster replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 622 | edge offload | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 623 | global admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5128 | edge site node pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5129 | edge inference gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5130 | edge sync interval | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *DPU SmartNIC & Programmable IO (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0021 | dpu pf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| AUDIT-0022 | dpu vf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0023 | dpu sf tx rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0024 | dpu bandwidth share | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0025 | dpu qos group | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0026 | sriov vf count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0027 | smartnic flow steering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| AUDIT-0028 | smartnic offload enable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| AUDIT-0029 | dpu cpu budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0030 | dpu memory budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0031 | dpu service placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0032 | dpu failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

### *Distributed Cluster Managers (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 209 | task admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 210 | task priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 211 | resource reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 212 | cluster quota | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 213 | task preemption | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 214 | binpack pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 10 decisions |
| 215 | spread pressure | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| 216 | gang schedule | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 60 decisions |
| 217 | worker allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 218 | maintenance evacuation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 90% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 219 | oversubscription | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 220 | resource reclaim | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| 221 | queue fairness | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 222 | deadline pressure | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 223 | scheduler retry | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *GPU Fabric & RDMA (18 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `fabric` (a high-speed fabric (RDMA, DPU, SmartNIC)), reached through the fabric manager's API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 145 | nvlink placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 146 | nvswitch route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 147 | gpu fabric quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 148 | rdma bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 149 | rdma route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 150 | nic affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 151 | collective concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 152 | collective algorithm | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 153 | collective chunk size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 154 | rank placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 155 | gpudirect policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 156 | congestion response | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 157 | rail selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 158 | fabric failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 159 | communication priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 160 | fabric isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1068 | collective protocol | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1069 | nic hca selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

### *HPC & Distributed Compute (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 193 | job slots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 194 | mpi ranks | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 195 | rank mapping | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 196 | node allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 197 | job walltime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 198 | job priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 199 | job preemption | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 200 | checkpoint restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 201 | parallel io budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 202 | collective budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 203 | accelerator share | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 204 | cpu gpu ratio | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 205 | memory per rank | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 206 | scratch budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 207 | scheduler fair share | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 208 | backfill policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1070 | node power saving | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 1071 | cpu frequency per job | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 1072 | partition limits | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Kubernetes Dynamic Device Allocation (8 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `gpu` (GPU serving with a response-time target), reached through nvidia-smi (clock ceiling and power limit) or the serving autoscaler. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0001 | dra device class selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0002 | dra claim capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| AUDIT-0003 | dra claim sharing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |
| AUDIT-0004 | dra device taint | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| AUDIT-0005 | dra device eviction | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| AUDIT-0006 | dra binding readiness | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| AUDIT-0 | dra binding failure | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 007 | response | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | tower | |
| AUDIT-0008 | dra device configuration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *OpenShift & Machine API (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `node` (a fleet of machines or VMs that boot in minutes), reached through the node pool's size command (Karpenter, Cluster Autoscaler, MachineSet, cloud ASG). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 85 | machine remediation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 88 | machine health gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 89 | mcp pause | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 90 | mcp max unavailable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 91 | node config rollout | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 92 | operator reconcile budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 93 | cluster version pacing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 94 | infra machine admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 95 | machine failure domain | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Quantum Computing Control Simulation (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `qpu` (a quantum or specialised accelerator queue), reached through the accelerator's job queue. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 593 | qubit mapping | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 594 | circuit admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 595 | shot allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 596 | circuit scheduling | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 597 | gate scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 598 | pulse amplitude | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 599 | pulse duration | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 600 | pulse phase | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 601 | pulse frequency | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| 602 | coupling control | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 603 | reset scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 604 | measurement scheduling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 605 | dynamical decoupling | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 606 | noise aware routing | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| 607 | error mitigation budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 608 | quantum queue priority | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Work Admission & Demand Shaping (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1 | api concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 2 | queue concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 3 | queue backpressure | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 4 | job admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 5 | batch admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 6 | inference admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 7 | load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 8 | priority gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 9 | tenant admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 10 | burst limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 11 | deadline admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 12 | work budget | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 13 | request queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 14 | retry admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 15 | background work gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 16 | maintenance work gate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1074 | request concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1123 | adaptive concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| 1124 | priority request queue under limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *Linux Kernel & OS Control (20 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5001 | cgroup cpu weight | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5002 | cgroup cpu quota | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| V5-5003 | cgroup memory high | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 10 decisions |
| V5-5004 | cfs scheduler latency | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5005 | cpufreq dvfs governor | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5006 | transparent hugepages | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5007 | io scheduler depth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5008 | numa auto balancing | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5009 | dirty ratio writeback | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5010 | vm swappiness | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5011 | tcp buffer autotune | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5012 | block nr requests | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5013 | ksm memory dedup | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5014 | net rx coalesce | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5015 | cpu cstate governor | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5016 | zswap pool fraction | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5017 | page cache reclaim | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5018 | irqbalance affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 60 decisions |
| V5-5019 | net qdisc codel target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5020 | hugepage pool reserve | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 20 decisions |

### *Serverless & Functions (FaaS) (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5021 | fn concurrency limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5022 | fn provisioned concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5023 | fn scale to zero idle | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5024 | fn memory size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5025 | fn prewarm pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5026 | fn reserved concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5027 | fn burst admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5028 | fn snapstart restore | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5029 | fn timeout budget | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *Virtualization & Hypervisor (9 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `cpu_host` (a CPU host whose clock and power can be set), reached through Linux cpufreq and RAPL. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5030 | vcpu allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5031 | memory ballooning | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5032 | vm overcommit density | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5033 | live migration pacing | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5034 | numa vcpu pinning | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5035 | sriov vf allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5036 | vgpu partition | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5037 | vm memory dedup ksm | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5038 | hypervisor cpu pinning | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |

### *Gut: Data-plane Intake & Waste (8 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5045 | ingress intake pacing | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5046 | dead letter retry cap | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5047 | gc compaction pacing | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5048 | ttl eviction waste | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5049 | backpressure threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5050 | queue drain rate | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5051 | retry budget window | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5052 | compaction debt drain | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |

### *Skin: Perimeter & Admission (5 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5053 | rate limit token bucket | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5054 | connection admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5055 | waf inspection sampling | capacity | at its limit on native (the full actuator): read and left native | Compute; stack; tower | global |
| V5-5056 | tls session cache | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5057 | ddos shield admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |

### *Media Streaming & CDN (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5072 | abr bitrate ladder cap | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5073 | player buffer target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5074 | transcoder worker pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| V5-5075 | live transcode gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5076 | vod transcode batch power | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| V5-5077 | packager workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 85% full; reads the last 60 decisions |
| V5-5078 | origin shield cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5079 | edge cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5080 | segment duration target | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5081 | cdn request coalescing | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5082 | drm license server pool | capacity | starts the machines the forecast needs before the rush lands and | Compute; stack; tower | machines started at 85% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5083 | thumbnail sprite batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 80% full; reads the last 10 decisions |
| V5-5084 | media ingest admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5085 | encoder lookahead frames | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |

### *Game Servers & Real-Time Interactive (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5086 | agones fleet buffer size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5087 | agones fleet min replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5088 | agones fleet max replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5089 | game server tick rate | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5090 | players per server cap | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5091 | matchmaking admission rate | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5092 | matchmaking wait window | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5093 | relay server pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5094 | voice chat mixer pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5095 | game node pool autoscaler | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5096 | replay render batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5097 | cloud gaming gpu clocks | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

### *Blockchain & Ledger Nodes (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `storage` (block, file or object storage), reached through the storage system's QoS and tiering API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5098 | node db cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5099 | mempool max size | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5100 | max peer connections | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5101 | script validation threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5102 | block relay admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5103 | rpc work queue depth | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5104 | utxo cache flush interval | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5105 | indexer batch workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5106 | archive node storage tier | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5107 | validator host power profile | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |

### *CI/CD & Build Farms (10 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Compute / AI / Cloud.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5108 | runner scale set min runners | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5109 | runner scale set max runners | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5110 | build concurrency limit | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5111 | build cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | global |
| V5-5112 | test shard parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5113 | job queue admission | admission | at its limit on native (nothing paced or turned away): read and left native | Compute; stack; tower | global |
| V5-5114 | artifact retention ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Compute; stack; tower | global |
| V5-5115 | build node cpu frequency | power | at its limit on native (the full clock): read and left native | Compute; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5116 | remote execution workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Compute; stack; tower | machines started at 70% full; reads the last 20 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5117 | warm runner pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Compute; stack; tower | machines started at 75% full; reads the last 40 decisions |

# Realm: Physics / Robotics / Autonomous (137 muscles of its own, 404 in its organism with the spine)

## *Automotive EV & Mobile Powertrain (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `ev_traction` (an electric vehicle traction motor), reached through the vehicle's motor controller (CAN; modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1083 | state of charge limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1084 | smart charging schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| AUDI T-0045 | traction torque limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0046 | regen braking level | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0047 | battery charge limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0048 | battery discharge limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0049 | battery thermal target | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0050 | motor thermal limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0051 | vehicle speed envelope | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| AUDI T-0052 | energy recovery target | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0053 | auxiliary power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0054 | fast charge current | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0055 | fast charge voltage | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| AUDI T-0056 | vehicle safe state | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Aviation & Autonomous Flight (28 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `flight_axis` (a flight-control axis), reached through the flight controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 481 | throttle envelope | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 482 | attitude target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 483 | attitude rate target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 484 | velocity target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 485 | altitude target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 486 | waypoint authority | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 487 | flight hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 488 | return to home | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 489 | land action | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 490 | mission admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 491 | geofence response | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 492 | failsafe selection | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 493 | battery reserve threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 494 | actuator saturation envelope | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 495 | flight mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 496 | flight termination safe state | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 1079 | attitude rate gains | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1080 | horizontal speed limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1081 | vertical speed limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 1082 | acceleration jerk limits | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5143 | px4 mpc xy vel max | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5144 | px4 mpc z vel max up | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5145 | px4 mpc acc hor | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5146 | px4 mc rollrate max | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5147 | ardupilot wpnav speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5148 | ardupilot atc accel r max | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| V5-5149 | drone battery failsafe reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics; stack; tower | 1.5 wobbles of margin on its line |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5150 | swarm formation spacing | capacity | its own peak speed while tasks wait, back to its everyday speed | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | | |

### *Robotics Fleet & Warehouse Automation (17 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `robot_fleet` (a fleet of robots or vehicles taking tasks), reached through the fleet manager's dispatch API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 449 | robot dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 450 | task assignment | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 451 | traffic reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 452 | robot route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 453 | charging dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 454 | battery reserve | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 455 | elevator request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 456 | door request | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 457 | conveyor speed | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 458 | agv speed | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 459 | warehouse zone admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 460 | robot quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 461 | fleet failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 462 | human safe stop | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |
| 463 | fleet concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| V5-5151 | amr fleet charge threshold | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Physics; stack; tower | 1.5 wobbles of margin on its line |
| V5-5152 | amr fleet active robots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

### *Robotics Motion Control (26 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `robot_joint` (a robot joint servo), reached through the robot controller (ROS 2, EtherCAT, the drive's fieldbus). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 433 | joint position | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 434 | joint velocity | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 435 | joint acceleration | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 436 | joint effort | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 437 | cartesian velocity | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 438 | trajectory speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 439 | trajectory acceleration | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 440 | jerk limit | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 441 | collision margin | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 442 | force limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 443 | gripper force | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 444 | locomotion speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | when its tracking strains (alone; inside a body, its everyday speed) | | |
| 445 | steering angle | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 446 | braking force | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 447 | balance correction | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1076 | trajectory tolerances | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 1077 | velocity acceleration scaling | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1078 | controller update rate | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | when its tracking strains (alone; inside a body, its everyday speed) | | |
| V5-5135 | ros2 executor threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Physics; stack; tower | machines started at 70% full; reads the last 40 decisions |
| V5-5136 | ros2 qos history depth | setpoint | left native: native already holds it inside its band (nothing to win) | Physics; stack; tower | global |
| V5-5137 | ros2 control loop rate | setpoint | left native: native already holds it inside its band (nothing to win) | Physics; stack; tower | global |
| V5-5138 | humanoid balance margin | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5139 | humanoid gait speed | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| V5-5140 | humanoid joint torque limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5141 | humanoid step height | capacity | its own peak speed while tasks wait, back to its everyday | Physics; stack; | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | tower | |
| V5-5142 | manipulator payload speed scaling | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

### *Spacecraft & Flight Software (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `reaction_wheel` (a spacecraft reaction wheel), reached through the attitude controller (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 497 | space command admission | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 499 | flight task schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 500 | space mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 501 | payload duty cycle | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 502 | communication allocation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 503 | space power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 504 | space thermal command | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 505 | attitude command envelope | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 506 | reaction wheel allocation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 507 | rcs authority | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 508 | safe mode transition | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 509 | watchdog recovery | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 510 | instrument activation | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 511 | fault isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Rail Traction & Train Control (14 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `rail_traction` (a train's traction and auxiliary systems), reached through the train control and management system and ATO (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2054 | traction effort limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2055 | regenerative braking share | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2056 | coasting speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2057 | dwell time schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2058 | hvac duty cycle | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2059 | train auxiliary power budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2060 | platform approach speed | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2061 | acceleration rate setpoint | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2062 | wheel slip protection mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2063 | traction motor thermal derate | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2064 | catenary voltage limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2065 | timetable recovery margin | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2066 | headway target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2067 | door release hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

### *Marine Propulsion & Vessel Automation (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `marine_propulsion` (a ship's propulsion shaft and power plant), reached through the vessel automation and power management system (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2068 | shaft speed target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2069 | propeller pitch limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2070 | shaft power limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2071 | bow thruster duty | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2072 | ballast pump mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2073 | engine load sharing setpoint | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2074 | slow steaming speed target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2075 | auxiliary engine staging | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2076 | shore power mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2077 | hotel load budget | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2078 | trim target | capacity | its own peak speed while tasks wait, back to its everyday speed near its heat limit or when its tracking strains (alone; inside a body, its everyday speed) | Physics; stack; tower | global |
| 2079 | rudder rate limit | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Elevators & Vertical Transport (12 muscles)*

Each is a motor-driven axis: a robot joint, a flight-control surface, a reaction wheel, a traction motor; its class is `elevator_hoist` (an elevator or escalator drive and its group control), reached through the lift controller and destination dispatch (modelled only). Omni-Compass reads its tracking error and its load. Its home realm is Physics / Robotics / Autonomous.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2146 | hoist speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2147 | acceleration limit | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2148 | regen drive mode | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2149 | standby power mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2150 | destination dispatch schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2151 | car parking mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2152 | door dwell hold | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2153 | escalator speed target | capacity | at its limit on native (its everyday speed is the machine's limit): read and left native | Physics; stack; tower | global |
| 2154 | group capacity mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2155 | motor thermal derate | power | at its limit on native (full torque): read and left native | Physics; stack; tower | global |
| 2156 | brake test schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |
| 2157 | fire recall mode | admission | at its limit on native (nothing paced or turned away): read and left native | Physics; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

# Realm: Energy / Facility / Industrial (213 muscles of its own, 480 in its organism with the spine)

## *Building & Critical Environment HVAC (15 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `building` (a building zone), reached through the building management system. Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1094 | setpoint reset trim and respond | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 1095 | duct static pressure setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 1096 | optimal start stop | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0081 | zone temperature target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| AUDIT-0082 | zone airflow | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0083 | ahu fan speed | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0084 | damper position | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0085 | economizer position | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0086 | boiler setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0087 | heat pump mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0088 | humidity target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| AUDIT-0089 | occupancy ventilation | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0090 | building demand limit | power | at its limit on native (the site envelope at full): read and left native | Energy; stack; tower | global |
| AUDIT-0091 | thermal storage dispatch | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| AUDIT-0092 | hvac emergency mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Energy Storage & Microgrid (19 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `microgrid` (a microgrid battery and loads), reached through the inverter or energy management system (IEEE 2030.5, SunSpec, OpenADR). Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 513 | battery charge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 10 decisions before it holds; keeps 0% of its line clear |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 514 | battery discharge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 515 | battery soc reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0.75 wobbles of margin on its line |
| 516 | grid import limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 517 | grid export limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 5% of its line clear |
| 518 | pv curtailment | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 519 | ev charge power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 520 | heat pump power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 521 | electrolyzer power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 5 decisions before it holds; keeps 0% of its line clear |
| 522 | microgrid demand limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 523 | peak shaving | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 524 | time of use schedule | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 525 | energy load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 526 | flex load admission | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 527 | storage dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 528 | microgrid emergency reserve | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 1085 | volt var | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1086 | volt watt | capacity | holds the charge for the line and clips only the draw that | Energy; stack; | 0 wobbles of margin on its line |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | would cross it; a UPS lends only the charge above its outage floor | tower | |
| 1087 | constant power factor reactive power | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |

### *Facility & Grid Optimization (15 muscles)*

Each is a battery, UPS or microgrid store with a load and a grid connection; its class is `facility` (a facility battery behind the meter), reached through the energy management system. Omni-Compass reads its draw on the grid connection and its reserve. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 561 | facility power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 562 | utility demand limit | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 563 | demand response | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 564 | electricity price gate | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 565 | carbon intensity gate | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 566 | renewable dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 567 | generator start stop | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 568 | site battery dispatch | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 569 | pue target | setpoint | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 1 wobbles of margin on its line |
| 570 | cooling power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 571 | it power budget | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | 0 wobbles of margin on its line; reads its line 3 decisions before it holds; keeps 0% of its line clear |
| 572 | rack power allocation | power | at its limit on native (full battery power): read and left native | Energy; stack; tower | global |
| 573 | facility peak guard | capacity | holds the charge for the line and clips only the draw that would cross it; a UPS lends only the charge above its outage floor | Energy; stack; tower | 0 wobbles of margin on its line |
| 574 | grid frequency response | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 575 | facility islanding | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Grid Transmission & Distribution (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `feeder_voltage` (a distribution feeder's voltage), reached through the distribution management system (IEC 61850, DNP3; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0033 | capacitor bank switch | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0034 | voltage regulator tap | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0035 | transformer tap | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0036 | inverter real power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0037 | inverter reactive power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0038 | feeder voltage target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDIT-0039 | feeder load transfer | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0040 | distribution storage dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0041 | demand response dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0042 | frequency droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0043 | grid protection mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0044 | grid restoration sequence | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Industrial PLC & Process Automation (18 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 465 | plc cycle authority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 466 | machine cell admission | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 467 | valve position | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 468 | pump flow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 469 | compressor speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 470 | heater power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 471 | furnace setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 472 | pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 473 | temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 474 | mass flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 475 | tank level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 476 | conveyor rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 477 | feed rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 478 | purge vent action | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 1088 | controller mode | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1089 | alarm limits | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1090 | safety interlock trip | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1091 | opc ua writes | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

### *Semiconductor Fab & Precision Manufacturing (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `chamber` (a controlled chamber (clean room, kiln, reactor)), reached through the PLC or DCS. Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 1092 | run to run control | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 1093 | idle sleep mode | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDIT-0057 | tool job dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDIT-0058 | wafer route | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0059 | chamber recipe selection | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0060 | chamber temperature | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0061 | chamber pressure | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0062 | gas flow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0063 | rf power | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0064 | vacuum pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0065 | robot transfer rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0066 | lot priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0068 | tool quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Water Wastewater & Pumping (12 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `water` (a water or pumping process), reached through the SCADA system (Modbus, DNP3). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0069 | pump speed water | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0070 | valve position water | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0071 | reservoir level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDI T-0072 | line pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0073 | flow target water | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| AUDI T-0074 | aeration rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0075 | chemical dose rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| AUDI T-0076 | filtration backwash | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0077 | lift station dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0078 | leak isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0079 | water demand shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| AUDI T-0080 | water emergency shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Healthcare Critical Environments (13 muscles)*

Each is a data hall or building zone cooled by chillers, air handlers or fans; its class is `hospital` (a hospital's critical rooms (operating rooms, isolation, pharmacy, imaging suites)), reached through the hospital's building management system (BACnet). Omni-Compass reads its temperature against its limit. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2001 | operating room air change setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2002 | isolation room pressure target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2003 | patient room temperature setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2004 | surgical suite humidity target | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2005 | ahu supply air temperature | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2006 | pharmacy cold room setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2007 | sterile storage humidity setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Energy; stack; tower | global |
| 2008 | imaging suite cooling capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2009 | chiller plant staging | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2010 | exhaust fan capacity | capacity | left to its own staging (native already has the cooling it needs: nothing to win) | Energy; stack; tower | global |
| 2011 | ward night setback mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2012 | medical gas plant demand limit | power | at its limit on native (the site envelope at full): read and left native | Energy; stack; tower | global |
| 2013 | emergency power load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Agriculture & Irrigation (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `irrigation` (pumps, pressure and climate on a farm or in a greenhouse), reached through the pump station controller, the pivot panel or the climate computer (Modbus, the vendor cloud). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2026 | irrigation pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2027 | mainline pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2028 | soil moisture target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2029 | pivot speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2030 | fertigation dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2031 | greenhouse temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2032 | greenhouse co2 target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2033 | vent position setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2034 | grain dryer temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2035 | cold storage temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2036 | barn ventilation rate | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2037 | milking vacuum level setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2038 | well drawdown level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2039 | drip zone dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Oil & Gas Pipelines (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `pipeline` (a pipeline segment with its compressors, pumps and valves), reached through pipeline SCADA (DNP3, Modbus, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2040 | compressor discharge pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2041 | pump station suction pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2042 | line pack target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2043 | pipeline flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2044 | compressor unit staging | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2045 | vfd pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2046 | terminal tank level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2047 | leak detection shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2048 | batch interface dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2049 | heater outlet temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2050 | drag reducing agent dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2051 | valve position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2052 | cathodic protection voltage | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2053 | pressure protection priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Mining & Mineral Processing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `mill` (a grinding, flotation or materials-handling circuit), reached through the plant DCS or PLC (OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2092 | sag mill load setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2093 | mill speed target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2094 | crusher gap setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2095 | flotation aeration rate | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2096 | cyclone feed pressure target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2097 | thickener underflow density target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2098 | reagent dose | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2099 | conveyor speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2100 | dewatering pump level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2101 | ventilation on demand airflow | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2102 | stockpile level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2103 | slurry pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2104 | tailings discharge shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2105 | ore blend dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *District Heating & Cooling (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `district_heat` (a district heating or cooling network), reached through the network control system and substation controllers (Modbus, M-Bus). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2106 | supply temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2107 | return temperature target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2108 | differential pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2109 | network pump speed | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2110 | heat pump staging | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2111 | chp dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2112 | thermal storage level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2113 | peak boiler heater output | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2114 | substation flow limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2115 | peak demand shed | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2116 | outdoor reset setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2117 | cooling network supply temperature | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2118 | cooling storage level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

### *Power Generation & Turbine Control (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `turbine` (a generating unit's governor, boiler or excitation loop), reached through the plant DCS and governor (OPC UA, IEC 61850; modelled only). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2119 | turbine speed droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2120 | unit load setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2121 | boiler steam pressure setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2122 | feedwater level target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2123 | agc participation limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2124 | excitation voltage setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2125 | hydro gate position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2126 | combustion air ratio setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2127 | cooling water flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2128 | inlet guide vane position | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2129 | reserve dispatch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2130 | turbine ramp rate limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2131 | emissions shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2132 | nuclear rod position target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

### *Renewable Generation & Inverter Control (13 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `inverter` (a plant of grid-support inverters or wind turbines), reached through the plant controller and inverter settings (SunSpec, IEC 61850, IEEE 1547). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2133 | inverter volt var setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2134 | inverter volt watt setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2135 | active power curtailment | power | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2136 | plant reactive power target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2137 | wind turbine yaw offset | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2138 | pitch angle limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2139 | rotor speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2140 | inverter frequency droop setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2141 | ramp rate limit | capacity | at its limit on native (the full actuator): read and left native | Energy; stack; tower | global |
| 2142 | tracker stow mode | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2143 | string mppt voltage setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2144 | noise mode schedule | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2145 | ice detection shutdown | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

### *Pharmaceutical & Food Manufacturing (14 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `batch_reactor` (a GMP batch reactor, fermenter or food process), reached through the batch control system (ISA-88, OPC UA). Omni-Compass reads how far the process sits from its band. Its home realm is Energy / Facility / Industrial.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2158 | reactor temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2159 | agitator speed setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2160 | fermenter dissolved oxygen target | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2161 | ph setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2162 | pasteurizer holding temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2163 | freezer tunnel temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2164 | cip cycle dispatch | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2165 | cleanroom pressure cascade setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2166 | lyophilizer shelf temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2167 | chromatography flow setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2168 | steam sterilizer cycle priority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2169 | oven zone temperature setpoint | setpoint | left native: native already holds it inside its band (nothing to win) | Energy; stack; tower | global |
| 2170 | refrigerant compressor authority | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |
| 2171 | batch hold quarantine | admission | at its limit on native (nothing paced or turned away): read and left native | Energy; stack; tower | global |

# Realm: Distribution / Specialized (217 muscles of its own, 484 in its organism with the spine)

## *Cache & Memory Services (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 305 | cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 306 | cache ttl | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 307 | cache eviction policy | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 308 | cache replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 309 | cache sharding | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 310 | cache prefetch | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 311 | cache writeback rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 312 | cache admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 313 | hot key isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 314 | cache connection limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 315 | cache memory limit | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 316 | cache compression | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 317 | cache warmup | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 318 | cache failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 319 | cache flush rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1106 | memory size threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Commerce & Payment Systems (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `commerce` (a customer-facing service with bursts), reached through the Kubernetes API. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 401 | payment admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 402 | payment concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 403 | payment retry | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 404 | payment timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 405 | fraud review gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 406 | authorization route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 407 | processor route weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 408 | transaction queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 409 | idempotency window | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 410 | order reservation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 411 | inventory hold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 412 | checkout load shed | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 413 | refund queue rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 414 | settlement batch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 415 | payment failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |

### *Data Analytics & ETL (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `batch` (batch and queued jobs), reached through the job queue (Kubernetes Jobs, Slurm, Spark). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 338 | executor size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 339 | dynamic allocation min | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 340 | dynamic allocation max | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | fewer than native runs; hands back what it no longer needs at once | | |
| 341 | shuffle partitions | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 342 | shuffle bandwidth | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 343 | etl concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 344 | stage parallelism | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 345 | query slots | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 85% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 346 | spill threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 347 | cache fraction | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 348 | batch interval | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 349 | stream back pressure | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 350 | data locality wait | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 351 | speculation policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 352 | analytics admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Database & Transactions (19 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `database` (a database cluster), reached through the database operator (replicas, connection pools). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 273 | db replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 275 | db memory | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 276 | db cache | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 90% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 277 | query concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 278 | read route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 279 | shard placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 280 | db failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 281 | replication lag gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 282 | db write throttle | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 283 | db pool resize | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 284 | transaction concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 285 | lock timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 286 | checkpoint rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 287 | vacuum compaction rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1099 | parallel workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 1100 | background writer | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 1101 | synchronous replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1102 | buffer pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 40 decisions |
| 1103 | io capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Messaging & Streaming (22 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 289 | partition count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 290 | partition placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 291 | producer quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 292 | consumer quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 293 | broker io quota | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 294 | message retention | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 295 | queue depth limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 296 | consumer concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 297 | producer batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 298 | fetch batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 299 | rebalance rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 300 | replication factor | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |
| 301 | retry backoff | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 302 | dead letter divert | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 303 | stream priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 304 | broker failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1104 | prefetch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 1105 | memory disk alarm | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| V5-5124 | mqtt broker max | capacity | starts the machines the forecast needs before the rush | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | connections | | lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| V5-5125 | mqtt max inflight messages | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5126 | mqtt session expiry | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5127 | broker cluster nodes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

### *Runtime & Application (21 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 353 | worker count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 354 | thread pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 355 | jvm heap | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 356 | gc budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 357 | connection pool runtime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 358 | application cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 359 | runtime memory | capacity | starts the machines the forecast needs before the rush lands | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 360 | async concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 361 | event loop workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| 362 | process count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 363 | request timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 364 | background workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 365 | runtime cpu budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 366 | runtime io budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 367 | runtime restart | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 1108 | go runtime | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 1109 | keepalive connection reuse | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| V5-5131 | dns resolver cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5132 | dns negative cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5133 | mail queue concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5134 | mail retry interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

### *Search, Indexing & Vector DB (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 321 | index workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 75% full; reads the last 20 decisions |
| 322 | index refresh rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 323 | segment merge rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 324 | search concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 325 | search timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 326 | shard count | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 327 | shard replication | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 328 | shard rebalance | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 329 | vector search k | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 330 | vector batch size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 331 | embedding workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 332 | index memory budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 333 | query route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 60 decisions |
| 334 | hot shard isolation | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 335 | search admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 1107 | circuit breakers | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Service Mesh & API Reliability (16 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 241 | circuit breaker | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 242 | retry budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 243 | service timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 244 | service concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 245 | connection pool | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 246 | traffic divert | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 247 | traffic mirror | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 248 | canary weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 249 | outlier ejection | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 250 | health threshold | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 251 | dns traffic weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 252 | session affinity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |
| 253 | request hedging | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 254 | fault injection gate | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 255 | service failover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 1097 | load balancing policy | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Telecom RAN & Edge Radio (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `ran` (a radio access network cell or site), reached through the RAN controller (O-RAN interfaces). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| AUDIT-0009 | ran connection admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| AUDIT-0010 | ran ue handover | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0011 | ran cell traffic steering | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0012 | ran slice resource budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| AUDIT-0013 | ran prb allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| AUDIT-0014 | ran scheduler weight | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | start, never fewer than native runs; hands back what it no longer needs at once | | |
| AUDIT-0015 | ran tx power | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| AUDIT-0016 | ran antenna tilt | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0017 | ran carrier enable | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0018 | ran cell sleep | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0019 | ran du cu placement | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| AUDIT-0020 | ran fronthaul budget | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

### *Workflow, Logistics & Fulfillment (15 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `workflow` (a workflow or pipeline engine), reached through the workflow engine's concurrency settings. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 417 | workflow admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 418 | workflow worker rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 419 | task queue rate | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 420 | workflow retry | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 421 | workflow backoff | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 422 | workflow timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 423 | inventory allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 424 | fulfillment route | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 425 | warehouse queue | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 426 | carrier selection | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 427 | dispatch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
| 428 | shipment batch | capacity | starts the machines the forecast needs before the rush lands and | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| | | | clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | | |
| 429 | route replan | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 430 | sla escalation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 431 | compensation action | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 40 decisions |

### *Medical Imaging & Clinical Systems (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `clinical` (clinical systems: imaging archives, records, interface engines, monitoring gateways), reached through the application's own scaling and admission settings (Kubernetes, the vendor console). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2014 | pacs archive tier target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| 2015 | dicom router concurrency | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2016 | ehr application replicas | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| 2017 | hl7 interface queue limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2018 | fhir api rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2019 | reconstruction gpu workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2020 | modality worklist timeout | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2021 | patient monitoring gateway capacity | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2022 | lab analyzer batch window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2023 | telehealth session admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2024 | clinical backup window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2025 | imaging prefetch priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Ports & Maritime Logistics (12 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `port` (a container terminal's cranes and vehicles taking moves), reached through the terminal operating system and fleet controllers. Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2080 | quay crane allocation | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2081 | yard crane fleet size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2082 | berth window admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2083 | truck gate rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2084 | horizontal transport fleet size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2085 | reefer plug power budget | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| 2086 | shore power connection capacity | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2087 | rail mounted gantry dispatch | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2088 | container dwell priority | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| 2089 | vessel arrival pacing | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| 2090 | stacking height target | setpoint | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| 2091 | equipment charging window | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Content Delivery & Edge Cache (6 muscles)*

Each is a regulated process: a temperature, pressure, flow, level, voltage or composition held at a setpoint; its class is `process` (an industrial process loop), reached through the PLC or DCS (OPC UA, EtherNet/IP, PROFINET). Omni-Compass reads how far the process sits from its band. Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5039 | cdn cache ttl | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5040 | edge cache size | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5041 | origin shield concurrency | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5042 | prefetch aggressiveness | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5043 | tiered cache promotion | capacity | at its limit on native (the full actuator): read and left native | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5044 | edge pop admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

### *Financial Markets & Trading Systems (14 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `server` (an application service on servers or pods), reached through the Kubernetes API (HPA target and floor, pod resize). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5058 | matching engine threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5059 | order gateway sessions | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5060 | order admission rate limit | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| V5-5061 | pre trade risk check workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5062 | market data fanout threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 10 decisions |
| V5-5063 | market data conflation interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5064 | fix session heartbeat interval | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5065 | kernel bypass poll cores | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| V5-5066 | tick store write batch | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |
| V5-5067 | post trade settlement workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 20 decisions |
| V5-5068 | end of day risk batch power | power | at its limit on native (the full clock): read and left native | Distribution; stack; tower | global |
| V5-5069 | colocation rack cooling setpoint | setpoint | cools the room ahead of the heat as it nears its limit or its cooling nears flat out; never warmer than native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5070 | backtest grid nodes | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5071 | surveillance alert queue workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |

### *Reliability, Security & Recovery (6 muscles)*

Each is a pool of servers, pods, GPUs, links or disks serving a stream of requests or jobs; its class is `network` (network routing and switching), reached through the network controller (SDN, routing API). Omni-Compass reads how far its queue and its load sit toward the service line (queue or lateness, and load above half). Its home realm is Distribution / Specialized.

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5118 | ids capture threads | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | global |
| V5-5119 | ids ring buffer size | setpoint | left native: native already holds it inside its band (nothing to win) | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| # | Muscle | Knob | What Omni-Compass does with it | Organisms | Its own card |
|---|---|---|---|---|---|
| V5-5120 | ids rule group admission | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |
| V5-5121 | waf inspection workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 70% full; reads the last 20 decisions |
| V5-5122 | siem ingest pipeline workers | capacity | starts the machines the forecast needs before the rush lands and clears work waiting in line within one machine start, never fewer than native runs; hands back what it no longer needs at once | Distribution; stack; tower | machines started at 80% full; reads the last 10 decisions |
| V5-5123 | endpoint scan scheduling | admission | at its limit on native (nothing paced or turned away): read and left native | Distribution; stack; tower | global |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix F. The Azure Account, Click by Click

> Evaluation and simulation use only; any other use requires a signed, paid Omni-Compass Enterprise License. See `LICENSE`.

This connects GitHub to an Azure account so the workflow `aks-metered` can build a real Azure Kubernetes Service cluster, run native Kubernetes (with Azure's own cluster autoscaler) against the same with Omni-Compass on top, count the machines Azure bills every 15 seconds, and delete everything when it is done. The design is written before the run in `docs/K8S_COMPASS_PREREGISTRATION.md`, "The bill on a real cloud". It costs about USD 10 to 30 of Azure time; a new account's free credit usually covers it. About ten minutes, once.

## 1. An Azure account

1. Open **azure.microsoft.com/free** and click **Start free** (or **Try Azure for free**).
2. Sign in with a Microsoft account, or make one.
3. Fill in the form. Azure asks for a card to confirm who you are; the free credit is used first.
4. When it says the account is ready, go to **portal.azure.com**.

## 2. Azure's own terminal (nothing to install)

1. In the Azure portal, at the top of the page, click the >_ icon (Cloud Shell), to the right of the search bar.
2. If it asks, choose **Bash** (not PowerShell).
3. If it asks about storage, choose **No storage account required** (or accept the default and **Create**).
4. A black terminal opens at the bottom of the page.

## 3. One line: a login for GitHub

Paste this line into that terminal and press **Enter**:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
az ad sp create-for-rbac --name omni-compass-github --role contributor
    ↪  --scopes /subscriptions/$(az account show --query id -o tsv) --
    ↪ sdk-auth
```

It prints a block of text that starts with `{` and ends with `}`. Select all of it, from `{` to `}`, and copy it. That block is a password for this one purpose; never paste it anywhere but the GitHub secret below.

## 4. Put it in GitHub

1. Open the repository on GitHub.
2. Click **Settings** (the tab at the top right of the repository, not your account settings).
3. On the left: **Secrets and variables**, then **Actions**.
4. Click **New repository secret**.
5. **Name:** `AZURE_CREDENTIALS`
6. **Secret:** paste the whole block from step 3.
7. Click **Add secret**.

## 5. Run it

**Actions** tab, then **aks-metered** on the left, then **Run workflow**, then the green **Run workflow** button. The defaults are the preregistered design (5 repetitions, native and native with Omni-Compass on top, 900 measured seconds, Standard_D2s_v4 in eastus, the governor at the tune-down on the machines alone: the pods exactly native's, so the bill can only fall or stay). About 6 hours on GitHub. Each cluster is deleted when its arm ends, and the resource group at the end of every repetition whatever happens, so nothing keeps billing.

If Azure answers **quota exceeded**: in the portal, search **Subscriptions**, open yours, and upgrade it to **Pay-As-You-Go** (the credit stays), or ask for a quota of 10 vCPUs for the DSv4 family in the region.

## 6. Afterwards

The result is the job `aggregate`'s receipt: billed machine-hours and the bill at list price, native against native with Omni-Compass on top, beside response time, failures and CPU, each with its 95% interval. To remove GitHub's access when you are done: in the Azure portal search **App registrations**, open **omni-compass-github**, **Delete**; and delete the `AZURE_CREDENTIALS` secret in GitHub.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix G. The Google Cloud Account, Click by Click

This connects GitHub to a Google Cloud account so two workflows can run the same tests Azure runs:

- `gke-metered`: a real Google Kubernetes Engine cluster, native Kubernetes (with Google's own cluster autoscaler) against the same with Omni-Compass on top, the machines Google bills counted every 15 seconds, the organisms with the cluster inside at 1, 10 and 100 copies (`organism`, `organism_scale`), everything deleted at the end; every set at the tune-down with the governor on the machines alone, so the bill can only fall or stay. The same design as Azure's `aks-metered` (`docs/K8S_COMPASS_PREREGISTRATION.md`).
- `cloud-organism` with `cloud` = `gcp`: the big organisms (the whole tower and the four stacked, the real cluster inside, at 100 copies) on one rented 8-processor machine, the same test Azure's `big-organism-detached` runs.

About fifteen minutes, once. New accounts get **$300 of free credit for 90 days**; Google does not charge the card during the free trial.

## 1. A Google Cloud account with the free credit

1. On the laptop, open **cloud.google.com/free** and click **Get started for free**.
2. Sign in with a Google account (a Gmail address works), or make one.
3. Choose the country, tick the terms, click **Continue**.
4. Enter a card. Google uses it to confirm who you are; the $300 credit is used first.
5. Click **Start my free trial**. The Google Cloud console opens (**console.cloud.google.com**).
6. **Turn on the full account** (the credit stays and is still used first; the free trial otherwise limits how many machines run at once and refuses GPUs): at the top of the console click **Activate** (or open the menu ☰, **Billing**, then **Upgrade** / **Activate full account**) and confirm.

## 2. A project for Omni-Compass

1. At the top left of the console, click the project picker (next to "Google Cloud").

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

2. Click **New project**. Name: `omni-compass`. Click **Create**.
3. When it is made, pick it in the project picker. Under the name you see its **Project ID** (for example `omni-compass-123456`). Write it down: it is the second secret below.

## 3. Google's own terminal (nothing to install)

1. At the top right of the console, click the >_ icon (**Activate Cloud Shell**).
2. A terminal opens at the bottom of the page. If it asks to authorize, click **Authorize**.

## 4. One paste: a login for GitHub

Paste this whole block into that terminal and press **Enter** (it takes about a minute):

```
P=$(gcloud config get-value project)
gcloud services enable container.googleapis.com compute.googleapis.com
    ↪  iam.googleapis.com
gcloud iam service-accounts create omni-compass-github --display-name
    ↪ "Omni-Compass GitHub"
for r in roles/container.admin roles/compute.admin roles/iam.serviceAc
    ↪ countUser; do
  gcloud projects add-iam-policy-binding "$P" --member "serviceAccount
    ↪ :omni-compass-github@$P.iam.gserviceaccount.com" --role "$r" --q
    ↪ uiet &gt;/dev/null
done
gcloud iam service-accounts keys create key.json --iam-account "omni-c
    ↪ ompass-github@$P.iam.gserviceaccount.com"
cat key.json; rm key.json
```

It prints a block that starts with `{` and ends with `}`. Select all of it, from `{` to `}`, and copy it. That block is a password for this one purpose: never paste it anywhere but the GitHub secret below. (If the project picker showed no project, run `gcloud config set project YOUR-PROJECT-ID` first.)

## 5. Put the two secrets in GitHub

1. Open the repository on GitHub, click **Settings** (the repository's tab, at the top right), then on the left **Secrets and variables**, then **Actions**.
2. **New repository secret**. Name: `GCP_CREDENTIALS`. Secret: paste the whole block from step 4. **Add secret**.
3. **New repository secret**. Name: `GCP_PROJECT`. Secret: the Project ID from step 2. **Add secret**.

That is all. The runs start from the repository's **Actions** tab on the newest engine (branch `v5`), and no result goes up before it has been read.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 6. What it costs

Google's list prices (us-central1, on demand): a worker of the Kubernetes test (`e2-standard-2`) about $0.067 an hour, up to 4 workers an arm, about $5 to $15 for a full five-repetition set; the big-organism machine (`e2-standard-8`) about $0.27 an hour, about $10 for its 39-hour ceiling. All of it comes out of the $300 credit first. Every cluster and machine is deleted when its run ends, whatever happens.

## 7. Afterwards

To remove GitHub's access: in the console menu ☰ **IAM & Admin**, **Service Accounts**, tick **omni-compass-github**, **Delete**; and delete the two secrets in GitHub.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix H. The Amazon Web Services Account, Click by Click

This connects GitHub to an Amazon Web Services (AWS) account so two workflows can run the same tests Azure runs:

- `eks-metered`: a real Amazon Elastic Kubernetes Service cluster, native Kubernetes (with its own cluster autoscaler) against the same with Omni-Compass on top, the machines Amazon bills counted every 15 seconds, the organisms with the cluster inside at 1, 10 and 100 copies (`organism`, `organism_scale`), everything deleted at the end; every set at the tune-down with the governor on the machines alone, so the bill can only fall or stay. The same design as Azure's `aks-metered` (`docs/K8S_COMPASS_PREREGISTRATION.md`).
- `cloud-organism` with `cloud` = `aws`: the big organisms (the whole tower and the four stacked, the real cluster inside, at 100 copies) on one rented 8-processor machine, the same test Azure's `big-organism-detached` runs.

About fifteen minutes, once. New accounts get **$100 of credit at sign-up and up to $100 more** for trying services, used before the card.

## 1. An AWS account

1. On the laptop, open **aws.amazon.com/free** and click **Create a Free Account** (or **Create an AWS account**).
2. Enter an e-mail address and an account name (`Omni-Compass`), click **Verify email address**, and type the code Amazon e-mails you.
3. Choose a password for the root user.
4. Contact information: **Business** (The Omni-Compass LLC) or **Personal**, the address and phone.
5. **Choose the Paid plan** if Amazon asks you to pick a plan. Your credits still pay first; the Free plan blocks services these tests need (the Kubernetes service among them).
6. Enter a card, then confirm the phone number with the code Amazon sends.
7. Support plan: **Basic support - Free**. Click **Complete sign up**, then **Go to the AWS Management Console** and sign in as the root user.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## 2. A login for GitHub (an IAM user)

1. In the search bar at the top of the console type **IAM** and open it.
2. On the left, **Users**, then **Create user**. User name: `omni-compass-github`. Do **not** tick console access. **Next**.
3. **Attach policies directly**. Tick **AdministratorAccess** (the tests build whole Kubernetes clusters, which needs EKS, EC2, CloudFormation and roles; this user is used only by GitHub, and you delete it when the tests are done). **Next**, **Create user**.
4. Click the new user **omni-compass-github**, open the **Security credentials** tab, and under **Access keys** click **Create access key**.
5. Choose **Application running outside AWS** (or **Other**), **Next**, **Create access key**.
6. The page shows an **Access key** (starts with `AKIA`) and a **Secret access key**. Keep the page open for the next step; the secret is shown only once. These two are passwords for this one purpose: never paste them anywhere but GitHub.

## 3. Put the two secrets in GitHub

1. Open the repository on GitHub, click **Settings** (the repository's tab, at the top right), then on the left **Secrets and variables**, then **Actions**.
2. **New repository secret**. Name: `AWS_ACCESS_KEY_ID`. Secret: the Access key. **Add secret**.
3. **New repository secret**. Name: `AWS_SECRET_ACCESS_KEY`. Secret: the Secret access key. **Add secret**.

That is all. The runs start from the repository's **Actions** tab on the newest engine (branch `v5`), and no result goes up before it has been read.

## 4. What it costs

Amazon's list prices (us-east-1, on demand): a worker of the Kubernetes test (`m5.large`) about $0.096 an hour, up to 4 workers an arm, plus about $0.10 an hour for each cluster's control plane, about $10 to $25 for a full five-repetition set; the big-organism machine (`m6i.2xlarge`) about $0.38 an hour, about $15 for its 39-hour ceiling. The credits pay first. Every cluster and machine is deleted when its run ends, whatever happens.

If Amazon refuses a machine for "vCPU limit" (new accounts start with a small limit): in the console search **Service Quotas**, open **Amazon Elastic Compute Cloud (Amazon EC2)**, **Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances**, **Request increase at account level**, ask for 64, **Request**. Amazon usually answers within a day.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## 5. Afterwards

To remove GitHub's access: in IAM, **Users**, tick **omni-compass-github**, **Delete**; and delete the two secrets in GitHub.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix I. The Install Manifests

Every file an installation applies or builds, printed in full from the repository: the watch install (namespace, service account, the read-only role and the watch Deployment), the cloud mode's grant and its governing Deployment, the pod-target grant of level 2 (never in the cloud mode), and the two container images. Each file carries the legal notice at its head; the `<...>` values are the installation's own.

`deploy/install/omni-compass.yaml`

*60 lines.*

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Omni-Compass: one-file install. Stage 1 (shadow): read-only identity
    ↪ , observe mode. It cannot write to the cluster.
# Image: build with `docker build -f deploy/Dockerfile -t &lt;registry&gt;/o
    ↪ mni-compass:&lt;tag&gt; .` and set it below.
# Reset, any time: kubectl -n omni-compass exec deploy/omni-compass --
    ↪  touch /tmp/omni.kill
# Promote to stage 2/3 only by the steps in docs/WIRING_GUIDE.md (each
    ↪  adds exactly the permissions that stage needs).
apiVersion: v1
kind: Namespace
metadata: {name: omni-compass}
---
apiVersion: v1
kind: ServiceAccount
metadata: {name: omni-compass, namespace: omni-compass}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-observe}
rules:
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
  - apiGroups: [""]
    resources: [nodes, pods, configmaps]
    verbs: [get, list]
  - apiGroups: [metrics.k8s.io]
    resources: [nodes, pods]
    verbs: [get, list]
  - apiGroups: [autoscaling]
    resources: [horizontalpodautoscalers]
    verbs: [get, list]
  - apiGroups: [apps]
    resources: [deployments]
    verbs: [get, list]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: {name: omni-compass-observe}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-observe}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: omni-compass, namespace: omni-compass, labels: {app:
    ↪ omni-compass}}
spec:
  replicas: 1
  selector: {matchLabels: {app: omni-compass}}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  template:
    metadata: {labels: {app: omni-compass}}
    spec:
      serviceAccountName: omni-compass
      securityContext: {runAsNonRoot: true, runAsUser: 10001, seccompP
    ↪ rofile: {type: RuntimeDefault}}
      containers:
        - name: controller
          image: omni-compass:local            # &lt;- your registry imag
    ↪ e
          args: ["--mode", "observe", "--interval", "60", "--closure",
    ↪  "/app/law/closure.json",
                "--audit", "/var/log/omni/audit.jsonl", "--kill-file"
    ↪ , "/tmp/omni.kill"]
          resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {cp
    ↪ u: 500m, memory: 256Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
    ↪ e: tmp, mountPath: /tmp}]
      volumes: [{name: audit, emptyDir: {}}, {name: tmp, emptyDir: {}}
    ↪ ]
```

`deploy/install/rbac-machines.yaml`

*25 lines.*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud mode's grant (the Operator's Manual, section 14.3): the wa
    ↪ tch role of omni-compass.yaml plus two verbs. Park a
# machine (a PreferNoSchedule taint) and wake it (remove the taint, un
    ↪ cordon): patch nodes. Mark a parked machine's pods
# first to go (the pod-deletion-cost annotation): patch pods. Nothing
    ↪ else: no delete, no eviction, no autoscaler write,
# no pod resize, no cloud credential.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-machines}
rules:
  - apiGroups: [""]
    resources: [nodes]
    verbs: [patch]
  - apiGroups: [""]
    resources: [pods]
    verbs: [patch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding

metadata: {name: omni-compass-machines}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-machines}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

`deploy/install/omni-compass-cloud.yaml`

*68 lines.*

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud mode (the Operator's Manual, chapter 19): Omni-Compass on
    ↪ top at the tune-down, on the machines alone. It moves
# no pod autoscaler (--sensed names no service), conveys no processor
    ↪ and caps no power; it parks only machines the
# cloud's autoscaler already made, after its own trial, and wakes them
    ↪  on a breach. Apply after omni-compass.yaml (the
# namespace, the service account and the watch role) and rbac-machines
    ↪ .yaml. It ships with --dry-run: it decides and
# writes nothing until that line is removed. Set the image (&lt;registry&gt;
    ↪ /omni-compass-cloud:&lt;version&gt;), TARGET_URL (the
# service's URL) and --slo-ms (the service's line), and --max-nodes (t
    ↪ he work pools' top). Three containers in one pod:
# the governor, the probe (its response-time feed) and the watchdog (i
    ↪ ts lease), sharing one process namespace.
apiVersion: apps/v1
kind: Deployment
metadata: {name: omni-compass, namespace: omni-compass, labels: {app:
    ↪ omni-compass}}
spec:
  replicas: 1                          # one writer: never more than o
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
  ↪ ne
strategy: {type: Recreate}           # never two governors at once,
  ↪ even during a rollout
selector: {matchLabels: {app: omni-compass}}
template:
  metadata: {labels: {app: omni-compass}}
  spec:
    serviceAccountName: omni-compass
    shareProcessNamespace: true          # the watchdog sees the gov
  ↪ ernor's process (the lease, chapter 20.2)
    terminationGracePeriodSeconds: 60    # room for the hand-back on
  ↪  SIGTERM: every parked machine woken, read back
    securityContext: {runAsNonRoot: true, runAsUser: 10001, seccompP
  ↪ rofile: {type: RuntimeDefault}}
    tolerations: [{key: CriticalAddonsOnly, operator: Exists}]     #
  ↪  runs on the system pool, never on a machine it parks
    nodeSelector: {omni-role: system}
    containers:
      - name: controller
        image: omni-compass-cloud:local
        args: ["--give-back-only", "--mode", "nodepool", "--law", "c
  ↪ ompass", "--dial", "save", "--sensed", "none/none",
               "--active-nodes-only", "--interval", "60", "--floor-i
  ↪ nterval", "5",
               "--min-nodes", "2", "--max-nodes", "4", "--max-node-s
  ↪ tep", "1",
               "--node-scale-cmd", "bash /app/scripts/kind_nodepool.
  ↪ sh {n}",
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
              "--node-restore-cmd", "bash /app/scripts/kind_nodepoo
↪ l.sh 100000",
              "--latency-file", "/feed/latency.csv", "--slo-ms", "5
↪ 00",
              "--closure", "/app/law/closure.json",
              "--audit", "/var/log/omni/audit.jsonl", "--kill-file"
↪ , "/tmp/omni.kill",
              "--dry-run"]
      env:
        - {name: WORKER_SEL, value: "omni-role=work"}      # the m
↪ achines it may park
        - {name: MIN_NODES, value: "2"}                    # never
↪  fewer in service
      resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {cp
↪ u: 500m, memory: 256Mi}}
      securityContext: {allowPrivilegeEscalation: false, readOnlyR
↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
      volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
↪ e: tmp, mountPath: /tmp}, {name: feed, mountPath: /feed}]
    - name: probe                  # the response-time feed (chapt
↪ er 18): real requests to the service, every 5 s
      image: omni-compass-cloud:local
      command: ["python", "/app/scripts/latency_probe.py"]
      args: ["$(TARGET_URL)", "/feed/latency.csv"]
      env:
        - {name: TARGET_URL, value: "http://&lt;service&gt;.&lt;namespace&gt
↪ svc.cluster.local/"}
        - {name: INTERVAL, value: "5"}
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

```
            - {name: N, value: "5"}
            - {name: DURATION, value: "31536000"}
          resources: {requests: {cpu: 10m, memory: 32Mi}, limits: {cpu
    ↪ : 100m, memory: 64Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: feed, mountPath: /feed}]
        - name: watchdog               # the lease (chapter 20.2): if
    ↪ the governor dies or hangs, it runs the governor's own
          image: omni-compass-cloud:local    # recorded restore (--res
    ↪ tore-only), which wakes every parked machine
          command: ["python", "/app/tools/omni_switch.py", "watchdog",
    ↪  "--every", "10"]
          env:                         # the restore is the governor's
    ↪  own command line: it needs the governor's environment
            - {name: WORKER_SEL, value: "omni-role=work"}
            - {name: MIN_NODES, value: "2"}
          resources: {requests: {cpu: 10m, memory: 64Mi}, limits: {cpu
    ↪ : 500m, memory: 256Mi}}
          securityContext: {allowPrivilegeEscalation: false, readOnlyR
    ↪ ootFilesystem: true, capabilities: {drop: [ALL]}}
          volumeMounts: [{name: audit, mountPath: /var/log/omni}, {nam
    ↪ e: tmp, mountPath: /tmp}]
      volumes: [{name: audit, emptyDir: {}}, {name: tmp, emptyDir: {}}
    ↪ , {name: feed, emptyDir: {}}]
```

deploy/rbac-target.yaml

*19 lines.*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Additional write access for --mode target: patch and annotate HPAs o
    ↪ nly. Apply only after an observe period.
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: {name: omni-compass-target}
rules:
  - apiGroups: [autoscaling]
    resources: [horizontalpodautoscalers]
    verbs: [patch, update]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: {name: omni-compass-target}
roleRef: {apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name
    ↪ : omni-compass-target}
subjects: [{kind: ServiceAccount, name: omni-compass, namespace: omni-
    ↪ compass}]
```

`deploy/Dockerfile`

*19 lines.*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# Omni-Compass controller image. kubectl comes from the official Kuber
    ↪ netes image (no package manager, no curl).
FROM registry.k8s.io/kubectl:v1.32.2 AS kubectl
FROM python:3.12-slim
COPY --from=kubectl /bin/kubectl /usr/local/bin/kubectl
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY omnicompass omnicompass
COPY omni_controller omni_controller
COPY hardware hardware
# the frozen, benchmarked closure-law setting (--closure /app/law/clos
    ↪ ure.json)
COPY tuning/GLOBAL_LEAGUE_PREREGISTRATION.json law/closure.json
USER 10001
ENTRYPOINT ["python", "-m", "omni_controller.controller"]
```

`deploy/Dockerfile.cloud`

*17 lines.*

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
# Evaluation and simulation use only; any commercialization, monetizat
    ↪ ion or other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE, NOTICE and DISCLOSURES
    ↪ .md.
# All patents, copyrights and trademarks filed in the USA. www.omni-co
    ↪ mpass.com
# The cloud mode's image (the Operator's Manual, chapter 19): the ship
    ↪ ped image (deploy/Dockerfile) plus what moving the
# machines alone needs: jq (the machine lever reads the cluster's JSON
    ↪  with it), the machine lever's script, the
# response-time probe or the Prometheus feed adapter, and the switch t
    ↪ ool the in-pod watchdog runs (the lease). Build the shipped imag
    ↪ e first and pass it: --build-arg BASE=&lt;registry&gt;/omni-compass:&lt;v
    ↪ ersion&gt;.
ARG BASE=omni-compass:local
FROM ${BASE}
USER root
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends jq &amp
    ↪  rm -rf /var/lib/apt/lists/*
COPY scripts/kind_nodepool.sh /app/scripts/kind_nodepool.sh
COPY scripts/latency_probe.py /app/scripts/latency_probe.py
COPY tools/omni_switch.py /app/tools/omni_switch.py
COPY tools/prom_feed.py /app/tools/prom_feed.py
USER 10001
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

APPENDIX

# Appendix J. The Implementer's Checklists

One checklist a phase (section 13.3). Each line is checked and initialled before the next phase begins, and the sheets are kept with the installation record (section 14.7).

## Phase 0: the copy

- [ ] `python3 verify.py` ends `VERIFICATION: PASS`; the receipt kept.
- [ ] `python3 tools/omni_version.py` names the engine version; the commit recorded.
- [ ] `python3 tools/seal.py --check` passes.
- [ ] `LICENSE`, `NOTICE` and `DISCLOSURES.md` read by counsel; the license in force for this installation named.

## Phase 1: the account and the cluster

- [ ] The provider's allowances read for the region and the machine families used; increases granted.
- [ ] A system pool of one machine, tainted `CriticalAddonsOnly=true:NoSchedule`, labelled `omni-role=system`.
- [ ] One or more work pools labelled `omni-role=work`, under the provider's cluster autoscaler, floor at least 2.
- [ ] The autoscaler's profile written down (scale-down timing, scan interval); the same in both arms of every paired run.
- [ ] The autoscaler seen to add a machine and delete one on its own.
- [ ] metrics-server answering (`kubectl top nodes`).
- [ ] The service's HPA on CPU, with container requests; its target, minimum and maximum written down by the service owner.

## Phase 2: identity

- [ ] Namespace `omni-compass` and service account `omni-compass` created.
- [ ] The watch role applied (`deploy/install/omni-compass.yaml`).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- [ ] For the cloud mode, the machines role applied (`deploy/install/rbac-machines.yaml`) and nothing else.
- [ ] `kubectl auth can-i` output of section 14.4 matches the table exactly; kept.
- [ ] No cloud credential given to the governor.

## Phase 3: watch

- [ ] The shipped image built and pushed, tagged with the engine version and commit.
- [ ] The governor running `--mode observe`; its audit shows `decision` records (`nodes_recommended`) and no `write` record.
- [ ] The audit on a persistent volume, shipped to the log store.
- [ ] At least one full cycle of the service's own load watched (a day; a week for a weekly peak).

## Phase 4: the feed and the line

- [ ] The line written down by the service owner: milliseconds, the percentile, what counts as a failure, where measured.
- [ ] The feed in place (the probe or an adapter), writing `elapsed_seconds,latency_ms,ok`.
- [ ] The feed advancing every 5 seconds or faster; its clock monotonic over a day.
- [ ] A stopped endpoint seen in the audit as blind or breached, every machine held.
- [ ] A stopped feed seen in the audit as blind, nothing given back.

## Phase 5: the paired run

- [ ] The run's design written before it starts: arms (native, and native with Omni-Compass on top), load, duration, repetitions, the line, the machine family, the autoscaler profile.
- [ ] The run made on one engine version; its commit recorded.
- [ ] The receipt read row by row: every row GAIN, EVEN or LOSS by the rules of chapter 11.
- [ ] Every LOSS explained as wiring and fixed, and the run repeated; none cut.
- [ ] In the cloud mode: HPA replicas equal to native's within the noise; the bill EVEN or GAIN.
- [ ] The receipt signed by the service owner and by finance (the bill rows).

## Phase 6: govern

- [ ] The cloud image built (`deploy/Dockerfile.cloud`) and pushed.
- [ ] `deploy/install/omni-compass-cloud.yaml` applied with the image, the service's URL, the line and the pools' top.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

- [ ] A day of `--dry-run`: every `write` record marked `"dry_run": true` read; each command asks for a count of machines in service between the floor and the work machines that exist, one machine from the last.
- [ ] `--dry-run` removed; the first parked machine read in the audit with the trial that allowed it.
- [ ] The watchdog running in the governor's pod (the manifest's third container): the pod reads `3/3` ready.
- [ ] The OFF switch drilled by its holder: every machine woken, every record gone, within 60 seconds; the drill recorded.
- [ ] The alarms of section 20.5 wired and each proven by a test.

## Phase 7: day two, every quarter

- [ ] The OFF switch drilled and recorded.
- [ ] The watchdog's hand-back drilled (the governor's process killed outright in a quiet hour, section 28.2) and recorded.
- [ ] The audit searched for `refused`, `error`, `blind` and an unclosed `proprioception` drift; each explained.
- [ ] The engine version checked against the receipts kept; no receipt read across versions.
- [ ] The installation record complete.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix K. The Kubernetes Governor's Settings

Every setting of the Kubernetes governor (`python -m omni_controller.controller`), grouped by who writes it. A setting not given on the command line takes its default. Settings in the last group are the factory tune (section 3.1): they are listed so that a command line can be read in full, and an installation leaves them as shipped; a command line that changes one runs a changed tune, whose receipts are its own (section 3.6).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

## K.1 The settings an installation writes

| Setting | Default | The cloud mode's value | What it is |
|---|---|---|---|
| `--slo-ms` | 0 (no line) | the service's line | the 95th-percentile response time the service must meet, in milliseconds (section 18.1) |
| `--latency-file` | none | `/feed/latency.csv` | the feed: rows `elapsed_seconds,latency_ms,ok` (section 18.2) |
| `--min-nodes` (and `MIN_NODES` in the environment of the governor and the watchdog) | 2 | the operator's floor, at least 2 | machines always kept in service |
| `--max-nodes` | 0 (read live) | the work pools' top | the most machines the machine lever counts on; it never asks for more than exist |
| `--node-scale-cmd` | none | `bash /app/scripts/kind_nodepool.sh {n}` | the command that keeps `{n}` machines in service (section 25.2 for its contract) |
| `--node-restore-cmd` | none | `bash /app/scripts/kind_nodepool.sh 100000` | the command a reset runs: every parked machine woken |
| `WORKER_SEL` (environment) | every machine that is not the control plane | `omni-role=work` | the label selector of the machines that may be parked |
| `NODE_CUSHION` (environment) | 0 | 0 | whole machines the operator holds back from the lever |
| `--sensed` | every HPA | `none/none` | the services whose autoscalers the feed speaks for; `none/none` names none, so no autoscaler is moved |
| `--dial` | `max` | `save` | the max, or the tune-down (section 3.2) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | Default | The cloud mode's value | What it is |
|---|---|---|---|
| `--replica-ceiling` | -1 (read live) | not used (no autoscaler is moved) | how far a sensed autoscaler's cap may be raised; 0 keeps the operator's cap |
| `--audit` | `omni_audit.jsonl` | `/var/log/omni/audit.jsonl` | the audit file (section 20.4) |
| `--kill-file` | `/tmp/omni.kill` | `/tmp/omni.kill` | the file whose presence turns the governor OFF (section 20.3) |
| `--dry-run` | off | on for the first day, then removed | decide and record every command, write none |
| `--kubectl` | `kubectl` | `kubectl` | the client the governor reads and writes through |
| `--site-limit-w`, `--power-cmd` | none | absent | a site's power limit and the command that prints its watts (level 7; never in a cloud) |
| `--cooling-cmd`, `--cooling-min-c`, `--cooling-max-c`, `--cooling-restore-c` | none; 18, 27, 22 °C | absent | the facility's cooling setpoint command and band (section 7.10) |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## K.2 The settings the cloud mode fixes

| Setting | Default | The cloud mode's value | What it is |
|---|---|---|---|
| `--mode` | `observe` | `nodepool` | `observe` writes nothing; `target` moves autoscaler targets; `nodepool` also moves machines |
| `--law` | `governor` | `compass` | the law the governor holds the service with |
| `--interval` | 60 s | 60 s | the full decision period (section 3.4) |
| `--floor-interval` | 15 s | 5 s | the fast checks' period, between decisions, in `nodepool` mode |
| `--max-node-step` | 2 | 1 | the most machines one decision moves |
| `--active-nodes-only` | off | on | count only machines in service |
| `--closure` | none | `/app/law/closure.json` | the shipped setting file the image carries |
| `--iterations` | 0 (run until stopped) | 0 | decisions before exiting; the paired-run harness sets it |
| `--restore-only` | off | used only by the watchdog | put every setting back from the cluster's own records and exit |
| `--no-api-proxy` | off | off | read through a new client process every time instead of one local API proxy (costs more processor) |

## K.3 The switches of the other muscles (outside the cloud mode)

These turn on muscles of levels 2 to 7 (section 6.3) on machines already paid for. None of them is used in a cloud.

| Setting | What it turns on |
|---|---|
| `--cap-deployments ns/name[,...]` | conveyance: the idle processor of each machine to the serving pods on it |
| `--rightsize-deployments ns/name[,...]` | each pod's processor request follows its measured use |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Setting | What it turns on |
|---|---|
| `--coldstart-deployments ns/name[,...]`, `--coldstart-signal ns/configmap` | a service scaled to zero while no work waits and woken when work arrives |
| `--batch` | held Jobs labelled `omnicompass.io/batch=true` admitted when there is room |
| `--batch-pace` | Jobs labelled `omnicompass.io/pausable=true` paused under power or heat stress |
| `--rollout-guard ns/name[,...]` | a rollout paused while a change is not permitted |
| `--contain-namespaces ns[,...]`, `--contain-cpu-m` | an agent namespace held to a processor budget |
| `--security-configmap ns/name` | a security hold read from a ConfigMap key `hold` |
| `--strict-replicas` | level 4: the governor sets replica counts, the HPA kept as the fast reflex upward |
| `--pod-reflex-writes` | the fast pod reflex may raise an HPA's replica floor itself |
| `--rapl-cmd`, `--cpufreq-cmd`, `--cpufreq-policy-root`, `--cpufreq-require-schedutil`, `--cpu-max-khz` | the processor's own frequency governor under its power meter (section 7.9) |
| `--gpu-query-cmd`, `--gpu-power-cmd`, `--gpu-max-w`, `--gpu-temp-limit` | a graphics card's power reading and limit through commands (chapter 21 uses the card governor instead) |
| `--thermal-model` | the heat muscle's state from the harness's heat model (for modelled runs only) |
| `--give-back-only` | the cloud mode in one flag: every switch above is turned off whatever else is passed, the replica ceiling is 0, the dial is at the tune-down and the node pool is the only knob |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## K.4 The factory tune

`--allow`, `--verdict-samples`, `--verdict-every-s`, `--verdict-recheck-s`, `--verdict-trial-s`, `--compass-tau`, `--compass-center`, `--cruise-after-s`, `--idle-demand`, `--coast-step`, `--min-target-change`, `--headroom`, `--reflex-window-s`, `--strict-window-s`, `--latency-window-s`, `--slo-clear-s`, `--convey-on`, `--convey-off`, `--cap-min`, `--cap-min-change-m`, `--cap-headroom`, `--batch-load-max`, `--batch-power-max`, `--pace-high`, `--pace-low`, `--pace-heat`, `--rightsize-headroom`, `--rightsize-min-m`, `--coldstart-idle`.

Each has its shipped value as its default, and the receipts of an installation are read on those values. Two of them bear on planning and are stated here for that reason only. `--headroom` (half again, by default) is the spare the governor keeps above the processor requests of the pods when it counts the machines they need: it never parks a machine that would leave the pods' requests, plus that spare, without a place (section 24.2). `--allow` (2%) is the most a machine given back on trial may add to the service's response time before the trial takes it back (section 19.1).

Every setting that names a time is stated in seconds (the `-s` settings) and keeps its time at any decision period (`--interval`). The older count forms, `--verdict-every`, `--verdict-recheck`, `--cruise-after`, `--strict-window` and `--slo-clear`, are still read, as a number of decisions, and win where they are given. Whatever the period, the node pool gives back at most one machine a minute.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix L. The Audit, Record by Record

The audit (`audit.jsonl`, section 20.4) is one JSON object a line. Every object carries `time` (seconds since the Unix epoch, from the governor's clock) and one key that names its kind. The records an implementer reads in the cloud mode are below, in the order a decision writes them; the fields not listed are the engine's own state, recorded for the owner's analysis and not needed to operate the governor.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## L.1 Every decision

| Record (its key) | When | Fields an implementer reads |
|---|---|---|
| `verdict` | when a give-back trial begins, ends or is abandoned | `verdict`: `trial`, `step allowed`, `step refused: the muscle is slower there`, `trial abandoned (service not calm)` or `trial abandoned (too little work)`; `step` (machines given back on trial); `machines_given_back_allowed` (how many may stay given back); `machines_native` (the machines the cluster held when the governor started); on `step allowed` and `step refused`, `reading` (`GAIN`, `EVEN`, `LOSS` or `past the allowance`, the trial's reading against the reference) and `interval_ln` (the 95% intervals behind it, in log units) |
| `cruise` | when work has waited for a place for several decisions, and when it stops | `cruise`: `on` or `off`; `pending` (pods waiting); while cruise is on, every machine is in service and the governor stands back |
| `authority` | every decision | `senses.blind` (each sense, `true` when it is blind), `senses.latency_age_s` (seconds since the feed's newest row), `proprioception` (for the machines, the drift between the count last commanded and the count now in service, as a share of the command: 0 means the command landed), `node_gate.ok` and `node_gate.reason` (whether the release gate would let a machine go, and why not) |
| **Record (its key)** | **When** | **Fields an implementer reads** |
| `decision` | every decision | `nodes_observed` (machines in service), `nodes_recommended` (machines the governor wants in service), `pending` (pods waiting for a place), `latency_p95_ms` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Record (its key) | When | Fields an implementer reads |
|---|---|---|
| | | (the feed's 95th percentile over its window), `slo_clean` (the line met over the last decisions), `mode`; outside the cloud mode, `hpa_target_recommended` |
| `write` | each command the governor runs | `write` (the command, as its argument list), `why` (`node pool size` from a decision, `scheduling floor (pending pods)` from a fast check, `reset: restore node pool` from a reset), `dry_run` |
| `refused` | each command that exits non-zero | `refused.nodes` (the count asked), `refused.now` (the count in service), `refused.code` (the exit code), `why`, `pool_top` |
| `decision_ms` | after every decision | the decision's own duration in milliseconds |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## L.2 Other records

| Record | When | Fields |
|---|---|---|
| `error` | a decision or a fast check fails (the API unreachable, a read refused) | `error` (the exception), `stderr` (the client's last output), `consecutive_failures` |
| `decision` with a text value | `killed` (the kill file is present: every setting handed back, nothing written), `restore only: every setting handed back` (the watchdog's restore), `master switch: OFF; every setting handed back; exiting` (`SIGTERM` or the master switch) | the text itself |
| `overhead` | when the governor's process exits | `cpu_s` (processor seconds of the governor and every command it ran), `wall_s`, `cores_mean` (their ratio: the governor's own cost in cores) |
| `idle_floor` | outside the cloud mode, when the service is idle and the pool goes to its floor | `from`, `to`, `demand` |
| `compass` | every decision | the engine's own reading, for the owner's analysis |

## L.3 Five searches that answer most questions

```
A=/var/log/omni/audit.jsonl
jq -c 'select(.write) | {time, why, cmd: (.write | join(" ")), dry_run
    ↪ }' $A              # every command, with its reason
jq -c 'select(.verdict) | {time, verdict, step, machines_given_back_al
    ↪ lowed}' $A          # every trial and its outcome
jq -c 'select(.authority) | select([.authority.senses.blind[]?] | any)
    ↪  | {time, senses: .authority.senses}' $A   # blind
jq -c 'select(.authority.proprioception.nodes? &gt; 0) | {time, drift: .a
    ↪ uthority.proprioception}' $A          # unlanded
jq -c 'select(.error or .refused)' $A
    ↪                     # failures and refusals
```

A drift reading on one decision is normal: the command was given at the end of the last decision, and the machine script takes seconds. A drift that stays above zero across several decisions is the one-writer question

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

of section 20.8.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

APPENDIX

# Appendix M. The Three Clouds Side by Side

The same installation on each provider, task by task. The commands are those of chapters 15 to 17 and 26, condensed; the chapters give the context and the choices. Every Kubernetes command (`kubectl ...`) is the same on all three.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## M.1 The cluster

| Task | Azure (AKS) | Google Cloud (GKE) | Amazon Web Services (EKS) |
|---|---|---|---|
| Check the allowance for the machine family | `az vm list-usage -l <region> -o table` | `gcloud compute regions describe <region>` (quotas) | Service Quotas console, EC2 "Running On-Demand Standard instances" (vCPUs) |
| Create the cluster with its system pool | `az aks create ... --nodepool-name system --nodepool-taints CriticalAddonsOnly=true:NoSchedule --nodepool-labels omni-role=system` | `gcloud container clusters create ... --node-taints CriticalAddonsOnly=true:NoSchedule --node-labels omni-role=system` | `eksctl create cluster -f cluster.yaml` with a `system` managed node group carrying the label and the taint |
| Add a work pool under the autoscaler, labelled | `az aks nodepool add ... --labels omni-role=work --enable-cluster-autoscaler --min-count <n> --max-count <top>` | `gcloud container node-pools create work0 ... --enable-autoscaling --min-nodes <n> --max-nodes <top> --node-labels omni-role=work` | a `work0` managed node group with `labels: {omni-role: work}` and the autoscaler's discovery tags, and the Cluster Autoscaler installed by Helm (or Karpenter with a NodePool carrying the label) |
| The autoscaler's scale-down profile | `az aks update --cluster-autoscaler-profile scale-down-unneeded-time=<t> ...` | `gcloud container clusters update --autoscaling-profile balanced` or `optimize-utilization` | the Cluster Autoscaler's flags (`--scale-down-unneeded-time` and others), or Karpenter's `consolidationPolicy` and `consolidateAfter` |
| Credentials for `kubectl` | `az aks get-credentials -g <rg> -n <cluster>` | `gcloud container clusters get-credentials <cluster> --zone <zone>` | `aws eks update-kubeconfig --name <cluster> --region <region>` (eksctl writes it) |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

| Task | Azure (AKS) | Google Cloud (GKE) | Amazon Web Services (EKS) |
|---|---|---|---|
| metrics-server | built in | built in | installed from its release manifest, placed on the system pool |
| Delete everything | `az group delete -n <rg> --yes` | `gcloud container clusters delete <cluster> --zone <zone>` | `eksctl delete cluster -f cluster.yaml` |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## M.2 The image and the registry

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| The registry | `az acr create -g <rg> -n <acr> --sku Standard`, then `az aks update --attach-acr <acr>` | `gcloud artifacts repositories create omni-compass --repository-format=docker --location=<region>` | `aws ecr create-repository --repository-name omni-compass` (and `omni-compass-cloud`) |
| Build the shipped and the cloud image | `az acr build -t omni-compass:<v> -f deploy/Dockerfile .` and `az acr build -t omni-compass-cloud:<v> -f deploy/Dockerfile.cloud --build-arg BASE=<acr>.azurecr.io/omni-compass:<v> .` | `docker build` and `docker push` of both, the second with `--build-arg BASE=<repo>/omni-compass:<v>` | as Google, with `aws ecr get-login-password ... \| docker login` first |
| Tag immutability | `az acr repository update --write-enabled false` per tag, or the registry's retention and lock settings | immutable tags on the repository | `aws ecr put-image-tag-mutability --image-tag-mutability IMMUTABLE` |
| Signature admission (section 26.3) | Image Integrity (Ratify with Azure Policy) | Binary Authorization | Kyverno `verifyImages` or the Sigstore policy controller |

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## M.3 Identity, network and records

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| The governor's cloud credential | none | none | none |
| The cluster's own credential for making machines | the cluster's managed identity (Azure's) | GKE's own service agent (Google's) | the Cluster Autoscaler's IAM role for service accounts, or Karpenter's role (the operator's, created by eksctl) |
| Network policy enforcement | Azure Network Policy Manager, Calico or Cilium (`--network-policy` at creation) | GKE Dataplane V2 or Calico | the VPC CNI's network policy agent, or Calico |
| A persistent volume for the audit | storage class `managed-csi` | storage class `standard-rwo` | storage class `gp3` through the EBS CSI driver |
| The API server's audit log | diagnostic setting, category `kube-audit-admin`, to Log Analytics | Cloud Audit Logs, Admin Activity, in Cloud Logging | control plane logging, type `audit`, to CloudWatch Logs |
| The governor's audit, shipped | the Azure Monitor agent (Container insights) | Cloud Logging's agent (on by default) | the CloudWatch agent or Fluent Bit |

## M.4 The bill

| Task | Azure | Google Cloud | Amazon |
|---|---|---|---|
| Where machine cost lives | Cost Management, cost analysis or export | the Cloud Billing export to BigQuery | the Cost and Usage Report, Cost Explorer |
| Separating the governed pools | the cluster's node resource group, and a tag on each work pool | GKE cost allocation by namespace and node-pool label | a cost allocation tag on each node group's instances, activated in Billing |
| The paired run from GitHub (Appendices F to H) | the `aks-metered` workflow | the `gke-metered` workflow | the `eks-metered` workflow |

Product names, billing granularity and console paths are the providers' own and change; the provider's current documentation is the authority for each, and an installation's plan (chapter 24) records the terms it used.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix N. The Paired Run from GitHub: Every Input

Phase 5's paired run can be made from the repository's own workflows, on GitHub's runners, against the company's cloud account (Appendices F to H set up the account and the repository secrets). Each workflow builds a fresh cluster for every arm of every repetition, runs native and native with Omni-Compass on top in rotation on the same load, prices the machines that existed every 15 seconds, deletes every cluster it made whatever happens, and writes the receipt to the run's summary and as an artifact. The inputs below are the workflows' own; a run's title names its load and its dial.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

## N.1 The inputs every cloud shares

| Input | Default | What it is |
|---|---|---|
| `reps` | 5 | paired repetitions; one runs at a time |
| `arms` | `native compass` | the two arms, in rotation order: native, and native with Omni-Compass on top |
| `duration_s` | 900 | measured seconds per arm |
| `price_per_node_hour` | the family's list price on the day the workflow was written | the price the bill rows use; check the provider's price list on the day |
| `load_steps` | empty (a steady load) | load-generator replicas per step, in order, for a burst or a day-shaped load (for example `1 6 1 8 1 6`) |
| `max_workers` | 4 | the work pool's top; the cloud's autoscaler runs it from 1 to this many |
| `hpa_max` | empty (the manifest's 10) | the service's replica ceiling, the same in both arms; size it with the fleet (nine 200-millicore pods fit a two-processor machine) |
| `organism`, `load_max`, `organism_scale` | empty, 6, 1 | one of the repository's six organisms with this cluster inside it, its peak load, and its copies on one clock (1, 10 or 100); empty runs the service alone |
| `dial` | `save` | the dial of Omni-Compass on top: `save`, the tune-down, for every bill result; `max` is never read as a bill result (section 19.2) |

Every cloud workflow runs the governor in the cloud mode (`OMNI_MACHINES_ONLY=1`, which passes `--give-back-only`, chapter 19).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

## N.2 The inputs of each cloud

| Workflow | Input | Default | What it is |
|---|---|---|---|
| aks-metered | location | eastus | the Azure region |
| | vm_size | Standard_D2s_v4 | the work machines' size |
| | tier | free | the clusters' control-plane tier, free or standard (outside the bill; the way past free-tier capacity refusals) |
| | worker_pools, pool_prices | empty | several work pools, one machine family each with its own top (size:max,...), and each pool's list price (work0=<price>;work1=<price>;...): the way past a per-family allowance |
| | replica_ceiling | empty | outside the cloud mode only: how far the governor may raise the service's replica cap; the cloud mode moves no autoscaler |
| gke-metered | project | the GCP_PROJECT secret | the Google Cloud project |
| | zone | us-central1-a | the zone |
| | machine | e2-standard-2 | the work machines' type |
| eks-metered | region | us-east-1 | the AWS region |
| | instance | m5.large | the work machines' instance type |

The repository's secrets each workflow reads are AZURE_CREDENTIALS (Azure), GCP_CREDENTIALS and GCP_PROJECT (Google Cloud), and AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY (Amazon). They are set once in the repository's settings, never printed, and used only to build and delete the run's own clusters; the governor inside the clusters never sees them (section 14.5).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.
www.omni-compass.com

## N.3 A second Azure workflow, for machines already paid for

`aks-metered-full` runs the same paired design with the governor's full wiring on the service (the pod autoscaler's target, the pods' processor limits and the modelled power sense) instead of the machines alone. It exists to measure the governor's levers beside the cloud's own, not to make a bill result: in a cloud every extra pod is billed, and the cloud mode alone is read for the bill (chapter 19). Its inputs are the shared ones above, with a burst of `1 4 1 6 1 9` and three repetitions of 600 seconds by default.

## N.4 Reading the run

The run's summary carries the receipt, row by row, read by the rules of chapter 11 and section 19.6: GAIN, EVEN or LOSS for each judged row, with its 95% interval, and the shown rows beside them. The artifact `live-reps-aks` (or the cloud's equivalent) holds the same table as Markdown and JSON, with the repository's notice, licence and disclosures attached. A receipt is read only against the engine version of the commit it ran on (`python3 tools/omni_version.py --commit <sha>`, chapter 12), which each repetition's job prints at its start.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix O. The License

The license under which this book and the software are made available, as it stands in the file `LICENSE` at the root of the repository.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*
**www.omni-compass.com**

```
Omni-Compass Evaluation License, version 1.0 (revised 11 October 2026)
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0

Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
All patents, copyrights and trademarks filed in the USA. www.omni-comp
    ↪ ass.com

THIS IS NOT AN OPEN-SOURCE LICENSE. The Software is proprietary. It is
    ↪  published so that it can be evaluated, reproduced
and audited, and for no other purpose. It is licensed, not sold. Evalu
    ↪ ation and simulation are free of charge; every
other use, and in particular every commercialization or monetization o
    ↪ f any kind, requires a signed, paid Omni-Compass
Enterprise License.

ACCEPTANCE. By downloading, copying, forking, installing, running, sca
    ↪ nning or otherwise using any part of the Software,
You accept this license. If You do not accept it, You may not use the
    ↪ Software. If You use the Software on behalf of an
organization, You represent that You have authority to bind it, and "Y
    ↪ ou" includes that organization and its
Affiliates.

1. DEFINITIONS

   1.1 "Licensor" means The Omni-Compass LLC.
   1.2 "Software" means everything Licensor publishes or makes availab
    ↪ le under this license, in any form: the source
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    code and any object code; the Omni-Compass engine, the compass
 ↪ law, the governors, controllers, brains, shields,
    plugs and wiring; the harnesses, tools, scripts, workflows, con
 ↪ figurations, presets, catalogs, organisms and realms;
    the documentation, the manual, the book, the specifications, th
 ↪ e preregistrations and the receipts; the benchmarks,
    data, tables, reports and Results published by Licensor; and ev
 ↪ ery update, version and copy of any of them.
1.3 "Mechanism" means the methods, processes, control laws, algorit
 ↪ hms, architectures and systems embodied in or
    described by the Software, including everything described in th
 ↪ e patent applications Licensor has filed.
1.4 "Evaluation" means internal use by You, free of charge, solely
 ↪ to test, study and assess the Software in
    contemplation of an Enterprise License: reading it; running its
 ↪  tests, verifiers, benchmarks and simulations; and
    running it in observe, watch, dry-run or shadow mode, or in a t
 ↪ est environment, on systems You own or control,
    where it writes to no system that serves production traffic, re
 ↪ al users, customers, patients or passengers, and to
    no power system, vehicle, machine or physical process outside a
 ↪  test bench.
1.5 "Simulation" means running the Software against modelled, recor
 ↪ ded or synthetic systems, including the organisms,
    realms, traces and simulators published with the Software.
1.6 "Production Use" means any use that is not Evaluation or Simula
 ↪ tion, including any use in which the Software, the
    Mechanism or any output of either controls, governs, tunes, siz
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ es, schedules or otherwise influences a system that
       serves production workloads or real users, or any physical proc
    ↪ ess outside a test bench.
   1.7 "Commercial Use" means any use for, or in connection with, comm
    ↪ ercial advantage, revenue, cost reduction in
       operations, or monetary or other compensation, including consul
    ↪ ting, integration, managed, outsourcing, training,
       benchmarking-for-hire and advisory services, and any use by or
    ↪ for a for-profit operation beyond Evaluation.
   1.8 "Hosted Service" means making the Software, the Mechanism or an
    ↪ y of their functionality available to any third
       party over a network or as a service, including software-as-a-s
    ↪ ervice, platform-as-a-service, a managed or
       outsourced service, a cloud marketplace offering, or a feature
    ↪ of another product or service.
   1.9 "Derivative" means any modification, adaptation, translation, p
    ↪ ort, extension or re-implementation of the Software
       or of the Mechanism, and any work that incorporates, is based o
    ↪ n, or is built using the Software as a reference.
   1.10 "Results" means the measurements, tables, receipts, logs, repo
    ↪ rts and other outputs produced by running the
       Software.
   1.11 "Enterprise License" means a written agreement for the Softwar
    ↪ e signed by Licensor and a licensee, under which
       the licensee pays the fees it sets out; it may include order fo
    ↪ rms, riders and statements of work.
   1.12 "Marks" means the names "Omni-Compass" and "OmniCompass", the
    ↪ Omni-Compass compass rose with its ring of Greek
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
      letters, and every related name, logo and trade dress of Licens
    ↪ or.
   1.13 "Affiliate" means any entity that controls, is controlled by,
    ↪ or is under common control with You, where control
      means ownership of more than half of the voting interests or th
    ↪ e power to direct management.
   1.14 "You" means the individual or entity exercising rights under t
    ↪ his license, with its Affiliates where the
      Acceptance paragraph above includes them.

2. WHAT THIS LICENSE ALLOWS

   2.1 Copyright license. Subject to Your compliance with this license
    ↪ , Licensor grants You a personal, worldwide,
      royalty-free, non-exclusive, non-transferable, non-sublicensabl
    ↪ e and revocable license to download, copy, read,
      run and modify the Software solely for Evaluation and Simulatio
    ↪ n, and to reproduce and publish Results as section
      4 allows.
   2.2 Patent license. Subject to the same conditions, Licensor grants
    ↪  You a license of the same scope, under the patent
      rights Licensor holds or will hold in the filed applications an
    ↪ d any patent issuing from them, solely to the
      extent needed to carry out Evaluation and Simulation with the S
    ↪ oftware. No patent license is granted for any
      Production Use, Commercial Use or Hosted Service, for any Deriv
    ↪ ative used outside Evaluation or Simulation, or for
      any implementation of the Mechanism other than the Software its
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ elf.
   2.3 Forks. You may keep a copy or a fork of the Software on a code-
    ↪ hosting service, publicly or privately, only for
       Evaluation and Simulation, with this license and every notice i
    ↪ ntact. A fork confers no right beyond this license.
   2.4 Reservation. Every right not expressly granted is reserved by L
    ↪ icensor. No right, title, license or interest is
       granted by implication, estoppel, exhaustion or otherwise.

3. WHAT THIS LICENSE DOES NOT ALLOW

   Unless an Enterprise License expressly permits it, You may not, and
    ↪  may not let or help anyone else:
   (a) make any Production Use or Commercial Use of the Software, the
    ↪ Mechanism or any Derivative;
   (b) commercialize or monetize the Software, the Mechanism, any Deri
    ↪ vative or any Result in any way, directly or
       indirectly, including by charging for access, services, support
    ↪ , results, savings or outcomes they produce;
   (c) offer the Software, the Mechanism or any Derivative as, or as p
    ↪ art of, a Hosted Service;
   (d) sell, resell, license, sublicense, lease, rent, lend, distribut
    ↪ e, publish as a package or binary, or otherwise
       transfer the Software or any Derivative, except forks under sec
    ↪ tion 2.3;
   (e) incorporate the Software, any part of it or any Derivative into
    ↪  any product, service, platform, appliance, image,
       firmware or offering;
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   (f) use the Software, the Mechanism, the documentation or the Resul
    ↪ ts to design, build, train, tune, benchmark for
       sale, or improve any product or service that competes with Omni
    ↪ -Compass, or to re-implement the Mechanism;
   (g) use the Software or the Results to train, fine-tune or evaluate
    ↪  any machine-learning model intended to reproduce,
       imitate or substitute for the Mechanism;
   (h) remove, alter or obscure any notice, the SPDX identifiers, the
    ↪ patent and trademark notices, this license, the
       NOTICE or DISCLOSURES.md, or circumvent, disable or work around
    ↪  any limit, switch or safeguard in the Software,
       including the kill switch and the hand-back to native settings;
   (i) connect the Software so that it writes to a safety-critical sys
    ↪ tem, as section 10 describes;
   (j) use the Marks, except to refer to Omni-Compass accurately as TR
    ↪ ADEMARKS.md allows; or
   (k) use the Software in violation of any law, or in breach of secti
    ↪ on 11.

4. REPRODUCING AND PUBLISHING RESULTS

   You may reproduce the benchmarks published with the Software and pu
    ↪ blish Your own Results, provided that You: (a)
   name the Software's engine version and the exact commit You ran; (b
    ↪ ) report every measured row, the unfavourable ones
   included, with the evidence class the Software assigns to it; (c) m
    ↪ ake Your complete raw files and configuration
   available to Licensor on request; (d) do not state or imply any end
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ orsement, certification or affiliation by
   Licensor; and (e) carry the notice at the top of README.md. Publish
    ↪ ing Results is not Commercial Use so long as no
   compensation is received for it, and it grants no right beyond this
    ↪  license.

5. THE OMNI-COMPASS ENTERPRISE LICENSE

   5.1 Every Production Use, Commercial Use, Hosted Service, distribut
    ↪ ion, incorporation, OEM or embedded use, and every
       other commercialization or monetization of the Software, the Me
    ↪ chanism or any Derivative requires an Enterprise
       License, signed before the use begins and paid as it provides.
    ↪ Moving from Evaluation to Production Use requires
       an Enterprise License before the first write to any production
    ↪ system.
   5.2 An Enterprise License sets its own scope, fees, term, support,
    ↪ warranties, indemnities and limits
       (ENTERPRISE_LICENSE.md summarizes the standard terms; the signe
    ↪ d agreement alone is binding). For its licensee and
       its scope, a signed Enterprise License governs where it differs
    ↪  from this license.
   5.3 Enterprise licensing: The Omni-Compass LLC, main office infrast
    ↪ ructure, toll-free direct 1-888-339-9551, system route asset 1-8
    ↪ 88-DEX-YLL-1, www.omni-compass.com.

6. OWNERSHIP AND INTELLECTUAL PROPERTY
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
   6.1 Licensor and its licensors own the Software, the Mechanism, the
    ↪  Marks and all intellectual property rights in
       them, including in every Derivative to the extent it embodies t
    ↪ hem. Nothing in this license transfers ownership.
   6.2 All patents, copyrights and trademarks covering the Omni-Compas
    ↪ s engine, its mathematics and its software have
       been filed in the United States by The Omni-Compass LLC. No fil
    ↪ ing number is stated here, and no grant,
       registration or approval is claimed; the status of every filing
    ↪  is held by the office where it was filed. Every
       person who downloads, copies, forks, scans or runs the Software
    ↪  is on notice of these filings and of this
       license (PATENTS.md, TRADEMARKS.md).
   6.3 Any use outside this license, without an Enterprise License, is
    ↪  unlicensed and infringes Licensor's copyright and
       other rights, in addition to breaching this license.

7. PATENT AND RIGHTS DEFENSE

   If You or any of Your Affiliates bring or join any claim, countercl
    ↪ aim or proceeding alleging that the Software or
   the Mechanism infringes, misappropriates or violates any patent, co
    ↪ pyright or other intellectual property right, or
   challenge the validity or enforceability of any of Licensor's right
    ↪ s in them, every license granted to You under this
   license ends automatically on the date the claim or challenge is fi
    ↪ led.
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
8. FEEDBACK AND CONTRIBUTIONS

   8.1 If You give Licensor comments, suggestions, ideas, reports or o
    ↪ ther feedback about the Software, You grant
       Licensor a perpetual, irrevocable, worldwide, royalty-free righ
    ↪ t to use, disclose and exploit it for any purpose,
       without obligation of any kind.
   8.2 Contributions are accepted only under the contributor license a
    ↪ greement (.github/CLA.md).

9. CHANGES

   Licensor reserves the right to change the Software, its documentati
    ↪ on, its results and these terms at any time,
   without notice. A changed version of these terms applies to every c
    ↪ opy of the Software obtained, and to every use
   made, after it is published in the Software's repository; continued
    ↪  use after publication is acceptance of the
   changed terms. Licensor may discontinue the Software, or this licen
    ↪ se for future copies, at any time.

10. SAFETY

   The Software is not designed, tested or certified for safety-critic
    ↪ al use. Under this license it may not write to
   any system whose failure could cause death, injury, environmental d
    ↪ amage or significant property damage, including
   vehicles and aircraft, medical devices, power-grid protection and d
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ ispatch, industrial safety systems and life-safety
   systems; such systems may be studied only in Simulation. Any live c
    ↪ onnection to a physical system requires an
   Enterprise License, a written safety case accepted by the system's
    ↪ owner, and every review that system's regulator or
   owner requires. The machines' own controls and protections stay in
    ↪ place; the kill switch must remain operable at all
   times.

11. EXPORT CONTROL, SANCTIONS AND LAWFUL USE

   You will comply with every export-control, sanctions, anti-corrupti
    ↪ on and other law that applies to the Software and
   to Your use of it, including the U.S. Export Administration Regulat
    ↪ ions and the sanctions administered by the U.S.
   Treasury's Office of Foreign Assets Control. You represent that You
    ↪  are not located in, organized under the laws of,
   or ordinarily resident in a country or region subject to comprehens
    ↪ ive U.S. sanctions, and that You are not on, or
   owned or controlled by a party on, any U.S. restricted-party list.
    ↪ You will not use the Software for any end use
   prohibited by those laws, including the design, development or prod
    ↪ uction of weapons of mass destruction.

12. DATA AND PRIVACY

   The Software contains no telemetry. It sends nothing to Licensor an
    ↪ d makes no network connection of its own except to
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   the systems You point it at. Your data and Your Results stay on You
    ↪ r systems and are Yours, subject to section 4. If
   You send Licensor any material (for example in an issue or an e-mai
    ↪ l), You represent that You may lawfully do so and
   that it contains no personal data or confidential information of an
    ↪ y third party.

13. THIRD-PARTY COMPONENTS

   Files that carry a different license notice (for example tests/thir
    ↪ d_party/) and third-party data sets (for example
   the PlanetLab traces under fleet/traces/ and k8s_controlplane/trace
    ↪ s/) remain under their own terms
   (THIRD_PARTY_NOTICES.md). Third-party software the Software runs on
    ↪  or connects to (Kubernetes, KEDA, Karpenter,
   NVIDIA drivers, databases and others) is licensed by its owners und
    ↪ er their own terms.

14. TERM AND TERMINATION

   14.1 This license lasts until it ends under this section or section
    ↪  7.
   14.2 It ends automatically, without notice, if You breach it. If Yo
    ↪ ur breach is Your first, was inadvertent, and is
       not a breach of sections 3(a) to 3(g) or section 10, Your lice
    ↪ nse is reinstated if You cure the breach within
       thirty days after You learn of it or after Licensor notifies Y
    ↪ ou, whichever comes first.
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
   14.3 Licensor may end this license for any person at any time by wr
    ↪ itten notice, and for all persons for future
        copies under section 9.
   14.4 When this license ends, You must stop all use of the Software,
    ↪  delete or destroy every copy and Derivative in Your
        possession or control, and, on request, certify in writing tha
    ↪ t You have done so.
   14.5 Sections 1, 3, 4(b) to 4(e), 6, 7, 8, 11, 12, 15 to 18 and 19
    ↪ survive the end of this license.

15. NO WARRANTY

   TO THE FULLEST EXTENT PERMITTED BY LAW, THE SOFTWARE, THE MECHANISM
    ↪  AND THE RESULTS ARE PROVIDED "AS IS" AND "AS
   AVAILABLE", WITH ALL FAULTS AND WITHOUT WARRANTY OR CONDITION OF AN
    ↪ Y KIND, EXPRESS, IMPLIED OR STATUTORY, INCLUDING ANY
   WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITL
    ↪ E, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, OR
   THAT THE SOFTWARE WILL BE ERROR-FREE OR UNINTERRUPTED. NO RESULT IS
    ↪  A PROMISE OF ANY OUTCOME ON ANY OTHER SYSTEM;
   MODELLED RESULTS (EVIDENCE CLASS S) ARE STATEMENTS ABOUT THE MODEL,
    ↪  NOT MEASUREMENTS (DISCLOSURES.md). YOU USE THE
   SOFTWARE AT YOUR OWN RISK AND ARE RESPONSIBLE FOR ITS USE ON YOUR S
    ↪ YSTEMS.

16. LIMITATION OF LIABILITY

   TO THE FULLEST EXTENT PERMITTED BY LAW, LICENSOR, ITS MEMBERS, MANA
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

```
    ↪ GERS, EMPLOYEES, CONTRACTORS AND LICENSORS SHALL NOT
   BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXE
    ↪ MPLARY OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF
   PROFITS, REVENUE, SAVINGS, DATA, USE OR GOODWILL, OR ANY COST OF SU
    ↪ BSTITUTE GOODS OR SERVICES, ARISING OUT OF OR
   RELATING TO THE SOFTWARE, THE MECHANISM, THE RESULTS OR THIS LICENS
    ↪ E, UNDER ANY THEORY OF LIABILITY, EVEN IF ADVISED OF
   THEIR POSSIBILITY. LICENSOR'S TOTAL LIABILITY FOR ALL CLAIMS SHALL
    ↪ NOT EXCEED ONE HUNDRED U.S. DOLLARS (USD 100). THESE
   LIMITS APPLY EVEN IF A REMEDY FAILS OF ITS ESSENTIAL PURPOSE.

17. INDEMNITY

   You will defend, indemnify and hold harmless Licensor and its membe
    ↪ rs, managers, employees and contractors from and
   against every claim, loss, liability, damage, cost and expense (inc
    ↪ luding reasonable attorneys' fees) arising out of
   or relating to (a) Your use of the Software outside this license, (
    ↪ b) any Production Use, Commercial Use or Hosted
   Service without an Enterprise License, (c) Your Derivatives, (d) Yo
    ↪ ur Results as You publish them, or (e) Your
   violation of any law or third-party right.

18. U.S. GOVERNMENT END USERS

   The Software is "commercial computer software" and "commercial comp
    ↪ uter software documentation" as those terms are
   used in 48 C.F.R. 2.101, 12.212 and 227.7202. A U.S. Government end
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪  user acquires only the rights set out in this
   license, consistent with 48 C.F.R. 12.212 and 227.7202-1 through 22
    ↪ 7.7202-4.

19. GENERAL

   19.1 Governing law. This license is governed by the laws of the Sta
    ↪ te of Delaware and the federal laws of the United
        States, without regard to conflict-of-laws rules. The United N
    ↪ ations Convention on Contracts for the
        International Sale of Goods does not apply.
   19.2 Venue. The state courts of the State of Delaware and the Unite
    ↪ d States District Court for the District of
        Delaware have exclusive jurisdiction over any dispute arising
    ↪ out of or relating to this license, and You consent
        to their personal jurisdiction. Licensor may seek
        injunctive or other equitable relief in any court of competent
    ↪  jurisdiction, without posting a bond, to protect
        its intellectual property or confidential information; a breac
    ↪ h of sections 3 or 10 would cause Licensor
        irreparable harm.
   19.3 Enforcement costs. In any action to enforce this license, the
    ↪ prevailing party may recover its reasonable
        attorneys' fees and costs.
   19.4 Assignment. You may not assign or transfer this license or any
    ↪  right under it, by operation of law or otherwise;
        any attempt is void. Licensor may assign this license freely.
   19.5 Notices. Notices to Licensor go to The Omni-Compass LLC by tel
```

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.
All patents, copyrights and trademarks filed in the USA.

```
    ↪ ephone at its main office infrastructure (toll-free direct 1-888
    ↪ -339-9551; system route asset 1-888-DEX-YLL-1) or through www.om
    ↪ ni-compass.com. Licensor may give notice to
        You through the Software's repository or at any address You ha
    ↪ ve given it.
   19.6 Severability and waiver. If any provision is held unenforceabl
    ↪ e, it is enforced to the maximum extent permitted and
        the rest of this license remains in effect. No failure or dela
    ↪ y in enforcing any provision is a waiver of it.
   19.7 Entire license. This license, with the NOTICE and DISCLOSURES.
    ↪ md, is the entire agreement between You and
        Licensor about its subject; for a licensee and its scope, a si
    ↪ gned Enterprise License governs where it differs.
   19.8 Interpretation. "Including" means "including without limitatio
    ↪ n". Headings are for convenience only. The
        English text of this license controls over any translation.
   19.9 Copies. Every copy, fork, export, report, archive or printout
    ↪ of any part of the Software must carry this
        LICENSE, the NOTICE and DISCLOSURES.md unchanged.

All rights reserved. Revision history: version 1.0 (evaluation and sim
    ↪ ulation only; first published 2026); revised
10 October 2026 (the notice in one wording everywhere); revised 11 Oct
    ↪ ober 2026 (definitions, patent license for
evaluation, hosted-service and machine-learning restrictions, results
    ↪ publication, patent and rights defense, safety,
export control, data, U.S. Government end users, term, cure, indemnity
    ↪ , Delaware governing law and venue).

Enterprise licensing: The Omni-Compass LLC, main office infrastructure
    ↪ , toll-free direct 1-888-339-9551, system route asset 1-888-DEX-
    ↪ YLL-1, www.omni-compass.com
```

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix P. Licensing: Questions and Answers

Plain answers to the questions buyers, engineers and their lawyers ask. The binding terms are in `LICENSE`; where this page and `LICENSE` differ, `LICENSE` governs. Every declaration is in `DISCLOSURES.md`.

**Is Omni-Compass open source?** No. The source is published so that it can be evaluated, reproduced and audited. It is licensed under the Omni-Compass Evaluation License 1.0 (`LicenseRef-OmniCompass-Evaluation-1.0`), which permits evaluation and simulation only.

**What may I do without a commercial license?** Evaluation and simulation, free of charge (`LICENSE`, sections 1.4, 1.5 and 2): read the code; run `verify.py`, the simulations and the benchmarks; reproduce the published results and publish your own, every row included (section 4); run Omni-Compass in watch, dry-run or shadow mode, or in a test environment, on systems you own or control, where it writes to nothing that serves production traffic or real users and to no physical process outside a test bench. A limited patent license comes with it, for evaluation only (section 2.2).

**What needs the Omni-Compass Enterprise License?** Every commercialization or monetization of any kind; production use; operating any system beyond evaluation; offering it, or its function, as a hosted, managed or software-as-a-service offering; redistributing it; incorporating it, or any part or derivative of it, into a product, service, appliance or firmware; using it or its results to build or train a competing product (`LICENSE`, sections 3 and 5). The Enterprise License is a written agreement signed by The Omni-Compass LLC and paid for; its standard terms are summarized in `ENTERPRISE_LICENSE.md` (production, hosted service, OEM and embedded, managed service provider, research).

**May I run it as a service for others?** Not without an Enterprise License with a hosted-service rider (`LICENSE`, section 3(c); `ENTERPRISE_LICENSE.md`, section 1).

**May I connect it to a grid, a plant, a vehicle or a medical device?** Not under the evaluation license: such systems may be studied only in simulation. A live connection to any physical system needs an Enterprise License and a written safety case accepted by the system's owner (`LICENSE`, section 10).

**How is the Enterprise License priced?** Against the measured gain on the customer's own paired receipts (native against Omni-Compass on the same system, the same load and the same clock). Terms are set in each signed agreement; no price stated anywhere in this repository is an offer.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

**Are patents involved?** All patents, copyrights and trademarks covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC (`PATENTS.md`). The evaluation license grants no patent license beyond evaluation.

**May I use the name or the compass rose?** Only to refer to Omni-Compass accurately (`TRADEMARKS.md`).

**May I contribute?** Contributions require the contributor license agreement (`.github/CLA.md`): every contribution is assigned to The Omni-Compass LLC, with a patent license from the contributor.

**Does Omni-Compass send data anywhere?** No. The software contains no telemetry and makes no network connection of its own except to the systems an operator points it at (a Kubernetes API, `nvidia-smi`, a meter command).

**What do scanners report?** The license is declared in machine-readable form for Black Duck, FOSSA, Snyk and REUSE (`pyproject.toml`, `REUSE.toml`, `.fossa.yml`, `.snyk`, `sbom/`); third-party components are listed in `THIRD_PARTY_NOTICES.md`.

**Can the terms change?** Yes. The Omni-Compass LLC reserves the right to change the software, its documentation, its results and these terms at any time, without notice; a changed version applies to copies obtained and uses made after it is published (`LICENSE`, section 9). A signed Enterprise License governs its own term.

**Which law governs?** The laws of the State of Delaware and the federal laws of the United States, with exclusive venue in the Delaware state courts and the United States District Court for the District of Delaware (`LICENSE`, section 19). The Omni-Compass LLC is based in Florida.

**Who do I contact?** The Omni-Compass LLC, main office infrastructure, toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1, www.omni-compass.com.

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

APPENDIX

# Appendix Q. Third-Party Notices

Omni-Compass uses the following third-party components. Each remains under its own license; nothing in the Omni-Compass Evaluation License changes those terms. Installed versions are those resolved from `requirements.txt`.

| Component | Use | License |
|---|---|---|
| NumPy | numerical arrays | BSD-3-Clause |
| Matplotlib | charts in the reports | Matplotlib License (PSF-based, BSD-compatible) |
| ReportLab (open-source edition) | the printable book and reports | BSD-3-Clause |
| Numba | compiled inner loops of the motion-axis plant | BSD-2-Clause |
| PyTorch (on GPU machines only, not installed by `requirements.txt`) | the GPU workload of the benchmark | BSD-3-Clause |
| `tests/third_party/hpa_independent.py` | an independently written HPA reference used unmodified in tests | its own terms, stated in the file |
| Liberation Serif and Liberation Sans fonts (embedded in the PDF book) | typesetting | SIL Open Font License 1.1 |
| DejaVu Sans and DejaVu Sans Mono fonts (embedded in the PDF book) | typesetting | Bitstream Vera / DejaVu license (free) |
| PlanetLab / CoMon VM CPU traces, March 2011 (`fleet/traces/`, `k8s_controlplane/traces/`; source in their `PROVENANCE.json`) | held-out demand for the modelled fleet | their own terms (public research data); no Omni-Compass copyright is claimed in them |

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

| Component | Use | License |
|---|---|---|
| Google cluster-usage traces 2011 (`clusterdata-2011-2`), from which `results/traces/google2011/` is derived (`docs/TRACES_PREREGISTRATION.md`) | the public demand trace on the real cluster | Google's published terms for the traces; no Omni-Compass copyright is claimed in the trace itself |

Kubernetes, kind, `kubectl`, the NVIDIA driver and `nvidia-smi` are not distributed with Omni-Compass; it calls them where an operator has installed them. Their names are the property of their owners (`DISCLOSURES.md`, section 1).

*© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and simulation use only.*
*All patents, copyrights and trademarks filed in the USA.*

BACK MATTER

# Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

**The Omni-Compass LLC**

Owner and developer: AJ Dubra

Main office infrastructure: toll-free direct 1-888-339-9551, system route asset 1-888-DEX-YLL-1

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

© 2026 The Omni-Compass LLC. All rights reserved. **Evaluation and simulation use only.** Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` and `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` All patents, copyrights and trademarks filed in the USA. www.omni-compass.com